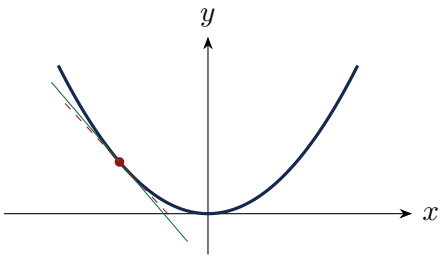
Foundations of Machine Learning

A first book

Fit a line. Know why. Do it by hand.



Prepared by

Krushna Parmar

Roll No. **202611032** • Course IT582

Grounded in the IT582 lectures. Every derivation from class is here. The writing is a book, not a slide dump: a question, a picture, the algebra, then something for you to do.

Contents

How to Read This Book **xi**

Lecture coverage map **xiii**

I The question: learn a function **1**

1 What is Machine Learning? **3**

1.1 A definition you can test . . . 3
1.2 A model is a function with knobs . . . 4
1.3 How wrong, as a single number . . . 4
1.4 The split that stops you lying . . . 5
1.5 A preview of how the knobs move . . . 6
1.6 Three choices, and there are only three . . . 6
1.7 The same idea, wearing other clothes . . . 7

2 Linear Regression **9**

2.1 One feature, two knobs . . . 9
2.2 Three points, computed dead . . . 9
2.3 Many features, still a dot product . . . 10
2.4 Linear in *what*, exactly? . . . 11
2.5 The bowl . . . 12

3 Why Squared Error? Least Squares as Gaussian MLE **15**

3.1 Write the noise . . . 15
3.2 The whole training set . . . 16

II Rolling downhill **19**

4 Gradient Descent **21**

4.1 The update . . . 21
4.2 Why the minus sign works on both sides . . . 21

4.3 A derivative you can see with arithmetic . . . 22
4.4 The slope of the cost . . . 22
4.5 Four values of α, one bowl . . . 23
4.6 Contours, arrows, a surface . . . 24
4.7 When to stop . . . 25

5 A Complete Gradient Descent Walkthrough 27
5.1 The bowl, the gradient, the update . . . 27
5.2 The four learning rates from class . . . 28
5.3 Stopping criteria (from the notes) . . . 32
5.4 Steepest ascent, finished (Lec 6 last page + Lec 7 first page) . . . 32
5.5 The batch update, written the way class writes it . . . 33

6 Gradients and Hessians in Full 37
6.1 The gradient of a function of many variables . . . 37
6.2 Why the gradient points the steepest way (a proof) . . . 37
6.3 The Hessian: second derivatives in a matrix . . . 38
6.4 Gradient & Hessian of a linear function . . . 39
6.5 Gradient & Hessian of a quadratic function . . . 39
6.6 Least squares: the normal equation . . . 40
6.7 Why the bowl really is a bowl . . . 41
6.8 Using curvature: Newton's method . . . 42

7 Batch and Stochastic Gradient Descent 45
7.1 Recap: the batch update uses all the data . . . 45
7.2 Batch gradient descent . . . 45
7.3 Stochastic gradient descent . . . 45
7.4 Batch vs Stochastic: the trade-off . . . 46

III Units, a tax on slopes 49

8 Vectorization and Feature Scaling 51
8.1 One example: from a sum to a dot product . . . 51
8.2 All n examples: the design matrix . . . 51
8.3 The cost, vectorized, then differentiated . . . 52
8.4 Gradient descent versus the closed form . . . 53
8.5 Why feature scaling exists — the house example . . . 53
8.6 The scaling formula, and the three rules on the last slide . . . 54

9 Regularization, Shrinkage, and the Ridge Normal Equation 59
9.1 Fitting quality: three pictures . . . 59

9.2 Two ways to address overfitting . . . 60
9.3 Ridge regression: the objective . . . 61
9.4 What if λ is insane? . . . 61
9.5 Ridge gradient descent, and why it is called shrinkage . . . 62
9.6 The regularized normal equation . . . 63
9.7 Choosing λ: the bias–variance trade-off . . . 64

10 From a Hard Constraint to Lambda 67
10.1 Overfitting as a budget . . . 67
10.2 The fence becomes a tax . . . 68
10.3 The path, drawn . . . 68

IV When matrices misbehave, and why least squares 71

11 Gradient of a Determinant, and Eigenvalues as Optimization 73
11.1 Minors, cofactors, adjugate — the vocabulary . . . 73
11.2 Gradient of the determinant . . . 73
11.3 Why $A \cdot \mathrm{adj}(A) = |A|I$ . . . 74
11.4 Gradient of $\log|A|$ . . . 75
11.5 Eigenvalues as an optimization problem . . . 76
11.6 A small matrix-calculus toolkit (rest of Lec 9) . . . 78

12 The Pseudoinverse and SVD 81
12.1 When the ordinary inverse is illegal . . . 81
12.2 SVD: $X = U\Sigma V^\top$ . . . 81
12.3 Inverting Σ the honest way . . . 82
12.4 A rank-1 matrix you can see . . . 84

V Likelihoods, priors, and bending the line 87

13 Probability From Zero, and Two Ways to Use It 89
13.1 What a probability even is (two honest answers) . . . 89
13.2 Random variable, distribution, parameter . . . 90
13.3 Expectation: the average you can compute before rolling . . . 90
13.4 Independence and the product rule (the engine of likelihood) . . . 91
13.5 A tiny worked example: a likelihood by hand . . . 91
13.6 The fork: frequentist vs Bayesian learning . . . 92

14 Probabilistic Models, MLE, and MAP 95
14.1 $P(x;\theta)$ versus $P(x \mid \theta)$ . . . 95

14.2 The likelihood, the log, the NLL . . . 95
14.3 Worked example: n coin tosses (Bernoulli MLE) . . . 96
14.4 From MLE to MAP: Bayes, in the notes' labels . . . 97
14.5 MAP, derived (Lecture 13 page 4) . . . 98
14.6 MAP with a Gaussian prior *is* ridge regression . . . 98
14.7 Coin again, now with a Beta prior . . . 102
14.8 What the 2026 reading list adds on top of class . . . 103

15 Smoothing, and Being Fully Bayesian 105
15.1 The zero-probability disaster . . . 105
15.2 Laplace smoothing: add one of each . . . 105
15.3 More than two outcomes: the Dirichlet . . . 107
15.4 Being fully Bayesian: stop picking, start integrating . . . 107
15.5 MLE, MAP, and fully Bayesian, side by side . . . 109

16 MLE and MAP for Probabilistic Linear Regression 111
16.1 Two questions, two estimators . . . 111
16.2 The probabilistic model of linear regression . . . 112
16.3 The Gaussian, univariate and multivariate . . . 113
16.4 The star derivation: MLE for the line *is* least squares . . . 113
16.5 Adding a belief: the MAP estimate . . . 114
16.6 The star derivation: MAP for the line *is* ridge . . . 115
16.7 Seeing it in code . . . 116
16.8 One xkcd, and why it fits . . . 117
16.9 The whole chapter on one line . . . 117

17 Basis Functions: Bending a Straight Line 119
17.1 Recap: the straight-line hypothesis . . . 119
17.2 More complex models from the same linear machine . . . 119
17.3 The linear basis function model . . . 120
17.4 A zoo of basis functions . . . 120
17.5 Why polynomial is "global" and Gaussian is "local" (the derivation) . . . 121
17.6 Choosing a basis: match the shape of the data . . . 122
17.7 Fitting a polynomial curve with a linear model . . . 122
17.8 The central challenge of machine learning: generalization . . . 123
17.9 Statistical learning theory: the assumptions that make it possible . . . 123
17.10 Under-fitting, over-fitting, and capacity . . . 124
17.11 One xkcd, and why it fits . . . 125
17.12 The chapter in one breath . . . 125

18 MLE Across Distributions: A Derivation Catalog 127

18.1 The universal recipe . . . 127
18.2 The discrete distributions . . . 128
18.3 The continuous distributions . . . 129
18.4 When they "change the terms": a distribution you have not memorised . . . 130
18.5 A quick-reference table . . . 131
18.6 Practice: derive these yourself . . . 132

VI Insem 2: generalization and classification 133

19 Capacity, Bias and Variance 135
19.1 Capacity and the hypothesis space . . . 135
19.2 From the textbook: training MSE vs test MSE (ISLR §2.2.1) . . . 137
19.3 The effective number of parameters . . . 137
19.4 Bias and variance of an estimator . . . 138
19.5 The bias–variance decomposition, derived . . . 139
19.6 Regularization as the dial . . . 140
19.7 Estimating bias and variance with the bootstrap . . . 142
Lecture slides for reference . . . 143

20 LASSO: Shrinkage that Selects 147
20.1 The objective . . . 147
20.2 The special case $\mathbf{X}^\top\mathbf{X} = \mathbf{I}$, derived step by step . . . 147
20.3 The geometry: why corners create zeros . . . 149
20.4 The p-norm family . . . 150
20.5 On real data: the regularization path . . . 151
20.6 From the textbook: the Bayesian view (ISLR §6.2.2) . . . 151
20.7 From the textbook: choosing λ (ISLR §6.2.3) . . . 151
Lecture slides for reference . . . 152

21 Cross-Validation and the Bootstrap 155
21.1 The problem: training error lies . . . 155
21.2 Train, validation, test . . . 156
21.3 The validation set approach . . . 156
21.4 Leave-one-out cross-validation (LOOCV) . . . 156
21.5 k-fold cross-validation . . . 157
21.6 From the textbook: the LOOCV shortcut, derived (ISLR eq. 5.2) . . . 159
21.7 LOOCV on the DC level, derived . . . 160
21.8 The bootstrap . . . 161
Lecture slides for reference . . . 163

22 Classification by Distance: Centroids and K-NN **167**
22.1 The setting . . . 167
22.2 Idea 1: distance from the class means . . . 167
22.3 Idea 2: nearest neighbour (1-NN) . . . 169
22.4 K-nearest neighbours . . . 170
22.4.1 Choosing K . . . 172
22.4.2 Breaking ties . . . 173
22.5 K-NN as a probability estimate (derivation) . . . 173
22.6 Distance functions . . . 174
22.7 Normalization: why feature scales matter . . . 175
22.8 Computational cost . . . 175
22.9 The curse of dimensionality . . . 176
22.10Class deck: the $[4a, 3b]$ grid question . . . 176
22.11From the textbook: the Bayes classifier (ISLR §2.2.3) . . . 176
22.12From the textbook: the curse of dimensionality, quantified (ESL §2.5) . . . 177
22.13Advantages and disadvantages . . . 177
Lecture slides for reference . . . 178

23 Naive Bayes **181**
23.1 Three facts from probability you need . . . 181
23.2 The classifier we want, and the obstacle . . . 182
23.3 The naive assumption . . . 182
23.4 Training: counting, on sir's example . . . 183
23.5 The zero problem . . . 184
23.6 Laplace smoothing . . . 184
23.7 Prediction, and why we use logarithms . . . 184
23.8 The algorithm, and continuous features . . . 186
23.9 From the textbook: Naive Bayes is a linear classifier (Cornell CS4780) . . . 186
23.10Summary . . . 187
Lecture slides for reference . . . 188

24 Linear Classifiers and the Perceptron **191**
24.1 Geometry you need: the dot product . . . 191
24.2 Hyperplanes and the linear classifier . . . 191
24.3 Margins: how far is a point from the boundary? . . . 192
24.4 The perceptron . . . 193
24.5 The convergence theorem (Block and Novikoff) . . . 195
24.6 When no line exists . . . 196
24.7 From the textbook: the weight-space view (Shewchuk, Berkeley CS189) . . . 197
24.8 Stacking perceptrons: why nonlinearity matters . . . 197

Lecture slides for reference . . . 199

25 Logistic Regression, Newton and Momentum 201

25.1 The problem: predicting a yes/no answer . . . 201
25.2 First attempt: just use linear regression . . . 201
25.3 The sigmoid function, and its three properties . . . 202
25.4 The logistic regression model . . . 204
25.5 From probability to decision: the threshold and the boundary . . . 204
25.5.1 Curved boundaries with extra features . . . 205
25.6 Odds and log-odds: what the weights mean . . . 206
25.7 Choosing the weights, part 1: why not squared error? . . . 206
25.8 Choosing the weights, part 2: maximum likelihood, step by step . . . 207
25.9 The gradient of the cost, every line . . . 208
25.10 Gradient descent for logistic regression . . . 210
25.11 Regularization, and the MAP view . . . 210
25.12 Newton's method: using curvature . . . 211
25.13 Gradient descent with momentum . . . 214
25.14 More than two classes: softmax regression . . . 215
25.15 From the textbook: where the sigmoid comes from . . . 216
25.16 From the textbook: the credit-card Default data (ISLR §4.3) . . . 216
25.17 From the textbook: judging a classifier (ISLR §4.4.3) . . . 217
25.18 Generative vs discriminative . . . 218
25.19 Exam drill from the class decks . . . 218
Lecture slides for reference . . . 219

26 Insem 2: The Map, the Traps, the Drill 223

26.1 Slide-by-slide map . . . 223
26.2 Corrections, all in one place . . . 224
26.3 The one-line drill . . . 225
26.4 Python you should be able to write cold . . . 226

VII Lab kits — the pocket cards 227

27 Feature Scaling, Conditioning, and Ridge — Lab 3 Kit 229

27.1 Scaling is a change of units . . . 229
27.2 Ridge: a tax on slopes, not on the intercept . . . 230

28 Basis Functions — Lab 4 Kit 233

28.1 One fit, three alphabets . . . 233
28.2 Choosing the alphabet . . . 234

VIII The rest of the map — uncut **237**

29 Learning Paradigms **239**

29.1 The map of machine learning . . . 239
29.2 The notation we will use . . . 239
29.3 Supervised learning . . . 240
29.3.1 Regression — when the output is a number . . . 240
29.3.2 Classification — when the output is a label . . . 241
29.4 Unsupervised learning . . . 242
29.4.1 Clustering — finding natural groups . . . 242
29.4.2 Dimensionality reduction — squashing without losing much . . . 242
29.4.3 A peek at topic modelling . . . 243
29.5 Reinforcement learning . . . 243
29.6 Semi-supervised and self-supervised learning . . . 243
29.7 Other flavours (briefly) . . . 244

30 One Story, Told Three Times **247**

30.1 The three routes . . . 247
30.2 The dependency map . . . 248
30.3 The checklist . . . 248

IX Toolkit **249**

A The Math You Need, From Zero **251**

A.1 Numbers, variables and functions . . . 251
A.2 Drawing a function: the graph . . . 251
A.3 Slope: how steep is a line? . . . 252
A.4 The derivative: slope of a curve . . . 252
A.5 Vectors: numbers with direction . . . 253
A.6 Matrices: tables that transform vectors . . . 255
A.7 Summation notation $\sum$ . . . 256

B What Stanford, Berkeley, and Princeton Do Next **257**

C Where to Learn This From Someone Else **259**

C.1 Readings, by what they are good at . . . 259
C.2 Video, by topic . . . 260
C.3 How to actually use this . . . 262

D Answers **263**

E Worked Solutions to Past-Exam Questions **277**

E.1 Mid-Semester Exam (06-10-2025) 277

E.2 End-Semester Exam (26-11-2025) 279

E.3 Short answers: questions beyond the current chapters 281

Closing & References **283**

How to Read This Book

This is not a pile of definitions. Each chapter starts with a situation you can hold in your head — three houses, a coin, a bowl you can sketch — and refuses to write the general formula until that situation has been computed by hand. If you catch yourself nodding without a number on the page, go back.

Pictures are part of the argument, not decoration. When a figure shows a square sitting on a residual, the area of that square *is* the term in the sum. When arrows sit on a contour, they are ∇J, pointing uphill. Read the caption after you have stared at the drawing.

Exercises sit at the end of every chapter, and every one of them is answered in Appendix D. The arrow beside a question is a link: click it and the PDF jumps to the worked answer; the answer links back to the question. That round-trip only helps if you use it in the right order, which is pen first, arrow second.

The boxes, and what each one is promising you

Six kinds of interruption appear in the margins and in the text. They are not decoration and they are not interchangeable, so it is worth thirty seconds to learn the grammar.

Derivation — blue panel.
The algebra, every line, nothing skipped. If a step says "clearly", that is a bug and you should be able to fill it in. These are what you will be asked to reproduce.

Another route to the same place — teal panel.
A second, independent derivation of something you have already proved. Skippable on a first pass. On a second pass it is the most valuable box on the page, because a result you can reach two ways stops feeling like a lucky manipulation and starts feeling forced.

Watch out — red panel.
A specific mistake this course produces, usually in a lab, usually silently.

In code — grey panel.
Runnable NumPy that checks the claim just made. Every one of these corresponds to a script in the repository, linked from the box, and every script asserts rather than prints.

Where this shows up — green rule.
The idea outside the classroom. Included when the application changes how you should think about the maths, not when it is merely a fun fact.

Go deeper — gold rule.
Somebody else's explanation of the same thing: a document with a section number, or a

lecture with a timestamped link. All links were checked live. Appendix C collects them.

Blue rules in the margin mark a formal statement; purple ones a question to answer before reading on.

The maths camp is Appendix A. Skip it until a symbol is opaque, then read only that section. If you are revising from a lecture PDF, the coverage map on the next pages is an index, not a reading order.

Lecture coverage map — nothing cut

The chapters are ordered as an argument, not as a lecture calendar. Nothing from the notes was dropped to make that work, and this table says where each lecture went. Rows marked + are additions rather than replacements: they sit alongside the class derivation, never in place of it.

Class	Here	What is kept, uncut
Intro / what ML is	Ch. 1	Two ways to program; Mitchell T/P/E; loss; train/test; GD preview; application gallery
Paradigms survey	Ch. 29	Supervised / unsupervised / RL / other flavours, all examples from class
Linear regression	Ch. 2	Hypothesis, design matrix, J, worked cost, convex bowl
+ Gaussian story	Ch. 3	*Added:* $y = X\theta + \varepsilon \Rightarrow$ MSE is MLE
Gradient descent	Ch. 4	Update rule, simultaneous step, ∇J derivation, α, 2-D/3-D, stopping
Lec 6 bowl	Ch. 5	$J = \theta_1^2 + \theta_2^2 + 4$, four α, closed form $(1-2\alpha)^k$, three stopping rules, $\|(2,3,4)\|$, steepest ascent
Lec 7 / Hessians	Ch. 6	∇, directional derivative, Hessian, linear/quadratic, normal equation, Hessian $\neq$ "grad of grad"
Lec 7 SGD	Ch. 7	Batch vs SGD vs mini-batch; $(y-h)x$ sign
Lec 8 LS	Ch. 6	$2Ax$, $2A$, $\hat{\theta} = (X^\top X)^{-1} X^\top y$
Lec 10	Ch. 8	$h = \theta^\top x$, $X\theta$, vectorized J, GD vs NE table, pinv / redundant / $d \leq n$, house scaling, μ/σ, train/test, outliers
Lec 11	Ch. 9	Under/overfit trio, both J conventions, $\lambda \to \infty$, shrink $1 - \alpha\lambda/n$, regularized NE
+ Caltech story	Ch. 10	*Added:* $w^\top w \leq C \to \lambda$
Lab 3 pocket card	Ch. 27	Same J as the lab, R-masked intercept, one NumPy solve
Lec 9	Ch. 11	Laplace $\nabla\|A\|$, $A\,\mathrm{adj}(A) = \|A\|I$, $\nabla \log\|A\|$, Rayleigh, trace identities
Lec 12 pinv	Ch. 12	SVD, Σ^+, rank-1 X, $X^\dagger = V\Sigma^+ U^\top$
Lec 12–13 MLE/MAP	Ch. 14	$P(x;\theta)$ vs $P(x \mid \theta)$, likelihood/NLL, Bernoulli MLE, MAP, Beta, Γ, $\hat{\theta}_{\mathrm{MAP}} = (s+\alpha-1)/(n+\alpha+\beta-2)$
Maths camp	App. A	Functions, slope, derivative, vectors, matrices, $\sum$ — all of it, moved to the back so the story can start
+ Where next	App. B	*Added:* logistic, kernels, bias–variance, backprop — named honestly, not faked
+ Answers	App. D	*Added:* worked solutions to every starred exercise

Revising from a lecture PDF? Enter by the left column. Learning this for the first time? Ignore the left column entirely and read Parts I–IV in order.

Every chapter ends in exercises. Starred ones are answered in Appendix D. Anything the book claims that a computer can check is checked — run `python notebooks/theory/run_all.py` and six scripts will re-derive the gradient, the closed form, the conditioning, the ridge path, the pseudoinverse, and the MAP estimate against your own machine.

One warning about repeats. The gradient-descent walkthrough looks like a second copy of the gradient-descent chapter, and the two ridge conventions look like the same formula twice. They are not. The walkthrough is where the arithmetic lives, and the two conventions differ by factors of n that will silently change your λ in Lab 3.

Part I

The question: learn a function

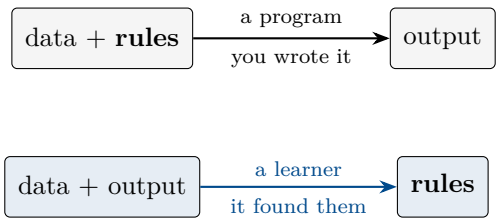
1

What is Machine Learning?

You already know how to make a computer do a job. You write the rule. *If the subject line contains the word lottery, dump the mail in spam.* The machine does not invent that rule. You did. That is ordinary programming: rules in, answers out.

Machine learning swaps the ends. You keep a drawer of letters that a human already marked spam or not. You never write the rule. You ask the machine to find a rule that would have labelled those letters correctly, and that still works on the next letter, which nobody has marked yet.

That swap is the whole subject. Everything later — lines, gradients, ridge — is a way of finding the rule and of refusing to lie about whether it worked. It is worth drawing, because the arrow genuinely reverses:



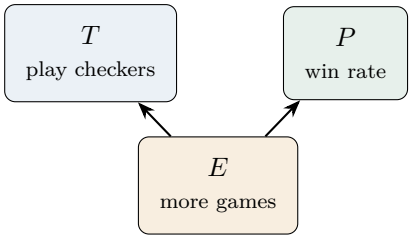
Top: ordinary programming. Bottom: machine learning. Same three nouns; the one you supply and the one you receive have traded places. The rule the learner hands back is a function with numbers in it, and those numbers are the only thing this course is really about.

1.1 A definition you can test

Arthur Samuel, in 1959, called this "the field of study that gives computers the ability to learn without being explicitly programmed." Useful as a slogan; useless as a test. Tom Mitchell, in 1998, gave a test.

A program learns from experience E on a class of tasks T, as measured by P, if its performance at T, measured by P, improves with E.

Three letters. You cannot hide. For a checkers program: T is playing checkers, P is the fraction of games won, E is games played against itself. If more games do not raise the win rate, the program is not learning. It is busy.



if P does not rise, you are not learning

Mitchell's test. Experience has to move the performance needle on the task. Otherwise the word "learn" is advertising.

1.2 A model is a function with knobs

Underneath the slogans there is one object: a function h_θ that eats an input x and emits a guess $\hat{y}$. The subscript θ is a list of numbers you are allowed to turn. Those numbers are the *parameters*. Learning is the act of turning them so the guesses line up with labelled examples.

The lectures' running example is movie reviews. Each review is a bag of words. Each English word w is given a sentiment score θ_w. The model's guess for a review is the sum of the scores of the words that appear:

$$h_\theta(\text{review}) \;=\; \sum_{w \in \text{review}} \theta_w .$$

There are on the order of 10^5 English words, so θ has on the order of 10^5 knobs. "Loved" wants a large positive score. "Tedious" wants a negative one. "The" should sit near zero, because it appears in raves and in pans.

loved +1.1 | the +0.1 | old −0.4 | good +0.6 | well +1.4 = +2.7 the guess

A review scored as a sum of word knobs. Training is the search for 10^5 numbers that make sums like this match the stars people actually gave.

Watch out

The sum ignores order. "Not good" and "good" share the word *good*, so they are not opposites in this model. The lectures flag this. It is a feature of the example, not a bug you should paper over. Real language models fix order later. We are here to learn how knobs get turned, not how English works.

If a symbol in h_θ is new, it is defined in Appendix A. The course's map of unsupervised learning, clustering, and reinforcement learning is still in this book, uncut, as Chapter 29. It waits until you have fitted one function, so those names have somewhere to sit.

1.3 How wrong, as a single number

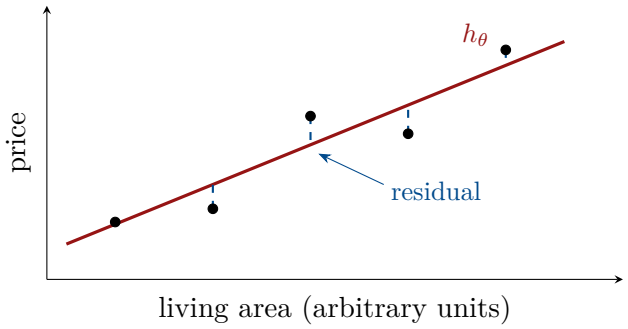
Knobs need a score. The score used in this course is the *squared-error loss*. You are given n labelled pairs $(x^{(i)}, y^{(i)})$. The model's guess on example i is $h_\theta(x^{(i)})$. The residual is the signed

miss $h_\theta(x^{(i)}) - y^{(i)}$. Square it, add them up, average:

$$J(\theta) \;=\; \frac{1}{2n}\sum_{i=1}^{n}\big(h_\theta(x^{(i)}) - y^{(i)}\big)^2.$$

The superscript (i) is an index, not a power. The $\frac{1}{2}$ is not mysticism. When you differentiate a square in Chapter 4, a 2 comes down and cancels it. Until then, treat $\frac{1}{2}$ as furniture.

Why square? A miss of $+3$ and a miss of -3 are equally bad, and a miss of 4 is four times as expensive as a miss of 2, not twice. That is a choice. Chapter 3 will tell you which noise story makes it the *only* honest choice. For now it is the score the lectures use, so it is the score we minimise.



Five houses, one line. Each blue stick is a residual. J is (half) the mean of the squares of those sticks. Training slides the red line until that mean will not go down.

In code the same object is four lines. Residuals first, then the average of their squares.

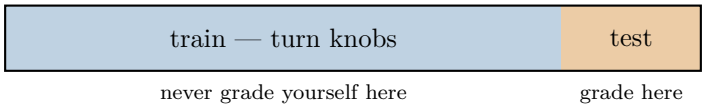
In code

```python
import numpy as np
def J(y_hat, y):
    n = len(y)
    return (1.0 / (2 * n)) * np.sum((y_hat - y)**2)

y     = np.array([1.0, 2.0, 3.0])
y_hat = np.array([1.2, 1.8, 3.1])
print(J(y_hat, y))   # 0.015
```

1.4 The split that stops you lying

If you fit θ on a set of reviews and then boast about J on those same reviews, you have measured memorisation. The lectures therefore cut the data. About 80% is the training set: that is where knobs get turned. The remaining 20% is the test set. You may look at test J only after θ is frozen. That number is a guess at what will happen to the next review, the one that does not exist yet.



Eighty percent to learn, twenty percent to be examined. Mixing the two is the most common way to fake a good model.

Why not try every function on earth and pick the one that interpolates the training points? Because of dimension. The lectures note that a function of $x \in \mathbb{R}^n$, learned with no restriction, can demand on the order of $\exp(n)$ examples. That is not a budget you have. So we restrict the family: straight lines, then lines with a tax on their slopes, then (later courses) neural nets. Less freedom, actually learnable.

1.5 A preview of how the knobs move

The algorithm that turns θ is gradient descent. Start somewhere. Repeatedly step downhill on J:

$$\theta^{(t+1)} \;=\; \theta^{(t)} - \alpha \nabla J(\theta^{(t)}).$$

∇J points uphill (Appendix A). Subtracting it points down. $\alpha > 0$ is the step size, the *learning rate*. Chapter 4 derives ∇J for a line and then refuses to wave hands about α.



convex — one bottom not convex — start matters

Left: linear regression. The loss is a bowl; downhill from anywhere is the same place. Right: a cartoon of a deep net. Several dips. Where you start is which dip you get.

1.6 Three choices, and there are only three

Stand back from the last four sections and count what you actually decided. It is a short list, and it does not get longer for the rest of this book.

Every method in this course is three choices.

1. A **hypothesis class** $\mathcal{H}$: the family h_θ is allowed to come from. Straight lines. Polynomials of degree at most three. Sums of word scores. Choosing $\mathcal{H}$ is choosing how much freedom the model gets.
2. A **loss** $J(\theta)$: one number saying how badly a particular h_θ fits. Smaller is better. Squared error, for us.
3. An **optimiser**: a procedure that searches $\mathcal{H}$ for the θ making J small. Gradient descent, for us, and later a closed-form shortcut.

Try removing one and see what breaks, because that is the argument for why there are exactly three.

Drop the loss and keep the other two. You have a family of candidate functions and a way to search, but no reason to prefer any member of the family over any other. Search for what? Every line is as good as every other line.

Drop the optimiser and keep the other two. Now you can grade any θ you are handed, which sounds like progress until you notice that θ ranges over $\mathbb{R}^{d+1}$. There are uncountably many candidates and you have no way to produce a good one. You can mark the exam but not sit it.

Drop the hypothesis class and keep the other two. This is the subtle one, and it is the reason the previous section talked about $\exp(n)$ examples. If h may be *any* function whatsoever, then there is always a function achieving $J = 0$: the one that memorises every training pair and returns nonsense elsewhere. The optimiser will find it, the loss will applaud, and you will have learned nothing. $\mathcal{H}$ is not a limitation you tolerate. It is the thing that makes generalisation possible at all.

Where this shows up

When a paper announces a new method, this is the fastest way to read it. Ridge regression: same $\mathcal{H}$, new J. Stochastic gradient descent: same $\mathcal{H}$ and J, new optimiser. A convolutional network: radically new $\mathcal{H}$, and often the same squared error and the same downhill step you are about to learn. Nine times out of ten, exactly one of the three slots changed, and identifying which one tells you what the paper is really claiming.

Go deeper

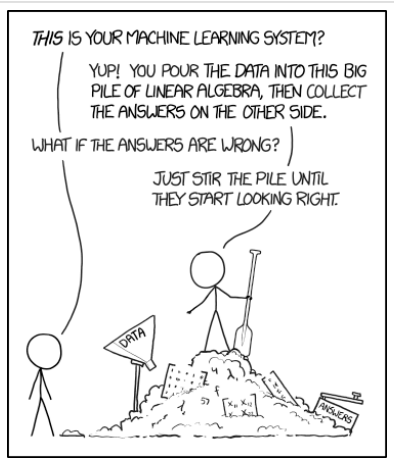
Read. Stanford CS229, main notes, Part I opening. Ng sets up h, the cost J, and LMS in the same order, on houses rather than reviews — worth reading straight after this chapter for a second pass at identical material. `https://cs229.stanford.edu/main_notes.pdf`

Watch. Cornell CS4780 (Kilian Weinberger), lecture 1, *Supervised learning setup*

1.7 The same idea, wearing other clothes

The first lecture is a gallery. Every item is "learn a function from examples" in a different costume.

What they call it	The function
Object detection	image $\mapsto$ boxes and labels
Face recognition	face $\mapsto$ identity
A self-driving stack	sensors $\mapsto$ steer / brake
Denoising, super-resolution	wrecked image $\mapsto$ clean image
Colourisation	greyscale $\mapsto$ colour
Emotion recognition	face $\mapsto$ a label



xkcd #1838 "Machine Learning" by Randall Munroe — https://xkcd.com/1838/ (CC BY-NC 2.5).

The pile of linear algebra in the comic is h_θ. The stirring is gradient descent on J. This book opens the pile.

Exercises

Pencil first, then click the ⇝ to jump to the worked answer in Appendix D. The answer links back to the question.

1.1 ⇝ State Mitchell's T, P, E for a system that predicts house prices from living area. Then state them for a spam filter. If you cannot name all three, you do not yet have a learning problem — you have a vibe.

1.2 ⇝ The bag-of-words model scores "not good" the same way it scores "good", up to the extra word *not*. Invent a two-word feature that would split those reviews apart without building a language model. What happens to the number of knobs?

1.3 ⇝ Three predictions $1.0, 2.0, 3.0$ against truths $0.0, 2.0, 5.0$. Compute each residual, each squared residual, and J with the course formula $\frac{1}{2n}\sum(\,\cdot\,)^2$.

1.4 ⇝ Why is a training J of zero not a victory by itself? Give a two-sentence story in which train $J = 0$ and the model is still useless on the next example.

1.5 ⇝ A classmate fits θ on all the data, then reports that same J as "test error." What, exactly, have they measured? What should they have done instead?

1.6 ⇝ Write the gradient-descent update from memory. Mark which symbol is a vector, which is a scalar, and which way ∇J points. Do not look back.

1.7 ⇝ For each of the following, say which of the three slots of Section 1.6 has changed relative to plain linear regression, and which two are untouched: (a) fitting a cubic instead of a line; (b) penalising $\|\theta\|^2$; (c) using one random example per step instead of all n; (d) solving $X^\top X\theta = X^\top y$ directly.

1.8 ⇝ Suppose $\mathcal{H}$ is genuinely unrestricted — any function $\mathbb{R} \to \mathbb{R}$ is allowed. Write down, explicitly, a function that achieves $J = 0$ on the five houses plotted earlier in this chapter and predicts -10^6 everywhere else. Why does no amount of cleverness in the optimiser protect you from this?

2

Linear Regression

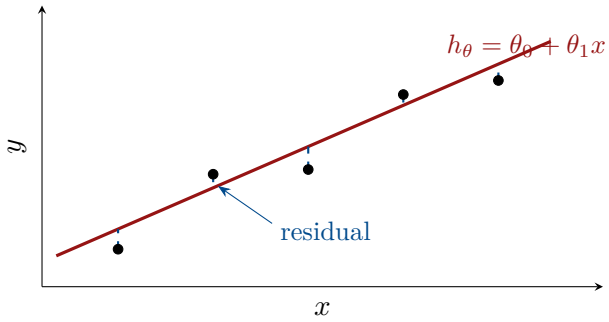
Chapter 1 gave you a score J and a promise that knobs would move. It did not say what h_θ *is*. Here it is a line. Everything in the rest of the book is this line, dressed up: more features, a tax on slopes, a probability story. If the line is vague, stop and do the three-point example in this chapter with a pencil. The algebra is not the point. The three numbers are.

2.1 One feature, two knobs

For a single input x — living area, say — the hypothesis is the line you met in school, with the letters renamed:

$$h_\theta(x) \;=\; \theta_0 + \theta_1 x.$$

θ_0 is the intercept, the guess when $x = 0$. θ_1 is the slope, the rupees (or dollars) per extra unit of area. Both are knobs. A line that "sits nicely" through a cloud of houses is a line whose J is small. That is the only sense of "nicely" this course owns.



The model is a line. A residual is a vertical gap, never a perpendicular gap: y is what we are blamed for, x is given.

2.2 Three points, computed dead

The notes use $(x, y) = (1, 1),\ (2, 2),\ (3, 3)$, so $n = 3$. Freeze $\theta_0 = 0$ for a page, so $h(x) = \theta_1 x$, and

$$J(\theta_1) \;=\; \frac{1}{6}\sum_{i=1}^{3}(\theta_1 x^{(i)} - y^{(i)})^2.$$

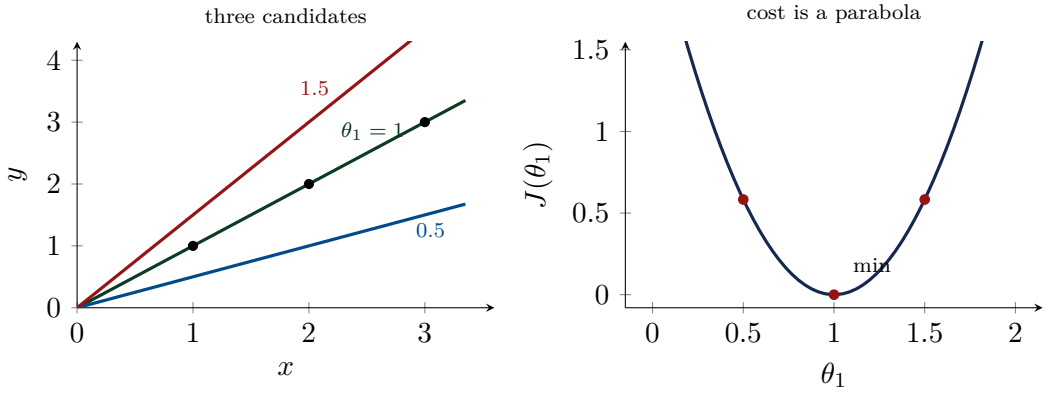
Try $\theta_1 = 1$. Predictions $1, 2, 3$. Residuals $0, 0, 0$. So $J(1) = 0$. The line *is* the data.

Try $\theta_1 = 0.5$. Predictions $0.5,\ 1,\ 1.5$. Residuals $-0.5,\ -1,\ -1.5$. Squares $0.25,\ 1,\ 2.25$. Sum

3.5. Then

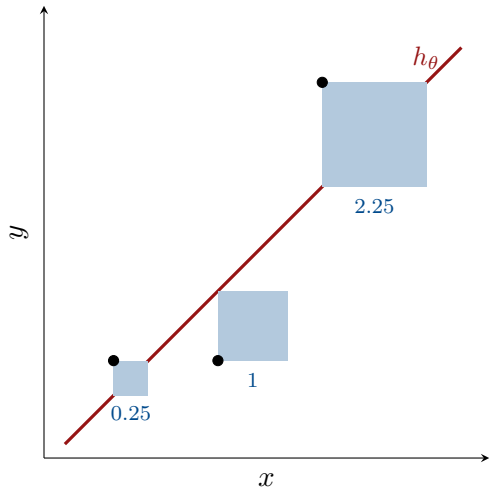
$$J(0.5) \;=\; \frac{3.5}{6} \;\approx\; 0.583.$$

Try $\theta_1 = 1.5$. Residuals $+0.5$, $+1$, $+1.5$. The squares are the same three numbers. $J(1.5) = J(0.5)$. Being too steep by half is exactly as expensive as being too shallow by half, on this particular cloud.



Left: the data and three slopes. Right: J against θ_1. You do not need a theorem yet. You can see a bowl with one bottom at $\theta_1 = 1$.

The squares in "least squares" are literal. Residuals of 0.5, 1, and 1.5 contribute areas of 0.25, 1, and 2.25. The worst point pays nine times what the best one pays, not three times. That single fact is why one outlier can own a least-squares fit, and it is the reason Chapter 9 eventually has to tax the parameters.



Each residual is the side of a literal square, drawn to scale. J adds these areas and divides by $2n$. Doubling a miss quadruples its bill.

2.3 Many features, still a dot product

With d inputs $x_1, \ldots, x_d$ the line becomes a weighted sum. Sneak in a fake feature $x_0 = 1$ so the intercept sits in the same sum:

$$h_\theta(x) \;=\; \theta_0 + \theta_1 x_1 + \cdots + \theta_d x_d \;=\; \sum_{j=0}^{d} \theta_j x_j \;=\; \theta^\top x.$$

The last step is a dot product (Appendix A). The "base fare" of a meter can be written as a rate times 1. That is all $x_0 = 1$ is.

Stack the n examples as rows of a matrix X (first column all ones) and the targets as a vector y. Then every prediction at once is $X\theta$, and the cost is the squared length of the residual vector:

$$\hat{y} \;=\; X\theta, \qquad J(\theta) \;=\; \frac{1}{2n}\|X\theta - y\|_2^2 \;=\; \frac{1}{2n}(X\theta - y)^\top (X\theta - y).$$

This is the same $\frac{1}{2n}\sum_i (h^{(i)} - y^{(i)})^2$ as before. The matrix line is how you stop writing loops. Lecture 10 will make a religion of that. Here it is only bookkeeping.

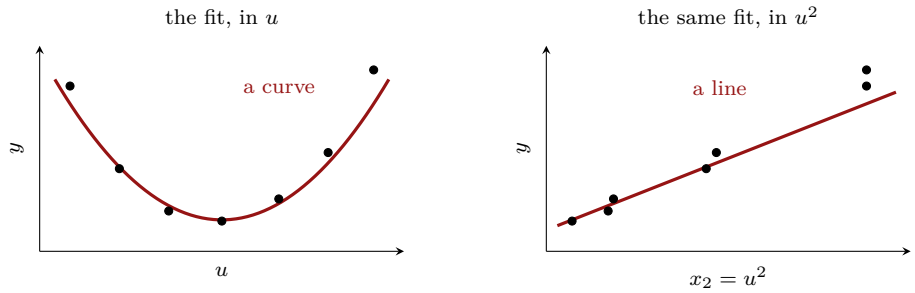
2.4 Linear in *what*, exactly?

Here is the misunderstanding that costs people the most, and it is over in one sentence: linear regression is linear in θ, not in x.

Nothing in $h_\theta(x) = \theta^\top x$ requires the entries of x to be raw measurements. They are whatever numbers you choose to put in the vector. If you have one measurement u and you build

$$x \;=\; \big(1,\; u,\; u^2,\; u^3\big)^\top,$$

then $h_\theta(x) = \theta_0 + \theta_1 u + \theta_2 u^2 + \theta_3 u^3$ is a cubic in u — a curve, visibly bending — and yet it is still linear regression, still a convex bowl, still solved by the same normal equation you are about to derive. The model does not know that column three is the square of column two. It sees four numbers.



One model, two coordinate systems. Plotted against the raw measurement u it is a parabola; plotted against the feature u^2 it is a straight line, because that is what it always was. Fitting curves is not a different technique — it is the same technique after you change what goes in the columns of X.

The general move is to pick a *feature map* ϕ and fit $h_\theta(u) = \theta^\top \phi(u)$. Take $\phi(u) = (1, u, u^2, u^3)$ and you fit cubics. Take $\phi(u) = (1, \log u)$ and you fit $\theta_0 + \theta_1 \log u$. Take $\phi(u) = (1, \sin u, \cos u)$ and you fit a wave. All of these are linear regression, and the machinery in the next eight chapters applies unchanged.

Watch out

Two things break the spell, and it is worth knowing which side of the line you are on. $h(u) = \theta_0 + \theta_1 u^{\theta_2}$ is *not* linear regression, because θ_2 sits in an exponent and the cost stops being a bowl. Neither is $h(u) = \theta_0 e^{\theta_1 u}$. The test is not "does the graph look straight"; it is "if I hold the data fixed and vary θ, is h a weighted sum of fixed numbers?"

Where this shows up

This is why linear models refuse to die. Almost every classical technique that sounds different — polynomial fitting, Fourier regression, splines, radial basis function networks, and the last layer of a trained neural network — is $\theta^\top \phi(u)$ with a different ϕ. The whole design effort moves into choosing ϕ, and the fitting stays a solved problem. Deep learning's actual novelty is that it learns ϕ too, instead of you picking it.

Go deeper

Watch. mathematicalmonk, *ML 9.1*, *Linear regression: nonlinearity via basis functions*

Read. Boyd & Vandenberghe, VMLS, Ch. 13. "Least squares data fitting" works the basis-function idea in detail, including piecewise-linear and polynomial features. https://web.stanford.edu/~boyd/vmls/vmls.pdf

$$\underbrace{\begin{bmatrix} 1 & x^{(1)} \\ 1 & x^{(2)} \\ 1 & x^{(3)} \end{bmatrix}}_{X} \underbrace{\begin{bmatrix} \theta_0 \\ \theta_1 \end{bmatrix}}_{\theta} = \underbrace{\begin{bmatrix} \theta_0 + \theta_1 x^{(1)} \\ \theta_0 + \theta_1 x^{(2)} \\ \theta_0 + \theta_1 x^{(3)} \end{bmatrix}}_{\hat{y}}$$

The design matrix. One multiply, three predictions. Subtract y and you have the residual vector whose squared length is the cost.

In code

The notes' three points, as a machine you can insult:

```
import numpy as np
x = np.array([1., 2., 3.]);  y = np.array([1., 2., 3.])
X = np.column_stack([np.ones(len(x)), x])

def predict(theta):
    return X @ theta
def cost(theta):
    r = predict(theta) - y
    return (1.0 / (2 * len(y))) * np.sum(r**2)

print(cost(np.array([0., 1.0])))   # 0.0
print(cost(np.array([0., 0.5])))   # 0.5833...
```

$X\theta$ is `X @ theta`. If this printout does not match the hand calculation above, your algebra and your code are having a fight. Believe the arithmetic.

2.5 The bowl

Free both knobs. $J(\theta_0, \theta_1)$ is now a surface. For a linear model with squared error it is a convex bowl: one lowest point, no other dips. From above, the level sets are concentric ellipses. The centre of the ellipses is the best (θ_0, θ_1). Convexity is why gradient descent, started anywhere, is entitled to the same answer. Deep nets do not get this gift.



Same object, two views. Training is the search for the red dot.

How do you find that dot without plotting the whole surface? You start somewhere and walk downhill. That walk is Chapter 4. The class numerical bowl $J = \theta_1^2 + \theta_2^2 + 4$ is the chapter after that, with every α the notes used. Nothing from those pages is skipped.

Another route: no calculus at all

Before you meet a single derivative, notice that the one-knob problem is already solvable by school algebra. With $h = \theta_1 x$ on data $(x^{(i)}, y^{(i)})$,

$$2n\,J(\theta_1) = \sum_i \left(\theta_1 x^{(i)} - y^{(i)}\right)^2 = \underbrace{\left(\textstyle\sum_i (x^{(i)})^2\right)}_{A}\theta_1^2 \;-\; 2\underbrace{\left(\textstyle\sum_i x^{(i)}y^{(i)}\right)}_{B}\theta_1 \;+\; \underbrace{\textstyle\sum_i (y^{(i)})^2}_{C},$$

which is $A\theta_1^2 - 2B\theta_1 + C$: a plain quadratic in one unknown, with $A > 0$ whenever any $x^{(i)} \neq 0$. Complete the square, exactly as you would for $ax^2 + bx + c$:

$$A\theta_1^2 - 2B\theta_1 + C = A\Big(\theta_1 - \tfrac{B}{A}\Big)^2 + \Big(C - \tfrac{B^2}{A}\Big).$$

The second bracket does not contain θ_1, so nothing you do can change it. The first is A times a square, so it is never negative and it is zero exactly once. Therefore

$$\theta_1^\star = \frac{B}{A} = \frac{\sum_i x^{(i)}y^{(i)}}{\sum_i (x^{(i)})^2}, \qquad J_{\min} = \frac{1}{2n}\Big(C - \tfrac{B^2}{A}\Big).$$

Check it on the three points: $A = 1 + 4 + 9 = 14$, $B = 1 + 4 + 9 = 14$, so $\theta_1^\star = 1$ and $J_{\min} = 0$, which is what you got by trying values. And $A > 0$ is precisely the statement that the parabola opens upwards — the one-dimensional ancestor of "$X^\top X$ is positive definite", which is the convexity condition you will meet in Chapter 6.

Why bother, if calculus is coming? Because $\theta_1^\star = B/A$ is already the normal equation. Written in matrix form, $A = X^\top X$ and $B = X^\top y$, and $\theta^\star = B/A$ becomes $\theta^\star = (X^\top X)^{-1} X^\top y$. You will derive that in three different ways over the next eight chapters. This is the first, and it used nothing beyond completing the square.

Exercises

Pencil first, then click the ⇝ to jump to the worked answer in Appendix D. The answer links back to the question.

2.1 ⇝ For the three points $(1,1), (2,2), (3,3)$ and $h = \theta_1 x$, compute $J(0)$ and $J(2)$ from the definition. Check they sit equally far from $\theta_1 = 1$ on the parabola.

2.2 ⇝ Keep θ_0 free. Write $J(\theta_0, \theta_1)$ as a sum of three squares for the same data. Do not minimise it yet. (You will, by calculus, in Chapter 6.)

2.3 ⇝ Build the design matrix X for those three points, including the column of ones. Compute $X\theta$ for $\theta = (0, 1)^\top$ and for $\theta = (1, 0)^\top$. What line is each?

2.4 ⇝ A data set has x in square feet and x also in square metres, both as columns, plus a ones column. What goes wrong with $X^\top X$? You do not need the pseudoinverse chapter — describe the failure in English.

2.5 ⇝ Why vertical residuals, not perpendicular distance to the line? (Hint: which variable is the one we are punished for missing?)

2.6 ⇝ Implement `cost` without `np.sum`, using a Python `for` loop. Check it matches the vectorised version on the three-point data. Time both on a random $n = 10^5$ problem. This is the argument for $X\theta$.

2.7 ⇝ Data: $(u, y) = (-2, 4.1), (-1, 1.0), (0, 0.2), (1, 0.9), (2, 3.8)$. Write the design matrix for the feature map $\phi(u) = (1, u, u^2)$. You do not have to solve it — just state its shape and say which column would carry the curvature.

2.8 ⇝ Which of these can be fitted by linear regression after a change of features, and which cannot? (a) $y = \theta_0 + \theta_1\sqrt{u}$; (b) $y = \theta_0 u^{\theta_1}$; (c) $y = \theta_0 + \theta_1 u + \theta_2 uv$ for two measurements u, v; (d) $y = 1/(\theta_0 + \theta_1 u)$. For the ones that cannot, say precisely which parameter is in the wrong place.

2.9 ⇝ Use the completing-the-square result to find $\theta_1^\star$ for $h = \theta_1 x$ on the data $(1, 2), (2, 3), (3, 7)$. Then evaluate J at your answer and at $\theta_1^\star \pm 0.1$, and confirm your answer really is the bottom.

3

Why Squared Error? Least Squares as Gaussian MLE

You already minimise $\sum_i (y^{(i)} - \theta^\top x^{(i)})^2$. That looks like a taste. It is not. If you are willing to say the leftovers are independent Gaussians, the same sum is what *maximum likelihood* tells you to minimise. Stanford CS229 puts this next to gradient descent. The lectures wait until Chapter 14. Read this now so that chapter is a reunion.

3.1 Write the noise

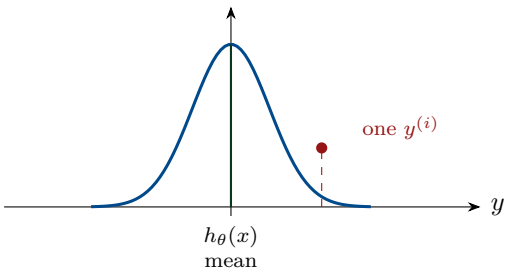
A house is never exactly on the line. Write

$$y^{(i)} \;=\; \theta^\top x^{(i)} + \varepsilon^{(i)}, \qquad \varepsilon^{(i)} \sim \mathcal{N}(0, \sigma^2) \quad \text{i.i.d.}$$

One observation then has density

$$p\big(y^{(i)} \mid x^{(i)}, \theta\big) \;=\; \frac{1}{\sqrt{2\pi\sigma^2}} \exp\Big(- \frac{\big(y^{(i)} - \theta^\top x^{(i)}\big)^2}{2\sigma^2} \Big).$$

The line is the mean of a bump. A point far from the line sits on the tail, where the bump is low.



Likelihood of one house is the height of the bump at the observed price. Far from the line, the height is tiny. Product of tiny numbers is why far points dominate J.

3.2 The whole training set

Independence turns heights into a product:

$$L(\theta) \;=\; \prod_{i=1}^{n} p(y^{(i)} \mid x^{(i)}, \theta) \;=\; (2\pi\sigma^2)^{-n/2} \exp\Big(-\frac{1}{2\sigma^2} \sum_{i=1}^{n} (y^{(i)} - \theta^\top x^{(i)})^2 \Big).$$

Maximum likelihood: pick θ that makes L as large as possible. The prefactor does not depend on θ. The exponential is largest when the sum of squares is smallest. So

$$\hat{\theta}_{\text{MLE}} \;=\; \arg\min_{\theta} \sum_{i=1}^{n} (y^{(i)} - \theta^\top x^{(i)})^2,$$

which is ordinary least squares. Same bowl as Chapter 2. Same critical point as the normal equation in Chapter 6.

Derivation

log is increasing, so $\arg\max L = \arg\max \log L$:

$$\log L(\theta) \;=\; -\frac{n}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \|y - X\theta\|_2^2.$$

Differentiate:

$$\nabla_\theta \log L \;=\; \frac{1}{\sigma^2} X^\top (y - X\theta) \;=\; 0 \quad\Longrightarrow\quad X^\top X\,\theta \;=\; X^\top y.$$

The $\frac{1}{2}$ and $\frac{1}{n}$ in the class J do not move the minimiser. They are cosmetics.

Change the noise, change the loss. Laplace leftovers give absolute deviations. A Bernoulli story (Chapter 14) gives log loss. Squared error is what you get when you believe a Gaussian bump around the line, nothing more romantic.

You do not have to take the equivalence on faith. Generate data with genuinely Gaussian noise, then check that the θ maximising the log-likelihood is byte-for-byte the θ minimising the squared error:

In code

```
import numpy as np

rng = np.random.default_rng(1)
n = 200
X = np.column_stack([np.ones(n), rng.normal(size=n)])
theta_true = np.array([1.5, -2.0])
sigma = 0.7
y = X @ theta_true + sigma * rng.normal(size=n)

def neg_log_lik(th, s):
    r = y - X @ th
    return 0.5*n*np.log(2*np.pi*s**2) + (r @ r) / (2*s**2)

def sse(th):
    r = y - X @ th
```

```
    return r @ r

th_ls = np.linalg.solve(X.T @ X, X.T @ y)       # least squares
grid  = th_ls + 0.05 * rng.normal(size=(500, 2))

# no nearby theta beats th_ls on EITHER objective, and both agree
assert all(sse(t) >= sse(th_ls) for t in grid)
assert all(neg_log_lik(t, sigma) >= neg_log_lik(th_ls, sigma) for t
    in grid)

# sigma rescales the NLL but cannot move the minimiser
for s in (0.1, 0.7, 5.0):
    assert np.isclose(neg_log_lik(th_ls, s), min(neg_log_lik(t, s)
        for t in grid))

print("ok: argmin SSE == argmin NLL, for every sigma")
```

The last loop is exercise 3.2 answered by machine: change σ and the value of the NLL moves, but the θ that minimises it does not. The full script is `notebooks/theory/ch_linreg_mle.py`, which also finite-differences $\nabla \frac{1}{2}\|y - X\theta\|^2 = X^\top(X\theta - y)$.

Another route: change the noise, change the loss

The derivation above runs one way — assume Gaussian noise, out comes squared error. It is more instructive run backwards. *Every* loss function you have ever seen is a negative log-likelihood for *some* noise assumption, and finding which one tells you when the loss is appropriate.

Take a general noise density $p(\varepsilon)$ and repeat the four steps: independence turns the joint density into a product, the logarithm turns the product into a sum, and the minus sign turns the maximisation into a minimisation, leaving

$$\hat{\theta} \;=\; \arg\min_{\theta} \sum_{i=1}^{n} \underbrace{\Big[-\log p\big(y^{(i)} - h_\theta(x^{(i)})\big) \Big]}_{\text{the loss on example } i}.$$

The bracket *is* the loss. So:

Noise density $p(r)$	$-\log p(r)$**, up to constants**	**Loss you know it as**
$\propto e^{-r^2/2\sigma^2}$ (Gaussian)	$r^2/2\sigma^2$	squared error
$\propto e^{-\lvert r\rvert/b}$ (Laplace)	$\lvert r\rvert/b$	absolute error (MAE)
Gaussian core, Laplace tails	Huber's function	Huber loss
$\propto (1 + r^2/\nu)^{-(\nu+1)/2}$ (Student t)	$\log(1 + r^2/\nu)$	a robust loss

Read the middle column as a function of r and the whole table becomes one sentence: *how fast the loss grows is how fast you think large residuals become implausible.* Squared error grows quadratically, so a residual of 10 is judged a hundred times worse than a residual of 1, and the fit will contort itself to avoid it — which is exactly the outlier sensitivity you saw in Chapter 8, now with a reason attached rather than an observation. Absolute error grows linearly, so a residual of 10 is only ten times worse, and the fit shrugs. Student-t

grows logarithmically: past a point, an extra unit of residual costs almost nothing, and the model is free to write the outlier off as noise.
None of these are new algorithms. They are the same four steps with a different density in the first one.

Go deeper

Watch. mathematicalmonk, *ML 4.1, Maximum Likelihood Estimation, part 1*
Fifteen minutes, a graphics tablet, no production values whatsoever, and the clearest available statement of the thing that trips everyone on first contact: the likelihood and the density are the same formula read as a function of different arguments. Continue into 4.2 if it lands.

Watch. Ali Ghodsi, Waterloo, *Lec 2.1, Regularization (deep learning course)*
Ghodsi derives the noise-assumption-to-loss correspondence for a research audience and then keeps going into what the prior is doing. More demanding than the rest of this list, and worth it.

Read. Stanford CS229, main notes, "Probabilistic interpretation". Two pages. The same derivation you just read, in the compressed form you will meet in papers. `https://cs229.stanford.edu/main_notes.pdf`

Exercises

Pencil first, then click the ⇝ to jump to the worked answer in Appendix D. The answer links back to the question.

3.1 ⇝ If $\varepsilon^{(i)}$ were Laplace (density $\propto e^{-|r|/\sigma}$), what sum would MLE minimise? Do not memorise a name — write the log-likelihood until the absolute values appear.

3.2 ⇝ Show that σ drops out of $\arg\max_\theta L(\theta)$ in the Gaussian model above. Where would σ come back if you also estimated it?

3.3 ⇝ A Gaussian prior $\theta \sim \mathcal{N}(0, \tau^2 I)$ multiplies $L(\theta)$ by $e^{-\|\theta\|^2/(2\tau^2)}$. What extra term appears in $-\log$ posterior? Which chapter of this book is that term?

3.4 ⇝ Take $n = 1$, $x = 1$, $y = 4$, $h = \theta$ (no intercept games). Sketch $L(\theta)$ and $J(\theta)$ on the same θ-axis. Where are their extrema?

Part II

Rolling downhill

4

Gradient Descent

You can plot $J(\theta_1)$ for the three-point line and see the bottom. You cannot plot $J(\theta)$ when θ has a hundred thousand coordinates. You need a rule that, standing at one θ, names a direction that lowers J, and a step size that does not throw you off the mountain.

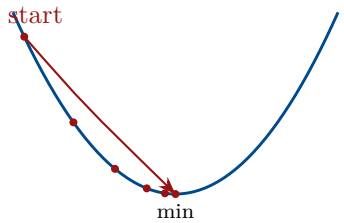
The rule is old: walk against the slope. The lectures call it gradient descent. This chapter derives the slope for linear regression, insists that both knobs update from the *old* θ, and shows — with the class bowl — what a stupid step size does. The full numerical walk-through of $J = \theta_1^2 + \theta_2^2 + 4$ is the next chapter. It is not a repeat. It is the same walk, with every iteration written out.

4.1 The update

For each coordinate θ_j,

$$\theta_j \;\leftarrow\; \theta_j - \alpha \frac{\partial J(\theta)}{\partial \theta_j}.$$

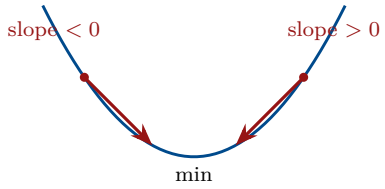
The partial derivative is the slope of J if you freeze every other knob and slide θ_j. Positive slope means J rises as θ_j rises, so you decrease θ_j. Negative slope: the other way. $\alpha > 0$ is the learning rate. Too small, you die of old age. Too large, you oscillate or leave the bowl. The symbol $\leftarrow$ means "replace."



A 1-D cartoon. Each red dot is one update. The path is not a physical ball — it is a sequence of points $\theta^{(0)}, \theta^{(1)}, \ldots$ you compute.

4.2 Why the minus sign works on both sides

Stand on the right face of a valley. The slope $\partial J/\partial\theta$ is positive (uphill to the right). Subtracting it moves you left, toward the floor. Stand on the left face. The slope is negative. Subtracting a negative adds, so you move right, toward the same floor. You do not need a separate "if" for each side. The minus sign is the if.



One formula, both faces. $\theta \leftarrow \theta - \alpha \cdot \text{slope}$.

Watch out

With two knobs you must compute *both* new values from the *old* (θ_0, θ_1), then assign. If you overwrite θ_0 first, the θ_1-gradient is taken at a point that is not the point you were standing on. That is a different algorithm. The notes draw it; so do we.

together

$t_0 = \theta_0 - \alpha\, \partial_{\theta_0} J(\theta_0, \theta_1)$
$t_1 = \theta_1 - \alpha\, \partial_{\theta_1} J(\theta_0, \theta_1)$
$\theta_0 = t_0,\ \theta_1 = t_1$

leaked

$t_0 = \theta_0 - \alpha\, \partial_{\theta_0} J$
$\theta_0 = t_0$ (already new)
$t_1 = \theta_1 - \alpha\, \partial_{\theta_1} J(\theta_0', \theta_1)$

Left: both partials see the same θ. *Right:* θ_0' *leaks into the* θ_1 *step. Do not ship the right-hand box.*

4.3 A derivative you can see with arithmetic

Before the long chain rule, a reminder the lectures actually compute.

$f(a) = 3a$. At $a = 2$, $f = 6$. At $a = 2.001$, $f = 6.003$. Slope $(0.003)/(0.001) = 3$. At $a = 5$ the same slope 3. So $f'(a) = 3$.

$f(a) = a^2$. At $a = 2$, $f = 4$. At $a = 2.001$, $f \approx 4.004001$, slope $\approx 4 = 2 \cdot 2$. At $a = 5$, slope $\approx 10 = 2 \cdot 5$. So $f'(a) = 2a$.

That is the power rule, not as a slogan, as a finite difference. The gradient of J is this, once per knob.

4.4 The slope of the cost

We need $\partial J/\partial \theta_j$ for $J(\theta) = \frac{1}{2n} \sum_i (h_\theta(x^{(i)}) - y^{(i)})^2$ with $h_\theta(x^{(i)}) = \sum_k \theta_k x_k^{(i)}$.

Derivation

$$\frac{\partial J}{\partial \theta_j} = \frac{\partial}{\partial \theta_j} \frac{1}{2n} \sum_{i=1}^{n} \Big(\sum_{k=0}^{d} \theta_k x_k^{(i)} - y^{(i)} \Big)^2 .$$

The sum over i and the $1/(2n)$ ride outside. For each term, the chain rule brings the power 2 down:

$$\frac{1}{2n} \sum_{i=1}^{n} 2\big(h_\theta(x^{(i)}) - y^{(i)}\big) \cdot \frac{\partial}{\partial \theta_j} \Big(\sum_k \theta_k x_k^{(i)} - y^{(i)} \Big).$$

Inside the last parenthesis, only the $k = j$ term knows about θ_j. Its derivative is $x_j^{(i)}$. The target $y^{(i)}$ is constant. So

$$\frac{\partial J}{\partial \theta_j} = \frac{1}{n} \sum_{i=1}^{n} \left(h_\theta(x^{(i)}) - y^{(i)} \right) x_j^{(i)}.$$

The 2 cancelled the $\frac{1}{2}$. That is why the $\frac{1}{2}$ was in J.

The update for linear regression is therefore

$$\theta_j \leftarrow \theta_j - \alpha \cdot \frac{1}{n} \sum_{i=1}^{n} \left(h_\theta(x^{(i)}) - y^{(i)} \right) x_j^{(i)}.$$

In English: each residual is blamed on feature j in proportion to how large x_j was on that example, then averaged. If house i was under-predicted and had a large living area, θ_{area} is pushed up.

Vectorised, the same thing is $\frac{1}{n} X^\top (X\theta - y)$. You will write that until you can see it in your sleep.

In code

The class bowl $J = \theta_1^2 + \theta_2^2 + 4$, start $(3, 4)$, $\alpha = 0.1$:

```
import numpy as np
theta = np.array([3., 4.])
for _ in range(30):
    theta = theta - 0.1 * (2 * theta)
print(theta)   # near (0, 0)
```

Same loop, linear regression, using the gradient we just derived:

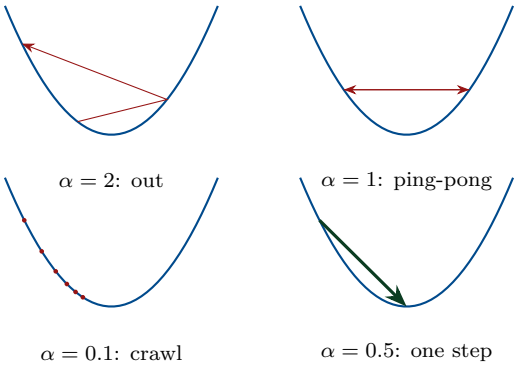
```
def gd(X, y, alpha=0.1, steps=500):
    theta = np.zeros(X.shape[1]); n = len(y)
    for _ in range(steps):
        theta = theta - alpha * (1/n) * (X.T @ (X @ theta - y))
    return theta
```

The line $\frac{1}{n}\sum(h - y)x_j$ *is* `(1/n)*(X.T @ (X@theta - y))`. Run `notebooks/theory/ch_linreg_mle.py` if you do not trust it.

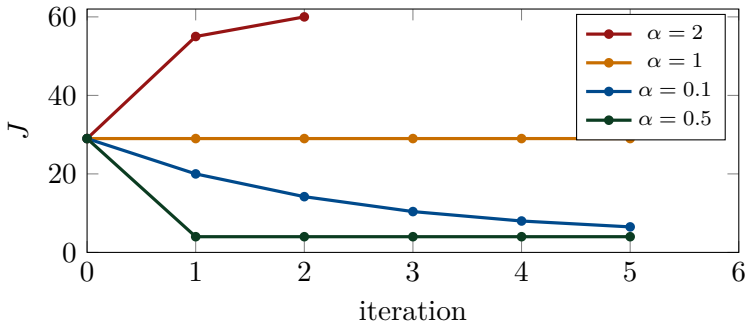
4.5 Four values of α, one bowl

The notes take $J(\theta) = \theta_1^2 + \theta_2^2 + 4$, so $\nabla J = 2\theta$, minimum at $(0, 0)$ with $J = 4$. Start at $(3, 4)$, where $J = 29$ and $\nabla J = (6, 8)$. One step multiplies θ by $(1 - 2\alpha)$. Stop here and compute the first step for $\alpha = 2$ yourself. Then look.

α	$1-2\alpha$	First steps	Name
2	-3	$(3,4) \to (-9,-12) \to (27,36)$, $J: 29 \to 229 \to 2029$	diverges
1	-1	$(3,4) \leftrightarrow (-3,-4)$, J stuck at 29	oscillates
0.1	0.8	$(3,4) \to (2.4,3.2) \to \cdots \to (0,0)$	crawls
0.5	0	$(3,4) \to (0,0)$ in one step	lands



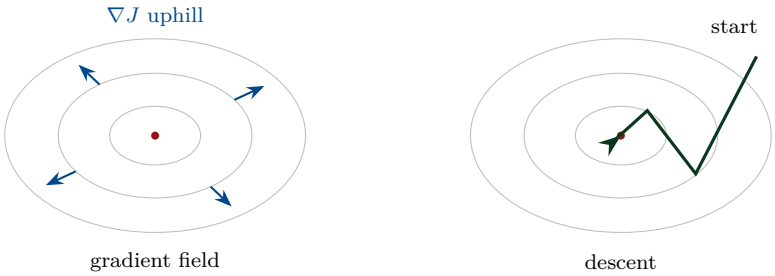
The four behaviours from class, drawn. $\alpha = 0.5$ is magic only because this J is a perfect quadratic. On housing data you do not get one-step perfection. You still get the same four kinds of failure.



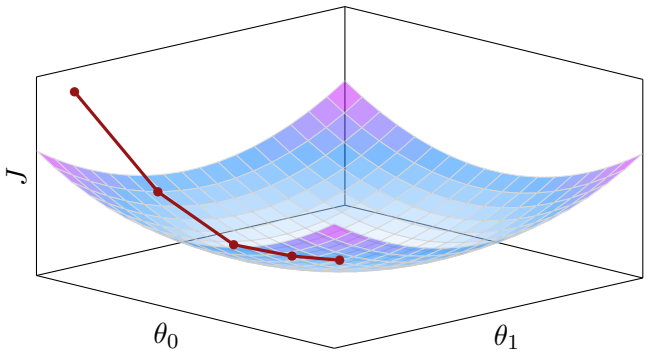
Cost against iteration. Divergence goes up. Oscillation is a flat lie. A small rate eases down. The lucky rate hits 4 and stays.

4.6 Contours, arrows, a surface

With two knobs, J is a surface. Its contours are the ellipses of Chapter 2. The gradient at a point is perpendicular to the contour through that point, pointing toward higher J. Descent walks the other way.



Left: arrows of steepest increase. Right: the path the lectures draw for a housing example, against those arrows, into the centre.



The same walk on the surface. Red is the iterate, not a marble with momentum. (Momentum is a later paper. Not this course.)



convex: any start, same floor

not convex: start picks the dip

Why the lectures care that J is convex here. Linear regression cannot hide a second minimum. A wavy loss can.

4.7 When to stop

The notes give three tests, any one of which, below a tolerance ϵ, is permission to halt:

1. $\|\theta^{(k+1)} - \theta^{(k)}\| \leq \epsilon$ — the knobs stopped moving;
2. $\|\nabla J(\theta^{(k)})\| \leq \epsilon$ — the floor is locally flat;
3. $|J(\theta^{(k+1)}) - J(\theta^{(k)})| \leq \epsilon$ — the score stopped improving.

The length $\|(2, 3, 4)\| = \sqrt{4 + 9 + 16} = \sqrt{29}$ is the same length as in Appendix A. A small ϵ on J can still leave θ wandering in a flat valley. Lab 3 will make you feel that.

Where this shows up

The update you just derived is, essentially unchanged, what trains every neural network in production. The differences are real but smaller than you would guess: the gradient is computed by backpropagation instead of by hand, α follows a schedule instead of being constant, and the loss is not convex so "the" minimum becomes "a" minimum. The line $\theta \leftarrow \theta - \alpha\nabla J$ survives all of it. If you can derive $\partial J/\partial\theta_j$ here, you have done the thing that a training loop does, once, by hand.

Go deeper

Watch. 3Blue1Brown, *Gradient descent, how neural networks learn*
The best animation of the gradient as a vector of "how much does each knob want to move". Watch it after you have derived the partial yourself, not before.

Read. Stanford CS229, main notes, "LMS algorithm". The same derivation with α and the update written per-example first, then batched — a useful order to see it in a second time. `https://cs229.`

> `stanford.edu/main_notes.pdf`

Exercises

Pencil first, then click the ⇝ to jump to the worked answer in Appendix D. The answer links back to the question.

4.1 ⇝ Derive $\partial J/\partial\theta_j$ for linear regression without looking. Name the line where the 2 cancels the $\frac{1}{2}$.

4.2 ⇝ Take $J = \theta_1^2 + \theta_2^2 + 4$, start $(3, 4)$, $\alpha = 0.25$. Compute θ after two steps by hand. What is $1 - 2\alpha$?

4.3 ⇝ Explain, in four sentences or fewer, why overwriting θ_0 before computing the θ_1 step is not "approximately gradient descent."

4.4 ⇝ A run has $\|\nabla J\|$ tiny but θ still drifting along a long thin valley. Which stopping rule fires first? Which one is lying?

4.5 ⇝ Write a two-knob update in NumPy that is *wrong* (sequential overwrite) and one that is right (simultaneous). On $J = \theta_1^2 + \theta_2^2 + 4$ with $\alpha = 0.4$, show they disagree after one step from $(3, 4)$.

4.6 ⇝ For the three-point line of Chapter 2, run 50 steps of GD from $\theta = (0, 0)$ with $\alpha = 0.1$ and with $\alpha = 1.5$. Report both final J values. One of them is a disaster. Which, and why, given the four cartoons above?

5

A Complete Gradient Descent Walk-through

Chapter 4 derived the update. This chapter is the lecture that refuses to stay abstract: $J = \theta_1^2 + \theta_2^2 + 4$, start at $(3, 4)$, four values of α, every row of the table written out. Do the next row yourself before you look. The three stopping rules, $\|(2, 3, 4)\|$, and the steepest-ascent proof are here because they were on the board, not because a template demanded a section.

Sir's bowl sits four units off the floor:

$$J(\theta) \;=\; \theta_1^2 + \theta_2^2 + 4.$$

The bottom is $(0, 0)$, where $J = 4$. Pretend you do not know that.

5.1 The bowl, the gradient, the update

Derivation

Differentiate one coordinate at a time:

$$\frac{\partial J}{\partial \theta_1} = 2\theta_1, \qquad \frac{\partial J}{\partial \theta_2} = 2\theta_2, \qquad \text{so} \qquad \nabla J(\theta) = \begin{bmatrix} 2\theta_1 \\ 2\theta_2 \end{bmatrix} = 2\theta.$$

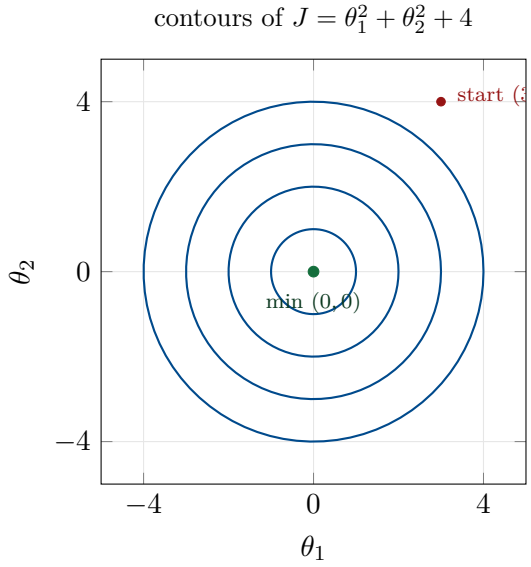
One gradient-descent step is then

$$\begin{aligned} \theta_1^{(k+1)} &= \theta_1^{(k)} - \alpha \cdot 2\theta_1^{(k)} = (1 - 2\alpha)\, \theta_1^{(k)}, \\ \theta_2^{(k+1)} &= \theta_2^{(k)} - \alpha \cdot 2\theta_2^{(k)} = (1 - 2\alpha)\, \theta_2^{(k)}. \end{aligned}$$

In vector form this is the one-line recurrence the whole chapter lives on:

$$\boxed{\theta^{(k+1)} \;=\; (1 - 2\alpha)\, \theta^{(k)}}.$$

Unrolling it, $\theta^{(k)} = (1 - 2\alpha)^k\, \theta^{(0)}$. The fate of the algorithm is *entirely* decided by the number $(1 - 2\alpha)$.



The class contour: concentric circles centred at the origin. Global minimum at $(0, 0)$*. Start at* $(3, 4)$*, where* $J = 9 + 16 + 4 = 29$*.*

$J - 4 = \theta_1^2 + \theta_2^2$ is the squared distance from the origin. Level sets of "distance from origin" are circles. The Hessian is $H = \nabla^2 J = 2I$, whose two eigenvalues are both 2 — same curvature in every direction — so the bowl is perfectly round. That is why a *single* α can be optimal for both coordinates at once. (Feature scaling in Chapter 8 is the trick that makes a *real* cost bowl this round.)

An Uttarayan kite on a perfectly calm terrace: the string's "pull toward the origin" is equally strong in every compass direction. If you yank too hard (α too big) the kite overshoots and the string wraps the other way. If you tug too softly you inch home for thirty steps. If you yank with exactly the right force, one tug lands you on the spool.

5.2 The four learning rates from class

We always start at $\theta^{(0)} = (3, 4)$, where $\nabla J = (6, 8)$ and $J = 29$. The notes try $\alpha \in \{2, 1, 0.1, 0.5\}$.

Case 1 — $\alpha = 2$: the cost explodes

Derivation

Update: $\theta \leftarrow \theta - 2 \cdot (2\theta) = -3\theta$. Each step *triples* the vector and flips the sign.

k	θ	∇J	J	$\theta^{k+1} = \theta - \alpha \nabla J$
0	$(3, 4)$	$(6, 8)$	29	$3 - 2 \cdot 6 = -9,\ 4 - 2 \cdot 8 = -12$
1	$(-9, -12)$	$(-18, -24)$	229	$-9 - 2(-18) = 27,\ -12 - 2(-24) = 36$
2	$(27, 36)$	$(54, 72)$	2029	$27 - 2 \cdot 54 = -81,\ 36 - 2 \cdot 72 = -108$

J goes $29 \to 229 \to 2029$. The notes' verdict: **cost is increasing**. In the closed form, $1 - 2\alpha = 1 - 4 = -3$, and $|-3| > 1$, so $\|(-3)^k \theta^{(0)}\|$ explodes.

Case 2 — $\alpha = 1$: perpetual oscillation

Derivation

Now $1 - 2\alpha = -1$, so $\theta^{(k+1)} = -\theta^{(k)}$. The algorithm hops between $(3, 4)$ and $(-3, -4)$ forever. J stays 29 on both sides — it neither diverges nor converges.

k	θ	∇J	J	next θ
0	$(3, 4)$	$(6, 8)$	29	$3 - 1 \cdot 6 = -3$, $4 - 1 \cdot 8 = -4$
1	$(-3, -4)$	$(-6, -8)$	29	$-3 - 1(-6) = 3$, $-4 - 1(-8) = 4$
2	$(3, 4)$	$(6, 8)$	29	$(-3, -4)$ again

The notes' exact phrase: **oscillating without divergence or convergence**.

Case 3 — $\alpha = 0.1$: slow, honest progress

Derivation

$1 - 2\alpha = 0.8$. Each step keeps 80% of the current vector — a contraction, but a lazy one.

k	θ	∇J	J	next
0	$(3, 4)$	$(6, 8)$	29	(2.4, 3.2)
1	(2.4, 3.2)	(4.8, 6.4)	20.48	(1.92, 2.56)
2	(1.92, 2.56)	(3.84, 5.12)	14.3104	(1.536, 2.048)
30	$\approx$ (0.0037, 0.005)	≈ 0	≈ 4	still creeping

(The $k = 2$ cost in the handwritten table is rounded to 14.24; the exact value is $1.92^2 + 2.56^2 + 4 = 14.3104$. Same story: downhill, slowly.) After 30 steps we are near the origin and $J \approx 4$, the floor of the bowl. The notes: **slow converge**.

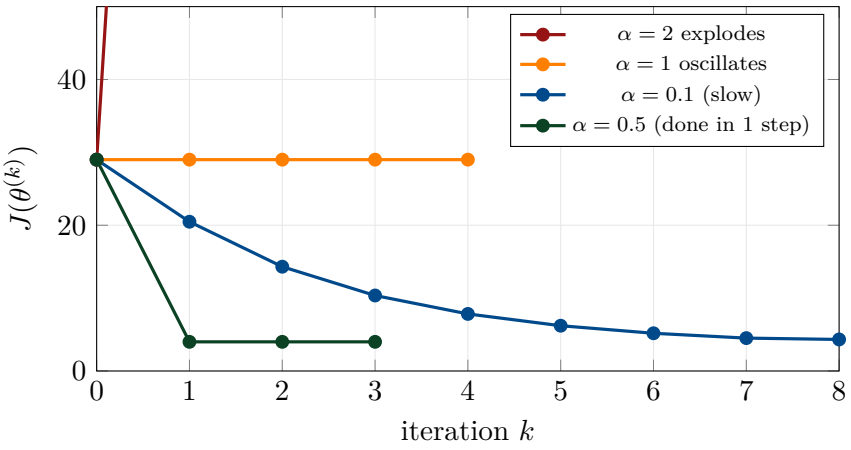
Case 4 — $\alpha = 0.5$: one-step bullseye

Derivation

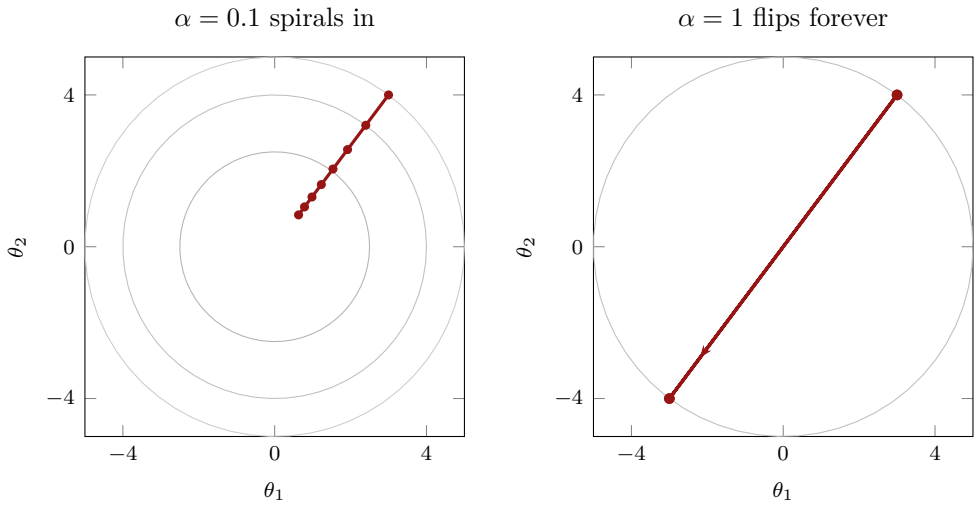
$1 - 2\alpha = 0$. The update is $\theta \leftarrow 0$ in a single shot:

$$3 - 0.5 \cdot 6 = 0, \qquad 4 - 0.5 \cdot 8 = 0.$$

This is not luck. For a quadratic $J = \frac{1}{2}\theta^\top H\theta + c$ the optimal fixed step is $\alpha^\star = 1/\lambda_{\max}(H)$. Here $H = 2I$ so $\lambda_{\max} = 2$ and $\alpha^\star = 1/2$. The class example is the rare bowl where "optimal" is also *exact*.



Four fates of the same start. The floor of this bowl is $J = 4$, not 0 — never wait for the cost to hit zero.



Left: $\alpha = 0.1$ walks radially toward the origin (circles stay circles). Right: $\alpha = 1$ teleports to the antipode and back.

$\theta^{(k)} = (1-2\alpha)^k\theta^{(0)}$ goes to 0 if and only if $|1-2\alpha| < 1$, i.e. $0 < \alpha < 1$. At $\alpha = 1$ we sit on the boundary (oscillation). Above 1 we diverge. At $\alpha = 1/2$ we hit in one step. This is the 1-D version of the general rule for a quadratic: GD with step α is a contraction on eigen-direction j iff $|1 - \alpha\lambda_j| < 1$. Bach's *Learning Theory from First Principles* and Boyd's convex notes both start from this spectral picture — it is the cutting-edge view of a 1950s algorithm, and it is already visible in this 2-parameter toy.

That last sentence is doing a lot of work, so let us actually prove it rather than assert it. The route is short and it explains every number in the table above.

The exact step-size threshold, from scratch

Step 1 — do the general one-dimensional quadratic. Take $J(\theta) = a(\theta - \theta^\star)^2$ with $a > 0$; the curvature is $J'' = 2a$. Its derivative is $J'(\theta) = 2a(\theta - \theta^\star)$, so one gradient step is

$$\theta^{(k+1)} = \theta^{(k)} - \alpha \cdot 2a\big(\theta^{(k)} - \theta^\star\big).$$

Step 2 — track the error, not the parameter. Subtract $\theta^\star$ from both sides and

write $e^{(k)} = \theta^{(k)} - \theta^\star$:

$$e^{(k+1)} = e^{(k)} - 2a\alpha\, e^{(k)} = (1 - 2a\alpha)\, e^{(k)}.$$

This is the whole trick. The messy iteration on θ is a plain geometric sequence on e.

Step 3 — unroll.

$$e^{(k)} = (1 - 2a\alpha)^k\, e^{(0)}.$$

A geometric sequence r^k goes to zero exactly when $|r| < 1$, grows without bound when $|r| > 1$, and does neither when $|r| = 1$. So

$$\boxed{\text{converges} \iff |1 - 2a\alpha| < 1 \iff 0 < \alpha < \frac{1}{a}}$$

Step 4 — read the class example off it. Our bowl is $J = \theta_1^2 + \theta_2^2 + 4$, so $a = 1$ in each coordinate and the condition is $0 < \alpha < 1$: exactly the interval the four experiments found. Setting $1 - 2a\alpha = 0$ gives $\alpha = 1/(2a) = 1/2$, the one-step jump. And $\alpha = 1$ gives $r = -1$, the eternal hop.

Step 5 — notice what the threshold depends on. The safe range is $1/a$: the *steeper* the bowl, the *smaller* the largest legal step. This is not an arbitrary fact about a toy. In Chapter 6 you found $\nabla^2 J = \frac{1}{n} X^\top X$, so the curvature is set by your data, and in Chapter 8 you will change that curvature deliberately by rescaling features. The number a in this box is the thread connecting those two chapters.

The same threshold in d dimensions

Nothing above needed the problem to be one-dimensional; it only needed the coordinates not to interact. For a general quadratic with Hessian H, diagonalise: H is symmetric, so it has an orthonormal eigenbasis $v_1, \dots, v_d$ with eigenvalues $\lambda_1 \geq \cdots \geq \lambda_d \geq 0$. Write the error in that basis, $e^{(k)} = \sum_j c_j^{(k)} v_j$. Because $Hv_j = \lambda_j v_j$, the gradient step acts on each coefficient separately:

$$c_j^{(k+1)} = (1 - \alpha\lambda_j)\, c_j^{(k)}.$$

So the algorithm is not one geometric sequence but d of them, running in parallel along the eigen-directions and never talking to each other. All of them must converge, so

$$|1 - \alpha\lambda_j| < 1 \ \text{ for every } j \qquad \iff \qquad 0 < \alpha < \frac{2}{\lambda_{\max}}.$$

Two consequences worth memorising. *The largest eigenvalue sets the speed limit* — one steep direction caps α for the whole problem, however flat everything else is. And *the smallest eigenvalue sets the actual speed*: with α pinned near $2/\lambda_{\max}$, the slowest coefficient shrinks by a factor $|1 - 2\lambda_{\min}/\lambda_{\max}|$ per step, which is close to 1 when $\lambda_{\max}/\lambda_{\min}$ is large. That ratio is the condition number, it is the subject of Chapter 11, and this is precisely why it matters.

Go deeper

Watch. 3Blue1Brown, *Gradient descent, how neural networks learn*

Read. Ruder, *An overview of gradient descent optimization algorithms*, §2–§4. Where momentum, RMSProp and Adam come from, each one an attempt to escape the $2/\lambda_{\max}$ speed limit derived

above. https://arxiv.org/abs/1609.04747

Choosing α is like adding chilli to a farsan mix. None: the snack is raw ($\alpha = 0.1$, you will be frying until midnight). A sensible pinch: done ($\alpha = 0.5$). A fistful: the kitchen is on fire ($\alpha = 2$). Exactly the wrong pinch: you keep taking the tray in and out of the oil and it never finishes ($\alpha = 1$).

5.3 Stopping criteria (from the notes)

Gradient descent is an infinite loop until you kill it. The notes give three honest kill-switches. Pick a tiny $\varepsilon > 0$ (say 10^{-6}) and stop when any one fires.

1. **Parameters stopped moving:** $\|\theta^{(k+1)} - \theta^{(k)}\| \le \varepsilon$.
2. **We are on flat ground** (underlined in class — this is the first-order optimality condition): $\|\nabla_\theta J(\theta^{(k)})\| \le \varepsilon$.
3. **The cost stopped dropping:** $\left|J(\theta^{(k+1)}) - J(\theta^{(k)})\right| \le \varepsilon$.

(i) asks "did the knobs twitch?" (ii) asks "is the slope already basically zero?" (iii) asks "did the scoreboard move?" They are *not* the same. On a long, flat valley, (iii) can fire while (ii) is still a little noisy; on a steep narrow ravine, (ii) can be huge even when (i) is small because you took a tiny α. In this class example they all agree near the origin.

What $\|\cdot\|$ actually means — the notes' numerical example

Derivation

The notes compute a Euclidean (L_2) norm on a 3-vector so the bars stop being decoration:

$$\vec{x} = \begin{bmatrix} 2 \\ 3 \\ 4 \end{bmatrix}, \qquad \|\vec{x}\| = \sqrt{2^2 + 3^2 + 4^2} = \sqrt{4 + 9 + 16} = \sqrt{29}.$$

Same recipe for a gradient: $\|\nabla J(3, 4)\| = \sqrt{6^2 + 8^2} = \sqrt{36 + 64} = 10$. At the start of the example, criterion (ii) with $\varepsilon = 10^{-3}$ is *nowhere near* firing ($10 \not\le 10^{-3}$). After the $\alpha = 0.5$ step it is exactly 0.

Watch out

"Cost hit zero" is *not* a stopping rule, and on this example it would never fire: the floor is 4, not 0. Always compare against a change, or against the gradient, never against an absolute $J = 0$ fantasy.

5.4 Steepest ascent, finished (Lec 6 last page + Lec 7 first page)

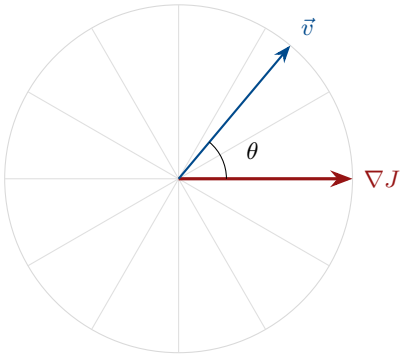
Chapter 6 already sketched this. Here is the notes' argument with every word filled in, because viva loves it.

Derivation

Claim. The direction of maximum rate of change of $J(\theta)$ is $\nabla_\theta J$.
Proof. Let $\vec{v}$ be a *unit* vector ($\|\vec{v}\| = 1$). The rate of change of J when you walk in direction $\vec{v}$ is the directional derivative

$$\frac{\partial J}{\partial \vec{v}} \;=\; \nabla J \cdot \vec{v} \;=\; \|\nabla J\|\,\|\vec{v}\|\,\cos\theta \;=\; \|\nabla J\|\,\cos\theta,$$

where θ is the angle between $\vec{v}$ and ∇J. At a fixed point, $\|\nabla J\|$ is a constant. $\|\vec{v}\| = 1$ is a constant. The only thing you can maximise is $\cos\theta$, whose maximum is 1, attained at $\theta = 0$. So $\vec{v}$ must be parallel to ∇J: *when you change in a direction along the gradient, you increase most rapidly.* ■
Therefore gradient *descent* walks along $-\nabla J$.



rate $= \|\nabla J\| \cos\theta$
biggest only at $\theta = 0$

Every unit $\vec{v}$ is a point on the circle. The climb is the projection of ∇J onto $\vec{v}$. The longest projection is ∇J itself.

5.5 The batch update, written the way class writes it

Once we return to linear regression, the same $\theta_j \leftarrow \theta_j - \alpha\partial_j J$ becomes, with $J = \frac{1}{2n}\sum_i (h^{(i)} - y^{(i)})^2$,

$$\theta_j \;:=\; \theta_j - \alpha\,\frac{1}{n}\sum_{i=1}^{n}\left(\sum_{k=0}^{d}\theta_k x_k^{(i)} - y^{(i)}\right)x_j^{(i)}.$$

The inner sum is the prediction for example i; the outer sum averages the residual-times-feature over all n training rows. That is **batch** GD, and the notes flag it in blue: **slow when the number of training data is large**. Stochastic GD (Chapter 7, next after Hessians) peels the outer sum down to *one* i and — watch the sign in the notes — writes

$$\theta_j \;:=\; \theta_j + \alpha\big(y^{(i)} - h_\theta(x^{(i)})\big)x_j^{(i)}$$

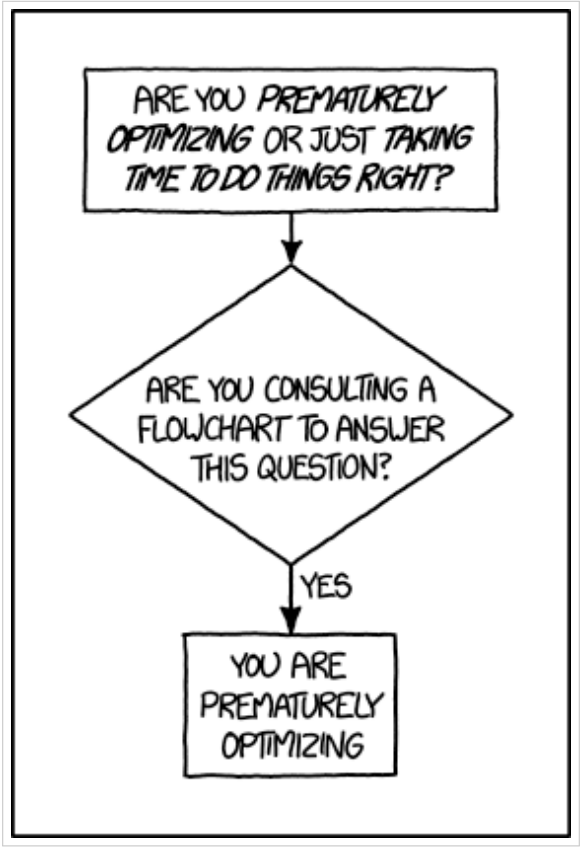
because $y - h = -(h - y)$. Same step, flipped residual. Faster per update; may not sit exactly on the minimum.

In code

The class example in five lines. Print J each step and you will reproduce the tables:

```
import numpy as np
```

```
def J(th): return th[0]**2 + th[1]**2 + 4
th = np.array([3.0, 4.0])
alpha = 0.1           # try 2, 1, 0.1, 0.5
for k in range(8):
    g = 2*th
    print(k, th, g, J(th))
    th = th - alpha*g
```



xkcd #1691 "Optimization" by Randall Munroe — https://xkcd.com/1691/ (CC BY-NC 2.5).

A person labels a local dip "optimum" while a deeper valley sits next door. On *this* chapter's bowl there is only one dip — but the cartoon is why we still write "may not converge to the minimum" next to SGD.

Exercises

Pencil first, then click the ⇝ to jump to the worked answer in Appendix D. The answer links back to the question.

5.1 ⇝ Without looking at the tables: start $(3, 4)$, $\alpha = 0.1$. Write θ and J for the first three iterates. Then check the lecture table.

5.2 ⇝ Prove that $\theta^{(k)} = (1 - 2\alpha)^k \theta^{(0)}$ for this J. For which α does this sequence converge? For which does $|1 - 2\alpha| = 1$?

5.3 ⇝ Compute $\|(2, 3, 4)^\top\|$. Which stopping rule uses this kind of length on $\Delta\theta$?

5.4 ⇝ Reproduce the steepest-ascent argument: directional derivative $= \|\nabla J\| \cos\theta$. Where is it maximised? Where minimised?

5.5 ⇝ Change the code listing to $\alpha = 2$ and print eight steps. Match the divergence table, or find the bug in your printout.

5.6 ⇝ Apply the threshold $0 < \alpha < 1/a$ to $J(\theta) = 25(\theta - 2)^2$. What is the largest legal α? What single α lands on the answer in one step? Now do the same for $J(\theta) = 0.01(\theta - 2)^2$ and comment on the two answers together.

5.7 ⇝ A quadratic in two variables has Hessian eigenvalues $\lambda_1 = 50$ and $\lambda_2 = 0.5$. State the largest α that does not diverge. With α set just under that value, by what factor does the error along the λ_2 direction shrink per step? Roughly how many steps to shrink it by a factor of ten?

5.8 ⇝ You run gradient descent and the parameter test $\|\Delta\theta\| \leq 10^{-6}$ fires while the gradient test $\|\nabla J\| \leq 10^{-6}$ does not. What have you almost certainly done wrong, and which of the two numbers should you trust?

6

Gradients and Hessians in Full

A partial derivative is a slope with the other knobs frozen. Stack those slopes and you have ∇J. Differentiate again and you have the Hessian — the thing that tells you whether the bowl is round or a valley. The lectures then set $\nabla J = 0$ and get the normal equation, which is gradient descent's destination written as a linear system. Every identity below is used later. None of it is scenery.

6.1 The gradient of a function of many variables

For a function $f : \mathbb{R}^n \to \mathbb{R}$ (many inputs, one number out), the **gradient** stacks all the partial derivatives into a vector:

$$\nabla_x f(x) = \left[\frac{\partial f}{\partial x_1},\ \frac{\partial f}{\partial x_2},\ \ldots,\ \frac{\partial f}{\partial x_n}\right]^\top.$$

If the input is a whole *matrix* A, the gradient is a matrix of the same shape, with $\big(\nabla_A f\big)_{ij} = \partial f / \partial A_{ij}$. Two rules we lean on: *linearity* $\nabla(f+g) = \nabla f + \nabla g$ and $\nabla(t\,f) = t\,\nabla f$. And a caution from the notes: $\nabla_x(Ax)$ is **not defined** — the gradient is only for functions that output a single number, but Ax outputs a vector.

$$f(x_1, x_2, x_3) \xrightarrow{\text{one slope per input}} \nabla f = \begin{bmatrix} \partial f/\partial x_1 \\ \partial f/\partial x_2 \\ \partial f/\partial x_3 \end{bmatrix}$$

The gradient is just "the slope in every direction, stacked into a vector".

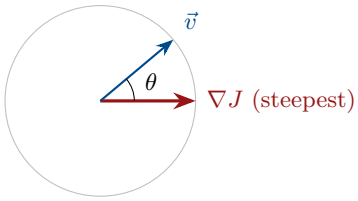
6.2 Why the gradient points the steepest way (a proof)

Derivation

Claim: ∇J points in the direction of fastest increase. **Proof (from the notes):** the rate of change of J in a unit direction $\vec{v}$ is the dot product

$$\frac{\partial J}{\partial \vec{v}} = \nabla J \cdot \vec{v} = \|\nabla J\|\,\|\vec{v}\| \cos\theta,$$

where θ is the angle between $\vec{v}$ and ∇J. Since $\|\nabla J\|$ is fixed and $\|\vec{v}\| = 1$, this is largest exactly when $\cos\theta = 1$, i.e. $\theta = 0$ — when $\vec{v}$ points *along* ∇J. So the gradient is the steepest-ascent direction, and $-\nabla J$ (used in gradient descent) is steepest *descent*. ■

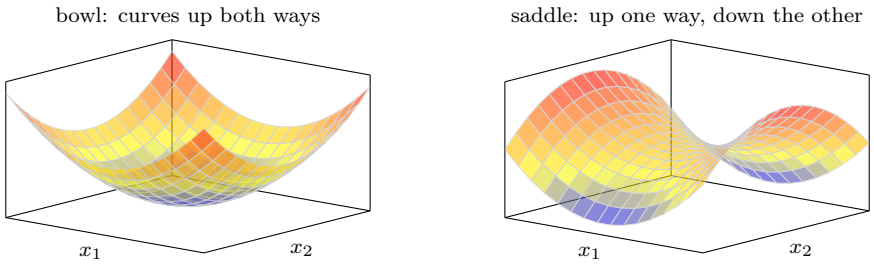


rate = $\|\nabla J\| \cos\theta$, biggest at θ=0

Moving along $\vec{v}$ climbs at rate $\|\nabla J\| \cos\theta$. The climb is fastest when $\vec{v}$ lines up with ∇J ($\theta = 0$).

6.3 The Hessian: second derivatives in a matrix

The **Hessian** $H = \nabla_x^2 f$ collects all the *second* partial derivatives: $H_{ij} = \dfrac{\partial^2 f}{\partial x_i \, \partial x_j}$. It is always **symmetric** ($H_{ij} = H_{ji}$) because the order of differentiation does not matter. Where the gradient gave slope, the Hessian gives *curvature* in every pair of directions.



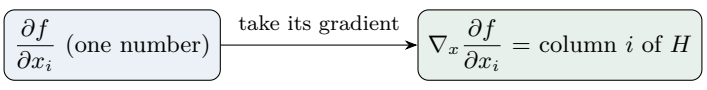
*Curvature made visible. A **bowl** ($z = x_1^2 + x_2^2$) curves up in every direction — Hessian eigenvalues all positive (a minimum). A **saddle** ($z = x_1^2 - x_2^2$) curves up one way and down the other — mixed-sign eigenvalues.*

Watch out

The Hessian is NOT "the gradient of the gradient" written naively. The gradient ∇f is a *vector* (outputs several numbers), and ∇ of a vector is not defined. The correct way (from the notes): take the gradient of *each component* $\partial f / \partial x_i$; that gives the i-th *column* of the Hessian. Assemble the columns:

$$\nabla_x^2 f(x) = \nabla_x \big(\nabla_x f(x)\big)^\top.$$

This is exactly the "build it column by column" idea used in the Lab 2 numerical Hessian.



Correct Hessian construction: differentiate each gradient component again to get one column of H; stack the columns.

6.4 Gradient & Hessian of a linear function

Derivation

Let $f(x) = b^\top x = \sum_{i=1}^n b_i x_i$ (a weighted sum). Differentiate w.r.t. x_k: only the $i = k$ term has x_k, and its derivative is b_k. So

$$\frac{\partial f}{\partial x_k} = b_k \quad \Longrightarrow \quad \boxed{\nabla_x\, b^\top x = b}.$$

This is the vector version of the scalar fact $\frac{d}{dx}(ax) = a$. Since the gradient b is a constant, all second derivatives vanish: the **Hessian of a linear function is** 0.

6.5 Gradient & Hessian of a quadratic function

Quadratics are the important case — our cost bowls *are* quadratics. The key result: for a symmetric matrix A, the function $f(x) = x^\top Ax$ has gradient $2Ax$ and Hessian $2A$. This is the exact vector version of the scalar $\frac{d}{dx}(ax^2) = 2ax$ and $\frac{d^2}{dx^2}(ax^2) = 2a$.

Derivation

See it on a 2×2 example. Let $A = \begin{bmatrix} a & b \\ b & c \end{bmatrix}$ (symmetric). Then

$$x^\top Ax = ax_1^2 + 2b\,x_1x_2 + c\,x_2^2.$$

Differentiate: $\partial/\partial x_1 = 2ax_1 + 2bx_2$ and $\partial/\partial x_2 = 2bx_1 + 2cx_2$. Stack them:

$$\nabla f = \begin{bmatrix} 2ax_1 + 2bx_2 \\ 2bx_1 + 2cx_2 \end{bmatrix} = 2 \begin{bmatrix} a & b \\ b & c \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = 2Ax.$$

Differentiating once more gives the constant matrix $\begin{bmatrix} 2a & 2b \\ 2b & 2c \end{bmatrix} = 2A$ — the Hessian.

Derivation

The general $n \times n$ proof (the full notes derivation). Write the quadratic as a double sum and differentiate with respect to one variable x_k:

$$f(x) = x^\top Ax = \sum_{i=1}^n \sum_{j=1}^n A_{ij}\, x_i x_j, \qquad \frac{\partial f}{\partial x_k} = \frac{\partial}{\partial x_k} \sum_i \sum_j A_{ij} x_i x_j.$$

Which terms survive? A term $A_{ij}x_ix_j$ depends on x_k only if $i = k$ or $j = k$. Splitting those out:

$$\frac{\partial f}{\partial x_k} = \underbrace{\sum_{i \neq k} A_{ik}x_i}_{\text{from } j=k} + \underbrace{\sum_{j \neq k} A_{kj}x_j}_{\text{from } i=k} + \underbrace{2A_{kk}x_k}_{\text{from } A_{kk}x_k^2}.$$

Fold the $2A_{kk}x_k$ back into the two sums (it supplies the missing $i = k$ and $j = k$ terms),

so each sum becomes a full sum:

$$= \sum_{i=1}^{n} A_{ik}x_i + \sum_{j=1}^{n} A_{kj}x_j.$$

Rename the dummy index $j \to i$ in the second sum, then use **symmetry** $A_{ik} = A_{ki}$:

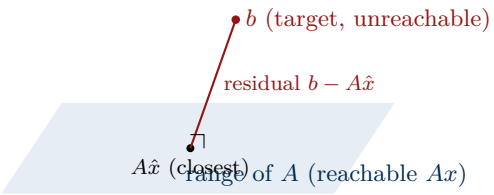
$$= \sum_{i} A_{ik}x_i + \sum_{i} A_{ki}x_i = 2\sum_{i} A_{ki}x_i = (2Ax)_k.$$

Since this holds for every k, $\nabla_x\, x^\top Ax = 2Ax$. ■

Vector/matrix form	Scalar analogy	Hessian
$\nabla_x\, b^\top x = b$	$\frac{d}{dx}(ax) = a$	0
$\nabla_x\, x^\top Ax = 2Ax$ (sym. A)	$\frac{d}{dx}(ax^2) = 2ax$	$2A$

6.6 Least squares: the normal equation

Here is the payoff. Suppose we want $Ax = b$ but no exact solution exists (there are more equations than unknowns, so b is "out of reach"). Least squares finds the x that makes Ax *as close as possible* to b — the foot of the perpendicular from b onto the set of reachable points.



Least squares: b sticks out of the plane of reachable vectors. The best $A\hat{x}$ is the point on the plane closest to b; the leftover residual is perpendicular to the plane.

Derivation

Minimise the squared distance $\|Ax - b\|_2^2$. Using $\|v\|_2^2 = v^\top v$ and expanding:

$$\begin{aligned}\|Ax - b\|_2^2 &= (Ax - b)^\top (Ax - b) \\ &= x^\top A^\top Ax - x^\top A^\top b - b^\top Ax + b^\top b \\ &= x^\top (A^\top A)x - 2\, b^\top Ax + b^\top b. \qquad \text{(the two middle terms are equal scalars)}\end{aligned}$$

Take the gradient using our two boxed rules — quadratic term ($A^\top A$ is symmetric) gives $2A^\top Ax$, linear term gives $2A^\top b$, constant gives 0:

$$\nabla_x \|Ax - b\|_2^2 = 2A^\top Ax - 2A^\top b.$$

Set it to zero (bottom of the convex bowl) and solve:

$$A^\top Ax = A^\top b \quad \Longrightarrow \quad \boxed{\hat{x} = (A^\top A)^{-1} A^\top b}.$$

These are the **normal equations**, and $\hat{x}$ is the least-squares solution.

This is linear regression in one shot. If A is the design matrix X (features plus a ones column) and b is the target vector y, then $\hat{\theta} = (X^\top X)^{-1} X^\top y$ is the best-fit parameter vector — the *same* answer gradient descent crawls toward in Chapter 4, but computed directly. (Gradient descent is still preferred when X is huge, because inverting $X^\top X$ gets expensive.)

Another route: read it off the picture instead

Go back to the drawing above, the one with b floating off the plane. That picture already contains the normal equation, and extracting it requires no differentiation whatsoever.
The set of vectors A can produce, as x ranges over everything, is the *column space* of A — the shaded plane. We want the point of that plane closest to b. From school geometry: the closest point on a plane to an outside point is the foot of the perpendicular. So at the optimum, the residual $r = b - A\hat{x}$ must be perpendicular to the plane.
Perpendicular to a plane means perpendicular to every vector lying in it, and every such vector is Av for some v. So for all v,

$$(Av)^\top r = 0 \quad \Longleftrightarrow \quad v^\top \big(A^\top r\big) = 0.$$

A vector whose dot product with *every* v vanishes is the zero vector. Hence $A^\top r = 0$, that is,

$$A^\top \big(b - A\hat{x}\big) = 0 \quad \Longrightarrow \quad A^\top A\hat{x} = A^\top b.$$

Same equation, obtained by staring at a triangle. It is worth having both derivations in your head, because they fail differently and they generalise differently. The calculus route tells you *why* the stationary point is a minimum rather than a maximum (you check the Hessian). The geometric route tells you *what the answer means*: $A\hat{x}$ is the shadow of y on the space your features can reach, and the residual is the part of y your features are constitutionally incapable of explaining. That reading is the one that makes the pseudoinverse of Chapter 12 obvious rather than magical.

6.7 Why the bowl really is a bowl

Chapter 2 asserted convexity by drawing a picture. Now you have the machinery to prove it, and the proof is three lines.

Derivation

Take the least-squares cost $J(\theta) = \frac{1}{2n}\|X\theta - y\|^2$. From the expansion above, its gradient is $\nabla J = \frac{1}{n} X^\top (X\theta - y)$, which is $\frac{1}{n} X^\top X\theta - \frac{1}{n} X^\top y$: a linear function of θ. Differentiating a linear function gives its coefficient matrix, so

$$\nabla^2 J(\theta) \;=\; \frac{1}{n} X^\top X.$$

Two things about this deserve to be said out loud.
It does not depend on θ. The curvature is the same everywhere on the surface. That is what makes the cost a *paraboloid* rather than some general lumpy landscape, and it is why a single global picture — the bowl — is honest.

It is positive semi-definite. For any vector v,

$$v^\top\Big(\tfrac{1}{n}X^\top X\Big)v = \tfrac{1}{n}(Xv)^\top(Xv) = \tfrac{1}{n}\|Xv\|^2 \;\geq\; 0,$$

because a squared length cannot be negative. A function whose Hessian is positive semi-definite everywhere is convex. Therefore the surface curves upward, or flat, in every direction, from every point — there are no saddles and no second valley, and any θ with $\nabla J = 0$ is a global minimum. ■

That last step is what licenses everything else in the book. Setting the gradient to zero is only a *necessary* condition for a minimum in general; a saddle point satisfies it too. Convexity is what upgrades it to sufficient, and convexity is what entitles gradient descent, started anywhere, to arrive at the same place.

When is the inequality strict — $\|Xv\|^2 > 0$ for every $v \neq 0$, making J strictly convex and the minimum unique? Exactly when $Xv = 0$ forces $v = 0$, that is, when the columns of X are linearly independent. Duplicate a feature column and you can find a $v \neq 0$ with $Xv = 0$; the bowl then has a perfectly flat trough along that direction, infinitely many minimisers, and a singular $X^\top X$. That is the failure Chapters 9 and 12 exist to handle, and you have just met it as a statement about eigenvalues rather than as an error message.

6.8 Using curvature: Newton's method

Gradient descent uses the first derivative and ignores the second. If you have the Hessian, you can do better, and seeing how makes it clear what the Hessian is *for*.

Derivation

Near a point $\theta^{(t)}$, approximate J by the quadratic that matches its value, slope and curvature there:

$$J(\theta) \;\approx\; J(\theta^{(t)}) + \nabla J(\theta^{(t)})^\top\big(\theta - \theta^{(t)}\big) + \tfrac{1}{2}\big(\theta - \theta^{(t)}\big)^\top H\big(\theta - \theta^{(t)}\big), \qquad H = \nabla^2 J(\theta^{(t)}).$$

Rather than stepping a little way downhill on this model, jump straight to *its* minimum. Write $s = \theta - \theta^{(t)}$ and differentiate the right-hand side using the two boxed rules — the linear term contributes ∇J, the quadratic term contributes Hs:

$$\nabla_s\big[\,\cdot\,\big] = \nabla J(\theta^{(t)}) + Hs = 0 \quad\Longrightarrow\quad s = -H^{-1}\nabla J(\theta^{(t)}).$$

So the update is

$$\boxed{\theta^{(t+1)} = \theta^{(t)} - H^{-1}\,\nabla J(\theta^{(t)})}$$

Compare with $\theta - \alpha\nabla J$. The learning rate, a single scalar you had to guess, has been replaced by a matrix you computed.

For least squares this is not an approximation at all. J *is* quadratic, so the model above is exact, and one Newton step from anywhere lands exactly on the answer:

$$\theta^{(1)} = \theta^{(0)} - \Big(\tfrac{1}{n}X^\top X\Big)^{-1}\tfrac{1}{n}X^\top\big(X\theta^{(0)} - y\big) = (X^\top X)^{-1}X^\top y.$$

> Newton's method applied to linear regression *is* the normal equation. They are the same computation wearing different names.

So why does anyone use gradient descent? Cost. A gradient is $O(nd)$; forming and inverting a $d \times d$ Hessian is $O(nd^2 + d^3)$. With d in the millions, as in any modern model, H^{-1} is not a matrix you are going to build. The whole design space of optimisers — momentum, RMSProp, Adam — lives in the gap between these two extremes, trying to buy some of Newton's curvature information at something closer to gradient descent's price.

Go deeper

Read. Petersen & Pedersen, *The Matrix Cookbook*, §2.4. Every gradient identity in this chapter, plus fifty you do not need yet. https://www.math.uwaterloo.ca/~hwolkowi/matrixcookbook.pdf

Watch. MIT 18.06 (Strang), lecture 16, *Projection matrices and least squares*
Strang derives the orthogonality route above on a blackboard, and proves $X^\top X$ invertible from independent columns in the same hour. If the "Another route" box felt slippery, watch this before re-reading it.

In code

The normal equation is one NumPy line, and we can *check* the gradient rules numerically (exactly the Lab 2 idea):

```
import numpy as np
# least squares: best-fit line for (1,1),(2,2),(3,3)
A = np.array([[1.,1],[1,2],[1,3]]);  b = np.array([1.,2,3])
theta_hat = np.linalg.inv(A.T @ A) @ A.T @ b      # (A^T A)^{-1} A^T b
print(theta_hat)                                   # -> [0, 1]  (intercept 0, slope 1)

# verify  grad(x^T A x) = 2 A x  with a finite difference
M = np.array([[2.,1],[1,3]])                       # symmetric
f = lambda x: x @ M @ x
x0 = np.array([1., 2.])
h = 1e-5
num = np.array([(f(x0+h*e)-f(x0))/h for e in np.eye(2)])
print(num, 2 * M @ x0)                             # [8 14]  matches 2Ax
```

For heavy problems prefer `np.linalg.lstsq(A, b)` — it solves the normal equations without forming the fragile inverse.

Exercises

Pencil first, then click the ⇝ to jump to the worked answer in Appendix D. The answer links back to the question.

6.1 ⇝ Write $\nabla(b^\top x)$ and $\nabla(x^\top A x)$ for symmetric A. Name the one-variable derivative each one generalises, then check the second with a 2×2 finite difference.

6.2 ⇝ Why is the Hessian symmetric? Name the theorem.

6.3 ⇝ Why is "the Hessian is the gradient of the gradient" the wrong sentence? Write the right one, with shapes attached to every symbol.

6.4 ⇝ Derive $A^\top A\,x = A^\top b$ from $f(x) = \|Ax - b\|^2$, expanding the square first. Then say what A and b are in linear regression, and what $\hat{x}$ becomes.

6.5 ⇝ Compute the Hessian of the least-squares J. What is it, in terms of X? Why does that single fact prove the bowl of Chapter 2 is convex?

6.6 ⇝ When is $(X^\top X)^{-1}$ illegal? Name two situations from the data, not from the algebra. What does Chapter 12 do about it?

6.7 ⇝ Redo the orthogonality argument for the concrete case $A = \left[\begin{smallmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{smallmatrix}\right]$, $b = (1, 3, 2)^\top$. Solve $A^\top A\hat{x} = A^\top b$, form the residual $r = b - A\hat{x}$ numerically, and verify $A^\top r = 0$ to rounding error. Then check that r is *not* zero — explain in one sentence why both facts are expected.

6.8 ⇝ Give a 3×2 matrix X whose columns are linearly dependent. Exhibit a nonzero v with $Xv = 0$, and use it to show $v^\top X^\top X v = 0$. What does that say about the shape of the cost surface in the direction v?

6.9 ⇝ Take $J(\theta) = \frac{1}{2}(\theta - 3)^2$. Write down H, then take one Newton step from $\theta^{(0)} = 100$. How many steps does gradient descent with $\alpha = 0.1$ need to get equally close? Explain the difference in one sentence using the word "curvature".

6.10 ⇝ For $n = 10^6$ examples and $d = 10^4$ features, estimate the cost of one gradient step and one Newton step in floating-point operations. State the ratio. This is the entire argument for first-order methods.

7

Batch and Stochastic Gradient Descent

The last chapter's update sums over every training row before it takes a step. That is honest and, on $n = 10^7$, unusable. The lectures split one choice — how many rows per step — into batch, stochastic, and mini-batch. The gradient formula does not change. The sum does.

7.1 Recap: the batch update uses all the data

The linear-regression update we derived sums over *every* training example:

$$\theta_j := \theta_j - \alpha \frac{1}{n} \sum_{i=1}^{n} \left(h_\theta(x^{(i)}) - y^{(i)}\right) x_j^{(i)}.$$

That $\sum_{i=1}^{n}$ means: to take *one* step, we look at all n points. That is **Batch** gradient descent.

7.2 Batch gradient descent

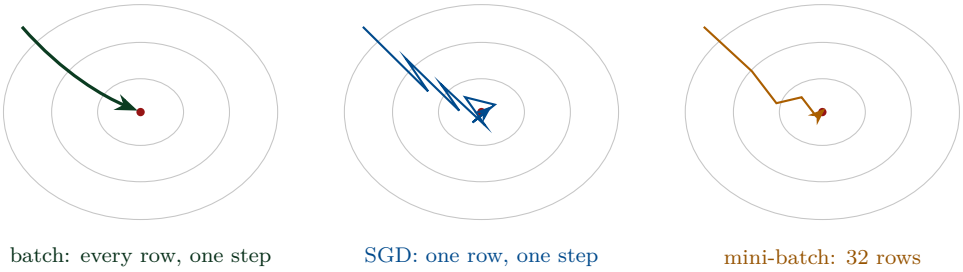
Batch GD uses the *whole* dataset to compute each step. Because it averages over everything, its steps are smooth and point reliably downhill. The price: if you have millions of examples, every single step is expensive — so it is *slow* on big data.

7.3 Stochastic gradient descent

Stochastic GD (SGD) goes to the opposite extreme: use *one* example at a time. Loop over the data and, for each example i, update immediately:

$$\theta_j := \theta_j + \alpha\left(y^{(i)} - h_\theta(x^{(i)})\right) x_j^{(i)}.$$

(This is the single-example version of the same rule — note $(y - h) = -(h - y)$, so the sign just flips the subtraction.) SGD is **faster** (a step costs one example, not n), but because each step trusts a single noisy point, it **may not settle exactly** at the minimum — it jitters around it.



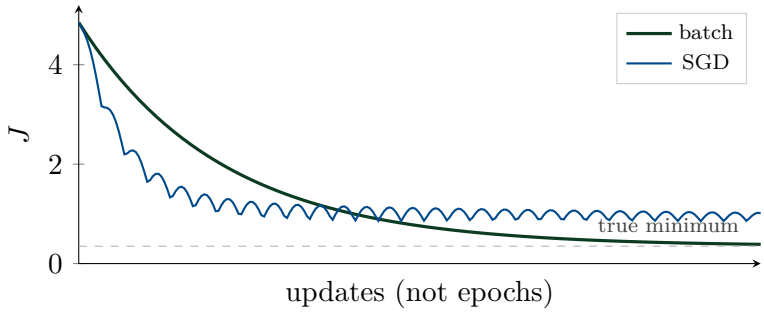
The same bowl, three step sizes of information. Batch averages all n residuals, so its direction is trustworthy and expensive. SGD trusts one row, so it is cheap and drunk — and near the minimum it never stops twitching, because a single row is never satisfied. Mini-batch buys most of the steadiness for a thirty-second of the cost.

The twitching at the end of the blue path is not a drawing flourish. It is the reason SGD needs either a decaying α or a tolerance you are willing to live with: the noise floor does not shrink just because you have arrived.

7.4 Batch vs Stochastic: the trade-off

	Batch GD	**Stochastic GD**
Data per step	all n examples	one example
Step direction	smooth, accurate	noisy
Speed per step	slow (esp. large n)	fast
Convergence	steady to the minimum	jitters near it

Plot the cost as the run proceeds and the trade-off stops being a table. Batch gives a curve you could publish. SGD gives a hairy band that descends much faster at the start and then refuses to flatten.



Cost against updates. Per update SGD is roughly n times cheaper, so on a wall clock the blue curve is far to the left of where it looks here. What it will not do is settle onto the dashed line.

In practice people use a **middle ground** — a small *mini-batch* (say 32 or 64 examples) per step — to get most of Batch's stability with most of SGD's speed. (The course notes formally define only Batch and Stochastic; mini-batch is the practical blend, and it is exactly what Lab 2 implements.)

In code

The only code difference is *how much data each update sees*. Same gradient, two loops:

```python
import numpy as np
def batch_gd(X, y, alpha=0.02, epochs=400):
    theta = np.zeros(X.shape[1]);  m = len(y)
    for _ in range(epochs):
        grad = (1/m) * (X.T @ (X @ theta - y))      # ALL examples per step
        theta = theta - alpha * grad
    return theta

def sgd(X, y, alpha=0.02, epochs=400):
    theta = np.zeros(X.shape[1])
    for _ in range(epochs):
        for i in np.random.permutation(len(y)):     # ONE example per step
            xi, yi = X[i:i+1], y[i:i+1]
            grad = xi.T @ (xi @ theta - yi)
            theta = theta - alpha * grad.ravel()
    return theta
# both recover the true line ~[1, 2] on data y = 2x + 1
```

Batch computes the gradient over the whole dataset before each step; SGD steps after every single example. A mini-batch version just loops over chunks (`X[s:s+bs]`) instead.

Deciding how to vote on a new restaurant: ***Batch*** *asks everyone in town before each decision — accurate but slow.* ***SGD*** *asks one random person and acts at once — fast but flighty.* ***Mini-batch*** *asks a small focus group — quick and reasonably reliable.*

Where this shows up

Why is the mini-batch size always 32, 64, 128, 256? Not because the mathematics prefers powers of two — the noise argument only cares about $\sqrt{b}$, which does not notice. It is hardware. A GPU executes threads in fixed-size groups (32 on NVIDIA parts), memory transactions are aligned to power-of-two boundaries, and matrix routines are tuned for tile sizes that divide evenly. A batch of 33 costs almost exactly what a batch of 64 costs, so nobody uses 33. The statistics chooses a range; the silicon chooses the number.

Go deeper

Read. Ruder, *An overview of gradient descent optimization algorithms*, §1–§2. Batch, mini-batch and stochastic set out side by side, then the whole family of methods that grew out of them. https://arxiv.org/abs/1609.04747

Read. Bottou, Curtis & Nocedal, *Optimization methods for large-scale machine learning*. Considerably harder, and the authoritative answer to "why does SGD work at all". https://arxiv.org/abs/1606.04838

Exercises

Pencil first, then click the ⇝ to jump to the worked answer in Appendix D. The answer links back to the question.

7.1 ⇝ Write the batch update and the SGD update for linear regression side by side. Account

for the sign flip $y - h = -(h - y)$ that the notes use on the SGD line.

7.2 ⇝ One pro and one con of each of batch and SGD. Then define mini-batch in one sentence, and say which of the two it inherits each property from.

7.3 ⇝ A data set has $n = 10^6$. How many gradient evaluations are in one batch step? In one SGD epoch? Which method can use a larger α, and why might you still prefer the noisier one?

7.4 ⇝ Mini-batch size 32 against 1 against n: sketch J versus iteration (not epoch) for all three on a convex bowl. No numbers needed — the shape of the noise is the answer.

7.5 ⇝ Run the two loops in the listing above on $y = 2x + 1$ with ten points. Print θ every ten epochs. Which one is still moving after it has essentially arrived, and what does that tell you about the stopping rules of Chapter 4?

Part III

Units, a tax on slopes

8

Vectorization and Feature Scaling

A $\sum$ over n is a loop. $X\theta$ is the same loop, handed to a library that has been beaten on for thirty years. Lecture 10 is that translation, then the reason a house's square-footage and its bedroom-count cannot share one α. Compact formulae, then the contour that looks like a ravine.

8.1 One example: from a sum to a dot product

The model for *one* observation is

$$h(x) = \sum_{j=0}^{d} \theta_j x_j.$$

Pack the knobs into a column and the features into a row that starts with a 1 (the fake feature that multiplies the intercept):

$$\theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_d \end{bmatrix}, \qquad x^\top = \begin{bmatrix} 1 & x_1 & \cdots & x_d \end{bmatrix}.$$

The sum is the inner product. The notes box it with a double red underline:

$$\boxed{h(x) = \theta^\top x}.$$

A single cashier adding $\theta_j x_j$ one spice at a time is the $\sum$. A whole counter of clerks hitting the same buttons at once is BLAS — the "optimized matrix libraries" on the slide. A farsan shop that weighs cumin, chilli, and salt together on one scale is $\theta^\top x$.

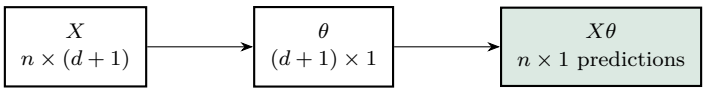
8.2 All n examples: the design matrix

For n rows the notes stack a one, then the d features, as an $n \times (d+1)$ matrix X. Row i is training example i; column 0 is all ones.

$$X = \begin{bmatrix} 1 & x_1^{(1)} & \cdots & x_d^{(1)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(i)} & \cdots & x_d^{(i)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(n)} & \cdots & x_d^{(n)} \end{bmatrix}_{n\times(d+1)}, \qquad \theta \in \mathbb{R}^{d+1}.$$

Every prediction at once:

$$\boxed{h_\theta(x) = X\theta} \in \mathbb{R}^n.$$



Dimensions have to kiss: $(n \times (d+1))$ *times* $((d+1) \times 1)$ *is* $(n \times 1)$*. If this fails, a feature was forgotten or the intercept column is missing.*

8.3 The cost, vectorized, then differentiated

The typed cost on the slide is the same J as Chapters 2–4:

$$J(\theta) = \frac{1}{2n} \sum_{i=1}^{n} \big(h_\theta(x^{(i)}) - y^{(i)}\big)^2.$$

Derivation

Replace h by $\theta^\top x^{(i)}$, then recognise a squared Euclidean norm (the notes' sidebar: $\|z\|_2^2 = z^\top z$):

$$J(\theta) = \frac{1}{2n} \sum_{i=1}^{n} (\theta^\top x^{(i)} - y^{(i)})^2 = \frac{1}{2n} (X\theta - y)^\top (X\theta - y).$$

Dimensions: $(X\theta - y)^\top$ is $1 \times n$ and $(X\theta - y)$ is $n \times 1$, so the product is a scalar. Expand the quadratic:

$$(X\theta - y)^\top (X\theta - y) = \theta^\top X^\top X\theta - (X\theta)^\top y - y^\top (X\theta) + y^\top y.$$

The two middle terms are equal scalars ($a^\top b = b^\top a$), and $y^\top y$ does not depend on θ so its gradient is 0. Rewrite $y^\top X\theta = (X^\top y)^\top \theta$ so both remaining pieces are in the Lecture 8 shapes $x^\top Ax$ and $b^\top x$:

$$\begin{aligned} \nabla_\theta J &= \frac{1}{2n} \nabla_\theta \big[\theta^\top (X^\top X)\theta - 2(X^\top y)^\top \theta\big] \\ &= \frac{1}{2n} \big[2X^\top X\theta - 2X^\top y\big] = \frac{1}{n} \big(X^\top X\theta - X^\top y\big). \end{aligned}$$

The two boxed identities used here are written in red in the corner of the closed-form slide:

$$\nabla_x(b^\top x) = b, \qquad \nabla_x(x^\top A x) = 2Ax \quad (A \text{ symmetric}).$$

Set the gradient to zero: $X^\top X\theta = X^\top y$, hence

$$\boxed{\hat{\theta} = (X^\top X)^{-1} X^\top y}.$$

8.4 Gradient descent versus the closed form

The comparison slide is a four-row scorecard. Red ticks and crosses in class:

Gradient descent	Closed form (normal equation)
Requires iterations	Non-iterative ✓
Need to choose α	No α ✓
Works well when n is large	Slower for large n (forming / inverting $X^\top X$) —
Supports incremental / streaming updates	$X^\top X$ may not even be invertible —

Three arrows off the "not invertible" box, copied from the red ink:

1. Use the **pseudoinverse**: `numpy.linalg.pinv` — Chapter 12.
2. **Remove a redundant feature** (two columns that are linear copies of each other make $X^\top X$ singular).
3. Keep $d \le n$ when you can: more features than rows is a classic way to lose full column rank.

In code

```
theta = np.linalg.solve(X.T @ X, X.T @ y)    # crashes if singular
theta = np.linalg.pinv(X) @ y                # Moore-Penrose fallback
theta = np.linalg.lstsq(X, y, rcond=None)[0]
```

8.5 Why feature scaling exists — the house example

The slide draws two contour plots. Left: elongated ellipses, GD bouncing off the walls. Right: circles, GD walking straight to the middle. The handwritten numbers next to it are a Ahmedabad-style house:

$$x_1 = \text{size in ft}^2 \in [0, 2000], \qquad x_2 = \#\text{bedrooms} \in [1, 5].$$

Derivation

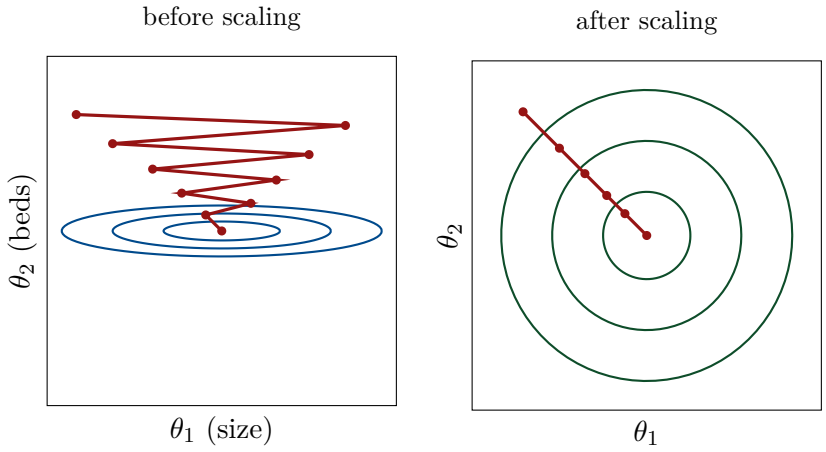
Drop the intercept for a moment, $h = \theta_1 x_1 + \theta_2 x_2$, and write the usual cost. The two partials are

$$\frac{\partial J}{\partial \theta_1} = \frac{1}{n}\sum_i (h^{(i)} - y^{(i)})\, x_1^{(i)}, \qquad \frac{\partial J}{\partial \theta_2} = \frac{1}{n}\sum_i (h^{(i)} - y^{(i)})\, x_2^{(i)}.$$

If a typical house has $x_1 \approx 1500$ and $x_2 \approx 3$, then *the same residual* $(h - y)$ is multiplied by 1500 in the θ_1 equation and by 3 in the θ_2 equation — a factor of about 500. The notes' conclusion, underlined:

- a small change in θ_1 makes a *large* change in $\theta_1 x_1$;
- the cost becomes extremely sensitive to θ_1.

That sensitivity is the long, thin ellipse. GD with one α cannot step comfortably in both directions, so it zig-zags.



Left: GD ricochets across a ravine (unscaled size vs. bedrooms). Right: the same α walks radially in. Slide takeaway: "Makes Gradient Descent converge faster."

8.6 The scaling formula, and the three rules on the last slide

For each feature $j = 1, \ldots, d$ (never the intercept column of ones),

$$x_j \leftarrow \frac{x_j - \mu_j}{\sigma_j},$$

where μ_j is the mean of x_j and σ_j is either the **standard deviation** (unit-variance / z-score, underlined in red) or the **range** (a cheap cousin of min–max). After this, the two partials have similar size, and "contours will become circular."

The last slide's three bullets, with the class underline:

- Rescales features to have zero mean and *unit variance*.
- Must apply the **same** scaling to training and testing: freeze μ_j, σ_j on train, reuse on val/test. Computing them on the whole dataset is leakage.
- Outliers can be an issue: one wild palace in the dataset drags μ and σ, squashing everyone else toward zero. Robust alternatives exist (median / IQR) but are not in this lecture.

The alternative: min–max scaling

The z-score is not the only way to make features comparable, and the lecture's parenthetical "or the range" is pointing at the other one. Min–max scaling squeezes each feature

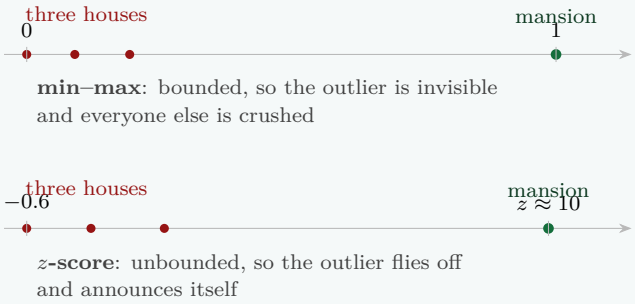
into a fixed interval, usually $[0, 1]$:

$$x_j \leftarrow \frac{x_j - \min_i x_j^{(i)}}{\max_i x_j^{(i)} - \min_i x_j^{(i)}}.$$

Work it on the houses. Areas $\{500, 1200, 2000\}$ square feet have min $= 500$, max $= 2000$, range 1500, so they become $\{0,\ 0.467,\ 1\}$. Bedrooms $\{1, 2, 4\}$ become $\{0,\ 0.333,\ 1\}$. Both features now live in $[0, 1]$, the two partial derivatives are comparable in size, and the bowl rounds out — which was the entire goal.

So which one? Be careful here, because the usual glib answer is wrong. Both formulas are *affine* — each is $x \mapsto (x - c)/s$ for constants c and s — so neither one can change the *relative* spacing of your points. If house B sits a third of the way between A and C before scaling, it sits a third of the way between them after, under either transform. Anybody who tells you min–max "distorts the distribution" and the z-score does not has not checked.

The real difference is what fixes the scale, and it shows up only when an outlier arrives. Add a single mansion of 50,000 square feet to the three houses. Under min–max the denominator is the range, which the mansion now owns single-handedly: it jumps from 1500 to 49,500, and the three ordinary houses land at 0, 0.014 and 0.030 — squeezed into a sliver at the bottom of the unit interval, while the mansion sits at exactly 1.0, looking like a perfectly ordinary maximum. Under the z-score the denominator is the standard deviation, which is an average over *all* the points, so it grows more slowly than the range does. The three houses keep a spread of about 0.07 in z units, and the mansion is thrown out to $z \approx 1.7$ with $n = 4$, or to $z \approx 10$ once there are a hundred ordinary houses diluting it.

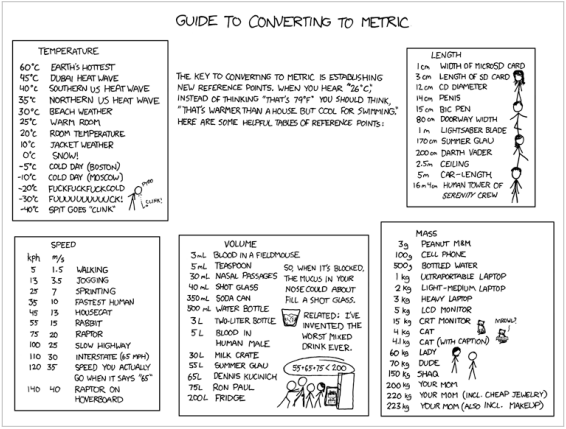


Same data, same relative spacing, different absolute scale. Min–max forces the outlier to be the boundary, which hides it and shrinks everybody else toward zero. The z-score lets it escape, which keeps the ordinary points at a usable magnitude and makes the outlier show up as a large z — something you can actually detect and act on.

Two practical consequences. Absolute magnitude matters because one learning rate and one λ are shared across all features, so a feature crushed to a range of 0.03 contributes almost nothing to the gradient and is effectively switched off. And a transform that hides outliers is worse than one that exposes them: $|z| > 4$ is a check you can run, whereas "this point is at 1.0" tells you nothing.

Use min–max when the bounds are real and meaningful — pixel intensities are genuinely in $[0, 255]$, a percentage is genuinely in $[0, 100]$ — or when a downstream step demands a bounded input. Use the z-score otherwise, which in this course is always. And when the

data really is contaminated, neither is right: use the median and the interquartile range, whose breakdown point is high enough that one bad row cannot move them at all. Both obey the same non-negotiable rule below: whichever numbers you compute, compute them on train and never recompute them.



xkcd #526 "Converting to Metric" by Randall Munroe — https://xkcd.com/526/ (CC BY-NC 2.5).

Scaling is a unit change. The physics does not change; the numbers become comparable. Apply the same conversion to the test set or you have built a model in metres and evaluated it in feet.

Where this shows up

Leakage through the scaler is one of the most common silent bugs in applied work, and it is silent precisely because it makes your numbers look *better*. A pipeline that calls `fit_transform` on the full dataset before splitting will report a validation score that is optimistic by a small, plausible-looking margin — exactly the size of margin nobody investigates. The fix is structural rather than vigilant: fit the scaler inside the cross-validation fold, so the arithmetic makes the mistake impossible instead of merely discouraged.

Go deeper

Read. Stanford CS229, main notes, "Normal equations" section. Same vectorised derivation, different notation — read it as a spot-check that your $\nabla J = \frac{1}{n} X^\top (X\theta - y)$ is right. https://cs229.stanford.edu/main_notes.pdf

Watch. Tübingen ML (von Luxburg), calculus track, *Mathematics for Machine Learning*

Run the numbers yourself: `notebooks/theory/ch_scaling.py` checks that mismatched units blow up the condition number, that standardising collapses it, and that one α can diverge on raw features while converging on scaled ones.

Exercises

Pencil first, then click the ⇝ to jump to the worked answer in Appendix D. The answer links back to the question.

8.1 ⇝ Write the vectorized model and cost using X and θ. Then derive the normal equation from that J, quoting $\nabla(x^\top A x) = 2Ax$ and $\nabla(b^\top x) = b$ and saying where the $y^\top y$ term went.

8.2 ⇝ Name three legal responses when $X^\top X$ is singular. Which one becomes Chapter 12?

8.3 ⇝ A house has area in $[0, 2000]$ and bedrooms in $\{1, 2, 3, 4\}$. Explain, using the factor x_j that sits inside $\partial J/\partial\theta_j$, why one α cannot serve both. What shape does the bowl become?

8.4 ⇝ Which two numbers do you freeze on the training set, and what are you forbidden to do with the test set?

8.5 ⇝ Show, with a single leaked row, what goes wrong when μ and σ are computed on train and test together. Make the damage numerical, not verbal.

8.6 ⇝ Apply min–max scaling by hand to the feature values $\{2, 5, 9, 14\}$. Then apply the z-score (you may leave σ as a surd). Which transform puts the four points at more nearly equal spacing, and why is that not automatically a good thing?

8.7 ⇝ Take the same four values and append a fifth, $x = 200$. Recompute both transforms. Report what happens to the image of the original four points under each, to two decimal places, and state the general lesson in one sentence.

8.8 ⇝ Why is the column of ones excluded from scaling? Say what would literally happen to that column if you applied the z-score formula to it, and what that would do to $X^\top X$.

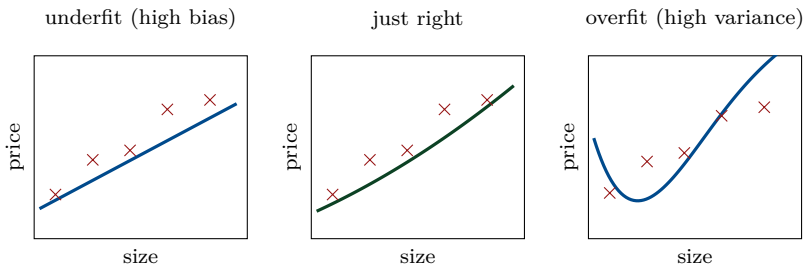
9

Regularization, Shrinkage, and the Ridge Normal Equation

Lecture 11 is the course's answer to a model that got too clever. We keep every feature — including the silly high powers — but we make the extra knobs expensive. That expense is λ. This chapter follows the annotated slides line by line: underfitting vs. overfitting, the $\lambda_3, \lambda_4 = 1000$ trick, the ridge objective (and the two λ conventions in the notes), what happens if λ is insane, the GD shrinkage factor $(1 - \alpha\lambda/n)$, and the regularized normal equation.

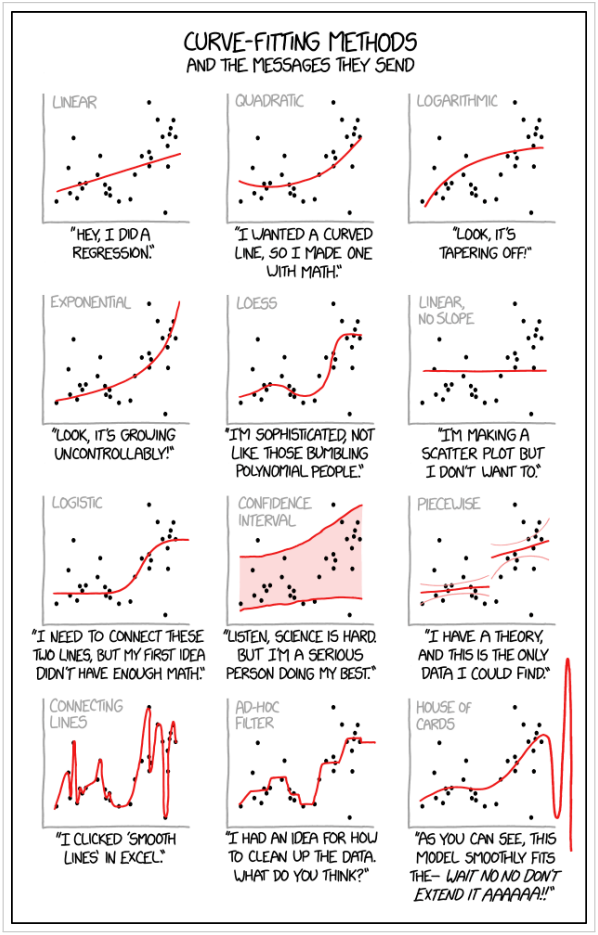
9.1 Fitting quality: three pictures

The slide "Fitting Quality" draws price against size with five data marks.



The three hypotheses from class: $h = \theta_0 + \theta_1 x$ (straight, misses the bend); quadratic (tracks the trend); degree-4 polynomial (forced through every mark, including a nonsense valley).

Underfitting: the hypothesis does not fit well — $J(\theta)$ stays large. **Overfitting:** $J(\theta)$ on the *training* marks is tiny, but the curve fails to generalise to a new house. It likes to happen when there are lots of features and a small amount of data. The red ink lists a kitchen-sink of house features: size, bedrooms, floor number, age, kitchen size, … plus polynomial extras $\theta_0 + \theta_1 x + \theta_2 x^2$ versus a cubic.



xkcd #2048 "Curve-Fitting" by Randall Munroe — https://xkcd.com/2048/ (CC BY-NC 2.5).

Overfitting in one comic: a model so flexible it will fit a dragon into a scatter plot. Regularization is how we tell the dragon to sit down.

9.2 Two ways to address overfitting

The slide "Addressing Overfitting" lists:

1. **Reduce the number of features** — by hand, or by a model-selection loop (keep / drop).
2. **Regularization** — keep all the features, but shrink the parameters.

The typed 4th-degree hypothesis has $\theta_3 x^3$ and $\theta_4 x^4$ underlined in red: those are the "too wiggly" extras. The handwritten regularizer attacks *exactly those two knobs*:

$$\min_{\theta} \frac{1}{2n} \sum_{i=1}^{n} \big(h_\theta(x^{(i)}) - y^{(i)}\big)^2 + \lambda_3 \theta_3^2 + \lambda_4 \theta_4^2, \qquad \lambda_3 = \lambda_4 = 1000.$$

With λ that large, the only way to keep the penalty from exploding is $\theta_3 \approx 0$, $\theta_4 \approx 0$, and the hypothesis collapses back to the quadratic $\theta_0 + \theta_1 x + \theta_2 x^2$. Regularization did feature selection without deleting columns.

> A method for automatically controlling the complexity of the learned hypothesis. It is incorporated into the cost function. It keeps all the features but reduces the magnitude of the

parameters. It works well when we have a lot of features, each contributing a bit to the label. Small parameter values give a simpler hypothesis and one less prone to overfitting.

9.3 Ridge regression: the objective

Once we tax *every* slope, not just θ_3 and θ_4, we get ridge regression. The typed formula on the slide is

$$J(\theta) = \frac{1}{2n}\sum_{i=1}^{n}\left(h_\theta(x^{(i)}) - y^{(i)}\right)^2 + \frac{\lambda}{2}\sum_{j=1}^{d}\theta_j^2. \tag{9.1}$$

Red brackets in class split this into a "Data" term and a "Regularization term," and a note identifies the second sum as $\|\theta\|_2^2$ — more precisely $\|\theta_{1:d}\|_2^2$, because the sum starts at $j = 1$. Four bullets:

- $\lambda \geq 0$ is the regularization parameter.
- **No regularization for θ_0.** The intercept is "the price of a mean-sized house," not a slope; taxing it would punish the average, not the wiggliness.
- $\sum_{j=1}^{d}\theta_j^2 = \|\theta_{1:d}\|_2^2$ is the magnitude of the parameter vector (minus the intercept).
- Equivalently $\|\theta_{1:d} - 0\|_2^2$: we are **pulling every slope toward zero**.

Two λ conventions live in the same lecture — do not mix them

The *typed* slide uses (9.1): the penalty is $(\lambda/2)\|\theta_{1:d}\|^2$, with *no* $1/n$. The *handwritten* cost on the GD slide wraps λ inside the $1/(2n)$:

$$J(\theta) = \frac{1}{2n}\left[\sum_{i=1}^{n}(h^{(i)} - y^{(i)})^2 + \lambda\sum_{j=1}^{d}\theta_j^2\right]. \tag{9.2}$$

Lab 3 of this course uses (9.2). The printed GD update on the same slide,

$$\theta_j \;:=\; \theta_j - \alpha\frac{1}{n}\sum_{i}(h^{(i)} - y^{(i)})x_j^{(i)} - \alpha\frac{\lambda}{n}\theta_j \quad (j > 0),$$

is the gradient of (9.2), not of (9.1). If you mix the J of one convention with the update of the other, your λ is off by a factor of n. Always state which J you differentiated.

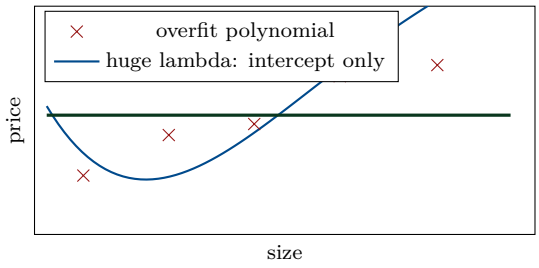
Ridge is the uncle at a farsan counter who will not let last year's giant wedding order dominate this year's recipe. Every fancy coefficient is nudged toward zero unless the data really insists. The fact that people eat something (the intercept θ_0) is never nudged. λ is how strict the uncle is. $\lambda = 0$ is "no uncle." $\lambda \to \infty$ is "only serve the house average, ignore size."

9.4 What if λ is insane?

The slide asks it in those words. Red ink: $\theta_1 \approx \theta_2 \approx \theta_3 \approx \theta_4 \approx 0$, because we penalise the parameters very highly, and the hypothesis collapses to

$$h_\theta(x) = \theta_0,$$

a horizontal line — underfitting of the most boring kind. The picture is a wiggly blue polynomial with a flat red line drawn through it labelled $h_\theta(x) = \theta_0$.



Too much regularization is underfitting in a tuxedo. The model is so terrified of slopes that it predicts the same price for a hut and a haveli.

9.5 Ridge gradient descent, and why it is called shrinkage

We work with the handwritten J (9.2) so the printed update matches. Split θ_0 from the rest because $x_0^{(i)} = 1$ always, and because θ_0 is not taxed.

Derivation

For $j \geq 1$,

$$\frac{\partial J}{\partial \theta_j} = \frac{1}{n}\sum_{i=1}^{n}\left(\sum_{k=0}^{d}\theta_k x_k^{(i)} - y^{(i)}\right)x_j^{(i)} + \frac{\lambda}{n}\theta_j.$$

(The extra $(\lambda/n)\theta_j$ is $\partial/\partial\theta_j$ of $(\lambda/(2n))\theta_j^2$.) Subtract α times that gradient:

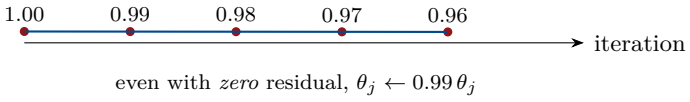
$$\theta_0 := \theta_0 - \alpha\frac{1}{n}\sum_{i=1}^{n}(h^{(i)} - y^{(i)})\underbrace{x_0^{(i)}}_{=1},$$

$$\theta_j := \theta_j - \alpha\frac{1}{n}\sum_{i=1}^{n}(h^{(i)} - y^{(i)})x_j^{(i)} - \alpha\frac{\lambda}{n}\theta_j \qquad (j > 0).$$

Rearrange the $j > 0$ line by collecting θ_j:

$$\theta_j \;:=\; \theta_j\left(1 - \alpha\frac{\lambda}{n}\right) - \frac{\alpha}{n}\sum_{i=1}^{n}(h^{(i)} - y^{(i)})x_j^{(i)}.$$

The notes name $C = 1 - \alpha\lambda/n$. For $\alpha > 0$, $\lambda > 0$, $n > 0$ we have $C < 1$ (they write $C \approx 0.99$ as a typical size, and $C \approx 0.9$ on the Lecture 12 recap). So each iteration multiplies θ_j by 0.99 *before* the data even speaks. That is **weight decay** / **shrinkage**: θ_j is made smaller than it used to be, every single step.



The data term can fight the shrink. The shrink never takes a day off. That is the whole point of λ.

Loshchilov & Hutter (arXiv:1711.05101) noticed that adaptive optimizers *break* this $(1-\alpha\lambda/n)$ factor if you shove $\lambda\theta$ into the same gradient as the data. True weight decay is "multiply the weights, then take the data step" — exactly the class rearrangement. The 2025–2026 optimizer wave (Muon and friends) still argues about how to decay matrix-shaped weights. The lecture already owns the 1-D version.

9.6 The regularized normal equation

The last slide of Lecture 11 types

$$\theta = (X^\top X + \lambda I)^{-1} X^\top y$$

and then derives it from the *un-normalized* SSE objective

$$\min_\theta \ \|y - X\theta\|_2^2 + \lambda\|\theta\|_2^2.$$

Derivation

Differentiate and set to zero. Using $\nabla_x\|Ax-b\|^2 = 2A^\top(Ax-b)$ and $\nabla_x\|\theta\|^2 = 2\theta$:

$$-2X^\top(y - X\theta) + 2\lambda\theta = 0 \qquad \Longrightarrow \qquad -X^\top y + X^\top X\theta + \lambda\theta = 0.$$

Factor θ. The $\lambda\theta$ term needs an identity so it can sit next to $X^\top X$:

$$(X^\top X + \lambda I)\theta = X^\top y \qquad \Longrightarrow \qquad \boxed{\hat\theta = (X^\top X + \lambda I)^{-1} X^\top y}.$$

Watch out

The I in that formula *does* tax θ_0 unless you replace it by $R = \mathrm{diag}(0, 1, \ldots, 1)$, which is what Lab 3 does and what the "no regularization for θ_0" bullet demands. The slide writes I because the algebra is cleaner; in code, mask the intercept. Also, this J is SSE, not $\frac{1}{2n}$SSE, so this λ is not the λ in (9.2) — they differ by a factor involving n. Same geometry, different numeric knob.

Why add λI? If $X^\top X$ has a zero eigenvalue (duplicate features, $d > n$), it is singular and you cannot invert. Adding λI lifts every eigenvalue by λ (Hastie 2020). For $\lambda > 0$ the matrix is positive definite, hence always invertible. Ridge is both a statistical shrinker and a numerical seatbelt. The $\lambda \to 0^+$ limit of this formula *is* the Moore–Penrose pseudoinverse of the next chapter.

In code

```
n, d1 = X.shape           # d1 includes intercept
R = np.diag(np.r_[0.0, np.ones(d1-1)])
# Lab-3 J = 1/(2n) (||Xth-y||^2 + lam ||th[1:]||^2)
th = np.linalg.solve(X.T @ X + lam * R, X.T @ y)
# GD shrinkage form, j>=1:
```

```
# th[j] = th[j]*(1 - alpha*lam/n) - (alpha/n) * (X[:,j] * (X@th - y)
    ).sum()
```

9.7 Choosing λ: the bias–variance trade-off

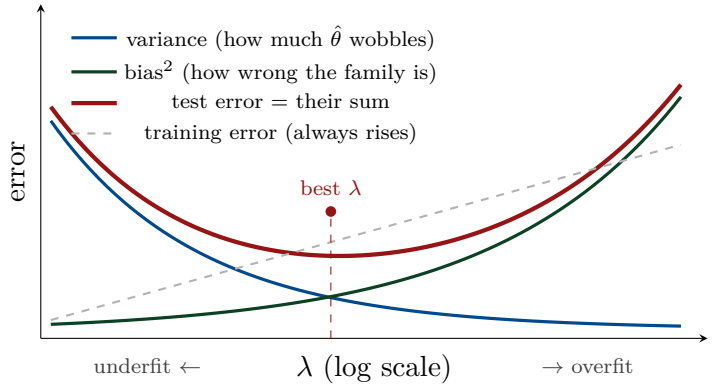
Everything so far says what λ *does*. It does not say what λ should *be*, and the honest answer — "try several and look" — deserves a picture, because the shape of that picture is always the same and knowing it saves you from misreading your own experiments.

Split the error you care about, the error on data you have never touched, into two parts.

Bias is how wrong you would be on average even with unlimited data, because the model you chose cannot represent the truth. A straight line fitting a curve has bias no amount of data will cure. Cranking λ up increases bias, because you are forbidding the model from using weights it might legitimately need.

Variance is how much your fitted $\hat{\theta}$ jumps around when you re-collect the training set. A model with many free parameters and few examples will chase whatever noise it happens to see, so two samples from the same source give two very different models. Cranking λ up decreases variance, because the penalty anchors the weights near zero regardless of what the noise says.

These two move in opposite directions, which is what makes it a trade-off rather than an optimisation.



The only graph you need to remember about λ. Test error is U-shaped because it is the sum of one falling curve and one rising curve. Training error, the dashed grey line, is monotone: it gets worse for every increase in λ, with no minimum anywhere. That is the whole reason you cannot pick λ by looking at training error — it would always tell you $\lambda = 0$.

Watch out

Read the axes carefully, because this figure is routinely misremembered. Note that the left end is not "good". At $\lambda = 0$ the training error is at its minimum and the test error is *high*: that is overfitting, and it looks like success from inside the training loop. Note also that the U is usually asymmetric and often shallow near the bottom, so "the best λ" is really "any λ in this decade", and reporting it to three significant figures is false precision.

The procedure, in full. Hold out a validation set that is neither trained on nor used for the final report. For λ on a logarithmic grid — $10^{-4}, 10^{-3}, \ldots, 10^{2}$, because the interesting behaviour spans orders of magnitude and a linear grid wastes almost all its points — fit on train, score on validation, and keep the λ with the lowest validation error. Then refit on train plus validation using that λ, and report on the test set once.

Where this shows up

Two failure modes account for most bad λ choices in practice. The first is picking λ on the test set and then reporting the test score, which turns the test set into a second training set and makes the reported number meaningless in a way nobody can detect from the outside. The second is searching a linear grid such as $\lambda \in \{0, 0.1, 0.2, \ldots, 1\}$; if the right answer is 10^{-3}, every point you tried was effectively zero and you will conclude that regularisation does not help.

Go deeper

Read. van Wieringen, *Lecture notes on ridge regression*, §1.5–1.6. The bias–variance decomposition of the ridge estimator done algebraically rather than by picture — including the striking result that there always exists a $\lambda > 0$ with strictly lower mean squared error than OLS. `https://arxiv.org/abs/1509.09169`

Read. Hastie, *Ridge regularization: an essential concept in data science*. Short, modern, and covers what happens when $d > n$, which is where the classical U-curve picture starts to break. `https://arxiv.org/abs/2006.00371`

Watch. Ali Ghodsi (Waterloo), Deep Learning Lec 2.1, *Regularization, and why the penalty is a Gaussian prior*

Check the shrinkage numerically with `notebooks/theory/ch_ridge_path.py`: it verifies that $\lambda = 0$ is exactly OLS, that $\|w\|$ falls monotonically as λ rises, and that the R-mask leaves θ_0 untouched.

Exercises

Pencil first, then click the ⇝ to jump to the worked answer in Appendix D. The answer links back to the question.

9.1 ⇝ Write the ridge J used in Lab 3, with θ_0 un-taxed, and the matching gradient-descent update including the factor $1 - \alpha\lambda/n$.

9.2 ⇝ Suppose you want to punish only θ_3 and θ_4, as the lecture slide does. What do you add to J, and what happens to those two knobs when their λ is 1000?

9.3 ⇝ Why is this called *shrinkage*? Point at the exact factor in the update and say why it is less than one.

9.4 ⇝ What happens to h as $\lambda \to \infty$? Draw the three-panel under / just-right / over-fit picture from memory before checking.

9.5 ⇝ Write the regularized normal equation. What does adding λI do to the eigenvalues of $X^\top X$, and why does that fix the singular case? Why must you mask row and column 0 when a formula writes I out of laziness?

9.6 ⇝ The notes carry two J conventions. Pick the Lab 3 one and translate a λ from the other convention so that gradient descent takes an identical step. Show the arithmetic, not a rule of thumb.

9.7 ⇝ Explain in two sentences why training error is monotone increasing in λ while test error is U-shaped. Your answer must use the words "feasible set" or "constraint" at least once.

9.8 ⇝ You search $\lambda \in \{0, 0.25, 0.5, 0.75, 1\}$ and find that validation error is flat and $\lambda = 0$ wins. Give two distinct explanations, and say which grid you should have searched instead.

9.9 ⇝ $X^\top X$ has eigenvalues $\{9, 4, 10^{-8}\}$. Compute the condition number before and after adding λI with $\lambda = 0.01$. Comment on what this does to the numerical stability of the solve, quite apart from any statistical benefit.

10

From a Hard Constraint to Lambda

Lecture 11 introduces ridge as an extra term in J. Caltech's *Learning From Data* tells the same algebra as a *story*, which we add here without replacing any class formula. Both λ conventions from the notes stay in Chapter 9; this chapter is the "why a tax exists" picture.

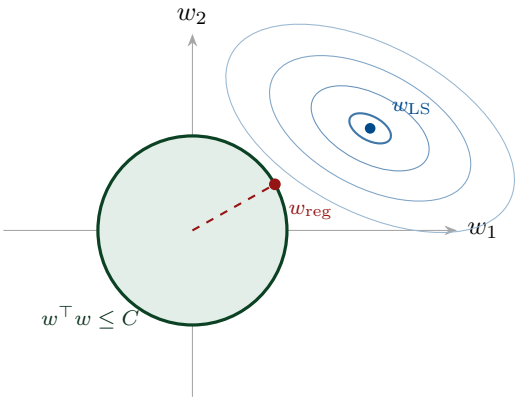
10.1 Overfitting as a budget

Suppose the features have been expanded — powers of x, or whatever Lecture 11 drew on the board. Unconstrained least squares is then free to hand you enormous coefficients that cancel each other out to thread every training point. Nothing in J forbids $w_3 = +8400$ next to $w_4 = -8399$.

So forbid it. Put a fence around how long w is allowed to be:

$$\min_w \|Zw - y\|_2^2 \qquad \text{subject to} \qquad w^\top w \;\le\; C.$$

C is a budget on squared length. The feasible set is a ball of radius $\sqrt{C}$ centred at the origin. Inside the ball nothing has changed; you are doing ordinary least squares. The interesting case is when the unconstrained optimum w_{LS} lies *outside* the ball. Then the best you can do is the point of the ball that sits on the lowest contour of J — and that point is where a contour is exactly tangent to the ball.



The whole of ridge regression in one picture. Blue ellipses are level sets of $\|Zw - y\|^2$, centred on the unconstrained answer. The green disc is the budget. The solution is the tangency point: the cheapest contour that still touches the ball. Shrink C and the red dot slides toward the origin along a curve — that curve is the ridge path.

Tangency is the entire content of the Lagrange condition below. At a tangency point the two surfaces share a normal direction, so $\nabla\|Zw-y\|^2$ must be parallel to $\nabla(w^\top w)$. Write "parallel" as "equal after scaling by some $\lambda \geq 0$" and you have already done the algebra.

10.2 The fence becomes a tax

A Lagrange multiplier $\lambda \geq 0$ turns the constraint into an unconstrained problem (the same move as eigenvalues-as-optimisation in Chapter 11):

$$\min_{w} \;\; \|Zw-y\|_2^2 + \lambda\big(w^\top w - C\big).$$

C is constant in w, so dropping it does not change the minimiser. Completing the square / setting the gradient to zero gives

$$\big(Z^\top Z + \lambda I\big)w \;=\; Z^\top y, \qquad w_{\text{reg}} \;=\; (Z^\top Z + \lambda I)^{-1} Z^\top y,$$

which is the regularized normal equation from class. Gradient descent on the augmented error is **weight decay**: each step multiplies w by something like $1-\alpha\lambda$ before adding the data gradient — the class factor $1-\alpha\lambda/n$ in Chapter 9 is that decay with the course's $1/n$ convention.

λ is not a mystical knob. It is the shadow price of the length budget C. Tiny λ (huge C): the fence is far away, you overfit. Huge λ (tiny C): you are crushed toward $w=0$ (or toward θ_0 only, once the intercept is exempted as class does). Cross-validation picks C without ever writing C — it picks λ instead.

Derivation

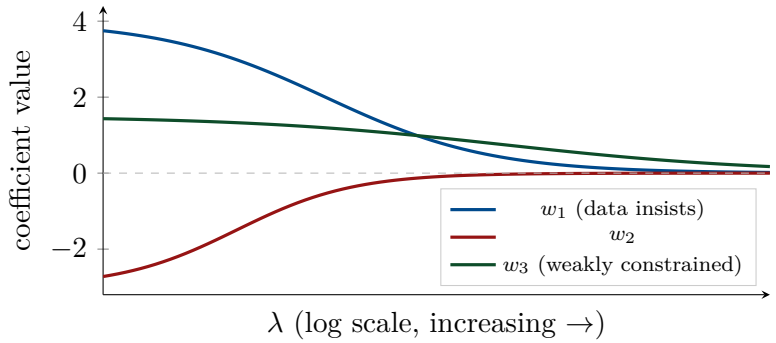
$\nabla_w\big(\|Zw-y\|^2 + \lambda\|w\|^2\big) = 2Z^\top(Zw-y) + 2\lambda w$. Set to zero: $Z^\top Z\,w + \lambda w = Z^\top y$. Factor $(Z^\top Z + \lambda I)w = Z^\top y$. If the notes use $J = \frac{1}{2n}(\|X\theta - y\|^2 + \lambda\|\theta_{1:d}\|^2)$, the same stationarity condition is $(X^\top X + \lambda R)\theta = X^\top y$ with $R = \text{diag}(0,1,\ldots,1)$ — that R is Chapter 27 and Chapter 9, kept.

Watch out

Class writes two J conventions (with and without certain n factors). This book does *not* throw either away: both are derived in Chapter 9. Pick one J when you *code* (Lab 3 uses the $\frac{1}{2n}$ form with un-regularised θ_0) and know how to translate.

10.3 The path, drawn

Sweep λ from 0 upward and every coefficient traces a curve. At $\lambda = 0$ you are at w_{LS}, wherever that is. As λ grows, each w_j is pulled toward zero, but not at the same rate: directions in which the data is informative (large singular values of Z) resist, and directions the data barely constrains collapse almost immediately. That is the same eigenvalue story as Chapter 11, seen from the side.



A ridge path. Every coefficient decays toward zero as the tax rises, but at its own rate. Cross-validation is the act of stopping somewhere along this x-axis — which is the same as choosing the budget C without ever writing C down.

In code

Computing that path is one solve inside a loop. This is also the fastest way to convince yourself that $\lambda = 0$ really is ordinary least squares:

```
import numpy as np

rng = np.random.default_rng(0)
n, d = 40, 3
Z = rng.normal(size=(n, d))
Z[:, 2] = Z[:, 1] + 0.01 * rng.normal(size=n)   # nearly redundant
    column
w_true = np.array([2.0, -1.0, 0.5])
y = Z @ w_true + 0.1 * rng.normal(size=n)

def ridge(lam):
    return np.linalg.solve(Z.T @ Z + lam * np.eye(d), Z.T @ y)

print(ridge(0.0))                          # ordinary least squares
print(ridge(10.0))                         # pulled toward zero
assert np.allclose(ridge(0.0), np.linalg.lstsq(Z, y, rcond=None)[0])
assert np.linalg.norm(ridge(10.0)) < np.linalg.norm(ridge(0.0))
```

The second assert is the budget, verified: raising λ shortens w. Run `notebooks/theory/ch_ridge_path.py` for the full sweep.

Go deeper

Watch. Yaser Abu-Mostafa, Caltech CS156, *Lecture 12, Regularization*
The lecture this chapter is built on. Abu-Mostafa derives the constraint-to-penalty equivalence on the board and then — the part worth staying for — argues about what λ *means*, which almost nobody else does.

Watch. Ali Ghodsi, University of Waterloo, *Lec 2.1, Regularization*
The same material with the constraint set treated as the primary object and the penalty as a consequence — the reverse of Abu-Mostafa's emphasis. Watch both and you will have seen the ball-and-contour picture drawn by two people who disagree about which half of it matters.

Read. van Wieringen, *Lecture notes on ridge regression*. Eighty pages on one estimator, and the best single source on why the eigenvalue shift $\lambda_i \to \lambda_i + \lambda$ is the whole story. https://arxiv.org/abs/1509.09169

Exercises

Pencil first, then click the ⇝ to jump to the worked answer in Appendix D. The answer links back to the question.

10.1 ⇝ Start from $\min\|Zw-y\|^2$ s.t. $w^\top w \leq C$. Introduce λ and arrive at $(Z^\top Z+\lambda I)w = Z^\top y$.

10.2 ⇝ In Lab 3, θ_0 is not taxed. Where does that appear in the Lagrange story — as a coordinate that is not inside C, or as something else?

Part IV

When matrices misbehave, and why least squares

11

Gradient of a Determinant, and Eigenvalues as Optimization

This is Lecture 9, rewritten from scratch so every symbol is unpacked. It sits here, after ridge and before the pseudoinverse, because the next chapters need $\nabla \log |A|$ (Gaussians) and eigenvalues (SVD stretches). Why does a machine-learning course suddenly differentiate a *determinant*? Because the multivariate Gaussian — the probabilistic model behind least squares, MLE, and MAP — has $\log |A|$ in its density. Differentiating that log-det is how you learn a covariance. The eigenvalue story on the same lecture is the other half: "the most stretched direction of a bowl is an eigenvector." Both are the same skill: turn a matrix identity into a gradient.

11.1 Minors, cofactors, adjugate — the vocabulary

Take a square matrix $A \in \mathbb{R}^{n \times n}$. Delete row i and column j; the leftover $(n-1) \times (n-1)$ determinant is the **minor** $M_{ij} = |A_{i,j}|$. Slap on a checkerboard sign and you have the **cofactor**

$$C_{ij} \;=\; (-1)^{i+j}\, M_{ij}.$$

Stack the cofactors into a matrix $C = [C_{ij}]$. The **adjugate** (also called the classical adjoint) is just that matrix, transposed:

$$\operatorname{adj}(A) \;=\; C^{\top}.$$

Laplace expansion says you can rebuild $|A|$ from *any* one column j:

$$|A| \;=\; \sum_{i=1}^{n} (-1)^{i+j} A_{ij}\, |A_{i,j}| \;=\; \sum_{i=1}^{n} A_{ij}\, C_{ij}.$$

11.2 Gradient of the determinant

Derivation

We want $\nabla_A |A|$, a matrix the same shape as A, whose (k, ℓ) entry is $\partial |A| / \partial A_{k\ell}$. Expand $|A|$ along column ℓ (the notes use j, then specialise to the entry we are differentiating):

$$|A| \;=\; \sum_{i=1}^{n} (-1)^{i+\ell} A_{i\ell}\, |A_{i,\ell}|.$$

The minor $|A_{i,\ell}|$ does *not* contain the entry $A_{k\ell}$ — we deleted that row and column to make it. So the only term in the sum that knows about $A_{k\ell}$ is the $i = k$ term:

$$\frac{\partial |A|}{\partial A_{k\ell}} \;=\; (-1)^{k+\ell}\, |A_{k,\ell}| \;=\; C_{k\ell} \;=\; \big(\mathrm{adj}(A)\big)_{\ell k}.$$

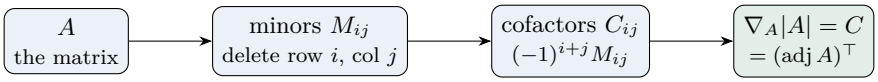
That last step is "adjugate = cofactor transpose", so the (ℓ, k) entry of $\mathrm{adj}(A)$ is $C_{k\ell}$. Therefore

$$\boxed{\nabla_A |A| \;=\; \big(\mathrm{adj}(A)\big)^{\top}}.$$

When A is invertible we also have the identity proved in the next section, $\mathrm{adj}(A) = |A| A^{-1}$, and so

$$\boxed{\nabla_A |A| \;=\; |A|\, (A^{-1})^{\top}}.$$

If A is symmetric, $(A^{-1})^{\top} = A^{-1}$, which is the extra note on the lecture's last pages.



Differentiating a determinant is "read off the cofactor of that entry." The gradient matrix is the cofactor matrix.

11.3 Why $A \cdot \mathrm{adj}(A) = |A| I$

This identity is the notes' long argument, and it is the reason we may replace $\mathrm{adj}(A)$ by $|A| A^{-1}$. Let $P = A \cdot \mathrm{adj}(A)$. The (i, j) entry is a row-of-A times a column-of-adjugate, which is a row of cofactors of *row* j:

$$P_{ij} \;=\; \sum_{k=1}^{n} a_{ik}\, C_{jk}.$$

Derivation

Diagonal, $i = j$. Then $P_{ii} = \sum_k a_{ik} C_{ik}$, which is Laplace expansion of $|A|$ along row i. So $P_{ii} = |A|$.

Off-diagonal, $i \neq j$. The same sum $\sum_k a_{ik} C_{jk}$ is the Laplace expansion of a *different* matrix: take A and replace row j by a copy of row i. That fake matrix has two identical rows, so its determinant is 0. Hence $P_{ij} = 0$.

Therefore P is the diagonal matrix with $|A|$ on every diagonal entry:

$$A \cdot \operatorname{adj}(A) = \begin{bmatrix} |A| & 0 & \cdots & 0 \\ 0 & |A| & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & |A| \end{bmatrix} = |A|\, I.$$

If $|A| \neq 0$ we may multiply both sides by A^{-1} and read $\operatorname{adj}(A) = |A|A^{-1}$.

The notes' 3×3 warning:

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 4 & 5 & 6 \end{bmatrix}$$

has two identical rows, so its determinant is 0. That is exactly why the off-diagonal argument works: the "wrong-row" expansion is the determinant of a cloned-row matrix, which is always zero.

The 2×2 check (green ink in class)

Derivation

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}, \qquad \operatorname{adj}(A) = \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}.$$

Multiply:

$$A \cdot \operatorname{adj}(A) = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix} = \begin{bmatrix} ad - bc & -ab + ab \\ cd - cd & -bc + ad \end{bmatrix} = \begin{bmatrix} |A| & 0 \\ 0 & |A| \end{bmatrix} = |A|I.$$

Every cancellation is a pair of identical terms with opposite sign. This is the whole identity, visible without n-dimensional courage.

11.4 Gradient of $\log|A|$

This is the formula that shows up in Gaussian MLE.

Derivation

Let $f(A) = \log|A|$. Chain rule, one entry at a time:

$$\frac{\partial \log|A|}{\partial A_{ij}} = \frac{1}{|A|} \cdot \frac{\partial |A|}{\partial A_{ij}} = \frac{1}{|A|} \cdot \left(|A|(A^{-1})^\top\right)_{ij} = \left((A^{-1})^\top\right)_{ij}.$$

So

$$\boxed{\nabla_A \log|A| = (A^{-1})^\top},$$

and if A is symmetric (a covariance always is), $\nabla_A \log|A| = A^{-1}$.

$\frac{d}{dx}\log x = 1/x$. *The matrix version is* $\nabla_A \log|A| = A^{-\top}$. *The determinant plays the role of "the size of A"; the inverse plays the role of "one over A." Same slogan, extra transpose because the world of matrices is not commutative.*

Watch out

The Matrix Cookbook (Petersen & Pedersen) writes many of these identities with the *denominator layout* vs *numerator layout* convention, which flips transposes. The class convention is: gradient has the *same shape* as the variable, $(\nabla_A f)_{ij} = \partial f/\partial A_{ij}$. Stick to that and you will match the notes, CS229, and Murphy's Gaussian notes.

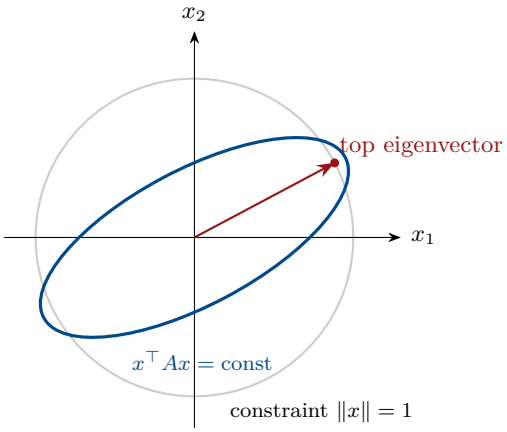
11.5 Eigenvalues as an optimization problem

This is the other half of Lecture 9, and it is the secret geometry of PCA, of the ridge ellipse, and of "which direction of the cost bowl is steepest."

Take a symmetric matrix A (so all eigenvalues are real, and eigenvectors can be chosen orthonormal). Among all unit-length vectors, which x makes the number $x^\top Ax$ as large as possible? The notes set this up as

$$\max_{x\in\mathbb{R}^n} x^\top Ax \qquad \text{subject to} \qquad \|x\|_2^2 = 1.$$

$x^\top Ax$ is the **objective**; $\|x\|_2^2 = 1$ is the **constraint** (stay on the unit sphere, otherwise you could just scale x to infinity).



The unit circle is the allowed set. Level sets of $x^\top Ax$ *are ellipses. The biggest value on the circle is where an ellipse just kisses it — and that contact point is an eigenvector.*

Derivation

Build the Lagrangian (a single number that folds the constraint into the objective with a multiplier λ):

$$\mathcal{L}(x,\lambda) \;=\; x^\top Ax \;-\; \lambda\big(x^\top x - 1\big).$$

Differentiate with respect to x, using $\nabla_x(x^\top Ax) = 2Ax$ (symmetric A, Chapter 6) and $\nabla_x(x^\top x) = 2x$:

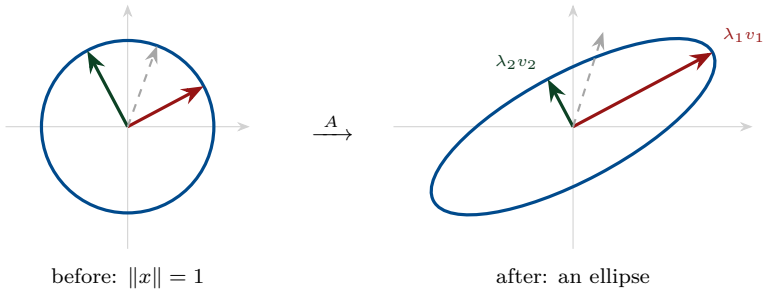
$$\nabla_x\mathcal{L} \;=\; 2Ax - 2\lambda x \;\longrightarrow\; 0 \qquad\Longrightarrow\qquad \boxed{Ax = \lambda x}.$$

The only stationary points of "$x^\top Ax$ on the unit sphere" are the eigenvectors of A. Plug $Ax = \lambda x$ back into the objective:

$$x^\top Ax = x^\top(\lambda x) = \lambda(x^\top x) = \lambda.$$

So the largest value of $x^\top Ax$ on $\|x\| = 1$ is the **largest eigenvalue**, attained at the corresponding unit eigenvector. The smallest value is the smallest eigenvalue. The notes' sentence: *the only points which can possibly maximise or minimise $x^\top Ax$ assuming $\|x\|_2^2 = 1$ are the eigenvectors of A.*

There is a second way to see the same fact, and it is the one worth carrying around. Instead of asking which x maximises a number, watch what A does to the whole unit circle. It stretches it into an ellipse. The eigenvectors are the axes of that ellipse, and each eigenvalue is the stretch factor along its own axis. A vector that is *not* an eigenvector gets rotated as well as stretched; an eigenvector is precisely a direction that comes out pointing the way it went in.



A turns the circle into an ellipse. The red and green arrows keep their direction and only change length — those are the eigenvectors, and the lengths they gain are λ_1 and λ_2. The dashed grey arrow, which started between them, comes out tilted: not an eigenvector. The longest radius of the ellipse is the largest value of $x^\top Ax$ on the circle, which is what the Lagrangian just proved.

The ratio $\lambda_{\max}/\lambda_{\min}$ is how elongated that ellipse is. It has a name, the *condition number*, and you have already suffered from it: it is exactly the ravine that makes unscaled gradient descent crawl in Chapter 8. A circle has condition number 1 and one learning rate works everywhere. A cigar has a large one, and the α that is safe along the thin axis is uselessly small along the long one.

The Hessian of a quadratic cost is a symmetric matrix. Its eigenvectors are the axes of the elliptical contours; its eigenvalues are how steep those axes are. Gradient descent with step α multiplies error along axis j by $(1 - \alpha\lambda_j)$ — that is Chapter 5's factor $(1 - 2\alpha)$, now in every direction. Feature scaling tries to equalise the λ_j. Ridge adds λ to every eigenvalue of $X^\top X$ (Hastie 2020, arXiv:2006.00371). PCA *is* this optimisation: first principal component = top eigenvector of the covariance. One Lagrangian, four chapters.

A wedding-season farsan shop has a round table of snacks (the unit sphere) and a profit function that likes some directions of the table more than others (the quadratic $x^\top Ax$). The most profitable seat at the table is not found by tasting every seat — it is an eigenvector. The Lagrange multiplier λ is the "price of staying seated on the table" rather than wandering off into the courtyard (where $\|x\|$ could grow without bound).

11.6 A small matrix-calculus toolkit (rest of Lec 9)

The notes close with five identities used constantly in least squares and in "functions of a matrix that spit out one number." We derive each in the class layout: $f : \mathbb{R}^{m\times n} \to \mathbb{R}$.

Derivation

(i) $f(X) = \operatorname{tr}(A^\top X) = \langle A, X\rangle$ (Frobenius inner product). Differentiating one entry: $\partial/\partial X_{ij}$ hits only the matching term A_{ij}, so

$$\frac{\partial f}{\partial X} = A.$$

This is the matrix version of $\frac{d}{dx}(a^\top x) = a$.
(ii) $f(x) = \operatorname{tr}(x^\top A x)$. For a vector x this is just the scalar $x^\top A x$, whose gradient is $(A + A^\top)x$ (no symmetry assumed). If A is symmetric this collapses to the $2Ax$ we already love.
(iii) $f(x) = \frac{1}{2}\|Ax-B\|_F^2$. The Frobenius norm is "$\sqrt{\sum_{i,j} a_{ij}^2}$" — Euclidean norm after you flatten the matrix. Expanding $\frac{1}{2}\|R\|_F^2 = \frac{1}{2}\operatorname{tr}(R^\top R)$ with $R = Ax - B$ and differentiating gives

$$\frac{\partial f}{\partial x} = A^\top(Ax - B),$$

the matrix least-squares gradient. (Vector least squares is the special case $B = b$.)
(iv) $f(x) = \frac{1}{2}\operatorname{tr}(B^\top x^\top x B)$. The product xB is being Frobenius-squared. Gradient:

$$\frac{\partial f}{\partial x} = x\,BB^\top.$$

(v) $f(x) = \frac{1}{2}\operatorname{tr}(B^\top x^\top A x B)$, the sandwich of (ii) and (iv):

$$\frac{\partial f}{\partial x} = \frac{1}{2}(A + A^\top)\,x\,BB^\top.$$

f	$\partial f/\partial x$	scalar cousin
$\operatorname{tr}(A^\top X)$	A	$a^\top x$
$\operatorname{tr}(x^\top A x)$	$(A + A^\top)x$	ax^2
$\frac{1}{2}\|Ax - B\|_F^2$	$A^\top(Ax - B)$	$\frac{1}{2}(ax - b)^2$
$\frac{1}{2}\operatorname{tr}(B^\top x^\top x B)$	$xBB^\top$	$\frac{1}{2}x^2b^2$
$\frac{1}{2}\operatorname{tr}(B^\top x^\top A x B)$	$\frac{1}{2}(A + A^\top)xBB^\top$	$\frac{1}{2}ax^2b^2$

In code

Check (i) and the log-det formula with a finite difference — Lab 2 energy, Lecture 9 identities:

```
import numpy as np
A = np.array([[2., 0.3],[0.3, 1.5]])
# d/dA log|A| should be A^{-T}
def logdet(M):
    return np.log(np.linalg.det(M))
H = 1e-6
num = np.zeros_like(A)
```

```python
for i in range(2):
    for j in range(2):
        E = np.zeros_like(A); E[i,j] = H
        num[i,j] = (logdet(A+E) - logdet(A)) / H
print(num)
print(np.linalg.inv(A).T)
```

Another route: the two-by-two shortcut

For a 2×2 matrix you never need the characteristic polynomial. Two facts do all the work, and both fall out of expanding $\det(A - \lambda I) = 0$ once and for all:

$$\lambda_1 + \lambda_2 = \operatorname{tr} A = a_{11} + a_{22}, \qquad \lambda_1 \lambda_2 = \det A.$$

Write $m = \frac{1}{2}(\lambda_1 + \lambda_2)$ for the mean of the two eigenvalues and $p = \lambda_1 \lambda_2$ for their product. The two numbers are then symmetric about m, so they are $m \pm$ something, and that something is pinned down by the product:

$$(m + s)(m - s) = m^2 - s^2 = p \quad \Longrightarrow \quad s = \sqrt{m^2 - p}, \qquad \boxed{\lambda = m \pm \sqrt{m^2 - p}}$$

Try it on the matrix from earlier in this chapter, $A = \left[\begin{smallmatrix} 3 & 1 \\ 1 & 3 \end{smallmatrix}\right]$. Trace is 6, so $m = 3$. Determinant is $9 - 1 = 8$, so $p = 8$. Then $s = \sqrt{9 - 8} = 1$ and the eigenvalues are $3 \pm 1 = \{4, 2\}$ — the same answer the polynomial gave, in about four seconds of arithmetic. This is worth internalising because in this course almost every matrix whose eigenvalues you care about is 2×2: the Hessian $(1/n)X^\top X$ of a two-feature problem, the curvature matrix whose $\lambda_{\max}$ sets your step-size bound in Chapter 5. The trick is due in this presentation to 3Blue1Brown; the underlying identities are as old as the subject.

Go deeper

Watch. 3Blue1Brown, *Eigenvectors and eigenvalues (Essence of Linear Algebra, Ch. 14)*
The one video to watch if you have watched none: eigenvectors as the directions a transformation does not rotate.

Watch. 3Blue1Brown, *A quick trick for computing eigenvalues*
The $m \pm \sqrt{m^2 - p}$ shortcut above, derived on screen.

Watch. Gilbert Strang, MIT 18.06, *Eigenvalues and eigenvectors*
The classical lecture. Strang's instinct is always to ask what the matrix *does*, which is the right instinct.

Read. Petersen & Pedersen, *The Matrix Cookbook*, §1 and §A. Not a text to learn from — a table to look things up in. Keep it open when you differentiate a matrix expression. https://www.math.uwaterloo.ca/~hwolkowi/matrixcookbook.pdf

Exercises

Pencil first, then click the ⇝ to jump to the worked answer in Appendix D. The answer links back to the question.

11.1 ⇝ Write $\nabla_A |A|$ in terms of the adjugate, and again for invertible A. Verify it on a 2×2 by finite difference using the listing above.

11.2 ⇝ Prove $A\,\mathrm{adj}(A) = |A|I$ in two sentences: one for the diagonal entries, one for the off-diagonal entries.

11.3 ⇝ What is $\nabla_A \log|A|$? What does it simplify to when A is symmetric, and why does that case matter for a covariance matrix?

11.4 ⇝ Set up "find the largest eigenvalue" as a constrained optimisation problem. Write the Lagrangian and show that a critical point satisfies $Ax = \lambda x$.

11.5 ⇝ Define the Frobenius norm. Why is it just the Euclidean length of a flattened matrix?

11.6 ⇝ Why does a Gaussian density contain $\log|A|$ at all? One paragraph, then compare with the likelihood you wrote in Chapter 3.

11.7 ⇝ Use $\lambda = m \pm \sqrt{m^2 - p}$ on $A = \left[\begin{smallmatrix} 2 & 1 \\ 1 & 2 \end{smallmatrix}\right]$ and on $B = \left[\begin{smallmatrix} 4 & 2 \\ 2 & 1 \end{smallmatrix}\right]$. For each, state $\lambda_{\max}$, then quote the largest step size gradient descent could survive on a cost with that Hessian (see Chapter 5). One of the two is singular — say which and say what that means for the normal equations.

12

The Pseudoinverse and SVD

Lecture 12 opens by recapping the ridge chapter's two formulae — the GD shrink $C = 1 - \alpha\lambda/n$ and the regularized normal equation — and then asks the question those formulae were dodging: *what if we refuse to add λI, and $X^\top X$ is still singular?* The answer is the Moore–Penrose **pseudoinverse**, built from the **singular value decomposition**. Barata & Hussein's tutorial (arXiv:1110.6882) is the standard deep dive; Strang's four-subspaces picture is the geometry; this chapter is the class version, with every annotation kept.

12.1 When the ordinary inverse is illegal

The ordinary least-squares formula

$$\hat{\theta} = (X^\top X)^{-1} X^\top y$$

needs $X^\top X$ invertible. If a column of X is a copy of another (the notes' later example: second column = 2× first column), then $X^\top X$ has a zero eigenvalue and there is no inverse. The red box in class groups $(X^\top X)^{-1} X^\top$ and names it

$$X^\dagger \qquad \text{(Moore–Penrose pseudoinverse).}$$

Then $\hat{\theta} = X^\dagger y$ still makes sense.

An inverse undoes a square, full-rank map. A pseudoinverse undoes as much of a rectangular or rank-deficient map as can honestly be undone, and refuses to invent information in the leftover directions. In those leftover directions it returns 0 — the shortest θ that still fits as well as possible.

12.2 SVD: $X = U\Sigma V^\top$

Any real matrix $X \in \mathbb{R}^{n \times d}$ factors as

$$X = U\Sigma V^\top,$$

where

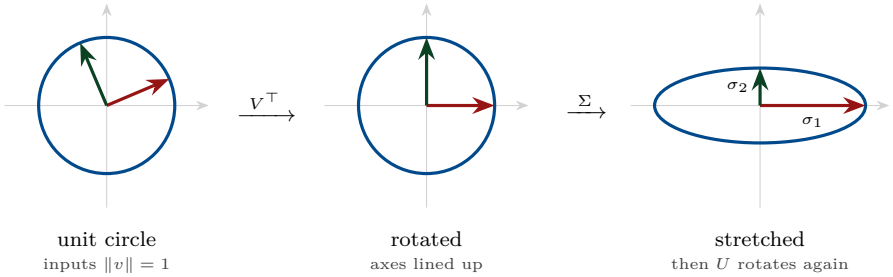
- U has orthonormal columns, the **left singular vectors** (eigenvectors of $XX^\top$);
- V has orthonormal columns, the **right singular vectors** (eigenvectors of $X^\top X$);

- Σ is rectangular-diagonal, holding the **singular values** $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0$ on the diagonal, and zeros everywhere else. $r = \text{rank}(X)$.

The notes' link to eigen-analysis (Lecture 9) is the identity

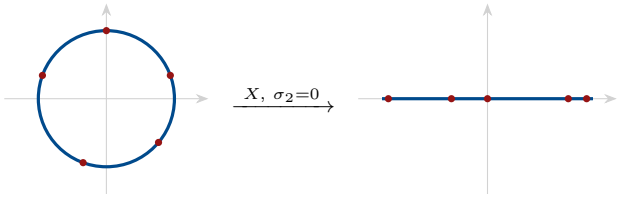
$$\sigma_i = \sqrt{\lambda_i(X^\top X)} = \sqrt{\lambda_i(XX^\top)}.$$

That is the algebra. The geometry is better. Every matrix, no matter how ugly, does exactly three things in sequence: it rotates, it stretches along perpendicular axes, and it rotates again. Feed a unit circle in and an ellipse comes out. The lengths of that ellipse's semi-axes are the singular values.



What every matrix does, in three moves. $V^\top$ turns the input so the special directions line up with the axes, Σ stretches each axis by σ_i, and U turns the result into place. The singular values are the semi-axes of the output ellipse — how much X magnifies each perpendicular direction.

Now the failure case has a shape. If $\sigma_2 = 0$, the ellipse has no thickness: it has collapsed to a line segment. An entire direction of input space was multiplied by zero and mapped onto the origin. No matrix can undo that, because everything along that direction arrived at the same place. Asking for X^{-1} is asking which of infinitely many inputs produced it.



A zero singular value is information destroyed. The circle becomes a segment, and the two red dots at the top and bottom land on the same output. This is exactly the state in which $X^\top X$ has a zero eigenvalue and refuses to be inverted. The pseudoinverse does not pretend otherwise — it simply declines to guess along the crushed direction.

12.3 Inverting Σ the honest way

The notes' numerical Σ is already singular:

$$\Sigma = \begin{bmatrix} 5 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

You cannot write $1/0$. The pseudoinverse of a diagonal matrix inverts the *non-zero* entries and leaves the zeros alone:

$$\sigma_i \leftarrow \frac{1}{\sigma_i}\ (\sigma_i \neq 0), \qquad 0 \leftarrow 0.$$

So

$$\Sigma^{+} = \begin{bmatrix} 1/5 & 0 & 0 \\ 0 & 1/2 & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

For the full matrix the notes write

$$\boxed{X^{\dagger} \;=\; V\,\Sigma^{+}\,U^{\top}}, \qquad \hat{\theta} = X^{\dagger} y.$$

(If X is square and invertible this collapses to X^{-1}, because $\Sigma^{+} = \Sigma^{-1}$ and U, V are rotations.)

Derivation

Why this θ solves least squares even when $X^{\top}X$ is singular. Write $X = U\Sigma V^{\top}$. Then

$$X^{\top}X = V\Sigma^{\top}\Sigma V^{\top},$$

whose eigenvalues are σ_i^2 (and zeros). The ordinary inverse would want $1/\sigma_i^2$; the pseudoinverse uses $1/\sigma_i^2$ when $\sigma_i \neq 0$ and 0 when $\sigma_i = 0$. Multiplying through gives $\hat{\theta} = V\Sigma^{+}U^{\top}y$, which is the minimum-norm minimizer of $\|X\theta - y\|_2^2$. Among all θ that achieve the same residual, this one is the shortest — it puts zero weight on the null-space directions.

Another route: two shortcut formulas, and when each applies

The SVD always works, and it is what `numpy.linalg.pinv` actually runs. But in the two situations you will meet most often, the pseudoinverse has a closed form you can write down without decomposing anything — and one of them you already know.

The tall case: more examples than features. If X is $n \times m$ with $n \geq m$ and the columns are independent, then $X^{\top}X$ is $m \times m$ and genuinely invertible, and

$$X^{\dagger} \;=\; (X^{\top}X)^{-1}X^{\top}.$$

Check one Moore–Penrose condition to see it is not a coincidence:

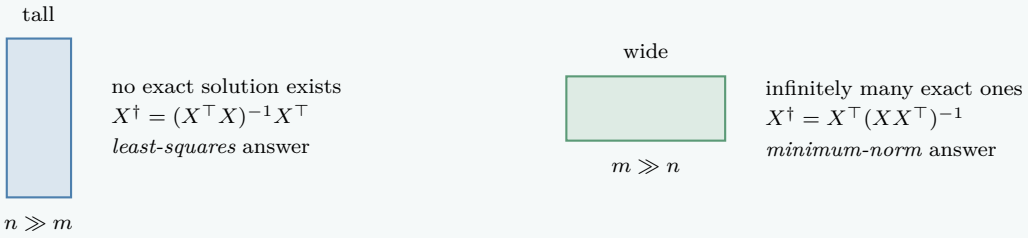
$$XX^{\dagger}X = X(X^{\top}X)^{-1}X^{\top}X = X\,I = X.\checkmark$$

Now look at what $\hat{\theta} = X^{\dagger}y$ becomes: $(X^{\top}X)^{-1}X^{\top}y$. That is the normal equation. The pseudoinverse is not a new solution to least squares — it is the old one, with the assumption of invertibility removed.

The wide case: more features than examples. If $n \leq m$ and the *rows* are independent, then it is $XX^{\top}$ that is invertible, and

$$X^{\dagger} \;=\; X^{\top}(XX^{\top})^{-1}.$$

Here $X\theta = y$ has infinitely many exact solutions — you have more knobs than constraints, so you can hit every training point perfectly in infinitely many ways — and $\hat{\theta} = X^{\dagger}y$ picks the one with the smallest $\|\theta\|$.



The same symbol $X^\dagger$ does two different jobs depending on the shape of the problem, and neither job is "inverting". Tall: you cannot satisfy every equation, so it returns the closest miss. Wide: you can satisfy all of them many ways, so it returns the smallest solution. When X is square and invertible both formulas collapse to X^{-1}.

Why the SVD form is still the one you should trust. Both shortcuts assume full rank. Duplicate a feature column, or measure the same quantity in feet and metres, and *neither* $X^\top X$ nor $XX^\top$ is invertible, so both formulas evaporate while $V\Sigma^+U^\top$ carries on working. Worse, near-dependence is not caught: if two columns are almost identical, $(X^\top X)^{-1}$ exists but is enormous, and computing it squares your condition number — $\kappa(X^\top X) = \kappa(X)^2$ — so you lose twice as many digits as you needed to. This is why numerical libraries never form $X^\top X$, and why `np.linalg.lstsq` beats `np.linalg.inv(X.T @ X) @ X.T @ y` on exactly the problems where it matters.

Go deeper

Watch. Steve Brunton, *Singular Value Decomposition: mathematical overview*
Later videos in the same playlist do least squares and the pseudoinverse directly, with code.

Read. Brunton & Kutz, *Data-Driven Science and Engineering*, §1.4. "Pseudo-inverse, least-squares, and regression" — the written version of the argument above, with the tall and wide cases worked separately. `http://databookuw.com/databook.pdf`

Read. Barata & Hussein, Moore–Penrose tutorial. If you want the four conditions proved rather than checked. `https://arxiv.org/abs/1110.6882`

Verify the claims numerically: `notebooks/theory/ch_svd_pinv.py` confirms the four Moore–Penrose conditions, that a zero singular value crushes a direction, and that $X^\dagger y$ really is minimum-norm on a rank-deficient problem.

12.4 A rank-1 matrix you can see

The notes' example:

$$X = \begin{bmatrix} 1 & 2 \\ 2 & 4 \\ 3 & 6 \end{bmatrix}.$$

Column two is exactly twice column one, so $\operatorname{rank}(X) = 1$. $X^\top X$ is 2×2 of rank 1, hence singular: the ordinary inverse does not exist. `np.linalg.pinv(X)` still returns a 2×3 matrix, and $\hat{\theta} = X^\dagger y$ splits the credit between the two identical directions in the shortest way (typically θ_2 takes a share of what θ_1 could have done alone).

In code

```python
X = np.array([[1.,2.],[2.,4.],[3.,6.]])
print(np.linalg.matrix_rank(X))          # 1
# np.linalg.inv(X.T @ X)                 # LinAlgError
print(np.linalg.pinv(X))                 # 2 x 3
U, S, Vt = np.linalg.svd(X, full_matrices=False)
S_plus = np.array([1/s if s > 1e-12 else 0.0 for s in S])
X_dag = Vt.T @ np.diag(S_plus) @ U.T
print(np.allclose(X_dag, np.linalg.pinv(X)))
```

$(X^\top X + \lambda I)^{-1} X^\top$ replaces $1/\sigma_i^2$ by $1/(\sigma_i^2 + \lambda)$, or equivalently shrinks singular values by $\sigma_i^2/(\sigma_i^2 + \lambda)$ (Hastie 2020; van Wieringen arXiv:1509.09169). Sending $\lambda \to 0^+$ recovers $X^\dagger$ on the non-zero σ_i and still 0 on the exact zeros. Ridge is Tikhonov regularization; the pseudoinverse is Tikhonov in the limit. Mathematics of Data Science (Strawn et al., arXiv:2607.11938, 2026) rebuilds this SVD story as the spine of modern data science — you already met it in FOML.

A stepwell (vav) has stairs that go down and stairs that go nowhere because a wall is in the way. SVD finds the stairs that actually descend ($\sigma_i > 0$) and the stairs that hit a wall ($\sigma_i = 0$). The pseudoinverse walks down the real stairs and refuses to invent a tunnel through the wall. Ridge says "pretend there is a little slope even on the wall, so nobody slips."

Exercises

Pencil first, then click the ⇝ to jump to the worked answer in Appendix D. The answer links back to the question.

12.1 ⇝ Write $X^\dagger$ from $X = U\Sigma V^\top$, stating exactly what you do to each singular value. Why is $1/0$ not a legal move, and what does putting a zero back there choose for you?

12.2 ⇝ Take the rank-1 X from this chapter. Compute $X^\dagger$ by hand, then check it against `np.linalg.pinv`. Confirm $XX^\dagger X = X$.

Part V

Likelihoods, priors, and bending the line

13

Probability From Zero, and Two Ways to Use It

The next three chapters are the *probabilistic* half of the course: maximum likelihood, priors, MAP, smoothing, and the fully Bayesian view. Every one of them rests on a handful of probability ideas — and if nobody ever taught you those cleanly, the algebra looks like magic. So we start from zero. No measure theory, no jargon. Just enough probability to read every formula in the coming chapters, and the one philosophical fork — *frequentist* versus *Bayesian* — that decides which formula you even write down.

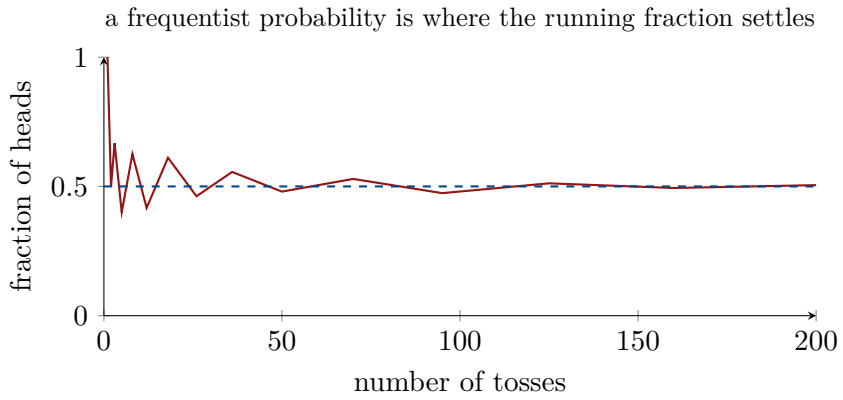
You do not need a statistics background to follow this. A "probability" is a number between 0 and 1 saying how likely something is. That is the whole starting point. We build likelihood, priors, and Bayes' rule on top of it, one small step at a time, with a coin in hand the entire way.

13.1 What a probability even is (two honest answers)

Ask "what does $P(\text{heads}) = 0.5$ mean?" and you get two different — both correct — answers. The split runs through this entire half of the course, so meet it now.

> **Frequentist:** a probability is a *long-run frequency*. $P(\text{heads}) = 0.5$ means: flip the coin a huge number of times and the *fraction* that come up heads settles to 0.5. It is a fact about repeatable experiments.
>
> **Bayesian:** a probability is a *degree of belief*. $P(\text{heads}) = 0.5$ means: given what I know, I am exactly as willing to bet on heads as on tails. It can describe a one-off event ("$P(\text{it rains tomorrow}) = 0.3$") where "long run" makes no sense.

Both obey the identical rules — every probability sits in $[0, 1]$, and the probabilities of all outcomes sum to 1 — so the arithmetic never changes. What changes is *what you are allowed to put a probability on*. That single permission is the whole frequentist–Bayesian fork, and we will cash it out in the last section.



Toss a fair coin again and again and the running fraction of heads is wild at first, then homes in on 0.5*. The frequentist calls that limiting fraction "the probability." The Bayesian is happy to call* 0.5 *a belief even before the first toss.*

13.2 Random variable, distribution, parameter

Three words appear in every formula ahead. Plain meanings:

- A **random variable** is a number whose value is uncertain until you look — the result of one coin toss $y \in \{0, 1\}$ (write Head = 1, Tail = 0), or tomorrow's temperature.
- A **distribution** is the list of how much probability each value gets. For one coin toss the whole distribution is two numbers: $P(y{=}1) = \theta$ and $P(y{=}0) = 1 - \theta$.
- A **parameter** is the knob that pins down *which* distribution you mean. For the coin the parameter is θ, the probability of heads. Learning = finding the parameter from data.

The coin's one-toss distribution has a name, the **Bernoulli**, and a one-line formula that packs both cases into one expression:

$$p(y \mid \theta) = \theta^{\,y}(1-\theta)^{\,1-y} \qquad \Rightarrow \qquad \begin{cases} y = 1: & \theta^1(1-\theta)^0 = \theta, \\ y = 0: & \theta^0(1-\theta)^1 = 1-\theta. \end{cases}$$

That trick — an exponent that switches a factor on or off — is how a single formula covers both outcomes, and it is the seed of the whole likelihood machine.

Do not let $\theta^y(1-\theta)^{1-y}$ scare you. It is just "θ if heads, $1-\theta$ if tails" written so you can multiply many tosses together without a pile of if-statements.

13.3 Expectation: the average you can compute before rolling

The **expectation** (or mean) of a random variable is each value times its probability, added up:

$$\mathbb{E}[X] = \sum_{x} x\, P(X{=}x).$$

For a single coin toss, $\mathbb{E}[y] = 1 \cdot \theta + 0 \cdot (1-\theta) = \theta$. The average of a 0/1 variable is exactly the probability of a 1 — which is why "fraction of heads" and "estimate of θ" are the same thing.

Expectation is the balance point of the distribution: the spot where the bars would balance on a finger.

Think of a thali. Each dish is a possible outcome; how much of your plate it fills is its probability. The expectation is the "average bite" — what you would get if you closed your eyes and scooped in proportion to how much of each dish is on the plate.

13.4 Independence and the product rule (the engine of likelihood)

Two events are **independent** if one tells you nothing about the other — separate coin tosses, separate dice. For independent events, probabilities **multiply**:

$$P(A \text{ and } B) = P(A)\,P(B).$$

So the probability of a whole sequence of independent tosses is the *product* of the per-toss probabilities. Three heads in a row from a θ-coin:

$$P(\text{H,H,H}) = \theta \cdot \theta \cdot \theta = \theta^3.$$

When every observation is drawn the same way and independently, we say the data is **i.i.d.** (independent and identically distributed). That assumption is what lets us turn a data set into one product:

$$p(y_1, \ldots, y_n \mid \theta) = \prod_{i=1}^{n} p(y_i \mid \theta).$$

That product, read as a function of θ with the data fixed, is the **likelihood** — the object the entire next chapter maximises. You have just built its engine.

Watch out

"Independent" is an *assumption*, not a law. Consecutive words in a sentence are not independent; neither are today's and tomorrow's stock prices. When the product rule is used, i.i.d. is being *assumed* — and every result inherits that assumption. Half of "why did my model fail" is a broken independence assumption.

13.5 A tiny worked example: a likelihood by hand

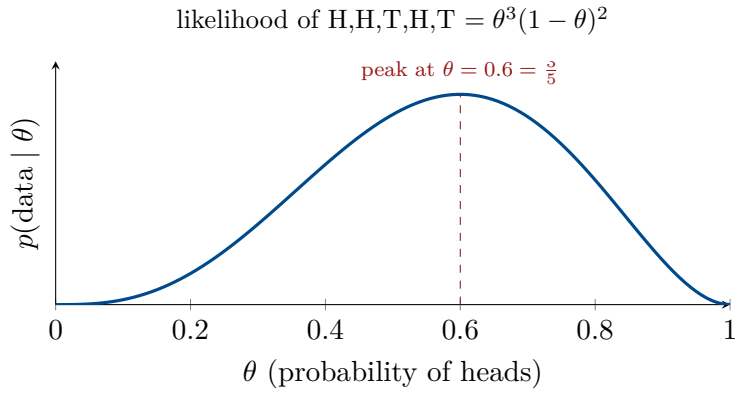
Suppose you toss a coin 5 times and see **H, H, T, H, T** — so $s = 3$ heads, $n = 5$. Under a θ-coin the probability of *exactly this sequence* is

$$p(\text{data} \mid \theta) = \theta \cdot \theta \cdot (1-\theta) \cdot \theta \cdot (1-\theta) = \theta^3(1-\theta)^2.$$

Now read it as a function of θ and try a few values:

$$\theta = 0.3:\ (0.027)(0.49) = 0.0132, \qquad \theta = 0.6:\ (0.216)(0.16) = 0.0346, \qquad \theta = 0.9:\ (0.729)(0.01) = 0.0073.$$

Of these three, $\theta = 0.6$ makes the data you actually saw the *most probable*. That is the whole idea of maximum likelihood in one line — and $0.6 = 3/5 = s/n$ is exactly the answer the next chapter derives with calculus.



Freeze the data, sweep θ: the likelihood is a hump that peaks at the fraction of heads. Maximum likelihood is just "stand at the top of this hump."

13.6 The fork: frequentist vs Bayesian learning

Now the payoff. We want to learn θ from data, and the two readings of probability give two different recipes.

Frequentist. θ is a *fixed unknown number*; only the data is random. There is no "distribution of θ" — it just has one true value. You estimate it by asking which θ makes the observed data most likely. This is **Maximum Likelihood** (MLE), and its notation is $P(x;\theta)$ — "x *parameterised by* θ", the semicolon marking θ as a knob, not a random variable.

Bayesian. θ is *itself uncertain*, so it gets its own distribution: a **prior** $p(\theta)$ before data, updated by the data into a **posterior** $p(\theta \mid x)$. Its notation is $P(x \mid \theta)$ — a genuine conditional bar, because θ is a bona-fide random variable. From the posterior you can take its peak (**MAP**) or average over all of it (**fully Bayesian**).

Frequentist	**Bayesian**
θ is one fixed number data is random	θ is uncertain give it a distribution
estimate by *max likelihood* $P(x;\theta)$	prior × likelihood → posterior $P(\theta \mid x)$
one best $\hat{\theta}$	*peak = MAP, average = fully Bayesian*

The fork the rest of this half of the course walks down. Left branch: one lecture (maximum likelihood). Right branch: two (MAP, then integrating over the whole posterior).

A frequentist weather forecaster says "in towns like this, it rained on 30% of such days." A Bayesian says "I am 30% sure it rains here, tomorrow." Same number, different licence: only the Bayesian is allowed to attach a probability to a single, never-repeated tomorrow — and, later, to the unknown θ itself.

Frequentists and Bayesians have argued for a century; xkcd #1132 (in the MLE chapter) is the famous cartoon of the fight. This book is a peace treaty: MLE for when you trust the data, priors for when you do not have enough of it, and — by the end — an integral that quietly uses both.

Check yourself

Q. Give the frequentist and Bayesian meanings of $P(\textbf{heads}) = 0.5$.
A. Frequentist: the long-run fraction of heads tends to 0.5. Bayesian: my degree of belief in heads is 0.5 (allowed even for a single toss).

Q. Why does i.i.d. let us multiply per-example probabilities?
A. Independent ⇒ joint = product; identically distributed ⇒ the same $p(\cdot \mid \theta)$ in every factor. Together: $p(\text{data} \mid \theta) = \prod_i p(y_i \mid \theta)$.

Q. What is the likelihood of H,H,T,H,T under a θ-coin, and where does it peak?
A. $\theta^3(1-\theta)^2$; it peaks at $\theta = 3/5$, the fraction of heads.

Q. What is the difference between $P(x;\theta)$ and $P(x \mid \theta)$?
A. Semicolon: θ is a fixed parameter (frequentist). Bar: θ is a random variable with its own distribution (Bayesian).

Q. In one line, what do MLE, MAP, and "fully Bayesian" do with θ?
A. MLE: pick the θ that maximises the likelihood. MAP: pick the peak of the posterior. Fully Bayesian: keep the whole posterior and average over it.

With this in hand, the next chapter turns the coin-likelihood into calculus (why the *log*, and the exact MLE), then adds a prior to get MAP. The chapter after that fixes MLE's zero-probability disaster with *smoothing*, and finally stops picking a single θ at all — it *integrates* over every θ the data still allows.

14

Probabilistic Models, MLE, and MAP

The second half of Lecture 12 and all of Lecture 13 change the costume, not the maths. Least squares was "make $X\theta$ close to y." Now y is a *random* vector drawn from a distribution that knows θ, and we ask which θ makes the data we actually saw the least surprising. That question is Maximum Likelihood. Putting a belief on θ before seeing y is Maximum A Posteriori — and it will turn out to *be* ridge.

14.1 $P(x;\theta)$ versus $P(x \mid \theta)$

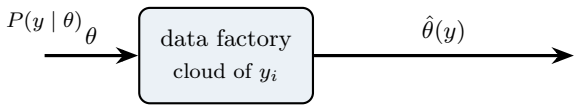
The notes draw a factory: θ goes in, data X (a cloud of points) comes out, and we send back an estimate $\hat{\theta}$. Two notations sit in boxes:

- $P(x;\theta)$ — probability of x **parameterized by** θ. Frequentist: θ is a knob of the machine, not a random variable.
- $P(x \mid \theta)$ — probability of x **conditioned on** θ. Bayesian: θ itself has a distribution, so the vertical bar is a genuine conditional.

In this course both are written $P(y \mid \theta)$ once we start multiplying. What matters is the i.i.d. assumption the notes underline:

$$\vec{y} = [y_1, \ldots, y_n]^\top \quad \text{with} \quad y_i \sim P(y \mid \theta) \quad \text{independent, identical.}$$

θ is the parameter. The observations are i.i.d. The job is to learn θ from y.



The notes' cartoon: a parameter goes into a random factory; data comes out; an estimator tries to recover the parameter.

14.2 The likelihood, the log, the NLL

Independence turns a joint into a product:

$$p(y \mid \theta) = p(y_1, \ldots, y_n \mid \theta) = \prod_{i=1}^{n} p(y_i \mid \theta).$$

That product, viewed as a function of θ with y frozen, is the **likelihood**. The notes' graph of $p(y \mid \theta)$ versus θ is a wavy curve; $\hat{\theta}$ is the θ under the highest peak.

Derivation

Products are hostile to calculus (the derivative of a product of n terms is a mess). Logs turn products into sums, and log is strictly increasing so it does not move the argmax:

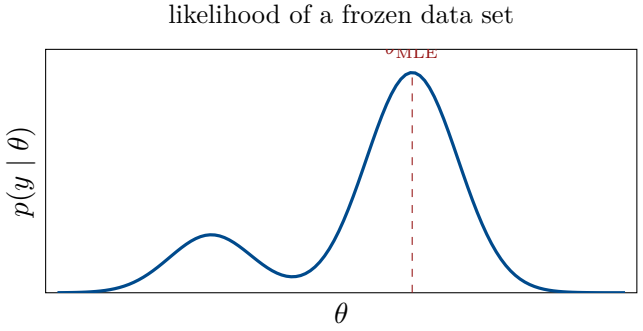
$$\ell(\theta) = \log p(y \mid \theta) = \log \prod_{i=1}^{n} p(y_i \mid \theta) = \sum_{i=1}^{n} \log p(y_i \mid \theta),$$

$$\hat{\theta}_{\text{MLE}} = \arg\max_{\theta} \ell(\theta) = \arg\max_{\theta} \sum_{i=1}^{n} \log p(y_i \mid \theta).$$

Machine-learning libraries minimise rather than maximise, so the notes flip the sign and define the **negative log-likelihood**

$$\text{NLL}(\theta) = -\ell(\theta) = -\sum_{i=1}^{n} \log p(y_i \mid \theta), \qquad \hat{\theta}_{\text{MLE}} = \arg\min_{\theta} \text{NLL}(\theta).$$

MLE: pick the knob θ that made today's data the most probable. NLL: the same thing, written as a loss. Squared error *is* an NLL — for a Gaussian with fixed variance, $-\log p(y_i \mid \theta) \propto (h_\theta(x^{(i)}) - y^{(i)})^2$. That is why least squares was hiding a probability model all along (CS229 notes; Murphy, *Probabilistic Machine Learning*).



The notes' picture: maximise $p(y \mid \theta)$ with respect to θ. If there are two peaks, MLE takes the taller one and ignores the prior (there is not one yet).

14.3 Worked example: n coin tosses (Bernoulli MLE)

This is Lecture 13, pages 2–3, algebra included, nothing skipped.

Derivation

Each toss $y_i \in \{0, 1\}$, Head $= 1$, Tail $= 0$, $P(\text{Head}) = \theta \in (0, 1)$. The Bernoulli pmf the notes write is

$$p(y_i \mid \theta) = \theta^{y_i}(1 - \theta)^{1 - y_i}.$$

(Check: if $y_i = 1$ this is θ; if $y_i = 0$ this is $1 - \theta$.) The log likelihood of the whole sample:

$$\ell(\theta) = \sum_{i=1}^{n} \log\left[\theta^{y_i}(1-\theta)^{1-y_i}\right] = \sum_{i=1}^{n} \left[y_i \log\theta + (1-y_i)\log(1-\theta)\right].$$

Differentiate. $\frac{d}{d\theta}\log\theta = 1/\theta$ and $\frac{d}{d\theta}\log(1-\theta) = -1/(1-\theta)$:

$$\frac{d\ell}{d\theta} = \frac{1}{\theta}\sum_{i=1}^{n} y_i - \frac{1}{1-\theta}\sum_{i=1}^{n}(1-y_i).$$

Set the derivative to zero. Write $s = \sum y_i$ (number of heads) and $\sum(1-y_i) = n - s$:

$$\frac{s}{\theta} - \frac{n-s}{1-\theta} = 0 \qquad \Longrightarrow \qquad s(1-\theta) = (n-s)\theta \qquad \Longrightarrow \qquad s - s\theta = n\theta - s\theta.$$

The θ terms on the right and left cancel in the notes' crossed-out version of the same algebra (they first put everything over $\theta(1-\theta)$):

$$\frac{(1-\theta)s - n\theta + \theta s}{\theta(1-\theta)} = 0 \qquad \Longrightarrow \qquad s - \theta s - n\theta + \theta s = 0 \qquad \Longrightarrow \qquad s - n\theta = 0.$$

Therefore

$$\boxed{\hat{\theta}_{\text{MLE}} = \frac{1}{n}\sum_{i=1}^{n} y_i},$$

the sample mean — "fraction of heads." If you saw 7 heads in 10 tosses, MLE says the coin is a 0.7 coin, full stop.

Watch out

That is also the problem. With $n = 2$ and two heads, MLE says $\hat{\theta} = 1$: the coin *never* tails. Tomorrow's first tail has probability zero under this model. The notes: if there is not enough data, θ might not be a good fit and can **overfit**. Fix: regularize with prior knowledge.

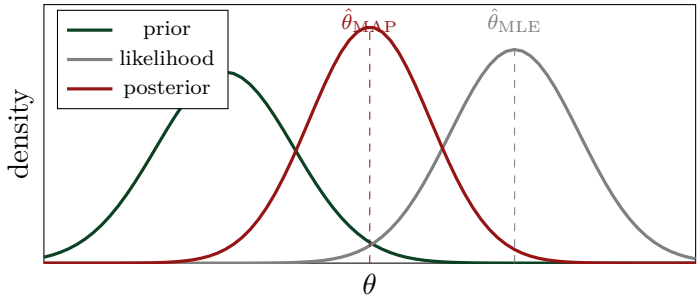
14.4 From MLE to MAP: Bayes, in the notes' labels

We put a distribution $p(\theta)$ on the parameter *before* seeing y. Bayes:

$$p(\theta \mid y) = \frac{p(y \mid \theta)\, p(\theta)}{p(y)}.$$

The notes label the four pieces in blue:

$p(\theta \mid y)$	**posterior** — belief about θ after the data
$p(y \mid \theta)$	**likelihood** — how well this θ explains y
$p(\theta)$	**prior** — belief about θ before the data
$p(y)$	**evidence** — boxed in class; a constant w.r.t. θ



The notes' three-curve picture. MAP is the compromise: the posterior peak sits between the prior peak and the likelihood peak. More data makes the black curve sharper and drags MAP toward MLE.

A uniform prior is "I refuse to have an opinion." The notes: a uniform prior is the same as using no prior, and MAP reduces to MLE.

14.5 MAP, derived (Lecture 13 page 4)

Derivation

$$\begin{aligned}\hat{\theta}_{\text{MAP}} &= \arg\max_{\theta} p(\theta \mid y) = \arg\max_{\theta} \log p(\theta \mid y) \\ &= \arg\max_{\theta} \log \frac{p(y \mid \theta)\, p(\theta)}{p(y)} \\ &= \arg\max_{\theta} \big[\log p(y \mid \theta) + \log p(\theta) - \log p(y)\big].\end{aligned}$$

Drop $\log p(y)$ (no θ). Expand the likelihood with i.i.d.:

$$\hat{\theta}_{\text{MAP}} = \arg\max_{\theta} \left[\sum_{i=1}^{n} \log p(y_i \mid \theta) + \log p(\theta)\right].$$

Flip to a minimisation of NLL:

$$\hat{\theta}_{\text{MAP}} = \arg\min_{\theta} \big[\text{NLL}(\theta) - \log p(\theta)\big].$$

The extra $-\log p(\theta)$ *is* a regularizer. If $p(\theta)$ is a zero-mean Gaussian on the slopes, $-\log p(\theta) \propto \|\theta_{1:d}\|_2^2$ and MAP *is* ridge (CS229; Murphy's conjugate-Gaussian notes; Berkeley CS189 ridge lecture). If $p(\theta)$ is uniform, $\log p(\theta)$ is constant and MAP = MLE, as the notes say.

14.6 MAP with a Gaussian prior *is* ridge regression

The last box said "MAP is ridge" and moved on. That sentence is the single most important claim in the course, and asserting it is not good enough. Here it is, every line, starting from nothing but two Gaussians.

Nothing in this derivation mentions squared error, and nothing mentions a penalty. Both will appear, uninvited, as consequences.

From two Gaussians to ridge, in seven steps

Step 1 — state the two beliefs. Assume, first, that each observation is the linear model plus independent Gaussian noise of variance σ^2:

$$y^{(i)} = \theta^\top x^{(i)} + \varepsilon^{(i)}, \qquad \varepsilon^{(i)} \sim \mathcal{N}(0, \sigma^2) \quad \Longrightarrow \quad p\big(y^{(i)} \mid x^{(i)}; \theta\big) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y^{(i)} - \theta^\top x^{(i)})^2}{2\sigma^2} \right).$$

Assume, second, that before seeing any data you expect the weights to be smallish and centred at zero, independently of each other:

$$\theta_j \sim \mathcal{N}(0, \tau^2) \quad \Longrightarrow \quad p(\theta) = \prod_{j=1}^{d} \frac{1}{\sqrt{2\pi\tau^2}} \exp\left(-\frac{\theta_j^2}{2\tau^2} \right).$$

The number τ^2 is how open-minded you are. Small τ^2: you are confident the weights are near zero. Large τ^2: you have barely an opinion.

Step 2 — write the log-likelihood. Independence turns the product into a sum, and $\log(\exp u) = u$:

$$\log p(y \mid X; \theta) = \sum_{i=1}^{n} \left[-\log \sqrt{2\pi\sigma^2} \; - \; \frac{(y^{(i)} - \theta^\top x^{(i)})^2}{2\sigma^2} \right] = \underbrace{-n \log \sqrt{2\pi\sigma^2}}_{\text{no } \theta} \; - \; \frac{1}{2\sigma^2} \sum_{i=1}^{n} \big(y^{(i)} - \theta^\top x^{(i)} \big)^2.$$

Step 3 — write the log-prior. Identically:

$$\log p(\theta) = \underbrace{-d \log \sqrt{2\pi\tau^2}}_{\text{no } \theta} \; - \; \frac{1}{2\tau^2} \sum_{j=1}^{d} \theta_j^2.$$

Step 4 — add them and bin the constants. MAP maximises $\log p(y \mid X; \theta) + \log p(\theta)$. The two underbraced terms do not contain θ, so they cannot influence which θ wins:

$$\hat{\theta}_{\text{MAP}} = \arg\max_{\theta} \left[-\frac{1}{2\sigma^2} \sum_{i} \big(y^{(i)} - \theta^\top x^{(i)} \big)^2 - \frac{1}{2\tau^2} \sum_{j} \theta_j^2 \right].$$

Step 5 — flip the sign. Maximising $-f$ is minimising f:

$$\hat{\theta}_{\text{MAP}} = \arg\min_{\theta} \left[\frac{1}{2\sigma^2} \|y - X\theta\|^2 + \frac{1}{2\tau^2} \|\theta\|^2 \right].$$

Step 6 — multiply by a positive constant. Scaling an objective by $2\sigma^2 > 0$ cannot move its minimiser:

$$\hat{\theta}_{\text{MAP}} = \arg\min_{\theta} \left[\|y - X\theta\|^2 + \frac{\sigma^2}{\tau^2} \|\theta\|^2 \right].$$

Step 7 — read the label. That is the ridge objective of Chapter 9, exactly, with

$$\boxed{\lambda \; = \; \frac{\sigma^2}{\tau^2}}$$

and therefore $\hat{\theta}_{\text{MAP}} = \hat{\theta}_{\text{ridge}} = (X^\top X + \lambda I)^{-1} X^\top y$. ■

Sit with what just happened. You wrote down two beliefs about the world — the noise is Gaussian, the weights are probably small — and out came a penalty term you had previously been told to add because it "discourages large weights". The penalty was never a trick. It was a prior all along, and it had a numerical value you could in principle have measured.

> **$\lambda = \sigma^2/\tau^2$ is a ratio of two uncertainties, and every property of ridge is legible in it.**
> *Noisy data, σ^2 large.* λ grows. Correct: if the observations are unreliable, lean harder on your prior belief and shrink more.
> *Confident prior, τ^2 small.* λ grows. Correct: you were sure the weights were near zero, so insist on it.
> *No prior at all, $\tau^2 \to \infty$.* $\lambda \to 0$ and ridge collapses to ordinary least squares. A flat prior is the same as no prior, exactly as the coin section said, and here it is again in continuous clothing.
> *Clean data, $\sigma^2 \to 0$.* $\lambda \to 0$. If the measurements are perfect there is nothing to protect against; fit them.

Watch out

The derivation above penalises every θ_j, including θ_0 if you let j run from zero. In practice the intercept is left unpenalised, which corresponds to putting a *flat* prior on θ_0 and a $\mathcal{N}(0, \tau^2)$ prior on the rest. This is not a fudge: shrinking the intercept toward zero is a claim that the target is near zero when all features are zero, which is a statement about your units, not about model complexity. Lab 3's R-mask (a diagonal matrix of ones with a zero in the top-left corner) is exactly this mixed prior, written as a matrix.

Other priors give other penalties

The argument used nothing about the Gaussian shape except that its log is a negative square. Change the prior and the whole recipe runs again, handing you a different regulariser.

Put a Laplace prior on the weights, $p(\theta_j) \propto \exp(-|\theta_j|/b)$. Then $-\log p(\theta) = \frac{1}{b}\sum_j |\theta_j| + \text{const}$, and Step 6 gives

$$\arg\min_\theta \; \|y - X\theta\|^2 + \lambda \sum_j |\theta_j|, \qquad \lambda = \frac{2\sigma^2}{b},$$

which is the Lasso. The reason the Lasso sets coefficients exactly to zero and ridge does not is now a statement about the two priors: the Laplace density has a sharp spike at zero, so it genuinely believes some weights *are* zero, whereas the Gaussian is smooth there and merely believes they are small.

Put a flat prior on everything and you get no penalty, which is MLE. The table is short and it is the whole subject:

Prior on θ_j	$-\log p(\theta)$ contributes	Known as
flat / uniform	nothing	MLE, ordinary least squares
$\mathcal{N}(0, \tau^2)$	$\frac{1}{2\tau^2}\sum_j \theta_j^2$	ridge, L_2, weight decay
Laplace$(0, b)$	$\frac{1}{b}\sum_j \lvert\theta_j\rvert$	Lasso, L_1

There is no separate theory of regularisation to learn. There is a theory of priors, and

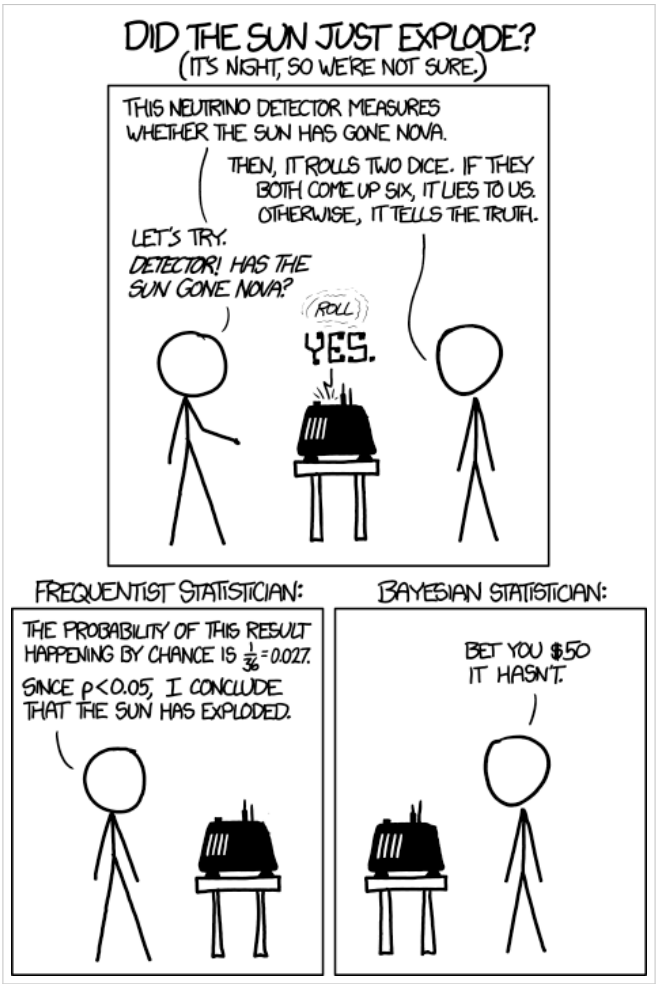
regularisers are what priors look like after you take a logarithm and change the sign.

Go deeper

Watch. Ali Ghodsi (Waterloo), Deep Learning Lec 2.1, *Regularization — the L_2-as-Gaussian-prior argument on a board*
He does the Laplace/L_1 case immediately afterwards, which is the "Another route" box above delivered live.

Watch. mathematicalmonk, *ML 6.3*, *Interpretation of MAP as a convex combination*
This is the one to watch if $\lambda = \sigma^2/\tau^2$ still feels arbitrary: he shows MAP as a weighted average of prior and data, with the weights set by how much data you have.

Read. Murphy, *Conjugate Bayesian analysis of the Gaussian distribution.* The same computation done properly, with the full posterior rather than just its mode. https://www.cs.ubc.ca/~murphyk/Papers/bayesGauss.pdf



xkcd #1132 "Frequentists vs. Bayesians" by Randall Munroe — https://xkcd.com/1132/ (CC BY-NC 2.5).

MLE is the frequentist who will not speak until the data does. MAP is the Bayesian who walked in with a prior and then updated. The sun-rising comic is the same fight this chapter just made algebraic.

14.7 Coin again, now with a Beta prior

The notes assume $\theta \sim \text{Beta}(\alpha, \beta)$ on $(0, 1)$:

$$p(\theta) = \frac{\Gamma(\alpha + \beta)}{\Gamma(\alpha)\Gamma(\beta)} \theta^{\alpha-1}(1-\theta)^{\beta-1}.$$

The Gamma function, boxed in blue on the last page of Lecture 13:

$$\Gamma(z) = \int_0^\infty t^{z-1} e^{-t}\, dt, \qquad \text{Re}(z) > 0, \qquad \Gamma(n) = (n-1)! \text{ for positive integers } n.$$

The notes then write "log probability … $\hat{\theta}_{\text{MAP}}$ =?" and stop. We finish the derivation — it is the surrounding topic the lecture was pointing at.

Derivation

Ignore the Γ-prefactor (no θ). The log posterior, up to a constant, is the Bernoulli log-likelihood plus the Beta log-prior:

$$\begin{aligned} \log p(\theta \mid y) &\propto \sum_{i=1}^{n} \big[y_i \log\theta + (1-y_i)\log(1-\theta) \big] + (\alpha-1)\log\theta + (\beta-1)\log(1-\theta) \\ &= \big(s + \alpha - 1\big)\log\theta + \big(n - s + \beta - 1\big)\log(1-\theta), \end{aligned}$$

where $s = \sum y_i$. Differentiate and set to zero:

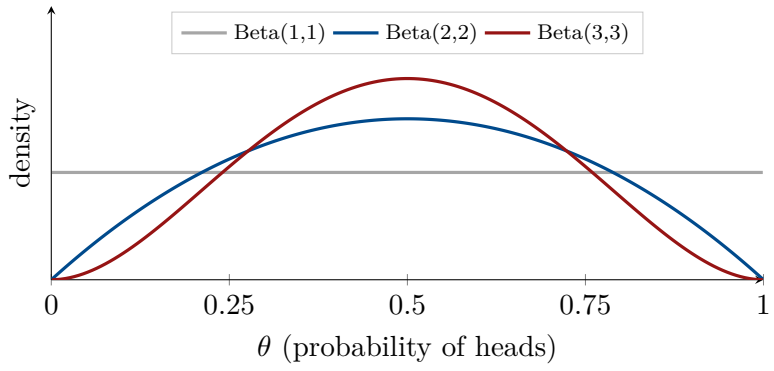
$$\frac{s+\alpha-1}{\theta} - \frac{n-s+\beta-1}{1-\theta} = 0 \qquad \Longrightarrow \qquad \boxed{\hat{\theta}_{\text{MAP}} = \frac{s+\alpha-1}{n+\alpha+\beta-2}}.$$

Compare to MLE's s/n. The prior contributed $\alpha - 1$ fake heads and $\beta - 1$ fake tails. With $\alpha = \beta = 2$ (a gentle bump around $1/2$), two heads in two tosses give

$$\hat{\theta}_{\text{MAP}} = \frac{2+2-1}{2+2+2-2} = \frac{3}{4},$$

not 1. The model is no longer willing to declare a two-headed coin.

A prior is not a mood. It is a curve over $\theta \in [0, 1]$ saying which coin biases you consider plausible before seeing a single toss. Three choices from the same Beta family, drawn to scale:



Beta$(1, 1)$ *is flat: every bias equally plausible, which is why it turns MAP back into MLE.* *Beta*$(2, 2)$ *is a gentle preference for a fair coin.* *Beta*$(3, 3)$ *is the same belief held more stubbornly. The prior's job is*

to decide how much evidence it takes to move you.

The formula says the prior contributes $\alpha-1$ imaginary heads and $\beta-1$ imaginary tails before the real tosses are counted. That is worth seeing as a number. Two heads in two tosses, under three different amounts of prior stubbornness:

Prior	fake heads	fake tails	$\hat{\theta}_{\text{MAP}}$ after HH
Beta$(1,1)$ (flat)	0	0	$2/2 = 1.00$
Beta$(2,2)$	1	1	$3/4 = 0.75$
Beta$(5,5)$	4	4	$6/10 = 0.60$

Only the flat prior is willing to announce, on the strength of two tosses, that the coin never lands tails. Every other row hedges, and the hedge is bigger the more stubborn the prior. As n grows, the fake counts stop mattering and every row converges on the same answer — which is the precise sense in which data eventually overrules belief.

A prior is **conjugate** for a likelihood if the posterior is the same family as the prior. Beta is conjugate to Bernoulli/Binomial: you start at Beta(α,β) and after s heads in n tosses you sit at Beta$(\alpha+s, \beta+n-s)$. MAP is just the mode of that posterior. Murphy's 2007 Gaussian-conjugacy note does the same trick for least squares: a Gaussian prior on θ plus a Gaussian likelihood gives a Gaussian posterior whose mode is ridge.

MLE is a friend who, after two good meals at a new farsan stall, declares it the best stall in Gujarat. MAP is the friend who walked in already believing "most stalls are average," so two good meals shift the score, but not to infinity. The Beta prior is that inherited belief; α, β are how stubborn it is.

In code

```python
def bernoulli_mle(y):
    return np.mean(y)

def bernoulli_map(y, alpha=2.0, beta=2.0):
    s, n = np.sum(y), len(y)
    return (s + alpha - 1.0) / (n + alpha + beta - 2.0)

y = np.array([1, 1])           # two heads
print(bernoulli_mle(y), bernoulli_map(y))   # 1.0  vs  0.75
```

14.8 What the 2026 reading list adds on top of class

The lecture stops at Beta–Bernoulli MAP. The papers downloaded for this edition are not a second textbook — they are the *next* sentences, so a student who finished FOML is not two years behind a 2026 reading group.

- **Double descent** (Nakkiran et al., arXiv:1912.02292). The class underfit / just-right / overfit trio is the *classical* U of test error. In over-parameterised models the test error can fall *again* after interpolating the training set. The slide is not wrong; it is incomplete once

d dwarfs n.

- **Grokking** (Power et al., arXiv:2201.02177). A network can sit in the overfit regime — train loss tiny, test loss awful — and then, much later, generalise. Regularization and time both matter; λ is not the only adult in the room.
- **SVD as the spine of data science** (Mathematics of Data Science, arXiv:2607.11938, 2026; Prince, *Understanding Deep Learning*, Feb 2026 PDF). Lecture 12's $U\Sigma V^\top$ is the same object behind PCA, ridge shrinkage $\sigma_i^2/(\sigma_i^2+\lambda)$, and modern linearised deep nets.
- **Muon** (Jordan et al., arXiv:2502.16982, 2025). The class shrink $\theta_j \leftarrow (1-\alpha\lambda/n)\theta_j$ is a *scalar* weight decay. Muon orthogonalises a *matrix* of weights each step — the 2025–2026 answer to "what is the Hessian-aware cousin of Lecture 11?" AdamW (arXiv:1711.05101) is the intermediate chapter: keep the multiply separate from the data gradient.

You do not need those papers to pass FOML. You *do* need them to hear a 2026 paper talk and recognise: that is our $X^\dagger$, that is our λ, that is our $\nabla \log|A|$, wearing a deeper network. That is the cutting edge this book is aiming at — not new slogans, but the class derivations used in anger.

Exercises

Pencil first, then click the ⇝ to jump to the worked answer in Appendix D. The answer links back to the question.

14.1 ⇝ $P(x;\theta)$ against $P(x \mid \theta)$: one sentence each, and which school of thought is entitled to which bar.

14.2 ⇝ Derive the Bernoulli MLE for s heads in n tosses, starting from the log-likelihood and differentiating.

14.3 ⇝ Write MAP twice: once as an arg max of a sum of logs, once as an arg min of a negative log-likelihood plus a penalty. When are MAP and MLE the same estimator, and what is the prior that makes it so?

14.4 ⇝ Finish the formula the notes left open: for a Beta(α,β) prior on a coin, $\hat{\theta}_{\text{MAP}} =$?

14.5 ⇝ Two tosses, both heads, with $\alpha=\beta=2$. Compute MLE and MAP. Which one refuses to claim certainty, and which term in the formula is doing that?

14.6 ⇝ What is $\Gamma(n)$ for a positive integer n? Write the integral definition too.

15

Smoothing, and Being Fully Bayesian

The last chapter ended with a warning: two heads in two tosses and maximum likelihood declares the coin *never* lands tails. That is not a quirk — it is a disease of counting, and it shows up everywhere from coins to spam filters. The cure is **smoothing**: pretend you saw a few outcomes you did not, so no possibility is ever assigned probability zero. And the beautiful part, which this chapter builds to slowly, is that smoothing is not a hack. It is exactly what a Bayesian with a Beta prior does — and if you refuse to pick a single θ at all and instead *average over every* θ, the same smoothed number falls out on its own.

Two ideas, one chapter: (1) never let an unseen event have probability zero — add a few "imaginary" observations first (*smoothing*); (2) instead of betting everything on one best θ, keep the whole range of plausible θ's and average — that is being *fully Bayesian*. Both turn out to be the same arithmetic.

15.1 The zero-probability disaster

Maximum likelihood estimates a probability by a bare fraction: $\hat{\theta}_{\text{MLE}} = s/n$, the fraction of heads. If an outcome never appeared, its estimated probability is 0 — and in a model that multiplies probabilities, a single 0 annihilates everything.

Watch out

One unseen word kills the whole sentence. A spam filter multiplies the probabilities of the words in an email. If the word "*quokka*" never appeared in the training spam, MLE says $P(\text{quokka} \mid \text{spam}) = 0$, so *any* email containing "quokka" gets probability exactly 0 of being spam — no matter how many other spammy words it has. One missing count vetoes all the evidence. That is the disaster smoothing exists to prevent.

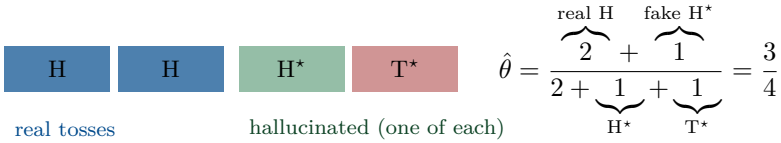
The fix is disarmingly simple: before you count, *pretend you already saw one of each outcome.* Then no count is ever zero.

15.2 Laplace smoothing: add one of each

Laplace's rule (also called add-one smoothing): estimate the probability of heads as

$$\hat{\theta}_{\text{Laplace}} = \frac{s+1}{n+2}$$

— the real heads s plus one imaginary head, over the real tosses n plus two imaginary tosses (one of each outcome). Two heads in two tosses no longer gives 1; it gives $\frac{2+1}{2+2} = \frac{3}{4}$. The coin is allowed to surprise you.



Laplace smoothing in one picture: glue on one imaginary head and one imaginary tail before counting. The "⋆" outcomes are the hallucinated pseudo-counts — prior belief wearing the costume of data.

Where does "add one" come from, and why *one*? Two derivations, and they meet.

Derivation

Route 1 — pseudo-counts. Pretend the data set arrived with one extra head and one extra tail already in it. Heads become $s+1$, total becomes $n+2$, and the plain fraction is $\frac{s+1}{n+2}$. That is all Laplace's rule *is*: a fraction computed after hallucinating one of each outcome.

Route 2 — a uniform prior, done Bayesian. Recall from the last chapter that a Beta(α, β) prior on θ updates, after s heads and $n - s$ tails, to the posterior Beta$(\alpha + s,\ \beta + n - s)$ (this is conjugacy — "just add the counts"). The *mean* of a Beta(a, b) is $a/(a + b)$, so the posterior mean is

$$\mathbb{E}[\theta \mid \text{data}] = \frac{\alpha + s}{\alpha + \beta + n}.$$

Take the *flat* prior Beta$(1, 1)$ — "every bias equally plausible", total ignorance — so $\alpha = \beta = 1$:

$$\mathbb{E}[\theta \mid \text{data}] = \frac{1 + s}{2 + n} = \frac{s + 1}{n + 2}.$$

Add-one smoothing $\frac{s+1}{n+2}$ is exactly the posterior mean of a coin under the flat Beta$(1, 1)$ prior. The "one imaginary head, one imaginary tail" are the prior's pseudo-counts. More generally, **add-k smoothing** $\frac{s+k}{n+2k}$ is the posterior mean under Beta(k, k): k imaginary outcomes of each kind. Smoothing was Bayesian statistics in a plain-fraction disguise all along.

Watch out

Two nearby numbers, do not confuse them. The Beta(α, β) *mode* (the peak, used by MAP) is $\frac{\alpha - 1}{\alpha + \beta - 2}$, so MAP under Beta$(2, 2)$ also gives $\frac{s+1}{n+2}$. The *mean* (used by the fully Bayesian prediction) is $\frac{\alpha}{\alpha + \beta}$, so the posterior mean under Beta$(1, 1)$ gives the same $\frac{s+1}{n+2}$. Two different priors, two different summaries, same famous number — a coincidence for this symmetric case, not a rule (see the skewed example below).

15.3 More than two outcomes: the Dirichlet

Coins have two outcomes. Dice have six; a word can be any of thousands; a classifier picks one of k classes. The Beta's big brother for k outcomes is the **Dirichlet** distribution. It puts a prior on a whole probability vector $(\theta_1, \ldots, \theta_k)$ with $\sum_j \theta_j = 1$, using one pseudo-count α_j per outcome.

With counts $n_1, \ldots, n_k$ (total $N = \sum_j n_j$) and a Dirichlet$(\alpha_1, \ldots, \alpha_k)$ prior, the posterior is Dirichlet$(\alpha_1 + n_1, \ldots, \alpha_k + n_k)$ — conjugacy again, "add the counts." Its mean gives the smoothed probability of outcome j:

$$\hat{\theta}_j = \frac{n_j + \alpha_j}{N + \sum_\ell \alpha_\ell}.$$

The **symmetric** Dirichlet$(1, \ldots, 1)$ (one imaginary count per category) gives Laplace smoothing for categories:

$$\hat{\theta}_j = \frac{n_j + 1}{N + k}.$$

This is exactly what a Naive Bayes text classifier does to word counts: add one to every word's tally so that a word never seen in a class does not veto that class. Beta is the $k = 2$ special case of the Dirichlet; add-one smoothing is the symmetric Dirichlet's posterior mean. One idea, scaled from a coin to a whole vocabulary.

A new thali restaurant with a 6-dish menu and only three customers so far cannot honestly say "nobody will ever order dish 4." The Dirichlet's imaginary customer tries one of every dish on the way in, so no dish is ruled out until the real crowd overwhelms that one polite ghost.

15.4 Being fully Bayesian: stop picking, start integrating

MLE and MAP both hand you a *single* θ — the top of the likelihood, or the top of the posterior — and then predict with that one number. The fully Bayesian says: why commit? The posterior $p(\theta \mid \text{data})$ describes *every* θ the data still allows, each with its own credibility. To predict the next toss, ask every θ what it thinks and average their answers, weighted by the posterior.

Derivation

The predictive distribution. The probability the next toss is heads, given the data, integrates the per-θ answer $P(\text{head} \mid \theta) = \theta$ against the posterior:

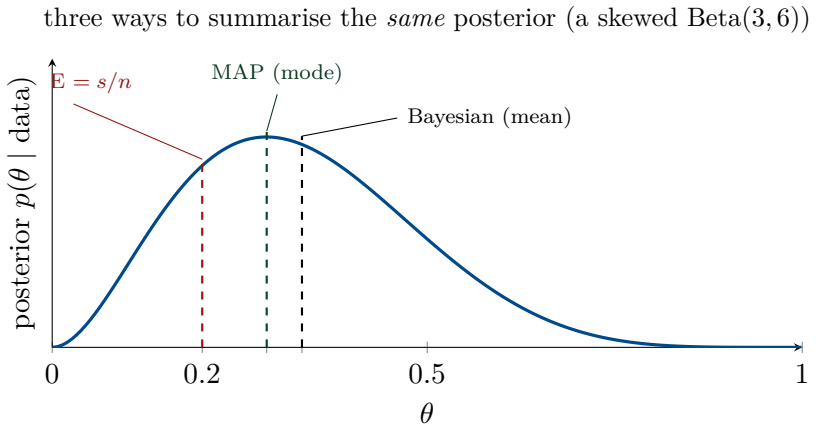
$$P(\text{next} = \text{head} \mid \text{data}) = \int_0^1 P(\text{head} \mid \theta)\, p(\theta \mid \text{data})\, d\theta = \int_0^1 \theta\, p(\theta \mid \text{data})\, d\theta = \mathbb{E}[\theta \mid \text{data}].$$

The prediction is the posterior *mean*. And we already know it: with a Beta(α, β) prior the posterior is Beta$(\alpha + s, \beta + n - s)$, whose mean is

$$\boxed{P(\text{next} = \text{head} \mid \text{data}) = \frac{s + \alpha}{n + \alpha + \beta}}.$$

The fully Bayesian prediction $\dfrac{s+\alpha}{n+\alpha+\beta}$ *is* the smoothed, pseudo-count estimate from the start of this chapter. Smoothing, the posterior mean, and "integrate over every model" are three names for the same number. Laplace's "add one" was never a trick — it was the honest average over all the coins the data had not ruled out.

Notice what the integral bought you: the prediction **no longer depends on a single model.** MLE bet everything on the one θ at the peak; if the data was thin, that peak was a wild guess and the bet was reckless. The Bayesian keeps the whole hump and averages, so a wide, uncertain posterior automatically produces a cautious, hedged prediction.



On a skewed posterior the three summaries genuinely differ: MLE is the likelihood's peak (ignores the prior), MAP is the mode (top of the posterior), and the fully Bayesian prediction is the mean (the balance point, pulled toward the long tail). The symmetric coin case hid this by making them coincide.

MLE and MAP answer "what is the single best θ?" The fully Bayesian answers a different question — "what will the next toss do?" — and to answer it, uses *all* the θ's at once. When data is plentiful the posterior is a narrow spike and all three agree. When data is thin the posterior is broad, and only the average is honest about how little you know.

Watch out

Integrating is the honest thing, but it is not always *easy*. We got a clean formula only because Beta is *conjugate* to the coin, so the posterior stayed a Beta we could average in closed form. For most models the integral $\int P(y \mid \theta)\, p(\theta \mid \text{data})\, d\theta$ has no formula, and you approximate it by *sampling* many θ's from the posterior (MCMC) and averaging their predictions. Conjugacy is the pleasant exception, not the rule.

15.5 MLE, MAP, and fully Bayesian, side by side

Method	What it computes	How it treats θ
MLE	$\arg\max_\theta \; p(\text{data} \mid \theta)$	a fixed knob; use the single peak
MAP	$\arg\max_\theta \; p(\text{data} \mid \theta)\, p(\theta)$	a random variable; use the posterior's *mode*
Fully Bayesian	$\int p(y \mid \theta)\, p(\theta \mid \text{data})\, d\theta$	a random variable; *average over the whole posterior*

In code

```python
import numpy as np

def coin_mle(s, n):                          # fraction of heads
    return s / n

def coin_map(s, n, a=2.0, b=2.0):            # mode of Beta(a+s, b+n-s)
    return (s + a - 1.0) / (n + a + b - 2.0)

def coin_bayes(s, n, a=1.0, b=1.0):          # posterior mean = predictive prob of heads
    return (s + a) / (n + a + b)

s, n = 2, 2                                  # two heads in two tosses
print(coin_mle(s, n))                        # 1.0   -> "never tails" (reckless)
print(coin_map(s, n))                        # 0.75  -> MAP with Beta(2,2)
print(coin_bayes(s, n))                      # 0.75  -> Laplace add-one = posterior mean
```

`coin_bayes` with `a=b=1` is Laplace's $\frac{s+1}{n+2}$ — and, by the last section, the fully Bayesian prediction. Same line of code, three names.

MLE is the intern who, after two lucky trades, bets the firm. MAP is the manager who says "markets are usually average, calm down." The fully Bayesian is the committee that polls every analyst, weights each by how credible the data makes them, and reports the average — slower, but the only one that never claims certainty it has not earned.

Check yourself

Q. Why is a probability estimate of exactly 0 dangerous in a model that multiplies probabilities?

A. One zero factor makes the entire product zero, so a single unseen event vetoes all other evidence (e.g. one unknown word zeroes a spam score).

Q. State Laplace's rule and give its two derivations.

A. $\hat{\theta} = (s+1)/(n+2)$. Route 1: add one imaginary head and one imaginary tail (pseudo-

counts). Route 2: posterior mean under a flat Beta$(1, 1)$ prior.

Q. How does add-one smoothing generalise to k categories, and what prior is that?
A. $\hat{\theta}_j = (n_j + 1)/(N + k)$; it is the posterior mean under a symmetric Dirichlet$(1, \ldots, 1)$ prior (Beta is the $k = 2$ case).

Q. Write the fully Bayesian predictive probability of the next head, and simplify for a Beta prior.
A. $\int_0^1 \theta \, p(\theta \mid \text{data}) \, d\theta = \mathbb{E}[\theta \mid \text{data}] = (s + \alpha)/(n + \alpha + \beta)$.

Q. When do MLE, MAP, and the Bayesian prediction agree, and when do they diverge?
A. They agree when data is plentiful (posterior a narrow spike). They diverge when data is thin or the posterior is skewed — then mode (MAP) $\neq$ mean (Bayesian), and MLE ignores the prior entirely.

Q. Why is the fully Bayesian integral usually hard, and what is done instead?
A. Only conjugate models give a closed form; in general you sample θ's from the posterior (MCMC) and average their predictions.

16

MLE and MAP for Probabilistic Linear Regression

The Bernoulli MLE/MAP chapter told the probabilistic story for a *coin* (Bernoulli): count the heads, maybe add a Beta prior, and out pops an estimate. Lecture 14 tells the *same* story for a *line*. And the punchline is one of the most satisfying facts in the whole course: the least-squares cost we minimised in the earlier linear-regression chapters was never an arbitrary choice. It is what you get when you assume the noise on your data is Gaussian and then ask "which line makes my data most probable?" Ridge, likewise, is what you get when you also walk in with a Gaussian belief that the slopes are small. Two of our workhorses — plain least squares and ridge — are secretly a likelihood and a likelihood-times-prior.

So far a "good" line was one with small squared error — we just *declared* that. This chapter shows the squared error is not a taste; it is a *consequence*. If you believe the true relationship is a line plus bell-curve noise, then "most probable line" **is** "smallest squared error". Change the noise assumption and the loss changes with it.

16.1 Two questions, two estimators

Every method in this chapter is one of two moves. We have data X and unknown parameters θ, and a model that ties them together, $p(x \mid \theta)$.

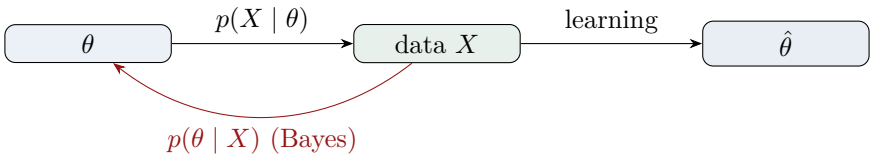
Maximum Likelihood (ML/MLE). Choose the θ that makes the observed data most probable:

$$\hat{\theta}_{\text{ML}} = \arg\max_{\theta} \; p(x \mid \theta).$$

Maximum A Posteriori (MAP). First put a *prior* belief $p(\theta)$ on the parameters, then choose the θ that is most probable *after* seeing the data:

$$\hat{\theta}_{\text{MAP}} = \arg\max_{\theta} \; p(\theta \mid x) = \arg\max_{\theta} \; p(x \mid \theta)\, p(\theta).$$

The second line uses Bayes' rule $p(\theta \mid x) = \dfrac{p(x \mid \theta)\, p(\theta)}{p(x)}$ and throws away $p(x)$ because it does not contain θ — it cannot change which θ wins the arg max. This is exactly the frequentist-vs-Bayesian split from the Bernoulli MLE/MAP chapter, now aimed at a regression line instead of a coin.



The forward arrow is the model: given θ, data is generated. Learning is the backward inference: given data, recover θ. MLE reads the forward arrow; MAP turns it around with Bayes.

16.2 The probabilistic model of linear regression

We keep the same n training pairs $\{(x_i, y_i)\}_{i=1}^n$ with $x_i \in \mathbb{R}^d$ and $y_i \in \mathbb{R}$. The new idea is to admit that data is *noisy*: the target is a line *plus* a random wobble.

$$\boxed{y_i = \theta^\top x_i + \varepsilon_i}, \qquad \varepsilon_i \sim \mathcal{N}(0, \sigma^2) \text{ (Gaussian noise).}$$

Each ε_i is drawn independently from a bell curve centred at 0. Because adding a constant $\theta^\top x_i$ just slides a bell sideways, the target itself is Gaussian:

$$\boxed{y_i \mid x_i, \theta \;\sim\; \mathcal{N}(\theta^\top x_i,\; \sigma^2)}.$$

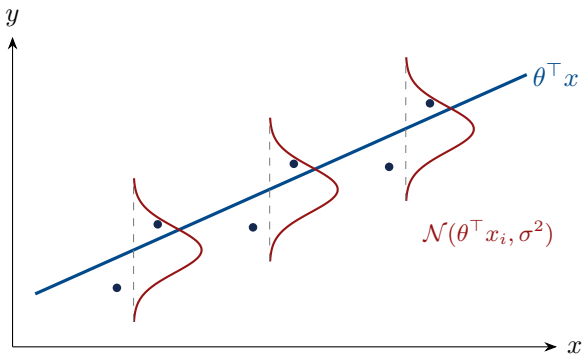
Derivation

Take the mean and variance of y_i, using that expectation is linear and that shifting by a constant does not change spread.

$$\mathbb{E}[y_i] = \mathbb{E}[\theta^\top x_i + \varepsilon_i] = \underbrace{\mathbb{E}[\theta^\top x_i]}_{\theta^\top x_i} + \underbrace{\mathbb{E}[\varepsilon_i]}_{0} = \theta^\top x_i,$$

$$\mathrm{Var}(y_i) = \mathrm{Var}(\theta^\top x_i + \varepsilon_i) = \underbrace{\mathrm{Var}(\theta^\top x_i)}_{0 \text{ (constant)}} + \mathrm{Var}(\varepsilon_i) = \sigma^2.$$

So the line $\theta^\top x_i$ is the *centre* of the bell and σ^2 is its *width*. The regression line is not the data — it is the ridge of the mountain of probability the data falls around.



The model $y_i \mid x_i \sim \mathcal{N}(\theta^\top x_i, \sigma^2)$: at every x a vertical bell curve hangs off the line, and each observed point is one draw from that bell. Fitting the line means sliding it so the bells sit as snugly over the points as possible.

16.3 The Gaussian, univariate and multivariate

Everything below rests on one formula, so we write it once and name its parts.

Univariate Gaussian (one number x, mean μ, variance σ^2):

$$\mathcal{N}(x;\mu,\sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left\{-\frac{(x-\mu)^2}{2\sigma^2}\right\}.$$

Multivariate Gaussian (a vector $x \in \mathbb{R}^d$, mean vector μ, covariance matrix $\Sigma \in \mathbb{R}^{d\times d}$):

$$\mathcal{N}(x;\mu,\Sigma) = \frac{1}{\sqrt{(2\pi)^d\,|\Sigma|}} \exp\left\{-\tfrac{1}{2}(x-\mu)^\top \Sigma^{-1}(x-\mu)\right\}.$$

The exponent is the star of the show. $-\frac{(x-\mu)^2}{2\sigma^2}$ is just a downward parabola in x; exponentiating a downward parabola gives the bell. The multivariate exponent $-\frac{1}{2}(x-\mu)^\top\Sigma^{-1}(x-\mu)$ is the same idea in many dimensions — it is exactly the quadratic form whose gradient and Hessian we computed back in the gradients-and-Hessians chapter. That is why the log of a Gaussian is a clean quadratic, and why maximising it will hand us back least squares.

Take log of any Gaussian and the ugly $\frac{1}{\sqrt{2\pi\sigma^2}}$ becomes a harmless constant, and the exp vanishes, leaving $-\frac{(x-\mu)^2}{2\sigma^2}$ — a plain squared distance. **Log turns a bell into a parabola.** That single fact is the bridge from probability to squared-error loss.

16.4 The star derivation: MLE for the line *is* least squares

Write the probability of a *single* observed target under the model. Because $y_i \mid x_i, \theta \sim \mathcal{N}(\theta^\top x_i, \sigma^2)$, we just plug $\mu = \theta^\top x_i$ into the univariate Gaussian:

$$p(y_i \mid x_i, \theta) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left\{-\frac{(y_i - \theta^\top x_i)^2}{2\sigma^2}\right\}.$$

Derivation

Step 1 — one point to all points (the i.i.d. product). The noises ε_i are independent, so the joint probability of the whole target vector is the product of the per-point probabilities:

$$p(y \mid X, \theta) = \prod_{i=1}^{n} p(y_i \mid x_i, \theta) = \left(\frac{1}{2\pi\sigma^2}\right)^{n/2} \exp\left\{-\sum_{i=1}^{n} \frac{(y_i - \theta^\top x_i)^2}{2\sigma^2}\right\}.$$

This number, read as a function of θ, is the **likelihood**.

Step 2 — take the log (monotone, so the arg max is unchanged).

$$\log p(y \mid X, \theta) = \underbrace{-\frac{n}{2}\log(2\pi\sigma^2)}_{\text{constant in } \theta} - \frac{1}{2\sigma^2}\sum_{i=1}^{n}(y_i - \theta^\top x_i)^2.$$

Step 3 — drop constants, flip the sign. Maximising the log-likelihood over θ only sees the last term, so

$$\log p(y \mid X, \theta) \;\propto\; -\frac{1}{2\sigma^2}\sum_{i=1}^{n}(y_i - \theta^\top x_i)^2, \qquad -\log p(y \mid X, \theta) \;\propto\; \frac{1}{2}\sum_{i=1}^{n}(y_i - \theta^\top x_i)^2.$$

Step 4 — read the answer.

$$\hat{\theta}_{\text{ML}} = \arg\max_\theta \log p(y \mid X, \theta) = \arg\min_\theta \tfrac{1}{2}\sum_{i=1}^{n}(y_i - \theta^\top x_i)^2.$$

That right-hand side is *exactly* the least-squares objective from the linear-regression chapter. Its minimiser is the normal-equation solution $\hat{\theta} = (X^\top X)^{-1}X^\top y$ from the gradients-and-Hessians chapter.

MLE gives the same solution as unregularized least squares. Gaussian noise ⇒ squared loss ⇒ the normal equation. The σ^2 scales the loss but not its minimiser, so you never even need to know the noise variance to fit the line.

Maximising likelihood is a shopkeeper asking: "for which price-line would today's sales be the least surprising?" The line that makes the actual receipts most ordinary — least surprising — is the fit. Surprise is measured in squared rupees off the line, which is why the least-surprising line is also the least-squares line.

Watch out

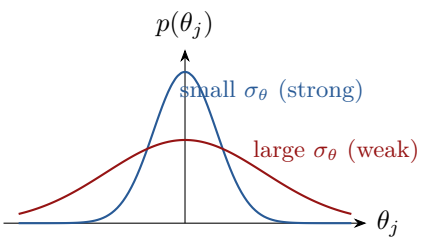
The equivalence needs the noise to be (i) Gaussian, (ii) zero-mean, (iii) the same variance σ^2 for every point (homoscedastic), and (iv) independent across points. Break any of these — say the noise grows with x — and "most probable line" stops being "least squares" and becomes a *weighted* least squares instead. The loss is downstream of the noise model; it was never a free choice.

16.5 Adding a belief: the MAP estimate

MLE listens only to the data. But often we *know* something before we see a single point — for a well-scaled problem, enormous slopes are implausible; the world is rarely that sensitive. We encode that with a **prior**: a Gaussian centred at zero on the parameter vector.

$$p(\theta) = \mathcal{N}(0,\ \sigma_\theta^2 I) \;\propto\; \exp\left\{-\frac{\theta^\top\theta}{2\sigma_\theta^2}\right\}.$$

Small σ_θ is a *strong* belief ("slopes are surely tiny"); large σ_θ is a *weak*, permissive belief.



The prior on each slope is a bell at 0. A narrow prior (blue) insists slopes stay near zero and will pull the fit hard; a wide prior (red) barely constrains it. This "pull toward zero" is precisely the ridge tax of the regularization chapter, now given a probabilistic name.

The MAP objective is the log-likelihood *plus* the log-prior:

$$J(\theta) = \log p(y \mid X, \theta) + \log p(\theta) = -\frac{1}{2\sigma^2}\sum_{i=1}^{n}(y_i - \theta^\top x_i)^2 - \frac{\theta^\top \theta}{2\sigma_\theta^2}.$$

16.6 The star derivation: MAP for the line *is* ridge

Derivation

Step 1 — write MAP as maximising log-posterior.

$$\hat{\theta}_{\text{MAP}} = \arg\max_{\theta} \Big[-\frac{1}{2\sigma^2}\sum_{i=1}^{n}(y_i - \theta^\top x_i)^2 - \frac{\theta^\top \theta}{2\sigma_\theta^2} \Big].$$

Step 2 — flip to a minimisation and multiply by the constant $2\sigma^2$. Multiplying every term by the positive constant $2\sigma^2$ does not move the optimum:

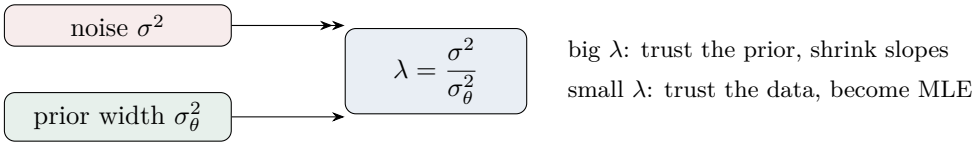
$$\hat{\theta}_{\text{MAP}} = \arg\min_{\theta} \ \ell(\theta), \qquad \ell(\theta) = \sum_{i=1}^{n}(y_i - \theta^\top x_i)^2 + \frac{\sigma^2}{\sigma_\theta^2}\,\theta^\top \theta.$$

Step 3 — name the ratio. Define

$$\boxed{\lambda = \frac{\sigma^2}{\sigma_\theta^2}} \quad \Rightarrow \quad \ell(\theta) = \sum_{i=1}^{n}(y_i - \theta^\top x_i)^2 + \lambda\,\theta^\top \theta.$$

That is the ridge objective of the regularization chapter, and its minimiser is the regularized normal equation $\hat{\theta}_{\text{MAP}} = (X^\top X + \lambda I)^{-1} X^\top y$.

MAP with a Gaussian prior gives ridge regression, with $\lambda = \sigma^2/\sigma_\theta^2$. The regularization strength is a *ratio of variances*: how noisy the data is (σ^2) versus how much you a-priori allow the slopes to roam (σ_θ^2). Noisy data or a tight prior ⇒ big λ ⇒ heavy shrinkage. Clean data or a loose prior ⇒ small λ ⇒ MAP relaxes back toward MLE.



λ is a tug-of-war between two variances. It is not a magic hyper-parameter you invented for the lab — it is the exact exchange rate between data noise and prior confidence.

The regularization chapter introduced ridge as "a tax on big slopes" and told you to tune λ by trying values. This chapter says *where that tax comes from*: it is a Gaussian prior that quietly votes for small slopes, and λ is just how loud that vote is compared to the data.

16.7 Seeing it in code

Both estimators are one linear solve. The only difference is the λI that MAP adds.

In code

```python
import numpy as np

def mle_line(X, y):                  # Gaussian noise  ==>  least squares
    # solves argmin ||X theta - y||^2  ==  (X^T X)^{-1} X^T y
    return np.linalg.solve(X.T @ X, X.T @ y)

def map_line(X, y, lam):             # Gaussian noise + Gaussian prior  ==>  ridge
    d = X.shape[1]
    return np.linalg.solve(X.T @ X + lam * np.eye(d), X.T @ y)

def neg_log_likelihood(X, y, theta, sigma2):
    r = y - X @ theta                # residuals
    n = len(y)
    return 0.5*n*np.log(2*np.pi*sigma2) + (r @ r)/(2*sigma2)
```

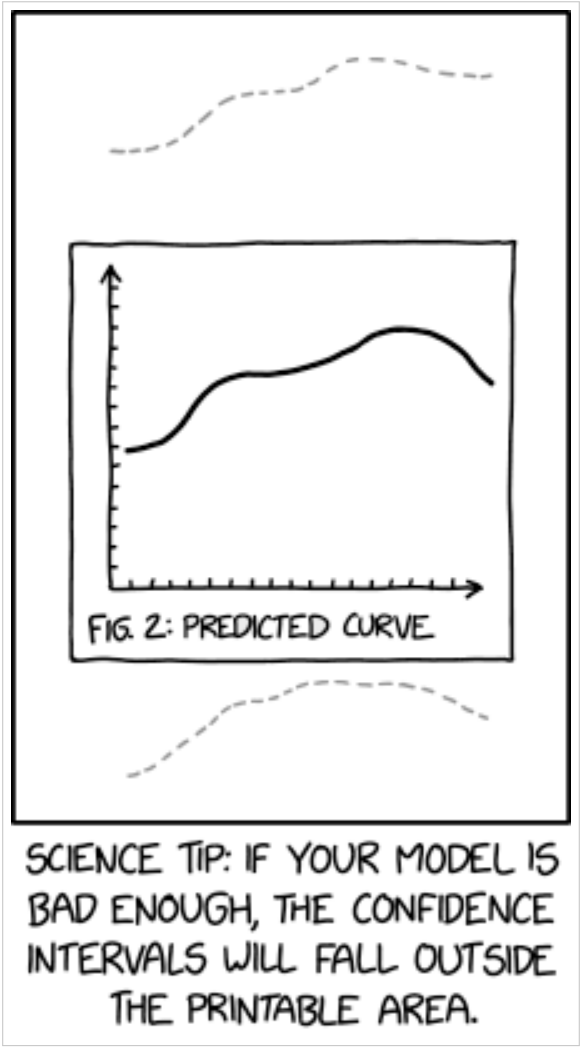
Read the maths straight off the code: `X.T@X` is $X^\top X$, `lam*np.eye(d)` is the λI that the Gaussian prior contributes, and `r@r` is $\sum(y_i - \theta^\top x_i)^2$ — the squared error that the log-Gaussian handed us. Set `lam=0` and `map_line` *is* `mle_line`: MLE is MAP with an infinitely wide prior ($\sigma_\theta \to \infty$, so $\lambda \to 0$).

In code

```python
# lambda is a RATIO of variances, not a free knob:
sigma2       = 0.25      # how noisy is the data
sigma_theta2 = 4.0       # how far do we a-priori let slopes roam
lam = sigma2 / sigma_theta2      # = 0.0625  -> gentle shrinkage
theta_hat = map_line(X, y, lam)
```

Halve the prior width σ_θ^2 and λ doubles — the fit shrinks harder, exactly as the tug-of-war figure promised.

16.8 One xkcd, and why it fits



xkcd #2311 "Confidence Interval" by Randall Munroe — https://xkcd.com/2311/ (CC BY-NC 2.5).

The comic jokes about an interval so wide it admits it knows nothing. That width is σ — the very noise parameter in $y \mid x \sim \mathcal{N}(\theta^\top x, \sigma^2)$. MLE fits the *centre* of that bell; the bell's width is the honesty about how much the data could still surprise us.

16.9 The whole chapter on one line

Probabilistic choice	Becomes this loss	Solved by
Gaussian likelihood $y \mid x \sim \mathcal{N}(\theta^\top x, \sigma^2)$	squared error $\sum(y_i - \theta^\top x_i)^2$	normal equation $(X^\top X)^{-1} X^\top y$
+ Gaussian prior $\theta \sim \mathcal{N}(0, \sigma_\theta^2 I)$	$+\lambda\theta^\top\theta$, $\lambda = \sigma^2/\sigma_\theta^2$	ridge $(X^\top X + \lambda I)^{-1} X^\top y$

> The four-line summary the lecture ends on:
> 1. MLE $\longrightarrow$ the unregularized solution.

2. MAP $\longrightarrow$ the regularized solution.
3. Gaussian likelihood $\longrightarrow$ squared loss.
4. Gaussian prior $\longrightarrow$ L_2 regularization.

Least squares spent twelve chapters thinking it was a geometry trick about perpendicular projections. Turns out it was doing statistics the whole time and never told anyone. Ridge is the same least squares that finally admitted it had opinions before the data arrived.

Check yourself

Q. Show that maximum likelihood for linear regression with Gaussian noise is least squares.
A. Write $p(y_i \mid x_i, \theta) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-(y_i - \theta^\top x_i)^2/2\sigma^2}$, take the i.i.d. product, take log, drop the θ-free constant and the $-1/2\sigma^2$ factor; the arg max of the log-likelihood is the arg min of $\sum (y_i - \theta^\top x_i)^2$.

Q. What is λ in terms of the model, and what does big λ mean?
A. $\lambda = \sigma^2/\sigma_\theta^2$. Big λ means noisy data and/or a tight zero-mean prior, so the slopes get shrunk hard toward 0.

Q. Why does σ^2 not appear in $\hat{\theta}_{\text{ML}}$?
A. It multiplies the whole log-likelihood by a positive constant, which cannot move the arg max. It sets the noise scale, not the best line.

Q. What happens to MAP as $\sigma_\theta \to \infty$?
A. $\lambda \to 0$, the prior becomes flat (no opinion), and MAP collapses onto MLE.

Q. Give the two solved forms.
A. $\hat{\theta}_{\text{ML}} = (X^\top X)^{-1} X^\top y$ and $\hat{\theta}_{\text{MAP}} = (X^\top X + \lambda I)^{-1} X^\top y$.

If a box here did not click, watch it derived. *For the Gaussian-noise-⇒-least-squares step, mathematicalmonk's "MLE for linear regression" does it on a board (https://www.youtube.com/watch?v=n5QZL1B8FaE). For the Gaussian-prior-⇒-ridge step and $\lambda = \sigma^2/\sigma_\theta^2$, ritvikmath's "Why L2 Regularization is Bayesian in Disguise" is the exact video (https://www.youtube.com/watch?v=Ovuszur2uzk). The full hand-picked list is in the closing "Watch these" section.*

17

Basis Functions: Bending a Straight Line

Linear regression fits a straight line, and the world is rarely straight. Lecture 15 shows the trick that lets a *linear* model draw *curves* — without changing a single thing about how we fit it. The secret is to stop feeding the raw input x into the model and instead feed *features of* x: its square, its logarithm, a bump centred near it, a soft step. The model stays linear in the parameters θ; it just becomes non-linear in x. Then, because non-linear models can bend hard enough to memorise noise, the chapter turns to the real question of the whole course: not "can we fit the training data?" but "will we do well on data we have never seen?"

"Linear regression" does not mean "straight line." It means "a weighted sum." If the things being summed are curved pieces — x^2, x^3, little Gaussian bumps — the sum is a curve, yet the maths of fitting is identical to fitting a line. We change *what goes in*, not *how we solve*.

17.1 Recap: the straight-line hypothesis

The hypothesis we have used since the linear-regression chapter, written with the bias term folded in as $x_0 = 1$:

$$y = \theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_d x_d = \sum_{j=0}^{d} \theta_j x_j,$$

and we fit it by minimising the sum of squared errors — the vertical gaps between the line and the points, each squared. Everything from here keeps that squared-error machinery; it only swaps the inputs.

17.2 More complex models from the same linear machine

The inputs x we feed the model can take many forms:

- the **original inputs** x_j;
- **transformations** of inputs ($\log x$, e^x, $\sqrt{x}$, …);
- **polynomial** terms ($x,\ x^2,\ x^3,\ \ldots$);
- **basis expansions** (Gaussian bumps, sigmoids, Fourier terms, …);

- **interactions** between variables, e.g. $x_3 = x_1 \cdot x_2$.

This is what lets us fit *non-linear* datasets with linear-regression techniques.

Think of a spice box (masalo dabbo). The dish (the fit) is still "a spoon of this plus a spoon of that" — a weighted sum. Basis functions just widen what can sit in the compartments: not only raw chilli (x) but roasted chilli (x^2), a pinch located at one spot (a Gaussian bump), a slow-building heat (a sigmoid). The cooking method never changes; the ingredients do.

17.3 The linear basis function model

Replace each raw input x_j by a **basis function** $\varphi_j(x)$ — any function of the input you like. The hypothesis becomes

$$\boxed{h_\theta(x) = \sum_{j=0}^{d} \theta_j \, \varphi_j(x)}.$$

The convention $\varphi_0(x) = 1$ keeps θ_0 as the bias, exactly as $x_0 = 1$ did. The *simplest* choice, $\varphi_j(x) = x_j$, gives back ordinary linear regression — so the straight line is just one member of a much larger family.

> **Linear in parameters, non-linear in inputs.** The model is called "linear" because it is linear in θ — each θ_j multiplies its feature and the results are added. The features $\varphi_j(x)$ may bend as wildly as they like. The cost is still a convex bowl in θ, so the normal equation and gradient descent work unchanged.

The design matrix makes it literally the same problem

Stack the basis outputs into a matrix Φ (rows are examples, columns are features):

$$\Phi = \begin{bmatrix} \varphi_0(x_1) & \varphi_1(x_1) & \cdots & \varphi_d(x_1) \\ \varphi_0(x_2) & \varphi_1(x_2) & \cdots & \varphi_d(x_2) \\ \vdots & \vdots & & \vdots \\ \varphi_0(x_n) & \varphi_1(x_n) & \cdots & \varphi_d(x_n) \end{bmatrix}, \qquad h = \Phi\theta.$$

Then the least-squares solution is the normal equation from the gradients-and-Hessians chapter, with X replaced by Φ:

$$\hat{\theta} = (\Phi^\top \Phi)^{-1} \Phi^\top y, \qquad \text{ridge:} \quad \hat{\theta} = (\Phi^\top \Phi + \lambda I)^{-1} \Phi^\top y.$$

Once the data is passed through the basis functions, fitting the fancy curved model is *exactly* the same computation as fitting the plain line — you just built a wider table Φ first. Nothing new to derive; only new columns.

17.4 A zoo of basis functions

Polynomial basis: $\varphi_j(x) = x^j$

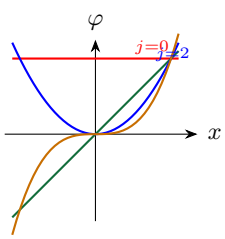
Powers of x. These are **global**: a small change in x nudges *every* basis function at once, because x^j never settles down.

Gaussian basis: $\varphi_j(x) = \exp\left\{-\dfrac{(x-\mu_j)^2}{2s^2}\right\}$

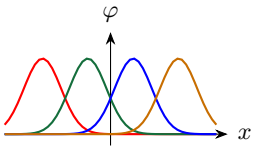
A bump centred at μ_j with width s. These are **local**: far from μ_j the bump is essentially zero, so moving x a little only wakes up the bumps near it. Here μ_j controls *location* and s controls *scale (width)*.

Sigmoidal basis: $\varphi_j(x) = \sigma\left(\dfrac{x-\mu_j}{s}\right)$, **with** $\sigma(a) = \dfrac{1}{1+e^{-a}}$

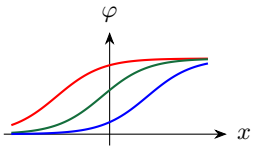
A soft step from 0 up to 1, switching near μ_j. Also **local** in the sense that the interesting action (the transition) happens in a small band; μ_j sets its location and s sets its scale (slope).



Polynomial x^j — global



Gaussian bumps — local



Sigmoids — local step

Three families. Polynomials keep growing, so every one of them reacts to any change in x (global). Gaussian bumps and sigmoid steps are flat almost everywhere except a small band, so only the nearby ones react (local). Locality is what lets a model fit a wiggle here without disturbing the fit there.

17.5 Why polynomial is "global" and Gaussian is "local" (the derivation)

The lecture makes "local vs global" precise with a one-line perturbation argument: nudge x by a tiny δ and see how much each basis function moves.

Derivation

Polynomial. For $\varphi_j(x) = x^j$, the derivative is $\varphi_j'(x) = j\,x^{j-1}$. Expand a small step $x \to x+\delta$:

$$(x+\delta)^j = x^j + j\,x^{j-1}\delta + O(\delta^2),$$

$$\Delta\varphi_j = (x+\delta)^j - x^j \approx j\,x^{j-1}\delta = \varphi_j'(x)\,\delta.$$

For a polynomial, $\varphi_j'(x) = jx^{j-1}$ is nonzero across the whole range of x, so a small change in x moves *all* φ_j by a nonzero amount. Hence it is **global**.

Gaussian. For $\varphi_j(x) = \exp\{-\frac{(x-\mu_j)^2}{2s^2}\}$, the same first-order rule gives

$$\Delta\varphi_j \approx \varphi_j'(x)\,\delta.$$

But if x is far from the centre μ_j, then $\varphi_j(x) \approx 0$ *and* its derivative is also tiny. So only the basis functions whose centres are *near* x change. Hence it is **local**.

Watch out

Global is not automatically bad — if the true trend really is a smooth global shape, a polynomial captures it with very few terms. But global also means a data point way out on the right can tilt the fit way out on the left. Local bases avoid that: fixing a wiggle in one region leaves the others alone. This is the same instability that made high-degree polynomials blow up at the edges in the regularization chapter.

17.6 Choosing a basis: match the shape of the data

There is no universally best basis; you pick one whose *shape* matches the structure you expect in the data.

Data property	Sensible basis
Smooth, global trend	Polynomial
Local nonlinearities	Gaussian
Periodic (repeating)	Fourier
Localised-frequency changes	Wavelets
Saturation / monotonic transitions	Sigmoid
Very complex	Neural networks

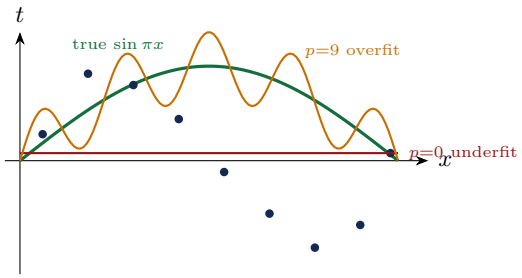
A neural network is what the last row admits when it gives up hand-picking basis functions and says "fine, let the model learn its own φ_j." Every fancy deep model in the course after this is, at heart, a basis-function model that chooses its own basis.

17.7 Fitting a polynomial curve with a linear model

Take the polynomial basis and fit

$$y = \theta_0 x_0 + \theta_1 x + \theta_2 x^2 + \cdots + \theta_p x^p = \sum_{j=0}^{p} \theta_j x^j.$$

On data generated as $t = \sin(\pi x) + \varepsilon$ (a sine plus noise), the *degree* p decides everything:



Same data, three degrees. $p = 0$ is a flat line that ignores the shape (underfitting). $p = 9$ threads every noisy point and oscillates wildly between them (overfitting). A moderate degree tracks the green truth. The model got more powerful and less trustworthy — that tension is the rest of the chapter.

In code

```python
import numpy as np

def poly_design(x, p):                # columns: 1, x, x^2, ..., x^p
    return np.vander(x, p+1, increasing=True)      # this is Phi

def gauss_design(x, centers, s):    # columns: bumps at each center (+ bias)
    B = np.exp(-(x[:,None] - centers[None,:])**2 / (2*s*s))
    return np.hstack([np.ones((len(x),1)), B])     # phi_0 = 1 for the bias

# fit is the SAME normal equation, just with Phi instead of X:
Phi   = poly_design(x_train, p=3)
theta = np.linalg.solve(Phi.T @ Phi, Phi.T @ y_train)
y_hat = poly_design(x_query, p=3) @ theta
```

`np.vander` builds the polynomial design matrix Φ in one call; swap in `gauss_design` and the fitting line does not change at all. That is the whole point of basis functions in code: *new columns, same solve.*

17.8 The central challenge of machine learning: generalization

More flexibility means we can always drive the *training* error to zero (enough basis functions will thread every point). But that is not the goal.

Generalization. The model must perform well on *new, previously unseen* inputs — not just the ones it trained on. The ability to do well on unobserved data is called *generalization*, and it is the actual objective of machine learning.

ML differs from pure optimisation. Optimisation only wants the training objective small. ML wants the *generalization error* (a.k.a. *test error*) small — the *expected* error on a new input drawn from the same source as the data. In linear regression with squared error the test error is

$$\frac{1}{2\,n^{(\text{test})}}\left\|X^{(\text{test})}\theta - y^{(\text{test})}\right\|_2^2,$$

the twin of the training error $\frac{1}{2\,n^{(\text{train})}}\left\|X^{(\text{train})}\theta - y^{(\text{train})}\right\|_2^2$ we actually minimise.

We can only *estimate* the generalization error, by measuring performance on a **test set** collected separately from the training set. We train by minimising the training error, but we care about the test error — and when the model only ever sees the training set, the honest question is: how will it do on the test case?

17.9 Statistical learning theory: the assumptions that make it possible

Why should doing well on the training set tell us *anything* about unseen data? Only because of assumptions linking the two sets.

The i.i.d. assumptions.

- Training and test data arise from the *same process*.
- Examples within each set are *independent*.
- Training and test sets are *identically distributed*.

We call this shared source the *data-generating distribution* p_{data}. The probabilistic framework plus i.i.d. is exactly what lets us reason about the relationship between training error and test error at all.

Derivation

Why are training and test errors unequal?

- For a *fixed* parameter θ chosen in advance, the expected training error equals the expected test error — both are averages over the same $p(x, y)$. If we sampled a training and a test set and used some pre-committed θ, they would match in expectation.
- **But we do not fix θ in advance.** We sample the training set *first*, then choose θ to make *that* set's error small. The parameters are tuned to the very set we then score them on.

So training error is an *optimistic* estimate: the model has already bent toward the training points, which is why test error is typically higher. The gap between them is the price of having peeked.

Grading your own practice paper with the answer key open gives a flattering "training score." The exam is the test set: same syllabus (identically distributed), new questions (unseen), no key. The gap between your practice score and your exam score is precisely the generalization gap.

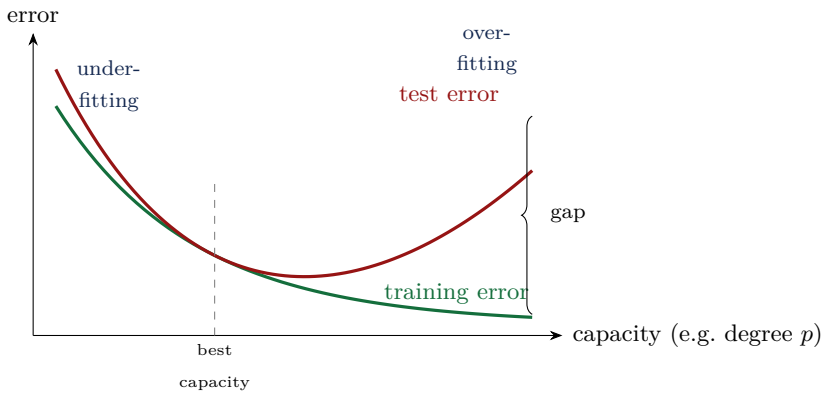
17.10 Under-fitting, over-fitting, and capacity

Two forces decide how well an algorithm does:

1. make the **training error** small; and
2. make the **gap** between training and test error small.

They map onto the two failure modes:

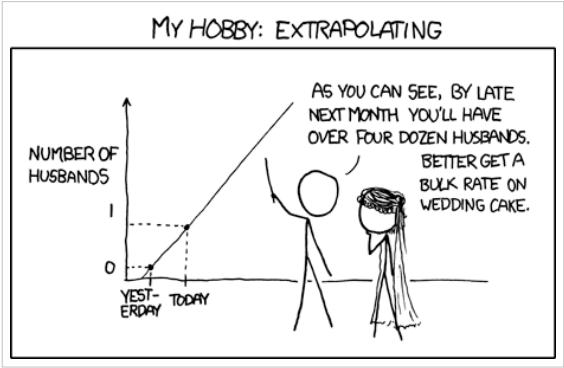
Underfitting: the model cannot even get a low training error — too rigid (e.g. $p = 0$ on a sine). **Overfitting:** the training error is low but the gap to the test error is large — too flexible (e.g. $p = 9$ threading noise). We steer between them by altering the model's **capacity** (how flexible it is — here, the number/kind of basis functions, or the ridge λ of the ridge and MLE/MAP chapters).



The signature picture of the whole field. Training error (green) falls forever as capacity grows. Test error (red) falls, bottoms out, then rises as the model starts fitting noise. Left of the dip: underfitting. Right of the dip: overfitting, where the green–red gap balloons. Ridge and choosing the basis are how we sit at the bottom.

More power is not more skill. A model that memorises the training set has low training error and no idea what to do with a new point. The job is to be *just* flexible enough to catch the real pattern and *not so* flexible that it also catches the noise.

17.11 One xkcd, and why it fits



xkcd #605 "Extrapolating" by Randall Munroe — https://xkcd.com/605/ (CC BY-NC 2.5).

Fit a trend to a few points, extend it confidently, and conclude the impossible. That is overfitting-plus-extrapolation in one panel: a flexible curve nailing the observed dots tells you almost nothing about the region where you have no data. Generalization is the discipline of not doing what this comic does.

17.12 The chapter in one breath

1. Swap raw inputs for basis functions $\varphi_j(x)$: the model $h_\theta(x) = \sum_j \theta_j \varphi_j(x)$ stays linear in θ, so fitting is the same normal equation with Φ in place of X.
2. Polynomials are global; Gaussians and sigmoids are local — shown by $\Delta\varphi_j \approx \varphi_j'(x)\delta$.
3. Pick the basis by the shape of the data (smooth → polynomial, bumpy → Gaussian, periodic → Fourier, …).
4. Flexibility lets us fit non-linear data — and lets us overfit. The real target is *generalization*:

low expected error on unseen data, estimated on a held-out test set.

5. i.i.d. from one p_{data} is what links train to test; training error is optimistic because θ was tuned on it.

6. Balance underfitting against overfitting by tuning *capacity* — degree, basis choice, or the ridge λ of the last two chapters.

Check yourself

Q. Why is $h_\theta(x) = \sum_j \theta_j \varphi_j(x)$ still called a linear model?
A. It is linear in the parameters θ; the φ_j may be non-linear in x. The cost stays convex in θ, so the normal equation still applies.

Q. What does $\varphi_0(x) = 1$ do?
A. It provides the bias term: $\theta_0 \varphi_0(x) = \theta_0$.

Q. Prove polynomials are global and Gaussians local.
A. $\Delta\varphi_j \approx \varphi_j'(x)\delta$. For x^j, $\varphi_j' = jx^{j-1} \neq 0$ everywhere $\Rightarrow$ all bases react (global). For a Gaussian, far from μ_j both φ_j and φ_j' are $\approx 0 \Rightarrow$ only nearby bases react (local).

Q. Define generalization error and how we estimate it.
A. The expected error on a new input from p_{data}; estimated by the error on a separately collected test set.

Q. Why is training error usually below test error?
A. θ is chosen to minimise the training error *after* seeing that set, so it is an optimistic estimate; for a θ fixed in advance the two would match in expectation.

Q. Underfitting vs overfitting, and the knob?
A. Underfitting: high training error (too rigid). Overfitting: low training error but large train–test gap (too flexible). Control via capacity — basis choice/degree or ridge λ.

Two videos that make this chapter stick. *Herman Kamper's "Polynomial regression and basis functions" shows the design matrix Φ being built column by column (https://www.youtube.com/watch?v=TSFMepJbHa0); Kilian Weinberger's Cornell lecture on the bias–variance decomposition derives the U-shaped test-error curve in full (https://www.youtube.com/watch?v=zUJbROOWavo). More in the closing "Watch these" section.*

18

MLE Across Distributions: A Derivation Catalog

The coin chapters derived one maximum-likelihood estimate — the Bernoulli $\hat{\theta} = s/n$. In an exam the distribution will *change*: Gaussian, Poisson, exponential, uniform, or one you have never memorised. The good news, and the whole point of this chapter, is that the **recipe never changes**. Memorise the five steps, not the answers, and you can derive the MLE for any distribution they hand you.

Do not try to memorise "$\hat{\lambda} = \bar{x}$ for Poisson, $\hat{p} = 1/\bar{x}$ for geometric". Memorise the *procedure*: write the likelihood, log it, differentiate, set to zero, solve. Every distribution below is the same five steps with a different formula plugged in.

18.1 The universal recipe

Given i.i.d. data $x_1, \ldots, x_n$ from a distribution with parameter θ:

1. **Likelihood** — multiply the per-point density/mass: $L(\theta) = \prod_{i=1}^{n} p(x_i \mid \theta)$.
2. **Log** — turn the product into a sum: $\ell(\theta) = \sum_{i=1}^{n} \log p(x_i \mid \theta)$.
3. **Differentiate** — $\dfrac{d\ell}{d\theta}$ (or the gradient for several parameters).
4. **Set to zero** — $\dfrac{d\ell}{d\theta} = 0$ (the score equation).
5. **Solve** for θ, and check it is a *maximum* (second derivative < 0, or a boundary/support argument when the derivative never vanishes).

Watch out

Step 5 is not a formality. For most distributions the derivative gives the answer, but for the *uniform* the derivative never hits zero and the maximum sits on the boundary of the support; for the *Laplace* the objective is not differentiable at the median. When the professor "changes the terms," these are exactly the traps being set.

18.2 The discrete distributions

Bernoulli — one coin (recap)

$p(x \mid \theta) = \theta^x(1-\theta)^{1-x}$, $x \in \{0,1\}$, $s = \sum x_i$.

Derivation

$\ell = \sum[x_i \log\theta + (1-x_i)\log(1-\theta)] = s\log\theta + (n-s)\log(1-\theta)$. $\dfrac{d\ell}{d\theta} = \dfrac{s}{\theta} - \dfrac{n-s}{1-\theta} = 0 \Rightarrow$ $\boxed{\hat\theta = \dfrac{s}{n}}$.

Binomial — m-toss batches

$p(k \mid \theta) = \binom{m}{k}\theta^k(1-\theta)^{m-k}$; observe counts $k_1, \dots, k_n$.

Derivation

The $\binom{m}{k}$ has no θ, so it drops in step 3. $\ell \overset{\theta}{=} \sum[k_i \log\theta + (m-k_i)\log(1-\theta)]$. Setting the derivative to zero, $\boxed{\hat\theta = \dfrac{\sum k_i}{nm}}$ — total successes over total trials. Same estimator as Bernoulli, just with m trials per observation.

Categorical / Multinomial — a K-sided die (a Lagrange multiplier)

K outcomes with probabilities $p_1, \dots, p_K$, $\sum_k p_k = 1$; counts $n_1, \dots, n_K$, $N = \sum n_k$.

Derivation

$\ell = \sum_k n_k \log p_k$, maximised *subject to* $\sum_k p_k = 1$. Use a Lagrange multiplier:

$$\mathcal{L} = \sum_k n_k \log p_k + \lambda\Big(1 - \sum_k p_k\Big), \qquad \frac{\partial\mathcal{L}}{\partial p_k} = \frac{n_k}{p_k} - \lambda = 0 \;\Rightarrow\; p_k = \frac{n_k}{\lambda}.$$

The constraint $\sum_k p_k = 1$ forces $\lambda = \sum_k n_k = N$, so $\boxed{\hat p_k = \dfrac{n_k}{N}}$ — each outcome's observed frequency. (Add a Dirichlet prior and this becomes the smoothed $\frac{n_k+\alpha}{N+\sum\alpha}$ of Chapter 15.)

Poisson — counts per interval

$p(k \mid \lambda) = \dfrac{\lambda^k e^{-\lambda}}{k!}$, $k = 0, 1, 2, \dots$

Derivation

$\ell = \sum[k_i \log\lambda - \lambda - \log k_i!]$. The $\log k_i!$ has no λ. $\dfrac{d\ell}{d\lambda} = \dfrac{\sum k_i}{\lambda} - n = 0 \Rightarrow$ $\boxed{\hat\lambda = \dfrac{1}{n}\sum k_i = \bar x}$ — the sample mean. (Sensible: for Poisson the mean *is* λ.)

Geometric — trials until the first success

$P(X = k \mid p) = (1-p)^{k-1}p,\ k = 1, 2, \ldots$

Derivation

$\ell = \sum[(k_i - 1)\log(1-p) + \log p] = (\sum k_i - n)\log(1-p) + n\log p.$ $\dfrac{d\ell}{dp} = -\dfrac{\sum k_i - n}{1-p} + \dfrac{n}{p} = 0.$

Cross-multiplying, $n(1-p) = p(\sum k_i - n) \Rightarrow n = p\sum k_i \Rightarrow$ $\boxed{\hat{p} = \dfrac{n}{\sum k_i} = \dfrac{1}{\bar{x}}}$.

18.3 The continuous distributions

Gaussian, unknown mean (variance known)

$p(x \mid \mu) = \dfrac{1}{\sqrt{2\pi\sigma^2}} \exp\!\left(-\tfrac{(x-\mu)^2}{2\sigma^2}\right).$

Derivation

$\ell = \text{const} - \dfrac{1}{2\sigma^2}\sum(x_i - \mu)^2.$ $\dfrac{d\ell}{d\mu} = \dfrac{1}{\sigma^2}\sum(x_i - \mu) = 0 \Rightarrow$ $\boxed{\hat{\mu} = \dfrac{1}{n}\sum x_i = \bar{x}}$. Maximising a Gaussian likelihood over the mean *is* minimising squared error — the exact fact behind least-squares regression (Chapter 14).

Gaussian, unknown variance (mean known)

Derivation

Treat $v = \sigma^2$ as the parameter. $\ell = -\dfrac{n}{2}\log(2\pi v) - \dfrac{1}{2v}\sum(x_i - \mu)^2.$ $\dfrac{d\ell}{dv} = -\dfrac{n}{2v} + \dfrac{1}{2v^2}\sum(x_i - \mu)^2 = 0 \Rightarrow$ $\boxed{\hat{\sigma}^2 = \dfrac{1}{n}\sum(x_i - \mu)^2}$.

Gaussian, both unknown (two-parameter MLE)

Derivation

Set both partials to zero. $\partial_\mu \ell = 0$ gives $\hat{\mu} = \bar{x}$ as above; substituting into $\partial_v \ell = 0$ gives

$$\boxed{\hat{\mu} = \bar{x}, \qquad \hat{\sigma}^2 = \frac{1}{n}\sum(x_i - \bar{x})^2}.$$

Watch out

The MLE variance divides by n, not $n-1$ — so it is *biased* (it slightly under-estimates σ^2). The "unbiased sample variance" with $n-1$ (Bessel's correction) is a *different* estimator, not the MLE. Exam trap: MLE ⇒ divide by n.

Exponential — waiting times

$p(x \mid \lambda) = \lambda e^{-\lambda x}$, $x \geq 0$.

Derivation

$\ell = n \log \lambda - \lambda \sum x_i$. $\frac{d\ell}{d\lambda} = \frac{n}{\lambda} - \sum x_i = 0 \Rightarrow \boxed{\hat{\lambda} = \frac{n}{\sum x_i} = \frac{1}{\bar{x}}}$ — the rate is one over the mean waiting time.

Uniform $U(0, \theta)$ — the derivative-free MLE (a rare gem)

$p(x \mid \theta) = \frac{1}{\theta}$ for $0 \leq x \leq \theta$, else 0.

Derivation

$L(\theta) = \prod \frac{1}{\theta} = \frac{1}{\theta^n}$ — but *only* when $\theta \geq x_i$ for *every* i; otherwise some factor is 0 and $L = 0$. So $L(\theta) = \theta^{-n}$ for $\theta \geq \max_i x_i$ and 0 below. Since θ^{-n} *decreases* in θ, we want the *smallest* legal θ:

$$\boxed{\hat{\theta} = \max_i x_i}.$$

Watch out

Here $\frac{d\ell}{d\theta} = -n/\theta$ is *never* zero — step 4 fails. The maximum is a *boundary* of the support, not a stationary point. If a question's likelihood has the parameter in its *range* ($0 \leq x \leq \theta$), suspect a boundary answer like $\max_i x_i$ (or $\min_i x_i$), and argue with monotonicity, not calculus.

Laplace — the MLE is the median (another gem)

$p(x \mid \mu, b) = \frac{1}{2b} \exp\big(-\frac{|x-\mu|}{b}\big)$.

Derivation

$\ell = -n \log(2b) - \frac{1}{b} \sum |x_i - \mu|$. Over μ, maximising ℓ means *minimising* $\sum |x_i - \mu|$ — the sum of absolute deviations. That is minimised at the **median**: $\boxed{\hat{\mu} = \text{median}(x_i)}$. (Contrast the Gaussian, whose $\sum (x_i - \mu)^2$ is minimised at the *mean*. Absolute-value loss → median; squared loss → mean.)

18.4 When they "change the terms": a distribution you have not memorised

The exam favourite is a density you have never seen, to prove you know the recipe rather than the answer. Example: the **Rayleigh** distribution, $p(x \mid \sigma) = \frac{x}{\sigma^2} \exp\big(-\frac{x^2}{2\sigma^2}\big)$, $x \geq 0$.

Derivation

Step 1–2 (log-likelihood).

$$\ell(\sigma) = \sum \left[\log x_i - 2\log\sigma - \frac{x_i^2}{2\sigma^2} \right] = \underbrace{\sum \log x_i}_{\text{no } \sigma} - 2n\log\sigma - \frac{1}{2\sigma^2}\sum x_i^2.$$

Step 3–4 (differentiate, set to zero).

$$\frac{d\ell}{d\sigma} = -\frac{2n}{\sigma} + \frac{1}{\sigma^3}\sum x_i^2 = 0.$$

Step 5 (solve). Multiply by σ^3: $-2n\sigma^2 + \sum x_i^2 = 0 \Rightarrow \boxed{\hat{\sigma}^2 = \frac{1}{2n}\sum x_i^2}$.

You never memorised Rayleigh — you *derived* it. That is the whole skill being tested.

Reading a new density: whatever multiplies inside an exp becomes a clean term after log; anything with no parameter in it (like $\log x_i$ or $\log k_i!$ or a $\binom{m}{k}$) is a constant and *vanishes* when you differentiate. Strip those first and the rest is one derivative.

18.5 A quick-reference table

Distribution	Density / mass	MLE
Bernoulli(θ)	$\theta^x(1-\theta)^{1-x}$	$\hat{\theta} = s/n$
Binomial(m, θ)	$\binom{m}{k}\theta^k(1-\theta)^{m-k}$	$\hat{\theta} = \frac{\sum k_i}{nm}$
Categorical	$\prod_k p_k^{n_k}$, $\sum p_k = 1$	$\hat{p}_k = n_k/N$
Poisson(λ)	$\lambda^k e^{-\lambda}/k!$	$\hat{\lambda} = \bar{x}$
Geometric(p)	$(1-p)^{k-1}p$	$\hat{p} = 1/\bar{x}$
Gaussian mean	$\mathcal{N}(\mu, \sigma^2)$	$\hat{\mu} = \bar{x}$
Gaussian var	$\mathcal{N}(\mu, \sigma^2)$	$\hat{\sigma}^2 = \frac{1}{n}\sum(x_i - \mu)^2$
Gaussian both	$\mathcal{N}(\mu, \sigma^2)$	$\hat{\mu} = \bar{x}$, $\hat{\sigma}^2 = \frac{1}{n}\sum(x_i - \bar{x})^2$
Exponential(λ)	$\lambda e^{-\lambda x}$	$\hat{\lambda} = 1/\bar{x}$
Uniform($0, \theta$)	$1/\theta$, $0 \le x \le \theta$	$\hat{\theta} = \max_i x_i$
Laplace(μ, b)	$\frac{1}{2b}e^{-\lvert x-\mu\rvert/b}$	$\hat{\mu} = \text{median}(x_i)$
Rayleigh(σ)	$\frac{x}{\sigma^2}e^{-x^2/2\sigma^2}$	$\hat{\sigma}^2 = \frac{1}{2n}\sum x_i^2$

Notice how many answers are just $\bar{x}$ in disguise. Poisson, Gaussian-mean, exponential (reciprocal), geometric (reciprocal) — the sample mean is the MLE's favourite child. When in doubt on an unseen distribution, differentiate honestly, but do not be surprised if $\bar{x}$ walks out again.

18.6 Practice: derive these yourself

Detailed solutions follow immediately — try each with the five-step recipe before reading.

Check yourself

Q. Shifted exponential. $p(x \mid \lambda) = \lambda e^{-\lambda(x-a)}$ **for** $x \geq a$**, with** a ***known.*** **Find** $\hat{\lambda}$**.**
A. $\ell = n \log \lambda - \lambda \sum (x_i - a)$; $\frac{d\ell}{d\lambda} = \frac{n}{\lambda} - \sum (x_i - a) = 0$; $\hat{\lambda} = \dfrac{n}{\sum (x_i - a)} = \dfrac{1}{\overline{x - a}}$. (Same as exponential on the shifted data.)

Q. Uniform $U(0, \theta)$ **numeric. Data** $\{0.2, 0.9, 0.5, 0.7\}$**. Give** $\hat{\theta}$ **and say why calculus fails.**
A. $\hat{\theta} = \max = 0.9$. $\frac{d\ell}{d\theta} = -n/\theta \neq 0$ ever, so the max is the boundary $\theta = \max_i x_i$, argued by monotonic decrease of θ^{-n}.

Q. Scaled Poisson terms. n **counts with** $\hat{\lambda} = \bar{x}$**. If every count doubles, what happens to** $\hat{\lambda}$**?**
A. $\bar{x}$ doubles, so $\hat{\lambda}$ doubles — the MLE tracks the sample mean linearly.

Q. Weibull-ish (given shape). $p(x \mid \lambda) = \lambda k x^{k-1} e^{-\lambda x^k}$**,** k **known,** $x \geq 0$**. Find** $\hat{\lambda}$**.**
A. $\ell = n \log \lambda + (\text{no-}\lambda) - \lambda \sum x_i^k$; $\frac{d\ell}{d\lambda} = \frac{n}{\lambda} - \sum x_i^k = 0$; $\hat{\lambda} = \dfrac{n}{\sum x_i^k}$ (exponential is the $k = 1$ case).

Q. Bernoulli with a re-parameterisation. A coin has $P(\textbf{head}) = e^{\eta}/(1+e^{\eta})$ **(logistic in** η**). With** s **heads in** n**, find** $\hat{\eta}$**.**
A. By invariance of the MLE, $\hat{\eta} = \log \frac{\hat{\theta}}{1-\hat{\theta}} = \log \frac{s}{n-s}$ — the log-odds of the observed fraction. (MLEs transform: $\widehat{g(\theta)} = g(\hat{\theta})$.)

Part VI

Insem 2: generalization and classification

19

Capacity, Bias and Variance

Every model in the first half of this book was fitted and then admired on the same data. This chapter asks the question that ruins that habit: *how good will the model be on data it has never seen?* The answer comes in three steps. A model has a **capacity**, a range of shapes it can bend into. The expected error on new data splits *exactly* into three pieces, **noise**, **bias**^{2} and **variance**, and capacity trades the last two against each other. **Regularization** is the dial that controls the trade. Every step is derived; sir's slides are at the end for reference.

19.1 Capacity and the hypothesis space

The hypothesis space

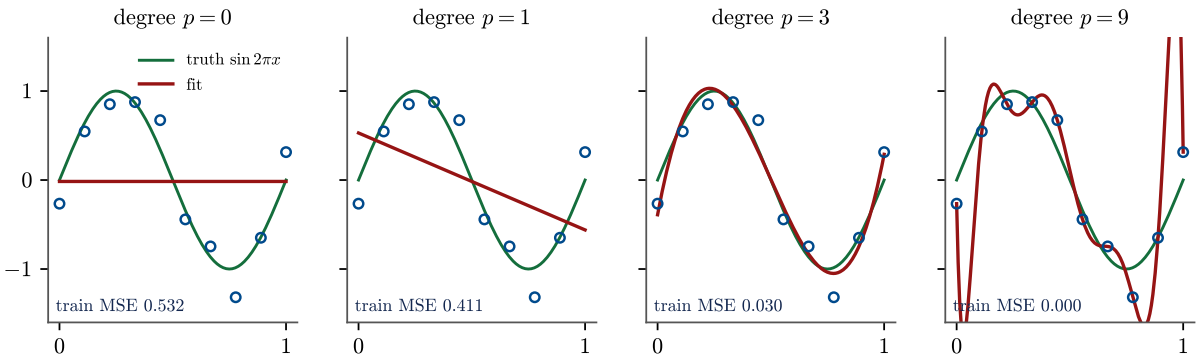
A learning algorithm does not search over all possible functions. It searches a fixed family, the **hypothesis space** $\mathcal{H}$. Linear regression with features $(1, x)$ searches straight lines: $\mathcal{H} = \{\theta_0 + \theta_1 x\}$. Add powers of x,

$$h_{\boldsymbol{\theta}}(x) = \theta_0 + \theta_1 x + \theta_2 x^2 + \cdots + \theta_p x^p = \sum_{j=0}^{p} \theta_j x^j,$$

and the family grows: every straight line is also a cubic with $\theta_2 = \theta_3 = 0$, but not the other way round. Note that this is still *linear regression*: the model is linear in the unknowns θ_j, even though it is curved in x.

Capacity

> The **capacity** of a model is its ability to fit a variety of functions: how large and flexible its hypothesis space is.
>
> - **Too little capacity**: cannot even fit the training data. *Underfitting*, also called high bias.
> - **Too much capacity**: fits the training data, including its noise, and fails on new data. *Overfitting*, also called high variance.
> - **Appropriate capacity**: matched to the true complexity of the task *and* to the amount of data.

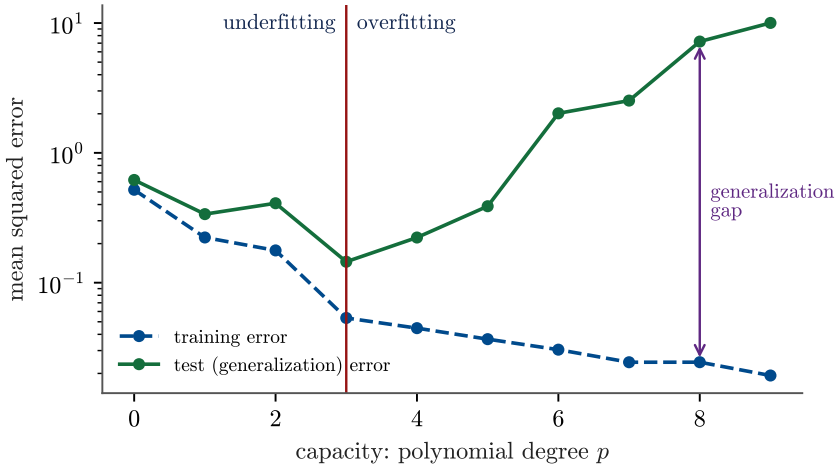


Ten noisy points from $y = \sin 2\pi x + \varepsilon$, fitted by least squares with degree $p = 0, 1, 3, 9$. Training error falls every time p grows; at $p = 9$ the curve passes through all ten points (training error 0) and is useless between them.

Mental model

Why does $p = 9$ fit 10 points exactly? It has 10 unknowns $\theta_0, \ldots, \theta_9$ and we have 10 equations (one per point). Ten equations in ten unknowns usually have an exact solution, so the curve goes through every point, noise included. A model that can fit *anything* tells you nothing when it fits *this*.

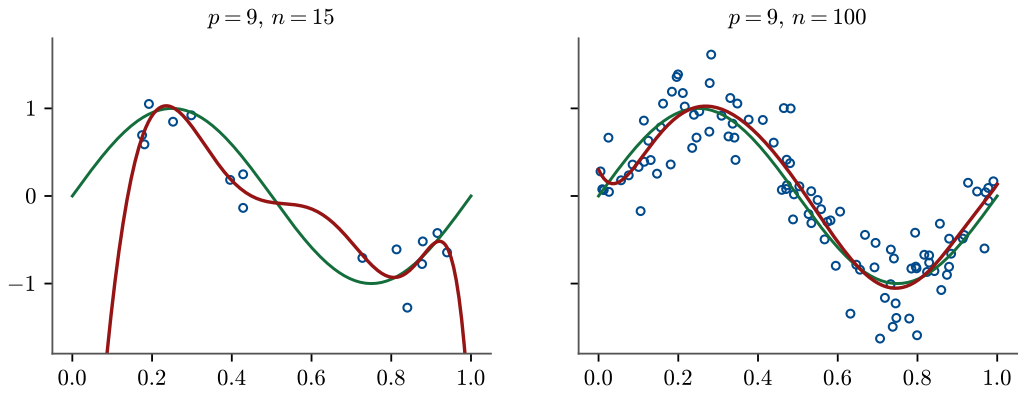
The U-shaped curve



Averaged over 200 random training sets of 10 points: training error (dashed) only goes down as degree rises. Test error (solid) falls, then rises: a U. The best capacity is at the bottom of the U; the gap between the curves is the generalization gap.

Three ways to reduce overfitting

1. **Reduce capacity**: fewer features, lower degree.
2. **Regularize**: keep the features but charge for large weights (ridge, LASSO).
3. **Get more data**: sir's rule of thumb is 5 to 10 times as many examples as parameters.



The same degree-9 model with 15 and with 100 points. More data pins the curve down. "Too much capacity" is always relative to the amount of data.

19.2 From the textbook: training MSE vs test MSE (ISLR §2.2.1)

ISLR states the problem in one line: we fit on training data, but what we care about is the error on *new* data,

$$\text{training MSE} = \frac{1}{n}\sum_{i=1}^{n}\big(y_i - \hat{f}(x_i)\big)^2, \qquad \text{test MSE} = \text{Ave}\,\big(y_0 - \hat{f}(x_0)\big)^2 \text{ over new } (x_0, y_0).$$

ISLR calls capacity **flexibility** and draws three simulations: a truly non-linear truth (best flexibility in the middle), a nearly linear truth (best flexibility low, the straight line wins), and a very wiggly truth with little noise (best flexibility high). The lesson: *the right capacity depends on the truth and on the data.* Training MSE always rewards more flexibility, so it cannot pick. ISLR also stresses that the test MSE can never fall below $\text{Var}(\varepsilon)$, the irreducible error, which is the σ^2 term of the decomposition derived below.

19.3 The effective number of parameters

Ask before reading on

Ridge regression with an enormous λ still has $p + 1$ coefficients, but they are all squashed near zero. Does it really have $p + 1$ degrees of freedom?

Counting coefficients stops measuring capacity once there is a penalty. Sir's slide gives a better count.

Many fitting methods produce fitted values that are a fixed matrix times the targets:

$$\hat{\mathbf{y}} = \mathbf{S}\,\mathbf{y}, \qquad \mathbf{S} \in \mathbb{R}^{n\times n} \text{ depends only on the inputs.}$$

Such a method is a **linear smoother**. Its **effective number of parameters** is the trace (sum of the diagonal entries) of $\mathbf{S}$: $d(\mathbf{S}) = \text{tr}(\mathbf{S})$.

S_{ii} measures how much observation i's own target y_i pulls on its own fitted value $\hat{y}_i$. Add these up and you get how much freedom the model has to chase the data.

Ordinary least squares: exactly d

Least squares gives $\hat{\boldsymbol{\theta}} = (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{y}$, so $\hat{\mathbf{y}} = \mathbf{X}\hat{\boldsymbol{\theta}} = \underbrace{\mathbf{X}(\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top}_{\mathbf{S}=\mathbf{H}}\mathbf{y}$ (the "hat" matrix). The trace has the cycling property $\operatorname{tr}(\mathbf{AB}) = \operatorname{tr}(\mathbf{BA})$. Move $\mathbf{X}$ from the front to the back:

$$\operatorname{tr}\mathbf{H} = \operatorname{tr}\left((\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{X}\right) = \operatorname{tr}(\mathbf{I}_d) = d,$$

the number of columns of $\mathbf{X}$. A cubic fit has $d = 4$ columns $(1, x, x^2, x^3)$, so $\operatorname{tr}\mathbf{S} = 4$ on any data.

Ridge: fewer than d

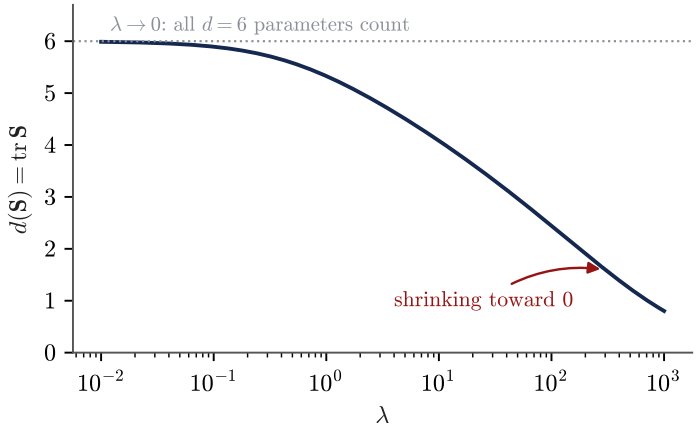
Ridge gives $\hat{\mathbf{y}} = \mathbf{X}(\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I})^{-1}\mathbf{X}^\top\mathbf{y}$. Write $\mathbf{X}$ in its singular value decomposition $\mathbf{X} = \mathbf{U}\mathbf{D}\mathbf{V}^\top$, where $\mathbf{U}, \mathbf{V}$ have orthonormal columns and $\mathbf{D} = \operatorname{diag}(d_1, \ldots, d_p)$. Then $\mathbf{X}^\top\mathbf{X} = \mathbf{V}\mathbf{D}^2\mathbf{V}^\top$ and $\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I} = \mathbf{V}(\mathbf{D}^2 + \lambda\mathbf{I})\mathbf{V}^\top$, so

$$\mathbf{S}_\lambda = \mathbf{U}\mathbf{D}\mathbf{V}^\top \cdot \mathbf{V}(\mathbf{D}^2 + \lambda\mathbf{I})^{-1}\mathbf{V}^\top \cdot \mathbf{V}\mathbf{D}\mathbf{U}^\top = \mathbf{U}\operatorname{diag}\Big(\frac{d_j^2}{d_j^2 + \lambda}\Big)\mathbf{U}^\top.$$

Taking the trace (and using $\mathbf{U}^\top\mathbf{U} = \mathbf{I}$):

$$d(\mathbf{S}_\lambda) = \sum_{j=1}^{p} \frac{d_j^2}{d_j^2 + \lambda}.$$

Each term lies between 0 and 1. $\lambda = 0$ gives p (ordinary least squares). $\lambda \to \infty$ gives 0. Ridge does not delete parameters; it *shrinks how much each one counts.*



Effective parameters of ridge as λ grows. Checked numerically: for a random 20×4 design, $\operatorname{tr}\mathbf{S}_\lambda = 4,\ 3.759,\ 2.504,\ 0.621$ at $\lambda = 0, 1, 10, 100$, matching the formula exactly.

19.4 Bias and variance of an estimator

What "expected value over datasets" means

Imagine collecting the training data again and again, each time a fresh random sample of the same size, and recomputing our estimate each time. The estimates differ from sample to sample. $\mathbb{E}[\hat{\theta}]$ is their long-run average; $\operatorname{Var}(\hat{\theta})$ is how spread out they are.

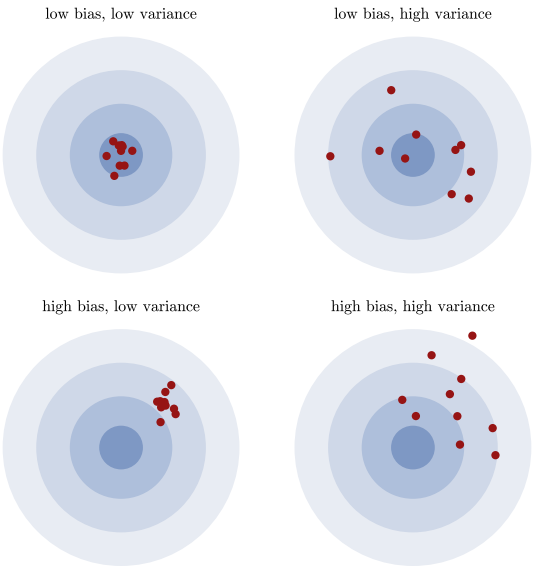
For an estimator $\hat{\theta}$ of a fixed unknown θ:

$$\text{Bias}(\hat{\theta}) = \mathbb{E}[\hat{\theta}] - \theta, \qquad \text{Var}(\hat{\theta}) = \mathbb{E}\big[(\hat{\theta} - \mathbb{E}[\hat{\theta}])^2\big].$$

Unbiased means $\mathbb{E}[\hat{\theta}] = \theta$: right on average. If $\mathbb{E}[\hat{\theta}] - \theta \neq 0$ the estimator is biased.

The DC level (sir's example)

n measurements $x^{(i)} = A + w^{(i)}$ of an unknown constant A, with independent noise $w^{(i)} \sim \mathcal{N}(0, \sigma^2)$. Estimate by the average $\hat{A} = \frac{1}{n}\sum_i x^{(i)}$. $\mathbb{E}[\hat{A}] = \frac{1}{n}\sum_i \mathbb{E}[x^{(i)}] = \frac{1}{n} \cdot nA = A$: unbiased. $\text{Var}(\hat{A}) = \frac{1}{n^2}\sum_i \text{Var}(x^{(i)}) = \frac{n\sigma^2}{n^2} = \frac{\sigma^2}{n}$ (independent terms, so variances add, and $\text{Var}(cX) = c^2\text{Var}(X)$). More data, less variance; still zero bias.



Each dot is the estimate from one possible training set. Bias is how far the centre of the cloud is from the bullseye; variance is how spread the cloud is.

19.5 The bias–variance decomposition, derived

The setup

The truth is $y = f(\mathbf{x}) + \epsilon$: an unknown function plus noise with $\mathbb{E}[\epsilon] = 0$ and $\text{Var}(\epsilon) = \sigma^2$. We fit a model h to a random training set $\mathcal{D}$, so $h(\mathbf{x})$ is itself random (it depends on which data we got). Let $\bar{h}(\mathbf{x}) = \mathbb{E}_{\mathcal{D}}[h(\mathbf{x})]$ be the *average model* over all possible training sets. The noise in a new test point is independent of the training data. We want the expected squared error at a fixed test input $\mathbf{x}$, $\mathbb{E}\big[(y - h(\mathbf{x}))^2\big]$, which is what our training loss $\frac{1}{2n}\sum(y^{(i)} - h(\mathbf{x}^{(i)}))^2$ approximates.

Two tools, used repeatedly:

- $(a+b)^2 = a^2 + 2ab + b^2$, and expectation is linear: $\mathbb{E}[a+b] = \mathbb{E}[a] + \mathbb{E}[b]$, $\mathbb{E}[c\,a] = c\,\mathbb{E}[a]$.
- For independent U, V: $\mathbb{E}[UV] = \mathbb{E}[U]\,\mathbb{E}[V]$. And $\mathbb{E}[U^2] = \text{Var}(U) + (\mathbb{E}[U])^2$.

Step 1: separate the noise

Add and subtract $f(\mathbf{x})$ (write f, h for $f(\mathbf{x}), h(\mathbf{x})$):

$$\begin{aligned}\mathbb{E}\big[(y-h)^2\big] &= \mathbb{E}\big[\big(\underbrace{(y-f)}_{a}+\underbrace{(f-h)}_{b}\big)^2\big] \\ &= \mathbb{E}\big[(y-f)^2\big] + 2\,\mathbb{E}\big[(y-f)(f-h)\big] + \mathbb{E}\big[(f-h)^2\big].\end{aligned}$$

First term. $y - f = \epsilon$, so $\mathbb{E}[\epsilon^2] = \mathrm{Var}(\epsilon) + (\mathbb{E}\epsilon)^2 = \sigma^2 + 0 = \sigma^2$. **Second term.** $\mathbb{E}[\epsilon\,(f-h)]$. The test noise ϵ is independent of $f - h$ (which depends only on the training data), so $= \mathbb{E}[\epsilon]\,\mathbb{E}[f-h] = 0 \cdot \mathbb{E}[f-h] = 0$. So far: $\mathbb{E}[(y-h)^2] = \sigma^2 + \mathbb{E}[(f-h)^2]$.

Step 2: separate bias from variance

Add and subtract the average model $\bar{h}$ inside the remaining term:

$$\mathbb{E}\big[(f-h)^2\big] = \mathbb{E}\big[\big((f-\bar{h}) + (\bar{h}-h)\big)^2\big] = (f-\bar{h})^2 + 2(f-\bar{h})\,\mathbb{E}[\bar{h}-h] + \mathbb{E}\big[(\bar{h}-h)^2\big].$$

$f - \bar{h}$ has no randomness left (both are fixed numbers at this $\mathbf{x}$), so it comes out of the expectation. **Cross term.** $\mathbb{E}[\bar{h}-h] = \bar{h} - \mathbb{E}[h] = \bar{h} - \bar{h} = 0$, by the definition of $\bar{h}$.

Result

$$\mathbb{E}\big[(y-h(\mathbf{x}))^2\big] = \underbrace{\sigma^2}_{\text{irreducible noise}} + \underbrace{\big(f(\mathbf{x}) - \bar{h}(\mathbf{x})\big)^2}_{\text{bias}^2} + \underbrace{\mathbb{E}\big[(h(\mathbf{x}) - \bar{h}(\mathbf{x}))^2\big]}_{\text{variance}}$$

Mental model

Three different villains. **Noise** lives in the data; no model can go below σ^2. **Bias** lives in the hypothesis space: even the average of infinitely many fits misses f, because $\mathcal{H}$ cannot represent it (systematic error). **Variance** lives in the finite sample: the fit you got depends on which n points you happened to draw (sensitivity to the training data).

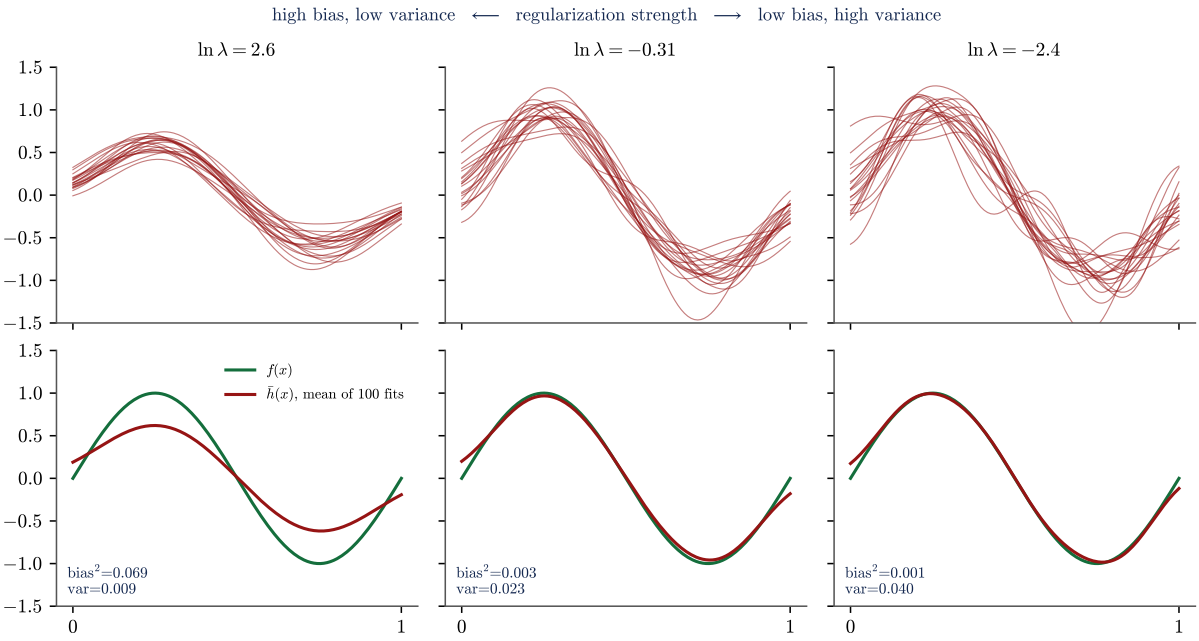
Watch out

The two cross terms vanish for *different* reasons. The first because the test noise is independent of the training set and has mean zero. The second because $\bar{h}$ is defined as the mean of h. In an exam, name both.

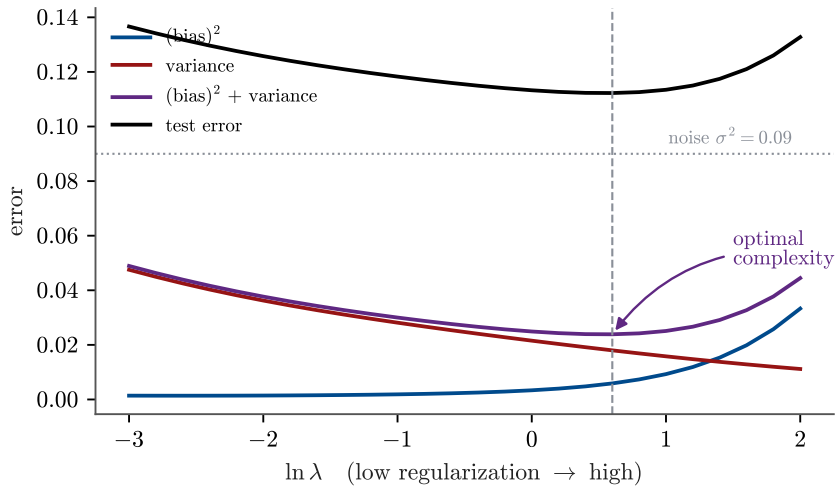
19.6 Regularization as the dial

The ridge objective from the slides: $J(\boldsymbol{\theta}) = \frac{1}{2n}\sum_i (h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) - y^{(i)})^2 + \frac{\lambda}{2}\sum_{j\geq 1}\theta_j^2$. Larger λ, smaller weights, smoother fits: lower variance, higher bias.

The illustration on sir's slide (from Bishop, *PRML* §3.2) can be rebuilt and measured. I regenerated it: 100 training sets of 25 points from $\sin 2\pi x$ with noise $\sigma = 0.3$, each fitted with 24 Gaussian basis functions plus a bias, under three penalties.



Top: 20 of the 100 fits. Bottom: their average $\bar{h}$ (red) against f (green). Heavy regularization ($\ln\lambda = 2.6$): fits agree (variance 0.009) but their average is flattened (bias$^2 = 0.069$). Light regularization ($\ln\lambda = -2.4$): the average is almost perfect (bias$^2 = 0.001$) but individual fits wander (variance 0.040).



Bias^2, variance, their sum, and the test error on 1000 fresh points. The test curve sits about 0.088 above bias^2+variance: that is the noise $\sigma^2 = 0.09$, exactly as the decomposition predicts. Both minima land at $\ln\lambda \approx 0.6$.

Sir's caption: *reducing training error drives down bias but ignores variance.* That is why we choose λ by validation error, not training error.

19.7 Estimating bias and variance with the bootstrap

In practice you have one dataset, not 100. Sir's slides fake the repetitions with the bootstrap (see the CV chapter): draw B resamples $\mathbf{Z}^{*b}$, refit, predict $S(\mathbf{Z}^{*b})$, then

$$\overline{S^*} = \frac{1}{B}\sum_{b=1}^{B} S(\mathbf{Z}^{*b}), \qquad \widehat{\mathrm{Var}}[S] = \frac{1}{B-1}\sum_{b=1}^{B}\big(S(\mathbf{Z}^{*b}) - \overline{S^*}\big)^2.$$

Watch out

The "Bias^2" line on slides 32 and 34 is written as the squared differences of the bootstrap predictions from their average, which is the variance formula again. Bias needs a reference point. The standard bootstrap bias estimate is $\widehat{\mathrm{bias}} = \overline{S^*} - S(\mathbf{Z})$ (average over resamples minus the value on the original data). In a simulation where the truth is known, bias^2 is $(\overline{S^*} - f(\mathbf{x}))^2$.

Check yourself

Q. Why is training error an optimistic estimate of test error?
A. The parameters were chosen to make exactly that error small; the same data cannot judge them fairly.

Q. Does more data reduce bias?
A. Not for a fixed hypothesis space: it reduces variance. To reduce bias, increase capacity.

Q. $d(\mathbf{S})$ for ridge as $\lambda \to \infty$?
A. 0 (or 1 if the intercept is unpenalized).

Go deeper

Read. *Elements of Statistical Learning*, §7.2–7.3, §7.6. Bias–variance and the effective number of parameters.

Read. Bishop, *PRML*, §3.2. The figure rebuilt above.

Read. Goodfellow et al., *Deep Learning*, §5.2. Capacity and the U-curve.

Watch. Kilian Weinberger, Cornell CS4780, *Lecture 19: Bias–Variance Decomposition*

Exercises

Pencil first, then click the ⇝ to jump to the worked answer in Appendix D. The answer links back to the question.

19.1 ⇝ For OLS with an intercept and 3 features on $n = 50$ points, what is $\operatorname{tr}\mathbf{S}$? For ridge with singular values $d = (4, 2, 1)$ (no intercept) and $\lambda = 4$, compute $d(\mathbf{S}_\lambda)$.

19.2 ⇝ An estimator $\tilde{A} = \frac{1}{n+1}\sum_{i=1}^{n} x^{(i)}$ of the DC level. Compute its bias and variance. Is it ever better than $\hat{A}$ in mean squared error bias^2 + var?

19.3 ⇝ In the decomposition, which term does each of these change, and in which direction: (a) doubling n; (b) moving from degree 3 to degree 9; (c) increasing λ; (d) cleaning measurement noise from the labels.

Lecture slides for reference

Capacity of a model

Model capacity is ability to fit variety of functions

- Model with Low capacity struggles to fit training set
- A High capacity model can overfit by memorizing properties of training set not useful on test set
  - When model has higher capacity, it overfits
- One way to control capacity of a learning algorithm is by choosing the hypothesis space
  - set of functions that the learning algorithm is allowed to select as being the solution
  - E.g., the linear regression algorithm has the set of all linear functions of its input as the hypothesis space
  - We can generalize to include polynomials in its hypothesis space which increases model capacity

Sir's lecture notes, slide 002. Capacity.

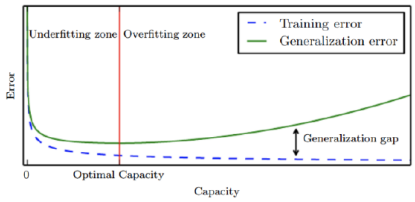
The Effective Number of Parameters

- Stack the outcomes $\boldsymbol{y} = \left[y^{(1)}, \cdots, y^{(n)}\right]^T$ and predictions $\hat{\boldsymbol{y}}$.
- For a linear fitting method $\hat{\boldsymbol{y}} = \boldsymbol{S}\boldsymbol{y}$
- $\boldsymbol{S}$ is an $n \times n$ matrix depends on the input vector $x^{(i)}$
- The number of effective parameters is defined as $d(\boldsymbol{S}) = trace(\boldsymbol{S})$

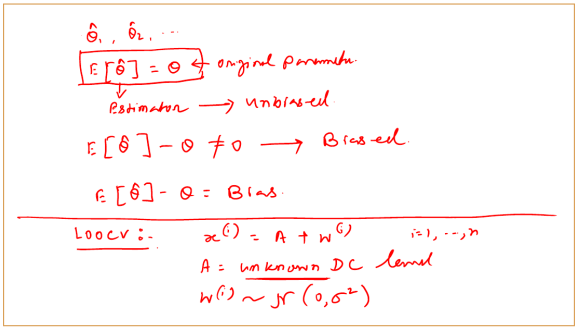
Sir's lecture notes, slide 006. Effective number of parameters.

Typical Generalization Error

- Relationship between capacity and error
- Typically generalization error has a U-shaped curve



Sir's lecture notes, slide 007. The U-shaped curve.



Sir's lecture notes, slide 020. Bias of an estimator.

Bias-Variance Decomposition of Squared Error

- Assume that $y = f(\boldsymbol{x}) + \epsilon$
  - Noise ϵ is sampled from a normal distribution with 0 mean and variance σ^2: $\epsilon \sim \mathcal{N}(0, \sigma^2)$
  - Noise lower-bounds the performance (error) we can achieve
- Recall the objective function

$$J(\boldsymbol{\theta}) = \frac{1}{2n}\sum_{i=1}^{n}\left(y^{(i)} - h_{\boldsymbol{\theta}}(\boldsymbol{x}^{(i)})\right)^2$$

- We can represent the function as an approximation of the expected value of the squared error: $E\left(y - h_{\boldsymbol{\theta}}(\boldsymbol{x})\right)^2$

Sir's lecture notes, slide 035. Decomposition setup.

Bias-Variance Decomposition of Squared Error

$$E\left(y - h_{\boldsymbol{\theta}}(\boldsymbol{x})\right)^2 = E\left[\left(y - f(x) + f(x) - h_{\boldsymbol{\theta}}(\boldsymbol{x})\right)^2\right]$$

Sir's lecture notes, slide 036. Decomposition, first two terms.

Sir's lecture notes, slide 037. Decomposition, third term.

Bias-Variance Illustration

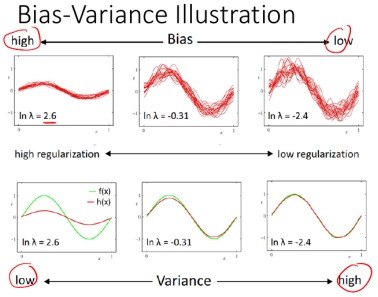
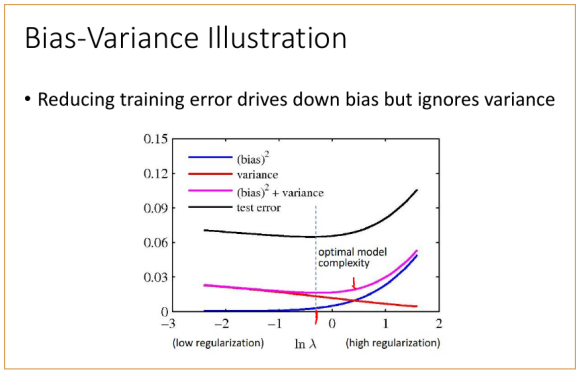


Illustration of dependence of bias and variance on model complexity (sinusoidal dataset): There are 100 datasets, each having 25 datapoints, there are 24 Gaussian basis functions in the model.

Top row: Result of fitting the model to the dataset for various values of $\ln \lambda$ (20 out of 100 fits are shown)

Bottom Row: Corresponding average of the 100 fits (red) along with the sinusoidal functions from which the dataset is generated (green).

Sir's lecture notes, slide 039. Bishop's illustration.



Sir's lecture notes, slide 040. Bias, variance, test error.

20

LASSO: Shrinkage that Selects

Ridge regression makes every coefficient smaller but never makes one exactly zero. Often we want the model to *throw features away*: fewer features are cheaper to measure, easier to explain, and less likely to fit noise. Change the penalty from θ_j^2 to $|\theta_j|$, and that is exactly what happens. The name says what it does: **L**east **A**bsolute **S**hrinkage and **S**election **O**perator (Tibshirani, 1996). We derive the solution case by case, see why it produces exact zeros, and work numbers. Sir's slides are at the end.

20.1 The objective

$$\hat{\boldsymbol{\theta}} = \arg\min_{\boldsymbol{\theta}} \underbrace{\|\mathbf{y} - \mathbf{X}\boldsymbol{\theta}\|_2^2}_{\text{fit the data}} + \underbrace{\lambda\|\boldsymbol{\theta}\|_1}_{\text{pay for size}}, \qquad \|\boldsymbol{\theta}\|_1 = \sum_{j=1}^{d} |\theta_j|.$$

$\lambda \geq 0$ sets the price. $\lambda = 0$ is ordinary least squares; huge λ forces everything to zero.

Why there is no general closed form

For ridge we set the gradient to zero and solved: $\hat{\boldsymbol{\theta}} = (\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I})^{-1}\mathbf{X}^\top\mathbf{y}$. Here the penalty contains $|\theta_j|$, which has a sharp corner at $\theta_j = 0$ and no derivative there. And $\theta_j = 0$ is precisely where we hope some coefficients will land. So "set the derivative to zero" does not work directly. The objective is still **convex** (a convex quadratic plus convex absolute values), so it has one minimum; in general we find it iteratively. In one important special case, though, we can solve it by hand.

20.2 The special case $\mathbf{X}^\top\mathbf{X} = \mathbf{I}$, derived step by step

Assume the columns of $\mathbf{X}$ are **orthonormal**: each has length 1 and any two are perpendicular. Then $\mathbf{X}^\top\mathbf{X} = \mathbf{I}$.

Step 1: expand the squared error

$\|\mathbf{y} - \mathbf{X}\boldsymbol{\theta}\|^2 = (\mathbf{y} - \mathbf{X}\boldsymbol{\theta})^\top(\mathbf{y} - \mathbf{X}\boldsymbol{\theta}) = \mathbf{y}^\top\mathbf{y} - 2\mathbf{y}^\top\mathbf{X}\boldsymbol{\theta} + \boldsymbol{\theta}^\top\mathbf{X}^\top\mathbf{X}\boldsymbol{\theta}$. With $\mathbf{X}^\top\mathbf{X} = \mathbf{I}$ the last term is $\boldsymbol{\theta}^\top\boldsymbol{\theta} = \sum_j \theta_j^2$. The middle term is $\mathbf{y}^\top\mathbf{X}\boldsymbol{\theta} = \sum_j (\mathbf{y}^\top\mathbf{x}_j)\,\theta_j$, where $\mathbf{x}_j$ is column j. So

$$J(\boldsymbol{\theta}) = \mathbf{y}^\top\mathbf{y} + \sum_{j=1}^{d} \Big(\theta_j^2 - 2\,(\mathbf{y}^\top\mathbf{x}_j)\,\theta_j + \lambda|\theta_j|\Big).$$

Step 2: the problem splits into d separate problems

Each θ_j appears only in its own bracket. To minimise a sum of brackets that share no variables, minimise each bracket on its own. So we solve, for each j, a one-variable problem:

$$\min_{\theta} \; g(\theta) = \theta^2 - 2\theta^0\theta + \lambda|\theta|, \qquad \theta^0 = \mathbf{y}^\top \mathbf{x}_j.$$

Note θ^0 is the ordinary least-squares coefficient: with $\mathbf{X}^\top\mathbf{X} = \mathbf{I}$, OLS gives $(\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{y} = \mathbf{X}^\top\mathbf{y}$, whose j-th entry is $\mathbf{y}^\top\mathbf{x}_j$.

Step 3: three cases

$|\theta|$ equals θ when $\theta > 0$ and $-\theta$ when $\theta < 0$. So try each region.

- **Case 1, answer is positive.** Then $g(\theta) = \theta^2 - 2\theta^0\theta + \lambda\theta$. Set $g'(\theta) = 2\theta - 2\theta^0 + \lambda = 0$, giving $\theta = \theta^0 - \frac{\lambda}{2}$. This is only valid if it really is positive: $\theta^0 > \frac{\lambda}{2}$.
- **Case 2, answer is negative.** Then $g(\theta) = \theta^2 - 2\theta^0\theta - \lambda\theta$. Set $2\theta - 2\theta^0 - \lambda = 0$, giving $\theta = \theta^0 + \frac{\lambda}{2}$. Valid only if negative: $\theta^0 < -\frac{\lambda}{2}$.
- **Case 3, neither.** If $-\frac{\lambda}{2} \le \theta^0 \le \frac{\lambda}{2}$, neither formula gives a value in its own region, so the minimum is at the corner: $\hat{\theta} = 0$. (Check: $g(\theta) - g(0) = \theta^2 + |\theta|\,(\lambda - 2\theta^0 \operatorname{sign}\theta) \ge 0$ because the bracket is non-negative here.)

Result: soft thresholding

$$\hat{\theta}_j = \begin{cases} \theta_j^0 - \frac{\lambda}{2} & \text{if } \theta_j^0 > \frac{\lambda}{2} \\ 0 & \text{if } |\theta_j^0| \le \frac{\lambda}{2} \\ \theta_j^0 + \frac{\lambda}{2} & \text{if } \theta_j^0 < -\frac{\lambda}{2} \end{cases} = \operatorname{sign}(\theta_j^0)\,\max\Big(|\theta_j^0| - \frac{\lambda}{2},\; 0\Big)$$

In words: take the least-squares value, **shrink its size by** $\lambda/2$, and if that would cross zero, **stop at zero**.

Watch out

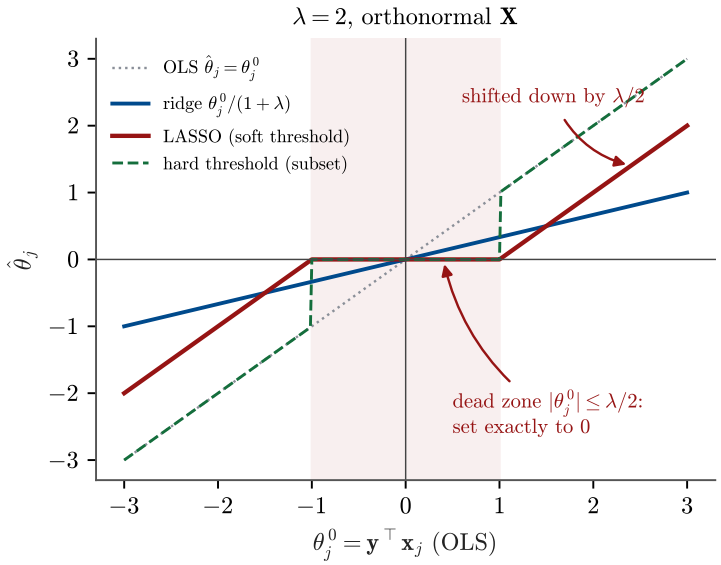
The test is on the OLS value θ_j^0, and it is $|\theta_j^0| \le \lambda/2$, not "negative values become zero". A coefficient $\theta_j^0 = -5$ with $\lambda = 2$ becomes -4, not 0. The slide's "if $\hat{\theta}_j < 0$" labels describe which case you are in, not a rule to zero out negatives.

Four observations, two orthonormal columns

$\mathbf{X} = \frac{1}{2}\begin{bmatrix} 1 & 1 \\ 1 & -1 \\ 1 & 1 \\ 1 & -1 \end{bmatrix}$. Check $\mathbf{X}^\top\mathbf{X}$: each column has squared length $4 \times \frac{1}{4} = 1$, and their dot product is $\frac{1}{4}(1 - 1 + 1 - 1) = 0$. So $\mathbf{X}^\top\mathbf{X} = \mathbf{I}$. $\mathbf{y} = (3, 1, 2, 0.6)$. OLS: $\theta_1^0 = \frac{1}{2}(3 + 1 + 2 + 0.6) = 3.3$, $\theta_2^0 = \frac{1}{2}(3 - 1 + 2 - 0.6) = 1.7$.

λ	$\lambda/2$	LASSO $\hat{\theta}_1$	LASSO $\hat{\theta}_2$	ridge $\frac{3.3}{1+\lambda}$	ridge $\frac{1.7}{1+\lambda}$
0.5	0.25	3.05	1.45	2.20	1.13
1	0.5	2.80	1.20	1.65	0.85
2	1	2.30	0.70	1.10	0.57
3.6	1.8	1.50	**0**	0.72	0.37
5	2.5	0.80	**0**	0.55	0.28

At $\lambda = 3.6$: $\lambda/2 = 1.8$ is larger than $|\theta_2^0| = 1.7$, so feature 2 is **removed**. Ridge never removes it, it only shrinks. Cross-check: scikit-learn's `Lasso` uses $\frac{1}{2n}\|\mathbf{y} - \mathbf{X}\boldsymbol{\theta}\|^2 + \alpha\|\boldsymbol{\theta}\|_1$, so our λ corresponds to $\alpha = \lambda/(2n)$; with $\alpha = 1/8$ it returns (2.8, 1.2), matching the $\lambda = 1$ row.



One coefficient, $\lambda = 2$. Ridge divides by $1 + \lambda$ (gentler slope, never zero). LASSO subtracts $\lambda/2$ (same slope, shifted, with a dead zone). Best-subset selection keeps or kills (a jump).

Mental model

Ridge charges a *percentage tax*: big coefficients pay a lot, tiny ones pay almost nothing, so none is pushed all the way to zero. LASSO charges a *flat fee* of $\lambda/2$: a coefficient whose least-squares value cannot afford the fee is set to zero. That is the whole reason LASSO selects features and ridge does not.

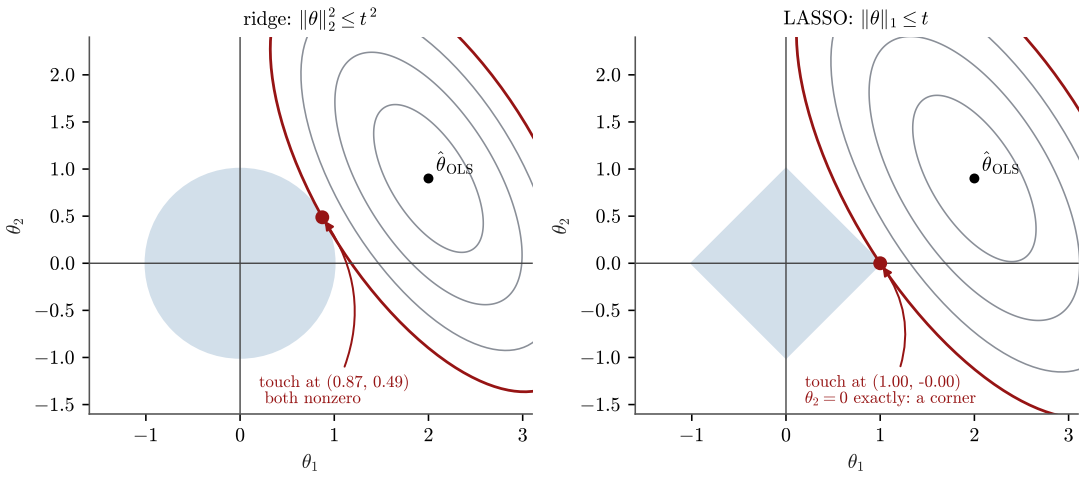
In general (non-orthonormal $\mathbf{X}$), **coordinate descent** solves LASSO by applying this soft threshold to one coordinate at a time, cycling until nothing changes. Sir calls it an iterative shrinkage method; it is what `sklearn.linear_model.Lasso` runs.

20.3 The geometry: why corners create zeros

The penalized problem is equivalent to a constrained one: for each λ there is a budget t such that

$$\min_{\boldsymbol{\theta}} \|\mathbf{y} - \mathbf{X}\boldsymbol{\theta}\|^2 \quad \text{subject to} \quad \|\boldsymbol{\theta}\|_1 \le t \qquad (\text{ridge: } \|\boldsymbol{\theta}\|_2^2 \le t^2).$$

The squared error has elliptical contours centred at the OLS solution. Grow an ellipse from the centre until it first touches the allowed region. The touching point is the answer.

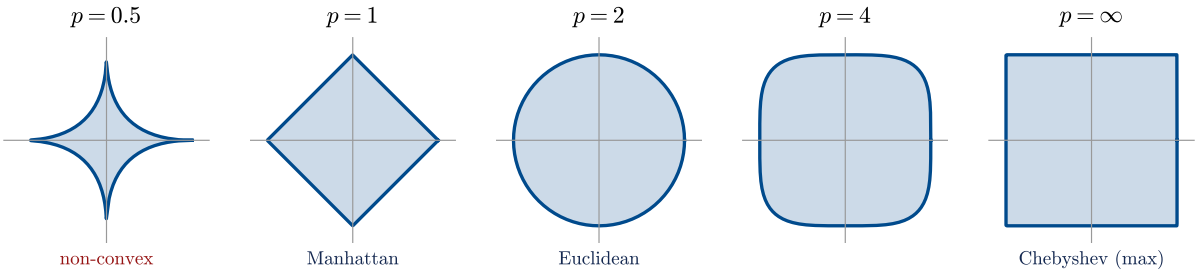


Same loss, same budget. The circle (ridge) is touched at a generic point $(0.87, 0.49)$*, both nonzero. The diamond (LASSO) is touched at its corner* $(1, 0)$*:* $\theta_2 = 0$ *exactly. Computed by constrained optimisation, not sketched.*

> **Ask before reading on**
>
> In 2 dimensions the diamond has 4 corners. In 100 dimensions, where does a growing ellipse most likely first touch the ℓ_1 ball?

On a corner or low-dimensional edge, where most coordinates are zero. In high dimensions the ℓ_1 ball is "mostly corners and edges"; the ℓ_2 ball has none.

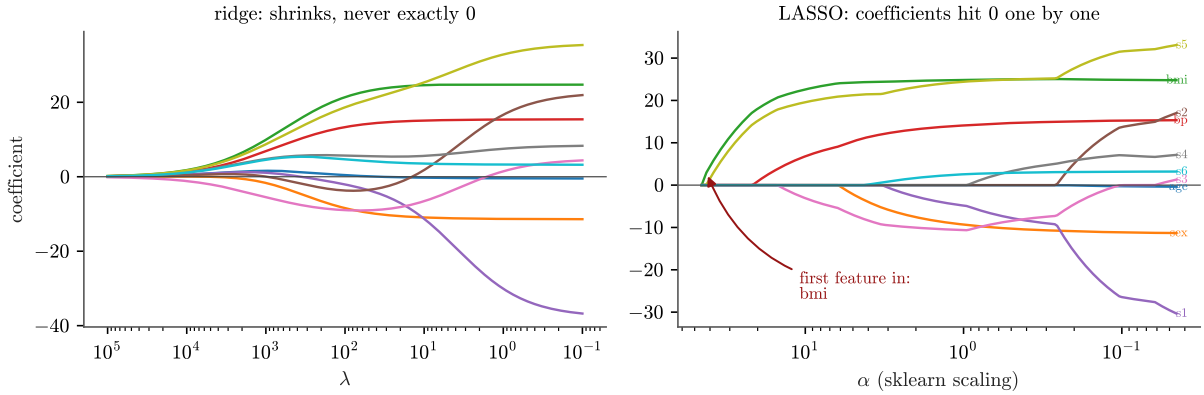
20.4 The p-norm family



Unit balls of $\|\mathbf{x}\|_p = (\sum |x_d|^p)^{1/p}$*. For* $p \le 1$ *there are corners on the axes (sparsity);* $p < 1$ *is not even convex.*

$p = 1$ is the smallest p that keeps the problem convex while still having corners, which is why LASSO is the workhorse. As $p \to 0$, $\sum |x_d|^p$ tends to the *number of nonzero entries* (the "ℓ_0 norm"). Penalising that directly is best-subset selection, which requires trying every subset of features: computationally hopeless for many features. LASSO is its convex relaxation.

20.5 On real data: the regularization path



Coefficient paths on the diabetes data (10 standardized features). Left, ridge: everything shrinks smoothly. Right, LASSO: reading from strong to weak penalty, `bmi` enters first and features join one at a time.

Watch out

Standardize features before LASSO or ridge, and do not penalize the intercept. The penalty treats every coefficient equally, but a coefficient's size depends on its feature's units: measure height in millimetres instead of metres and its coefficient shrinks 1000 times, so the penalty would barely touch it.

20.6 From the textbook: the Bayesian view (ISLR §6.2.2)

Each penalty is a prior

Model $y_i = \mathbf{x}_i^\top \boldsymbol{\theta} + \varepsilon_i$ with Gaussian noise of variance σ^2, and a prior $p(\boldsymbol{\theta}) = \prod_j g(\theta_j)$. The posterior is proportional to likelihood times prior, so its negative log is

$$-\ln p(\boldsymbol{\theta} \mid \text{data}) = \frac{1}{2\sigma^2}\|\mathbf{y} - \mathbf{X}\boldsymbol{\theta}\|^2 - \sum_j \ln g(\theta_j) + \text{const.}$$

Gaussian prior $g(\theta) \propto e^{-\theta^2/(2\tau^2)}$: the penalty is $\frac{1}{2\tau^2}\sum_j \theta_j^2$, **ridge**.
Laplace prior $g(\theta) \propto e^{-|\theta|/b}$: the penalty is $\frac{1}{b}\sum_j |\theta_j|$, **LASSO**. The most probable $\boldsymbol{\theta}$ (the posterior mode, MAP) is therefore the ridge or LASSO solution.

Mental model

The Laplace density has a sharp peak at 0, the Gaussian a rounded top. A sharp peak says "I strongly believe many coefficients are exactly zero"; a rounded one says "I believe all coefficients are small". LASSO's exact zeros are that sharp peak showing through.

20.7 From the textbook: choosing λ (ISLR §6.2.3)

1. Pick a grid of λ values (usually spaced on a log scale).
2. For each, compute the k-fold cross-validation error (CV chapter).

3. Choose the λ with the smallest CV error, then refit on all the data with that λ.

A common refinement, the **one-standard-error rule**: compute the standard error of the CV curve at its minimum, and pick the *largest* λ (simplest model) whose CV error is within one standard error of the minimum. The CV curve is noisy; this prefers a sparser model that is statistically just as good.

Check yourself

Q. Ridge or LASSO when you believe all 50 features matter a little?
A. Ridge.

Q. Two perfectly correlated features?
A. Ridge splits the weight between them; LASSO tends to keep one and zero the other.

Q. LASSO solution for $\theta^0 = 0.4$, $\lambda = 1$, orthonormal X?
A. $0.4 \leq 0.5$, so 0.

Go deeper

Read. Tibshirani, "Regression shrinkage and selection via the lasso" (1996). The original paper.

Read. Hastie, Tibshirani, Wainwright, *Statistical Learning with Sparsity*, ch. 2. Soft thresholding and coordinate descent.

Read. *Elements of Statistical Learning*, §3.4. Subset vs ridge vs LASSO in the orthonormal case.

Watch. Kilian Weinberger, Cornell CS4780, *Lecture 13: Linear / Ridge Regression*

Exercises

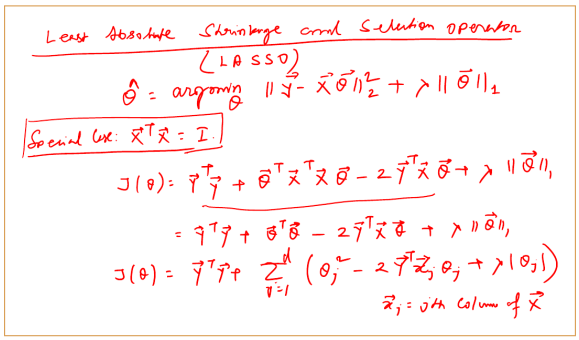
Pencil first, then click the ⇝ to jump to the worked answer in Appendix D. The answer links back to the question.

20.1 ⇝ Orthonormal $\mathbf{X}$, OLS values $\boldsymbol{\theta}^0 = (4, -0.6, 1.5, -2)$. Give the LASSO solution for $\lambda = 2$ and for $\lambda = 3.2$, and the ridge solution for $\lambda = 1$.

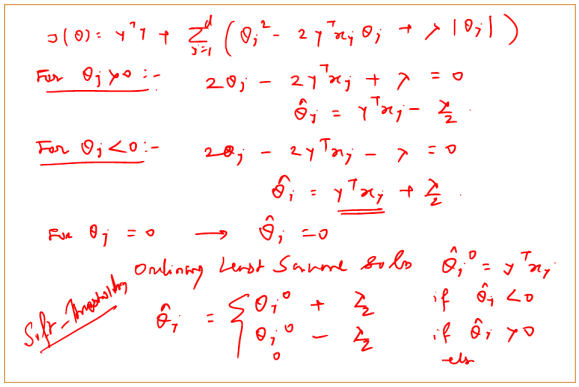
20.2 ⇝ Show that for $g(\theta) = \theta^2 - 2\theta^0\theta + \lambda|\theta|$ with $|\theta^0| \leq \lambda/2$, $g(\theta) \geq g(0)$ for every θ.

20.3 ⇝ What is the smallest λ for which the orthonormal LASSO solution is the all-zero vector? Express it with $\mathbf{X}^\top\mathbf{y}$.

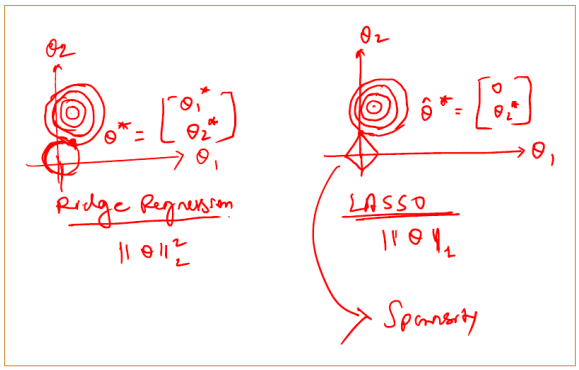
Lecture slides for reference



Sir's lecture notes, slide 008. LASSO objective and the special case.



Sir's lecture notes, slide 009. The three cases.



Sir's lecture notes, slide 015. Ridge vs LASSO picture.

21

Cross-Validation and the Bootstrap

A model is only useful if it works on data it has never seen. But the future is not in our file. This chapter is about two ways to *estimate* performance on unseen data using only the data we have. **Cross-validation** reuses the data as its own test set. The **bootstrap** reuses it as its own population. Every step is explained, every formula derived, and the class examples are worked by hand. Sir's slides are at the end for reference.

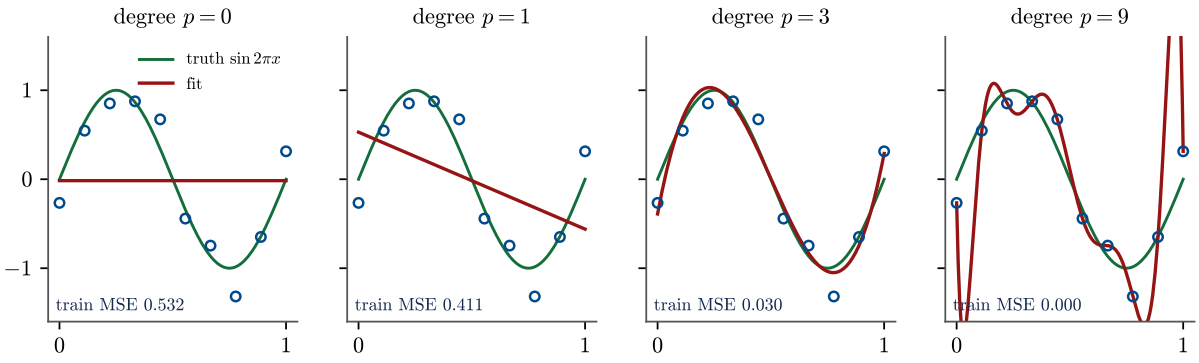
21.1 The problem: training error lies

A story with polynomials (from the class deck)

We want to learn $y = \mathbf{w}^\top f(x) + b$ where f makes polynomial features $x, x^2, x^3, \ldots$. Twenty noisy points were generated from a smooth curve. Fit polynomials of degree 1, 3, 9, 19:

- degree 1: a straight line, misses the bend (underfit);
- degree 3: follows the curve nicely;
- degree 9 and 19: wriggle through every point, and swing wildly between them (overfit).

The *training* error keeps going down as the degree goes up: the degree-19 curve has the smallest training error of all. So training error cannot tell us which degree is best. "Does a higher degree mean a better model?" No, but training error says yes.



The same phenomenon on ten points from $\sin 2\pi x$*: training error falls with every increase in degree, while the curve becomes useless.*

Mental model

Judging a model on the data it was fitted to is like marking your own exam with the answer key you studied from. The score is high, and it tells you nothing about the next exam. We need questions the model has not seen.

21.2 Train, validation, test

Split the data into three parts with three different jobs:

- **Training set**: used to fit the parameters (the weights).
- **Validation set**: used to choose *settings* that are not fitted directly: the polynomial degree, λ, K in K-NN. This is called model selection.
- **Test set**: used *once*, at the very end, to report how good the final model is (model assessment).

Never train or choose anything using the test data. The moment you use it to make a choice, it stops being an honest estimate.

21.3 The validation set approach

The recipe

Split the data into two parts. Train on the first part. Compute the error on the second part. Use that error as the estimate of test error.

Two drawbacks (class deck)

1. **The estimate depends on the split.** A different random split can give a noticeably different error, and even a different "best" degree.
2. **It wastes data.** Only part of the data trains the model, and only part evaluates it. With small datasets both parts are too small.

Cross-validation fixes both by rotating the roles.

21.4 Leave-one-out cross-validation (LOOCV)

The recipe

For every $i = 1, \dots, n$: train the model on all points except point i, then predict point i and record the error. Average the n errors:

$$\mathrm{CV}_{(n)} = \frac{1}{n}\sum_{i=1}^{n}\left(y_i - \hat{y}_i^{(-i)}\right)^2 \qquad \text{(regression)}, \qquad \mathrm{CV}_{(n)} = \frac{1}{n}\sum_{i=1}^{n}\mathbf{1}\left[y_i \neq \hat{y}_i^{(-i)}\right] \qquad \text{(classification)},$$

where $\hat{y}_i^{(-i)}$ is the prediction for point i from the model trained *without* point i, and $\mathbf{1}[\cdot]$ is 1 if the statement is true and 0 otherwise (so the classification version is the fraction of mistakes).

Class deck: picking a polynomial degree with LOOCV

With 100 data points: hold out point 1, train on the other 99, compute the squared error SE_1 on point 1. Hold out point 2, train on 99, compute SE_2. Continue to SE_{100}. Then LOOCV $= \frac{SE_1+SE_2+\cdots+SE_{100}}{100}$. Do this for degree 1, degree 2, degree 3, and so on. That is 100 fits per degree. Plot LOOCV against the degree and pick the minimum. In sir's sketch the minimum is at degree 2: the "good fit".

LOOCV for a classifier (class deck, KNN)

Ten points, four + and one − on the left, five − on the right. Hold each out and classify it from the other nine. 1-NN gets 5 of 10 wrong; 3-NN gets 1 of 10 wrong (full working in the K-NN chapter). So LOOCV prefers $K = 3$.

Cost

LOOCV fits the model n times. One linear-regression fit with n points and d features costs $O(nd^2 + d^3)$ (build $\mathbf{X}^\top\mathbf{X}$, then solve), so LOOCV costs about n times that. For expensive models that is too much. (For least squares and ridge there is an exact shortcut that needs only one fit; see the box below.)

21.5 k-fold cross-validation

The recipe

Split the data randomly into k groups of roughly equal size (the *folds*). For each fold in turn: train on the other $k-1$ folds, compute the error on the held-out fold. Average the k fold errors. Each point is used for validation exactly once and for training $k-1$ times.

$$\mathrm{CV} = \frac{1}{n}\sum_{i=1}^{n} L\big(y_i,\ \hat{f}^{-\kappa(i)}(\mathbf{x}_i)\big),$$

where $\kappa(i)$ is the fold containing point i and $\hat{f}^{-\kappa(i)}$ is the model trained without that fold.

Using 5-fold CV to choose a hyperparameter, step by step

To judge $\lambda = 0.1$ in ridge regression: (1) split the data into 5 folds; (2) hold out fold 1, train with $\lambda = 0.1$ on folds 2–5, compute the error on fold 1; (3) repeat holding out folds 2, 3, 4, 5; (4) the quality of $\lambda = 0.1$ is the average of the 5 errors. Repeat for $\lambda = 1$, $\lambda = 10$, and so on, and pick the λ with the smallest average. Finally retrain on all the data with the chosen λ.

In code

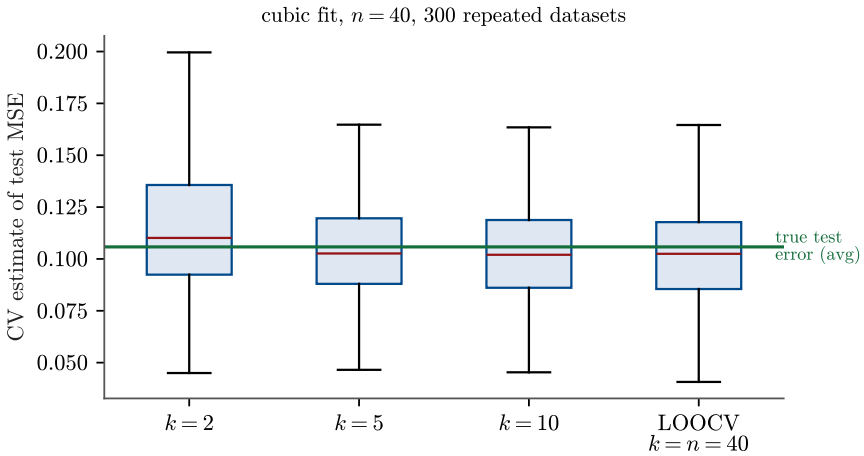
```
import numpy as np
def kfold_cv(X, y, fit, predict, k=5, seed=0):
    idx = np.random.default_rng(seed).permutation(len(y))   # shuffle once
```

```
    folds = np.array_split(idx, k)                                  # k
        nearly equal groups
    total = 0.0
    for f in folds:
        train = np.setdiff1d(idx, f)                                #
            everything except this fold
        model = fit(X[train], y[train])
        total += np.sum((predict(model, X[f]) - y[f]) ** 2) #
            squared errors on the held-out fold
    return total / len(y)                                           #
        average over all n points
```

Which k?

Typical choices are 5, 10 and n (LOOCV). The class decks and sir's slide 24 describe the trade-off:

- **Bias.** Each fold's model trains on only $\frac{k-1}{k}$ of the data. Less data usually means a worse model, so small k *overestimates* the error (pessimistic bias). LOOCV trains on $n-1$ points, almost all the data, so it is nearly unbiased.
- **Variance.** LOOCV's n models are trained on almost identical data, so their errors are highly correlated, and an average of correlated numbers does not settle down as much. This is the usual argument that LOOCV has higher variance.
- **Cost.** k fits for k-fold, n fits for LOOCV.



A check by simulation: cubic fits on 300 independent datasets of 40 points. $k = 2$ is pessimistic (mean 0.121 against a true 0.106); $k = 5, 10, n$ are all close.

Watch out

In that simulation LOOCV was *not* more variable than 10-fold. For stable methods such as least squares the variance argument is weak; it matters for unstable ones (deep trees, 1-NN). The practical advice is unchanged: use 5 or 10 folds, as most papers do.

LOOCV in one fit

For least squares, ridge and polynomial fits the predictions are $\hat{\mathbf{y}} = \mathbf{S}\mathbf{y}$ for a matrix $\mathbf{S}$. Then exactly

$$\mathrm{CV}_{(n)} = \frac{1}{n}\sum_{i=1}^{n}\left(\frac{y_i - \hat{y}_i}{1 - S_{ii}}\right)^2,$$

using the residuals of a single fit on all the data. A point with large S_{ii} (high leverage) pulled the full fit towards itself, so its residual is inflated to compensate.

21.6 From the textbook: the LOOCV shortcut, derived (ISLR eq. 5.2)

For least squares, LOOCV does not need n refits. The proof uses one matrix identity.

For an invertible matrix $\mathbf{A}$ and a vector $\mathbf{u}$ with $\mathbf{u}^\top\mathbf{A}^{-1}\mathbf{u} \neq 1$:

$$(\mathbf{A} - \mathbf{u}\mathbf{u}^\top)^{-1} = \mathbf{A}^{-1} + \frac{\mathbf{A}^{-1}\mathbf{u}\,\mathbf{u}^\top\mathbf{A}^{-1}}{1 - \mathbf{u}^\top\mathbf{A}^{-1}\mathbf{u}}.$$

(Check: multiply the right side by $\mathbf{A} - \mathbf{u}\mathbf{u}^\top$ and it collapses to $\mathbf{I}$.) It says how the inverse changes when one rank-one piece is removed.

Derivation

Let $\mathbf{A} = \mathbf{X}^\top\mathbf{X}$, $\mathbf{b} = \mathbf{X}^\top\mathbf{y}$, so $\hat{\boldsymbol{\theta}} = \mathbf{A}^{-1}\mathbf{b}$, $\hat{y}_i = \mathbf{x}_i^\top\hat{\boldsymbol{\theta}}$, and the leverage $h_i = \mathbf{x}_i^\top\mathbf{A}^{-1}\mathbf{x}_i$ (the i-th diagonal entry of the hat matrix). Removing row i removes $\mathbf{x}_i\mathbf{x}_i^\top$ from $\mathbf{A}$ and $\mathbf{x}_i y_i$ from $\mathbf{b}$.

Step 1. Sherman–Morrison with $\mathbf{u} = \mathbf{x}_i$, then multiply on the left by $\mathbf{x}_i^\top$:

$$\mathbf{x}_i^\top(\mathbf{A} - \mathbf{x}_i\mathbf{x}_i^\top)^{-1} = \mathbf{x}_i^\top\mathbf{A}^{-1} + \frac{h_i\,\mathbf{x}_i^\top\mathbf{A}^{-1}}{1 - h_i} = \frac{\mathbf{x}_i^\top\mathbf{A}^{-1}}{1 - h_i}.$$

Step 2. The leave-one-out prediction for point i:

$$\hat{y}_i^{(-i)} = \mathbf{x}_i^\top(\mathbf{A} - \mathbf{x}_i\mathbf{x}_i^\top)^{-1}(\mathbf{b} - \mathbf{x}_i y_i) = \frac{\mathbf{x}_i^\top\mathbf{A}^{-1}\mathbf{b} - h_i y_i}{1 - h_i} = \frac{\hat{y}_i - h_i y_i}{1 - h_i}.$$

Step 3. The leave-one-out residual:

$$y_i - \hat{y}_i^{(-i)} = \frac{y_i(1 - h_i) - \hat{y}_i + h_i y_i}{1 - h_i} = \boxed{\frac{y_i - \hat{y}_i}{1 - h_i}} \qquad \Longrightarrow \qquad \mathrm{CV}_{(n)} = \frac{1}{n}\sum_{i=1}^{n}\left(\frac{y_i - \hat{y}_i}{1 - h_i}\right)^2.$$

Check on four points

$x = (1, 2, 3, 4)$, $y = (1, 3, 2, 5)$, model $\theta_0 + \theta_1 x$. One fit: $\hat{y} = 1.1x$, so $\hat{y} = (1.1,\ 2.2,\ 3.3,\ 4.4)$, residuals $(-0.1,\ 0.8,\ -1.3,\ 0.6)$. Leverages $h = (0.7,\ 0.3,\ 0.3,\ 0.7)$ (the end points pull hardest on the line). Leave-one-out residuals: $-0.1/0.3 = -0.333$, $0.8/0.7 = 1.143$, $-1.3/0.7 = -1.857$, $0.6/0.3 = 2.000$. $\mathrm{CV}_{(4)} = \frac{1}{4}(0.111 + 1.306 + 3.449 + 4.000) = 2.217$. Refitting four times by brute force gives the same four residuals.

21.7 LOOCV on the DC level, derived

Sir works one estimator through LOOCV completely. It is the cleanest example of what holding a point out does.

The setting

We measure an unknown constant A (a "DC level") n times, with noise: $x^{(i)} = A + w^{(i)}$, where the noises $w^{(i)} \sim \mathcal{N}(0, \sigma^2)$ are independent. The natural estimate is the average, $\hat{A} = \frac{1}{n}\sum_i x^{(i)}$.

Step 1: the full-data estimator

$$\mathbb{E}[\hat{A}] = \frac{1}{n}\sum_{i=1}^{n}\mathbb{E}[x^{(i)}] = \frac{1}{n}\sum_{i=1}^{n} A = \frac{nA}{n} = A \qquad \text{linearity; } \mathbb{E}[w^{(i)}] = 0$$

$$\mathrm{Var}(\hat{A}) = \frac{1}{n^2}\sum_{i=1}^{n}\mathrm{Var}(x^{(i)}) = \frac{n\sigma^2}{n^2} = \frac{\sigma^2}{n} \qquad \text{independent: variances add.}$$

Unbiased, and the variance shrinks as n grows: more data, more reliable.

Step 2: the leave-one-out estimator

Leave point i out: $\hat{A}_{-i} = \frac{1}{n-1}\sum_{m\neq i} x^{(m)}$. The same two lines with $n-1$ instead of n:

$$\mathbb{E}[\hat{A}_{-i}] = A, \qquad \mathrm{Var}(\hat{A}_{-i}) = \frac{\sigma^2}{n-1} \;\geq\; \frac{\sigma^2}{n} = \mathrm{Var}(\hat{A}).$$

Still unbiased; a little more variable, because it uses one point fewer.

Step 3: the LOOCV error for point i

$$\begin{aligned} e^{(i)} = x^{(i)} - \hat{A}_{-i} &= \left(A + w^{(i)}\right) - \frac{1}{n-1}\sum_{m\neq i}\left(A + w^{(m)}\right) && \text{substitute the model} \\ &= A + w^{(i)} - \frac{(n-1)A}{n-1} - \frac{1}{n-1}\sum_{m\neq i} w^{(m)} \\ &= w^{(i)} - \frac{1}{n-1}\sum_{m\neq i} w^{(m)} && \text{the unknown } A \text{ cancels.} \end{aligned}$$

Mean: $\mathbb{E}[e^{(i)}] = 0 - 0 = 0$. Variance: $w^{(i)}$ is independent of the other noises, so the variances add:

$$\mathrm{Var}(e^{(i)}) = \sigma^2 + \frac{(n-1)\sigma^2}{(n-1)^2} = \sigma^2 + \frac{\sigma^2}{n-1} = \boxed{\frac{n}{n-1}\sigma^2}.$$

Mental model

Read $\frac{n}{n-1}\sigma^2$ as two pieces: the noise in the held-out point (σ^2, which nothing can remove) plus the noise in the model predicting it ($\frac{\sigma^2}{n-1}$). Sir's note: for small n the second piece is noticeable; for large n it fades. Simulated with $n = 6$, $\sigma = 2$: variance 4.800, theory $\frac{6}{5}\cdot 4 = 4.8$.

21.8 The bootstrap

The problem it solves

We computed a statistic from our sample: a mean, a regression coefficient, a prediction at one point. How uncertain is it? If we could collect many fresh samples from the population, we would recompute the statistic on each and look at the spread. Usually we cannot (class deck: it may be impossible to collect many samples, and we may not want to assume a distribution for a formula).

The idea

Treat the sample as a miniature population. Draw new samples *from the sample*, with replacement, each the same size as the original. Each such resample is a **bootstrap sample**.

> Training set $\mathbf{Z} = (z_1, \ldots, z_N)$, $z_i = (\mathbf{x}_i, y_i)$. For $b = 1, \ldots, B$: draw N points from $\mathbf{Z}$ with replacement, giving $\mathbf{Z}^{*b}$, and compute the statistic $S(\mathbf{Z}^{*b})$. Then
>
> $$\overline{S^*} = \frac{1}{B}\sum_{b=1}^{B} S(\mathbf{Z}^{*b}), \qquad \widehat{\mathrm{Var}}[S] = \frac{1}{B-1}\sum_{b=1}^{B}\left(S(\mathbf{Z}^{*b}) - \overline{S^*}\right)^2.$$
>
> The square root of that variance is the **bootstrap standard error**.

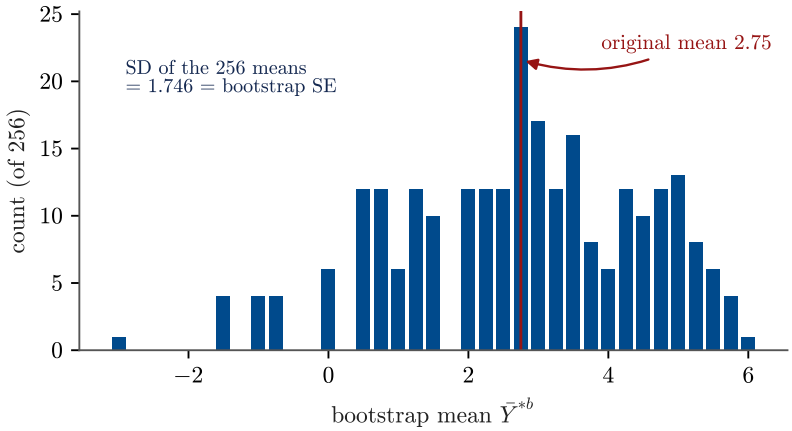
"With replacement" means a point can be picked more than once, and some points are not picked at all. On average a bootstrap sample leaves out about 36.8% of the original points. Why: a given point is missed on one draw with probability $1 - \frac{1}{N}$, so on all N draws with probability $(1 - \frac{1}{N})^N$, which tends to $e^{-1} = 0.368$. For $N = 4$ it is $(3/4)^4 = 0.316$.

Class example (Fox, 1997): mean difference between paired male and female scores

Four pairs: $(24, 18)$, $(14, 17)$, $(40, 35)$, $(44, 41)$. Differences $Y = (6, -3, 5, 3)$. Mean $\bar{Y} = \frac{6-3+5+3}{4} = 2.75$. Sample standard deviation (dividing by $n-1$): 4.031.
With $N = 4$ there are only $4^4 = 256$ equally likely ordered resamples, so we can list them *all* (the "ideal" bootstrap) instead of drawing B at random. A few from the slide:

resample	mean
$(6, 6, 6, 6)$	6.00
$(6, 6, 6, -3)$	3.75
$(6, 6, 6, 5)$	5.75
$(-3, 5, 6, 3)$	2.75
$(3, 3, 3, 3)$	3.00

Over all 256: the average of the means is exactly 2.75, and their standard deviation is $\boxed{1.746}$. That is the bootstrap standard error of the mean.
Compare with the textbook formula $s/\sqrt{N} = 4.031/2 = 2.016$. The bootstrap uses the "divide by N" spread ($\hat{\sigma} = 3.491$, giving $3.491/2 = 1.746$), so with tiny samples it runs a little low, by $\sqrt{(N-1)/N}$.



All 256 bootstrap means for Fox's four differences. Their spread is the bootstrap standard error.

From the textbook: ISLR's investment example (§5.2)

The best split between two investments

Invest a fraction α in asset X and $1-\alpha$ in asset Y. To minimise risk, minimise $\mathrm{Var}(\alpha X + (1-\alpha)Y)$. Using $\mathrm{Var}(aX+bY) = a^2\sigma_X^2 + b^2\sigma_Y^2 + 2ab\,\sigma_{XY}$:

$$V(\alpha) = \alpha^2\sigma_X^2 + (1-\alpha)^2\sigma_Y^2 + 2\alpha(1-\alpha)\sigma_{XY}.$$

Differentiate and set to zero: $V'(\alpha) = 2\alpha\sigma_X^2 - 2(1-\alpha)\sigma_Y^2 + 2(1-2\alpha)\sigma_{XY} = 0$. Collect the α terms: $\alpha(\sigma_X^2 + \sigma_Y^2 - 2\sigma_{XY}) = \sigma_Y^2 - \sigma_{XY}$, so

$$\alpha = \frac{\sigma_Y^2 - \sigma_{XY}}{\sigma_X^2 + \sigma_Y^2 - 2\sigma_{XY}}.$$

With ISLR's values $\sigma_X^2 = 1$, $\sigma_Y^2 = 1.25$, $\sigma_{XY} = 0.5$: $\alpha = \frac{0.75}{1.25} = 0.6$. In practice the variances are unknown, so we estimate $\hat{\alpha}$ from data. How uncertain is $\hat{\alpha}$? There is no simple formula, and that is exactly the bootstrap's job: resample the (return-X, return-Y) pairs 1000 times, recompute $\hat{\alpha}$ each time, and take the standard deviation of the 1000 values. ISLR's bootstrap gives $\mathrm{SE}(\hat{\alpha}) = 0.087$, very close to the 0.083 it finds by simulating 1000 fresh datasets from the true population: the bootstrap recovered the true uncertainty using one dataset.

Bootstrap prediction error, and its catch

The slide estimates prediction error as $\widehat{\mathrm{Err}}_{\mathrm{boot}} = \frac{1}{B}\frac{1}{N}\sum_b \sum_i L(y_i, \hat{f}^{*b}(\mathbf{x}_i))$: train on each bootstrap sample, test on the original data. The catch: each bootstrap sample contains about 63% of the original points, so the model is partly tested on its own training data, which makes the estimate optimistic. The fix is to score each point only with models that did *not* see it (the leave-one-out bootstrap), or blend with the training error (the ".632" estimator).

Check yourself

Q. CV or bootstrap for choosing λ?
A. CV, which estimates prediction error directly. Use the bootstrap for uncertainty (standard errors, confidence intervals) and for bagging.

Q. Why must preprocessing be done inside each fold?

A. Scaling or feature selection fitted on all the data lets the held-out fold leak into training, so CV becomes optimistic.

Q. $\mathrm{Var}(e^{(i)})$ **for LOOCV on the DC level with** $n = 5$**?**

A. $\frac{5}{4}\sigma^2$.

Q. Fraction of points left out of a bootstrap sample?

A. About $1/e \approx 36.8\%$.

Go deeper

Read. *An Introduction to Statistical Learning*, ch. 5. Validation set, LOOCV, k-fold, bootstrap, with the same polynomial example.

Read. *Elements of Statistical Learning*, §7.10–7.11. The right and wrong way to do CV; the .632 estimator.

Watch. Kilian Weinberger, Cornell CS4780, *Lecture 20: Model Selection / Regularization / Overfitting*

Exercises

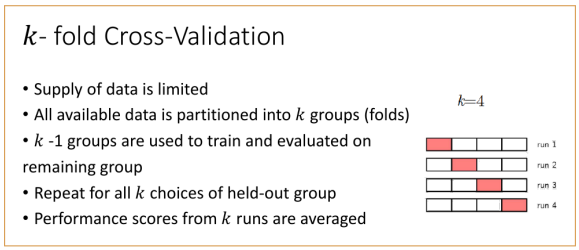
Pencil first, then click the ⇝ to jump to the worked answer in Appendix D. The answer links back to the question.

21.1 ⇝ For the DC level with $n = 11$, $\sigma^2 = 4$: give $\mathrm{Var}(\hat{A})$, $\mathrm{Var}(\hat{A}_{-\kappa(i)})$ and $\mathrm{Var}(e^{(i)})$.

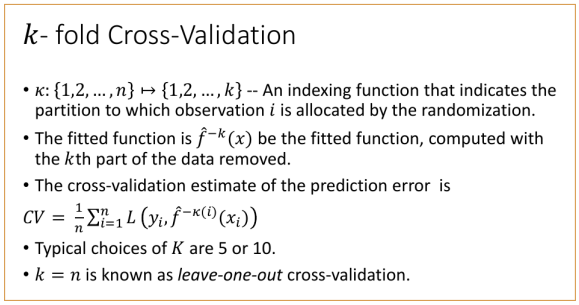
21.2 ⇝ With $N = 3$ data points $(1, 4, 7)$, list how many ordered bootstrap samples there are, and compute the ideal bootstrap SE of the mean.

21.3 ⇝ A model is evaluated with 5-fold CV on 100 points. How many models are fitted, on how many points each? Repeat for LOOCV.

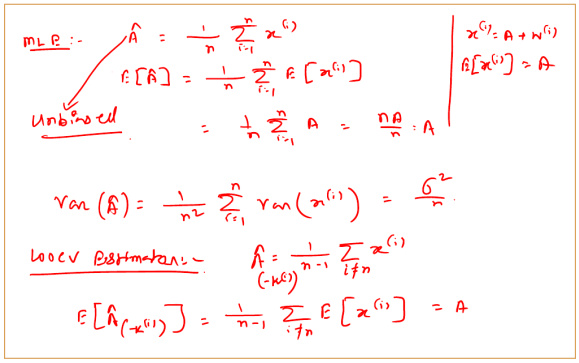
Lecture slides for reference



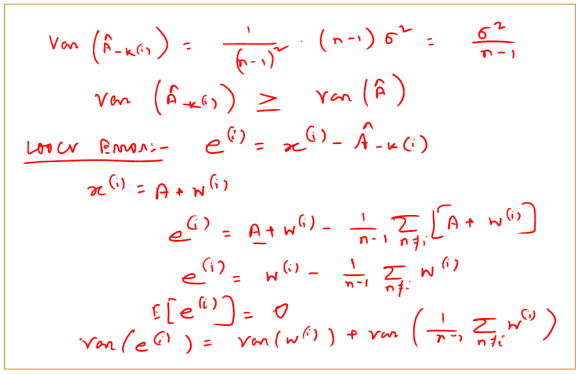
Sir's lecture notes, slide 012. k-fold cross-validation.



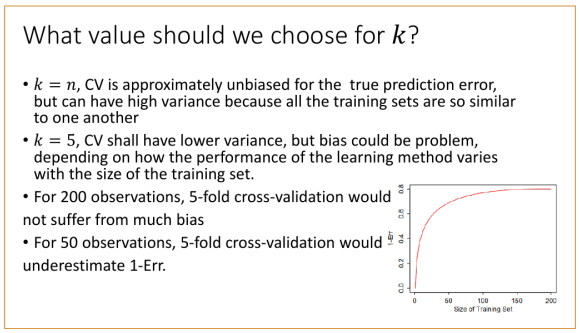
Sir's lecture notes, slide 013. The CV formula.



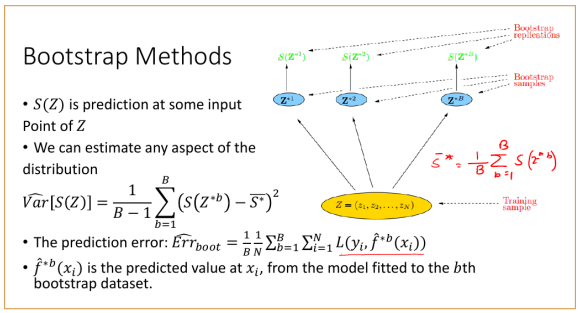
Sir's lecture notes, slide 021. DC level: the MLE.



Sir's lecture notes, slide 022. LOOCV estimator and error.



Sir's lecture notes, slide 024. Choosing k.



Sir's lecture notes, slide 030. The bootstrap.

Bootstrap Example

- Adapted from Fox (1997) "Applied Regression Analysis"
- Goal: Estimate mean difference between Male and Female
- Four pairs of observations are available:

Observ.	Male	Female	Differ. Y
1	24	18	6
2	14	17	-3
3	40	35	5
4	44	41	3

Mean = 2.75
Std Dev = 4.04

Sample	Y_1^*	Y_2^*	Y_3^*	Y_4^*	$\bar{y}^*$
1	6	6	6	6	6.00
2	6	6	6	-3	3.75
3	6	6	6	5	5.75
..					
100	-3	5	6	3	2.75
101	-3	5	-3	6	1.25
...					
255	-3	3	3	5	3.5
256	3	3	3	3	3.00

Sir's lecture notes, slide 031. Fox's example.

22

Classification by Distance: Centroids and K-NN

The oldest idea for classifying something new is also the most human: *a new thing is probably like the things it is close to.* A fruit that looks and weighs like the apples in your basket is probably an apple. This chapter turns that sentence into two algorithms, works every example by hand, and then asks the questions that matter in practice: which K, which distance, how expensive, and why feature scales can ruin everything. Sir's slides are at the end for reference.

22.1 The setting

We have n labelled training examples $(\mathbf{x}^{(1)}, y^{(1)}), \ldots, (\mathbf{x}^{(n)}, y^{(n)})$. Each $\mathbf{x}^{(i)}$ is a point in d-dimensional space (its features); each $y^{(i)}$ is a class label. A new point $\mathbf{x}$ arrives without a label. We must predict one. Everything in this chapter is a way to measure "close" and then copy labels from close points.

22.2 Idea 1: distance from the class means

The rule in words

Summarise each class by one representative point, its **mean** (average position, also called its centroid). Give the new point the label of whichever mean is nearer.

The rule in symbols

Use labels $y \in \{-1, +1\}$, with n_- negative and n_+ positive examples. The two means are

$$\boldsymbol{\mu}_- = \frac{1}{n_-} \sum_{i:\, y_i = -1} \mathbf{x}_i, \qquad \boldsymbol{\mu}_+ = \frac{1}{n_+} \sum_{i:\, y_i = +1} \mathbf{x}_i.$$

After computing the two means you can throw the training data away: two points are the whole model.

Derivation: the rule is secretly a straight line

Derivation

Define $f(\mathbf{x}) = \|\boldsymbol{\mu}_- - \mathbf{x}\|^2 - \|\boldsymbol{\mu}_+ - \mathbf{x}\|^2$ (squared distance to the negative mean minus squared distance to the positive mean). If $f > 0$, $\mathbf{x}$ is farther from $\boldsymbol{\mu}_-$, so predict $+1$.
Step 1: expand a squared distance. For any vectors, $\|\mathbf{a}-\mathbf{b}\|^2 = (\mathbf{a}-\mathbf{b})^\top(\mathbf{a}-\mathbf{b}) = \|\mathbf{a}\|^2 - 2\mathbf{a}^\top\mathbf{b} + \|\mathbf{b}\|^2$ (exactly like $(a-b)^2 = a^2 - 2ab + b^2$).
Step 2: apply it twice.

$$f(\mathbf{x}) = \big(\|\boldsymbol{\mu}_-\|^2 - 2\boldsymbol{\mu}_-^\top\mathbf{x} + \|\mathbf{x}\|^2\big) - \big(\|\boldsymbol{\mu}_+\|^2 - 2\boldsymbol{\mu}_+^\top\mathbf{x} + \|\mathbf{x}\|^2\big).$$

Step 3: cancel. The $\|\mathbf{x}\|^2$ appears once with $+$ and once with $-$. It disappears, and this is the whole point: the only term that could have curved the boundary is gone.

$$f(\mathbf{x}) = 2(\boldsymbol{\mu}_+ - \boldsymbol{\mu}_-)^\top\mathbf{x} + \big(\|\boldsymbol{\mu}_-\|^2 - \|\boldsymbol{\mu}_+\|^2\big) = \mathbf{w}^\top\mathbf{x} + b,$$

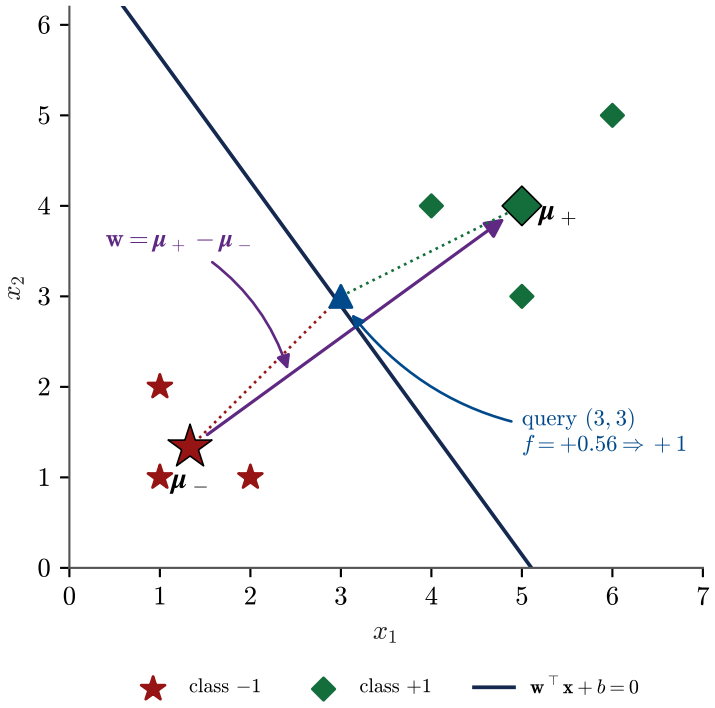
with $\mathbf{w} = 2(\boldsymbol{\mu}_+ - \boldsymbol{\mu}_-)$ and $b = \|\boldsymbol{\mu}_-\|^2 - \|\boldsymbol{\mu}_+\|^2$.
Step 4: read it. The prediction is $y = \text{sign}(\mathbf{w}^\top\mathbf{x} + b)$, and the boundary $\mathbf{w}^\top\mathbf{x} + b = 0$ is a straight line (a hyperplane in d dimensions).
Step 5: $\mathbf{w}$ is perpendicular to the boundary. Take any two points $\mathbf{x}_a, \mathbf{x}_b$ on the boundary: $\mathbf{w}^\top\mathbf{x}_a + b = 0$ and $\mathbf{w}^\top\mathbf{x}_b + b = 0$. Subtract: $\mathbf{w}^\top(\mathbf{x}_a - \mathbf{x}_b) = 0$. The vector $\mathbf{x}_a - \mathbf{x}_b$ lies *along* the boundary, and its dot product with $\mathbf{w}$ is zero, so $\mathbf{w}$ is perpendicular to every direction inside the boundary. Geometrically, the boundary is the **perpendicular bisector** of the segment joining the two means.

Three stars, three diamonds

Class -1: $(1,2), (2,1), (1,1)$. Mean: $\boldsymbol{\mu}_- = \left(\frac{1+2+1}{3}, \frac{2+1+1}{3}\right) = (1.333,\ 1.333)$. Class $+1$: $(4,4), (5,3), (6,5)$. Mean: $\boldsymbol{\mu}_+ = (5,\ 4)$.

$$\mathbf{w} = 2\big((5,4) - (1.333, 1.333)\big) = 2\,(3.667,\ 2.667) = (7.333,\ 5.333),$$
$$b = \|\boldsymbol{\mu}_-\|^2 - \|\boldsymbol{\mu}_+\|^2 = (1.778 + 1.778) - (25 + 16) = 3.556 - 41 = -37.444.$$

Query $(3,3)$**:** $f = 7.333(3) + 5.333(3) - 37.444 = 22 + 16 - 37.444 = +0.556 > 0$, so predict $+1$. Check directly: squared distance to $\boldsymbol{\mu}_-$ is $(1.667)^2 + (1.667)^2 = 5.556$; to $\boldsymbol{\mu}_+$ it is $2^2 + 1^2 = 5$. Closer to $\boldsymbol{\mu}_+$, and $5.556 - 5 = 0.556 = f$. The shortcut and the definition agree. **Query** $(2,4)$**:** $f = 14.667 + 21.333 - 37.444 = -1.444 < 0$, so predict -1 (distances 7.556 and 9).



The worked example: the boundary is the perpendicular bisector of the segment $\boldsymbol{\mu}_-\boldsymbol{\mu}_+$, and $\mathbf{w}$ points from the negative mean to the positive one.

Strengths and limits

- Simple, fast, and tiny to store (one point per class).
- Needs enough data per class to estimate each mean reliably.
- **Only straight boundaries**, because $\|\mathbf{x}\|^2$ cancelled. If a class is ring-shaped, its mean sits in the hole, and the method fails.
- Treats every class as a round blob. Replacing "distance to the mean" with "likelihood under a class-shaped Gaussian" leads to linear discriminant analysis (LDA).

22.3 Idea 2: nearest neighbour (1-NN)

The rule

Keep *all* the training points. Give the new point the label of the single closest training point:

$$\hat{y}(\mathbf{x}) = y^{(i^*)}, \qquad i^* = \arg\min_i \ \|\mathbf{x} - \mathbf{x}^{(i)}\|.$$

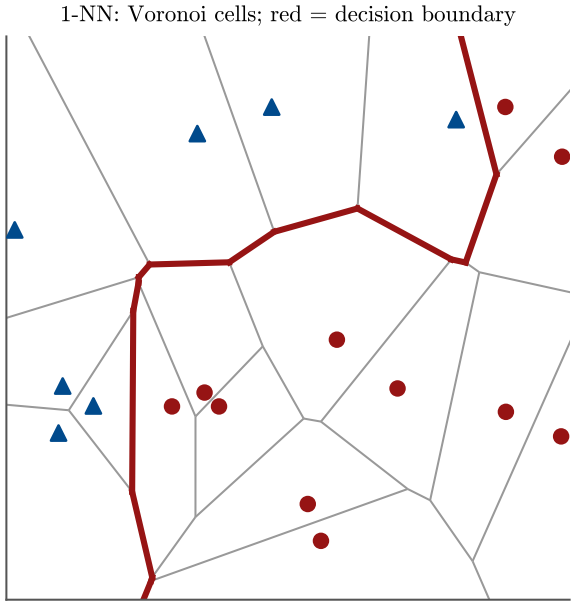
Every training point is a **prototype**. Nothing is "learned" in advance, so the method is called *memory-based*, *instance-based*, *lazy*, or *non-parametric* (the size of the model grows with the data).

The picture: Voronoi cells

For each training point $\mathbf{x}_i$, its **Voronoi cell** is the region of space closer to it than to any other training point:

$$V_i = \{\mathbf{x} : \ \|\mathbf{x} - \mathbf{x}_i\| \leq \|\mathbf{x} - \mathbf{x}_j\| \ \text{ for every } j \neq i\}.$$

1-NN predicts y_i everywhere inside V_i. Why are the cells convex polygons? The set of points closer to $\mathbf{x}_i$ than to one particular $\mathbf{x}_j$ is a half-plane (one side of their perpendicular bisector, by the same algebra as Idea 1). V_i is the intersection of these half-planes, one for each competitor, and an intersection of half-planes is a convex polygon.



1-NN on 18 points. Grey: all Voronoi edges. Red: only the edges between cells of different classes. Those form the decision boundary: a chain of perpendicular bisectors.

> **Ask before reading on**
>
> One wrongly labelled point sits deep inside the other class. With 1-NN, what happens to the region around it? What simple change would make the classifier ignore it?

It carves out an island of wrong predictions (its whole Voronoi cell). The fix is to ask more than one neighbour.

How good can 1-NN be?

Cover and Hart (1967) proved that with unlimited data, the error rate R of 1-NN satisfies $R^* \leq R \leq 2R^*$, where R^* is the **Bayes error**: the error of the best possible classifier that knows the true distribution. So a method with no training at all is, in the limit, at most twice as bad as the best anything can do.

22.4 K-nearest neighbours

The algorithm, step by step

1. **Choose** K, the number of neighbours to consult (for example $K = 3$).

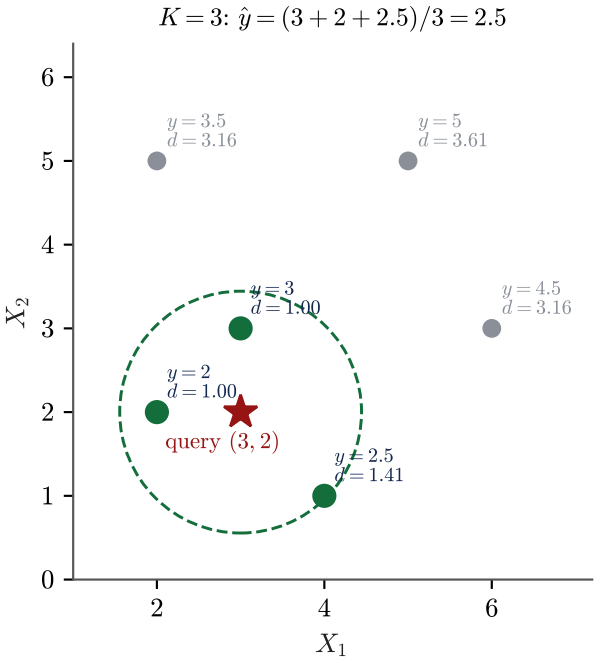
2. **Compute distances** from the query $\mathbf{x}$ to every training point.
3. **Rank** the training points from nearest to farthest.
4. **Select** the top K.
5. **Predict.** Classification: the most frequent label among the K. Regression: the average of their y values, $\hat{y} = \frac{1}{K}\sum_{j=1}^{K} y^{(j)}$.

Class deck: K-NN regression with $K = 3$ at $(3, 2)$

Data $(X_1, X_2; Y)$: $(2, 5; 3.5)$, $(5, 5; 5)$, $(3, 3; 3)$, $(6, 3; 4.5)$, $(2, 2; 2)$, $(4, 1; 2.5)$. Euclidean distances from $(3, 2)$:

point	distance	Y
$(3, 3)$	$\sqrt{0^2 + 1^2} = 1.000$	3
$(2, 2)$	$\sqrt{1^2 + 0^2} = 1.000$	2
$(4, 1)$	$\sqrt{1^2 + 1^2} = 1.414$	2.5
$(2, 5)$	$\sqrt{1^2 + 3^2} = 3.162$	3.5
$(6, 3)$	$\sqrt{3^2 + 1^2} = 3.162$	4.5
$(5, 5)$	$\sqrt{2^2 + 3^2} = 3.606$	5

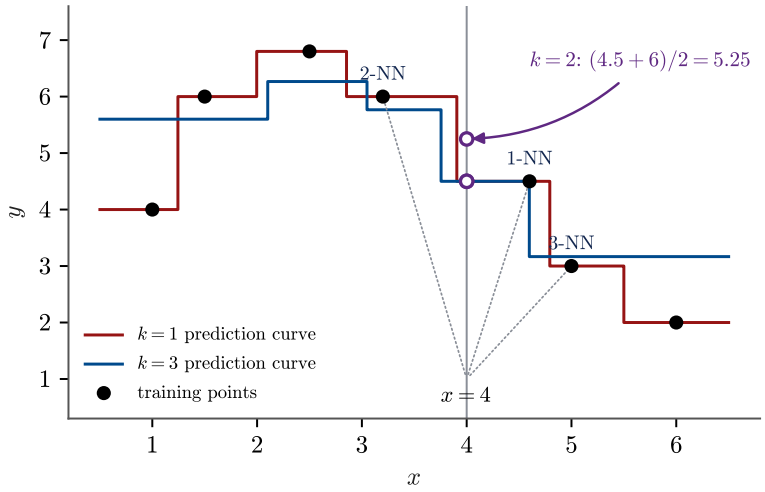
The three nearest are the first three rows. Prediction: $\hat{Y} = \frac{3+2+2.5}{3} = \boxed{2.5}$.



The same example: the dashed circle reaches exactly the three nearest points (green).

One dimension: K changes the answer

Points (x, y): $(1, 4)$, $(1.5, 6)$, $(2.5, 6.8)$, $(3.2, 6)$, $(4.6, 4.5)$, $(5, 3)$, $(6, 2)$. Query $x = 4$. Distances: $4.6 \to 0.6$, $3.2 \to 0.8$, $5 \to 1.0$, $2.5 \to 1.5$, $6 \to 2.0$, $1.5 \to 2.5$, $1 \to 3$. $K = 1$: 4.5. $K = 2$: $(4.5 + 6)/2 = 5.25$. $K = 3$: $(4.5 + 6 + 3)/3 = 4.5$. $K = 7$: every point, 4.614 (the global mean).

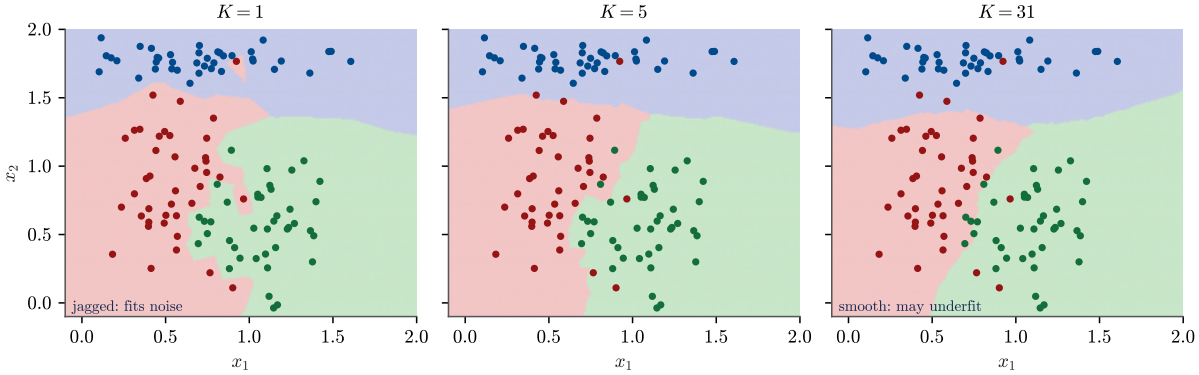


K-NN regression gives a step function: constant until the set of nearest neighbours changes.

Watch out

K-NN cannot extrapolate. Its prediction is an average of training targets, so it never leaves $[\min y,\ \max y]$, and beyond the last training point it is flat. For prediction outside the range of the data, use a model with a formula (linear regression).

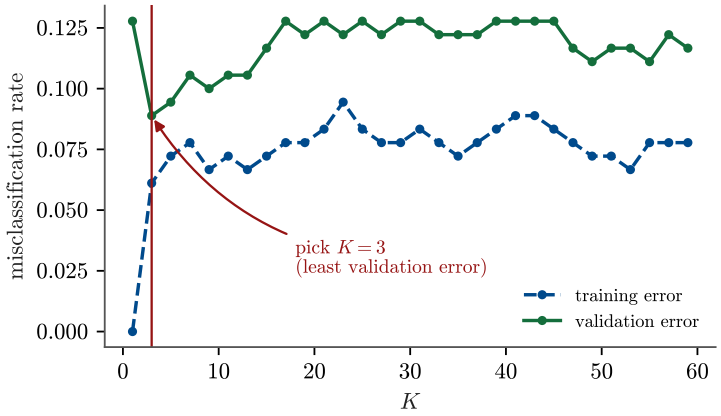
22.4.1 Choosing K



Three classes, $K = 1, 5, 31$. Small K follows every noisy point; large K smooths the boundary and eventually blurs the classes.

Mental model

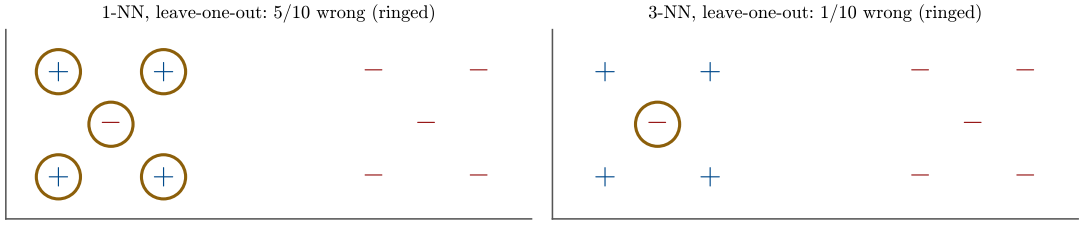
K is a capacity dial that runs *backwards*. $K = 1$: highest capacity, zero training error, an island for every outlier (overfitting). $K = n$: lowest capacity, always the majority class (underfitting). So K is chosen like the polynomial degree or λ: try a range, measure **validation** error, keep the K with the least.



1-NN has training error 0 (every point is its own nearest neighbour), so training error is useless for choosing K. Validation error picks $K = 3$ here.

Class deck: leave-one-out error of 1-NN and 3-NN

Ten points: four + on the left, one − between them, and five − on the right. Leave-one-out: hide each point in turn, classify it from the other nine, and count mistakes. **1-NN.** Each left + is closest to the − in the middle of the left group, so all four are misclassified. The middle − is closest to a +, so it is wrong too. The right-hand − points all have − neighbours: correct. Error 5/10. **3-NN.** A left + now sees the middle − *and* two other +: majority +, correct. The middle − sees three +: still wrong. Error 1/10. Using three neighbours outvoted the one awkward point.



The ringed points are the ones misclassified when held out.

22.4.2 Breaking ties

With even K the vote can split evenly. Sir's three rules: (1) toss a coin; (2) prefer the class with the larger prior $P(C_k) = N_k/N$ (the more common class overall); (3) fall back to the single nearest neighbour. In practice also: use odd K for two classes, or weight each vote by 1/distance.

22.5 K-NN as a probability estimate (derivation)

Why should "count the labels of the neighbours" be a sensible rule? Because it is Bayes' rule with estimated densities.

Derivation

Step 1: estimate the density. Draw a small region $\mathcal{R}$ around $\mathbf{x}$, just big enough to hold K of the N training points. Let V be its volume. The probability that a random point lands in $\mathcal{R}$ is $P = \int_{\mathcal{R}} p(\mathbf{x}')\, d\mathbf{x}'$. If $\mathcal{R}$ is small, the density hardly changes inside it, so

$P \approx p(\mathbf{x})\,V$. We also estimate P by the fraction of points that landed there: $P \approx K/N$. Equate:

$$p(\mathbf{x}) \approx \frac{K}{NV}.$$

Step 2: the same inside one class. Class C_k has N_k training points, and K_k of them are inside $\mathcal{R}$. The same argument gives $p(\mathbf{x} \mid C_k) \approx \dfrac{K_k}{N_k V}$. The prior is $P(C_k) = \dfrac{N_k}{N}$.

Step 3: Bayes' rule.

$$P(C_k \mid \mathbf{x}) = \frac{p(\mathbf{x} \mid C_k)\,P(C_k)}{p(\mathbf{x})} = \frac{\dfrac{K_k}{N_k V} \cdot \dfrac{N_k}{N}}{\dfrac{K}{NV}} = \frac{K_k}{K}.$$

N_k, N and V all cancel. The posterior probability of class k is simply the fraction of the K neighbours that belong to it. So the majority vote is "choose the class with the highest posterior".

22.6 Distance functions

Everything depends on what "close" means.

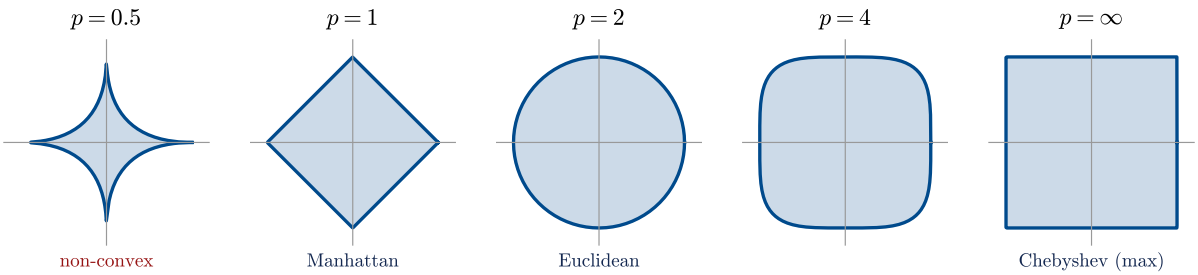
Distance	Formula	Use / property
Euclidean	$\sqrt{\sum_d \lvert x_d - x'_d\rvert^2}$	symmetric, spherical, all dimensions equal; sensitive to outliers and scale
Manhattan ($p = 1$)	$\sum_d \lvert x_d - x'_d\rvert$	city-block distance; one large difference counts less
Minkowski (p-norm)	$\left(\sum_d \lvert x_d - x'_d\rvert^p\right)^{1/p}$	$p = 1$ Manhattan, $p = 2$ Euclidean, $p \to \infty$ the largest single difference
Hamming	$\sum_d \mathbf{1}[x_d \neq x'_d]$	number of attributes that differ; categorical data, strings
KL divergence	$\sum_d x_d \log \frac{x_d}{x'_d}$	between histograms ($x_d > 0$, $\sum_d x_d = 1$); not symmetric

One pair of points, every distance

$\mathbf{x} = (1, 2, 3)$, $\mathbf{x}' = (4, 0, 3)$; absolute differences $(3, 2, 0)$. Manhattan: $3 + 2 + 0 = 5$. Euclidean: $\sqrt{9 + 4 + 0} = \sqrt{13} = 3.606$. $p = 3$: $(27 + 8)^{1/3} = 3.271$. $p = 10$: 3.005. $p = \infty$: $\max(3, 2, 0) = 3$. As p grows, the largest difference takes over. Hamming between the strings `karolin` and `kathrin`: positions 3, 4, 5 differ, so 3. KL between histograms $(0.5, 0.3, 0.2)$ and $(0.4, 0.4, 0.2)$: $0.5 \ln \frac{0.5}{0.4} + 0.3 \ln \frac{0.3}{0.4} + 0 = 0.0253$; the other way round 0.0258. Not equal, so KL is not a true distance.

Watch out

Sir's board writes KL with a leading minus sign. The standard definition is $D_{\mathrm{KL}}(\mathbf{x} \,\|\, \mathbf{x}') = \sum_d x_d \log \frac{x_d}{x'_d} \geq 0$, no minus (equivalently $-\sum_d x_d \log \frac{x'_d}{x_d}$).



The "unit circle" $\{\mathbf{x} : \|\mathbf{x}\|_p \leq 1\}$ for different p. For $p \leq 1$ it has corners on the axes, which is why $p \leq 1$ penalties promote sparsity (LASSO chapter).

22.7 Normalization: why feature scales matter

Class deck: area and price

Feature 1 ranges over 1 to 10 (say, rooms). Feature 2 ranges over 101 000 to 501 000 (say, price). Query $(5,\ 25\,000)$ against a training point $(7,\ 35\,000)$:

$$\sqrt{(5-7)^2 + (25\,000 - 35\,000)^2} = \sqrt{4 + 100\,000\,000} \approx 10\,000.0002.$$

The difference in feature 1 changed the distance in the eighth digit. Feature 1 is effectively ignored, purely because of its units.

For each feature j, compute its mean μ_j and standard deviation σ_j over the training set, then replace every value by

$$\tilde{x}_j = \frac{x_j - \mu_j}{\sigma_j}.$$

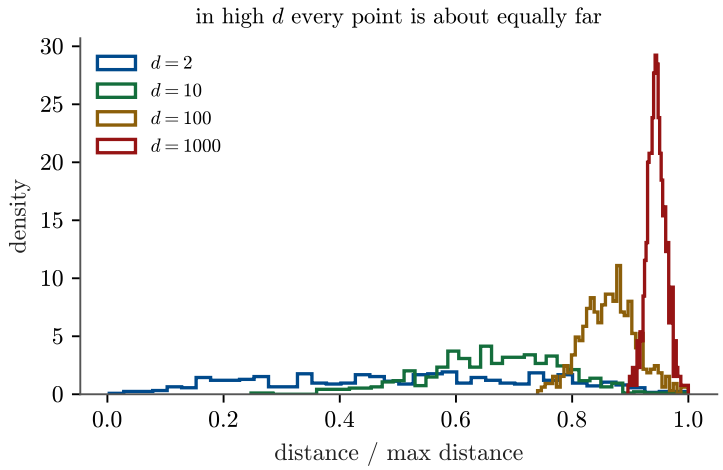
Every feature then has mean 0 and standard deviation 1, so each contributes on a comparable scale. Use the *training* μ_j, σ_j for test points too.

22.8 Computational cost

- **Training:** zero work. Just store the data. (That is why K-NN is called a lazy algorithm.)
- **Each prediction:** compute N distances in d dimensions, $O(Nd)$; sort them, $O(N \log N)$ (or pick the K smallest in $O(N \log K)$).
- **Memory:** the entire training set.

This is the reverse of most models, which are slow to train and fast to use. Data structures such as KD-trees and ball trees, or approximate search, cut the per-query cost.

22.9 The curse of dimensionality



500 random points in a d-dimensional cube; distances from a random query, divided by the largest. In $d = 2$ they range from near 0 to 1. In $d = 1000$ they all crowd near the maximum: the "nearest" neighbour is barely nearer than the farthest.

In high dimensions every point is roughly equally far from every other, so "nearest" stops meaning "similar". K-NN works best with few features, or after reducing dimension.

22.10 Class deck: the $[4a, 3b]$ grid question

1-NN on a grid

A point (x_1, x_2) is positive if $x_1 > x_2$. The training set is every point $(4a,\ 3b)$ with integers a, b: $(0,0), (4,0), (4,3), (8,6), (8,9), (12,9), \ldots$, each labelled by the rule. For each query, find the nearest grid point (nearest multiple of 4 for x_1, of 3 for x_2) and copy its label.

query	nearest grid point	its label	1-NN predicts
$(5.52,\ 2.41)$	$(4,\ 3)$	$4 > 3$: +	positive
$(8.47,\ 5.84)$	$(8,\ 6)$	$8 > 6$: +	positive
$(7,\ 8.17)$	$(8,\ 9)$	$8 < 9$: −	negative
$(6.7,\ 8.88)$	$(8,\ 9)$	$8 < 9$: −	negative

So the first two queries are labelled positive. Each answer comes from a single grid point, which is the 1-NN idea in its purest form.

22.11 From the textbook: the Bayes classifier (ISLR §2.2.3)

> If we knew the true $P(Y = k \mid \mathbf{x})$, the best possible rule would be: assign $\mathbf{x}$ to the class with the largest posterior. This is the **Bayes classifier**, and its error rate, the **Bayes error rate** $1 - \mathbb{E}\big[\max_k P(Y = k \mid X)\big]$, is the lowest any classifier can achieve. It plays the role that the noise σ^2 played in regression: an irreducible floor.

A Bayes error you can compute

Two equally likely classes on a line, $X \mid Y=0 \sim \mathcal{N}(0,1)$ and $X \mid Y=1 \sim \mathcal{N}(2,1)$. By symmetry the Bayes boundary is $x=1$. A class-0 point lands beyond 1 with probability $P(Z>1)=0.1587$, and a class-1 point below 1 with the same probability, so the Bayes error is $\boxed{0.159}$. No classifier, however clever, beats 15.9% here.

K-NN tries to *estimate* the Bayes classifier using the neighbours' label fractions K_k/K (derived above). Cover and Hart's result says that with infinite data, 1-NN's error is at most twice this floor.

22.12 From the textbook: the curse of dimensionality, quantified (ESL §2.5)

Derivation

Data are uniform in the unit cube $[0,1]^p$. To make a neighbourhood that captures a fraction r of the data (say $r=1\%$ for a local average), use a sub-cube with edge length e. Its volume e^p must equal r, so

$$e_p(r) = r^{1/p}.$$

dimension p	1	2	3	10	100
edge for $r=1\%$	0.01	0.10	0.215	0.631	0.955
edge for $r=10\%$	0.10	0.316	0.464	0.794	0.977

In 10 dimensions, capturing just 1% of the data needs a box covering 63% of the range of *every* feature. The "neighbourhood" is no longer local, so its average no longer describes the point. That is the curse of dimensionality in one formula.

22.13 Advantages and disadvantages

Advantages	**Disadvantages**
no assumption about the data distribution (non-parametric)	stores all training data; slow at test time
works for classification and regression, any number of classes	sensitive to feature scale, outliers and irrelevant features
adapts online: adding a point is free	poor in high dimensions; cannot extrapolate

Where this shows up

Sir's MNIST note: pixel-wise Euclidean 1-NN on 16×16 digits gets about 95.2%, an SVM 95.8%, a human 97.5%. Your Lab 6 got 95.1% with $K=3$ on 28×28 digits. Modern search engines and retrieval systems are fast approximate K-NN over learned feature vectors.

Check yourself

Q. Is distance-from-means parametric?
A. Yes: a fixed number of parameters (the means). K-NN is non-parametric.

Q. Why scale features before K-NN?
A. Distances add squared differences, so a large-unit feature drowns the others.

Q. Training error of 1-NN?
A. Zero, unless identical points carry different labels.

Q. Posterior from K-NN?
A. $P(C_k \mid \mathbf{x}) \approx K_k/K$.

Go deeper

Read. Cover and Hart, "Nearest neighbor pattern classification" (1967). The twice-Bayes bound.

Read. Bishop, *PRML*, §2.5.2. The $K/(NV)$ density estimate.

Watch. Kilian Weinberger, Cornell CS4780, *Lecture 3: k-nearest neighbors*

Exercises

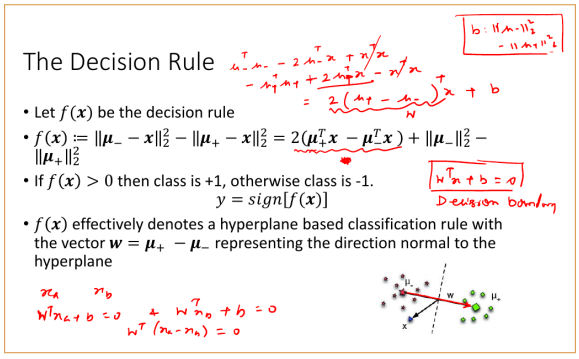
Pencil first, then click the ⇝ to jump to the worked answer in Appendix D. The answer links back to the question.

22.1 ⇝ Class -1: $(0,0),(2,0)$. Class $+1$: $(4,4),(6,2)$. Find $\mathbf{w}$, b for the distance-from-means rule and classify $(3,1)$.

22.2 ⇝ With the one-dimensional regression data of this chapter, predict at $x = 5.4$ for $K = 1, 2, 3$.

22.3 ⇝ For $\mathbf{x} = (0,0)$, $\mathbf{x}' = (3,4)$ compute the Manhattan, Euclidean and Chebyshev ($p = \infty$) distances. Which is always the largest of the three, and why?

Lecture slides for reference



Sir's lecture notes, slide 045. Distance from means: the decision rule.

1-NN Visualization

- Every training point is a prototype
- A test point x is assigned the label of the nearest prototype

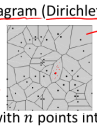
$\hat{y}(x) = y_i$ s.t. $i = \arg\min_i \|x - x_i\|_2^2$

- 1-NN induces a Voronoi diagram (Dirichlet tessellation).

Given a set of training points $\{x_1, \dots x_n\}$

Voronoi cell of x_i

$V_i = \{x : \|x - x_i\| \le \|x - x_j\| \; \forall j \neq i\}$



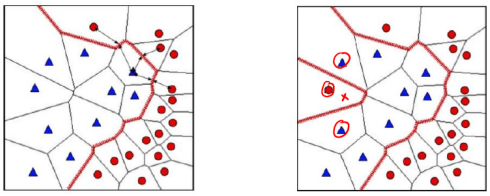
Voronoi cell

- The partitioning of plane with n points into convex polygons
- Each polygon contains exactly one seed point
- Every point in a given polygon is closer to its seed point than to any others

Sir's lecture notes, slide 049. 1-NN and Voronoi cells.

The Decision Boundary of 1-NN

- The decision boundary is composed of hyperplanes that form perpendicular bisectors of pairs of points from different classes



Sir's lecture notes, slide 050. 1-NN decision boundary.

K-NN classification –

Given training examples $\{x_i, y_i\}$

- x_i: features / examples
- y_i: label

test point x that we want to classify.

Algorithm:-

- Compute distance $D(x, x_i)$ to every training example x_i
- Select k closest instances $x_{i_1}, \dots, x_{i_k}$ and their labels $y_{i_1}, \dots y_{i_k}$
- Output the class y^* which is most frequent in $y_{i_1}, \dots y_{i_k}$

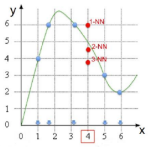
Sir's lecture notes, slide 052. K-NN algorithm.

K-NN for Regression

- Given: training examples $\{(\boldsymbol{x}^{(1)}, y^{(1)}), (\boldsymbol{x}^{(2)}, y^{(2)}), \cdots, (\boldsymbol{x}^{(n)}, y^{(n)})\}$
  - $\boldsymbol{x}^{(i)}$ attribute-value representation of examples
  - $y^{(i)}$ real-valued target (profit, rating on YouTube, etc.)
- Testing point $\boldsymbol{x}$ that we want to predict the target

Algorithm:

- compute distance $D(\boldsymbol{x}, \boldsymbol{x}^{(i)})$ to every training example
- select k closest instances $\boldsymbol{x}^{(1)}, \boldsymbol{x}^{(2)}, \dots, \boldsymbol{x}^{(k)}$ and their labels $y^{(1)}, y^{(2)}, \dots, y^{(k)}$
- output the mean of $y^{(1)}, y^{(2)}, \dots, y^{(k)}$: $\bar{y} = \frac{1}{k}\sum_{j=1}^{k} y^{(j)}$



Sir's lecture notes, slide 057. K-NN regression.

Distance fn:-

(i) Euclidean Distance:- x, x'

$D(x, x') = \sqrt{\sum_d |x_d - x'_d|^2}$

- Symmetric, spherical, treats all dimensions equally
- Sensitive to outliers

(ii) Hamming Distance:- $D(x, x') = \sum_d \mathbf{1}_{x_d \neq x'_d}$

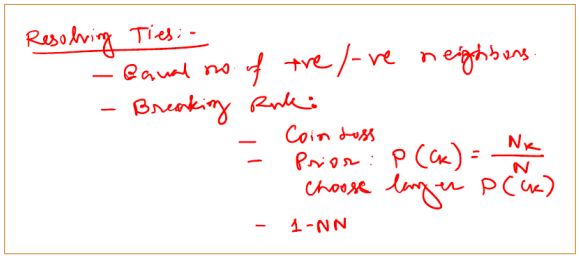
↳ Number of attributes where $x_d \neq x'_d$ differs

(iii) Minkowski Distance:- (p-norm)

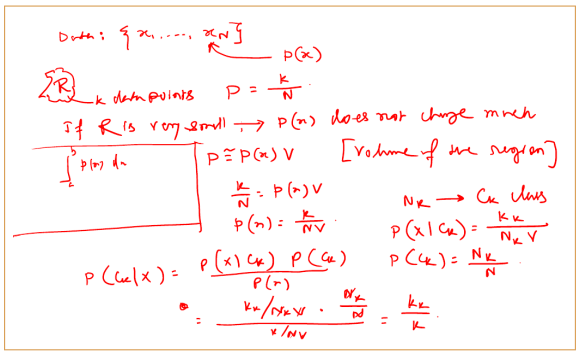
$D(x, x') = \sqrt[p]{\sum_d |x_d - x'_d|^p}$

$p = 2 \longrightarrow$ Euclidean Distance

Sir's lecture notes, slide 058. Distance functions.



Sir's lecture notes, slide 060. Resolving ties.



Sir's lecture notes, slide 061. K-NN as density estimation.

23

Naive Bayes

K-NN never wrote down a model of the data. Naive Bayes does the opposite: it tells a small probabilistic story about how each class *produces* its features, then turns the story around with Bayes' rule to ask "given these features, which class is most likely?". One bold assumption turns an impossible estimation problem into simple counting. We rebuild every piece from basic probability and work sir's example completely. His slides are at the end for reference.

23.1 Three facts from probability you need

Fact 1: conditional probability

$P(A \mid B)$ is the probability of A *among the cases where* B *happened*:

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)} \qquad \text{equivalently} \qquad P(A \cap B) = P(A \mid B)\, P(B).$$

Counting version: $P(A \mid B) = \dfrac{\#(A \text{ and } B)}{\#B}$. Example: of 4 days, 3 had "play = yes"; on all 3 of those the sky was sunny. So $P(\text{sunny} \mid \text{play}) = \frac{3}{3} = 1$.

Fact 2: Bayes' rule

Derivation

Write the joint probability two ways using Fact 1:

$$P(A \cap B) = P(A \mid B)\, P(B) \qquad \text{and} \qquad P(A \cap B) = P(B \mid A)\, P(A).$$

Set them equal and divide by $P(B)$:

$$\boxed{P(A \mid B) = \frac{P(B \mid A)\, P(A)}{P(B)}}$$

Read with A = hypothesis (the class Y) and B = evidence (the features X):

$$P(Y = y_k \mid X = \mathbf{x}) = \frac{P(X = \mathbf{x} \mid Y = y_k)\, P(Y = y_k)}{P(X = \mathbf{x})} \qquad \Big(\text{posterior} = \frac{\text{likelihood} \times \text{prior}}{\text{evidence}}\Big).$$

The **prior** $P(Y)$ is how common each class is before seeing anything. The **likelihood** $P(X \mid$

Y) is how typical these features are for that class. The **posterior** combines both. Bayes' rule lets us compute the hard thing we want ($P(Y \mid X)$) from two easier things we can count.

Fact 3: independence and conditional independence

Events are **independent** if $P(A \cap B) = P(A)P(B)$: knowing one tells you nothing about the other. They are **conditionally independent given** C if $P(A \cap B \mid C) = P(A \mid C)\, P(B \mid C)$: *once you know* C, knowing A tells you nothing more about B.

23.2 The classifier we want, and the obstacle

To classify $\mathbf{x} = (x_1, \ldots, x_d)$, compute $P(Y = y_k \mid \mathbf{x})$ for every class k and pick the largest. Look at the three pieces of Bayes' rule:

- $P(Y = y_k)$: **easy**. Count how often each class occurs.
- $P(X = \mathbf{x})$: **not needed**. It is the same number for every class (it does not mention k), so it cannot change which class has the largest posterior.
- $P(X_1 = x_1 \cap \cdots \cap X_d = x_d \mid Y = y_k)$: **necessary, but impractical**.

> **Ask before reading on**
>
> Ten yes/no features, two classes. How many numbers do you need to describe every $P(X_1, \ldots, X_{10} \mid Y)$? How many training rows would it take to see each feature combination even once?

Ten binary features can take $2^{10} = 1024$ combinations. For each class we need a probability for each combination (1023 free numbers, since they must sum to 1), so $2 \times 1023 = 2046$ numbers in total. A dataset of a few hundred rows never shows most combinations, so most of those numbers would be estimated as 0. With d binary features the count is $2(2^d - 1)$: it doubles with every new feature.

23.3 The naive assumption

Assume the features are **conditionally independent given the class**:

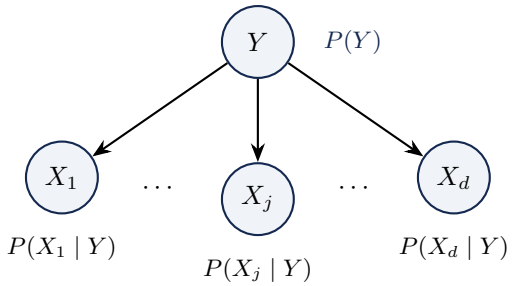
$$P(X_1, \ldots, X_d \mid Y) = P(X_1 \mid Y)\, P(X_2 \mid Y) \cdots P(X_d \mid Y) = \prod_{j=1}^{d} P(X_j \mid Y).$$

Now each feature needs its own small table per class. For binary features: d numbers per class instead of $2^d - 1$. Ten features: 20 numbers instead of 2046.

Watch out

Conditionally independent is not the same as independent. In spam, "free" and "money" appear together a lot overall, because both are common in spam. Naive Bayes only assumes that *within* spam (and within non-spam) one word tells you nothing more about the other. That is still false in real data, which is why the method is called naive. It works surprisingly well anyway, because we only need the *largest* posterior to be right,

not the exact numbers.



The naive assumption as a picture: every feature depends on the class, and on nothing else.

The graphical model draws the assumption: one parent Y with an arrow to every feature X_j, and *no arrows between features*. Each arrow is one table $P(X_j \mid Y)$.

23.4 Training: counting, on sir's example

Sky	Temp	Humid	Wind	Water	Forecast	Play?
sunny	warm	normal	strong	warm	same	yes
sunny	warm	high	strong	warm	same	yes
rainy	cold	high	strong	warm	change	no
sunny	warm	high	strong	cool	change	yes

The priors

3 of 4 rows say yes: $P(\text{play}) = \frac{3}{4}$, $P(\neg\text{play}) = \frac{1}{4}$.

The conditional tables, one count at a time

To get $P(\text{Sky} = \text{sunny} \mid \text{play})$: look only at the 3 "yes" rows (rows 1, 2, 4). In how many is the sky sunny? All 3. So $\frac{3}{3} = 1$. Then look only at the 1 "no" row (row 3): sky rainy, so $P(\text{sunny} \mid \neg\text{play}) = \frac{0}{1} = 0$.

	given play (3 rows)	given not play (1 row)
$P(\text{Sky} = \text{sunny})$	$3/3 = 1$	$0/1 = 0$
$P(\text{Sky} = \text{rainy})$	$0/3 = 0$	$1/1 = 1$
$P(\text{Humid} = \text{high})$	$2/3$	$1/1 = 1$
$P(\text{Humid} = \text{normal})$	$1/3$	$0/1 = 0$

These are the maximum-likelihood estimates: for counting data, the MLE of a probability is the observed fraction.

23.5 The zero problem

Ask before reading on

Tomorrow is rainy, warm and normal humidity. Multiply out the "play" score with the tables above. What do you get?

$P(\text{rainy} \mid \text{play}) = 0$, so the whole product for "play" is 0, no matter how strongly the other features point to playing. One combination that happened not to appear in four rows *vetoes* a class. Sir calls this a case of overfitting: the model treats "not seen in training" as "impossible".

23.6 Laplace smoothing

$$P(X_j = v \mid Y = y_k) = \frac{c_v + 1}{\sum_{v'} c_{v'} + |\text{values}(X_j)|}$$

- c_v: number of class-y_k examples with $X_j = v$.
- $\sum_{v'} c_{v'}$: number of class-y_k examples (all values counted).
- $|\text{values}(X_j)|$: how many values X_j can take (2 for sunny/rainy).

Adding 1 to every count adds $|\text{values}(X_j)|$ to the total, so the probabilities still sum to 1, and nothing is ever exactly zero.

Every smoothed number, with the arithmetic

Sky (2 values), class play (3 rows): sunny $\frac{3+1}{3+2} = \frac{4}{5}$, rainy $\frac{0+1}{3+2} = \frac{1}{5}$. Sky, class not play (1 row): sunny $\frac{0+1}{1+2} = \frac{1}{3}$, rainy $\frac{1+1}{1+2} = \frac{2}{3}$. Humid (2 values), play: high $\frac{2+1}{5} = \frac{3}{5}$, normal $\frac{1+1}{5} = \frac{2}{5}$. Humid, not play: high $\frac{1+1}{3} = \frac{2}{3}$, normal $\frac{0+1}{3} = \frac{1}{3}$. Temp (warm/cold) has exactly the same counts as Sky, so: warm|play $\frac{4}{5}$, cold|play $\frac{1}{5}$, warm|not $\frac{1}{3}$, cold|not $\frac{2}{3}$. Check: within each class each table adds to 1 ($\frac{4}{5} + \frac{1}{5} = 1$, $\frac{1}{3} + \frac{2}{3} = 1$).

Smoothing is a prior

Add-one smoothing is the posterior mean of a probability under a uniform prior (Beta(1,1) for two values, Dirichlet for more). The imaginary examples are *pseudo-counts*. This is derived in the smoothing chapter of the Likelihoods part.

23.7 Prediction, and why we use logarithms

The rule

$$h(\mathbf{x}) = \arg\max_{y_k} \; P(Y = y_k) \prod_{j=1}^{d} P(X_j = x_j \mid Y = y_k).$$

The log version

Derivation

log is an increasing function: if $a > b > 0$ then $\log a > \log b$. So the class with the largest product also has the largest log of the product. The log of a product is the sum of the logs:

$$\log\left[P(y_k)\prod_j P(x_j \mid y_k)\right] = \log P(y_k) + \sum_{j=1}^{d} \log P(x_j \mid y_k).$$

So $h(\mathbf{x}) = \arg\max_{y_k}\left[\log P(Y = y_k) + \sum_{j=1}^{d} \log P(X_j = x_j \mid Y = y_k)\right]$.

Why bother: a spam filter multiplies tens of thousands of small probabilities. The product becomes smaller than the smallest number a computer can store and rounds to exactly 0 for every class. Sums of logs stay perfectly manageable.

Sir's prediction for $\mathbf{x} = (\text{rainy},\ \text{warm},\ \text{normal})$

Using base-10 logs, as the slide does. Values: $\log\frac{3}{4} = -0.125$, $\log\frac{1}{5} = -0.699$, $\log\frac{4}{5} = -0.097$, $\log\frac{2}{5} = -0.398$, $\log\frac{1}{4} = -0.602$, $\log\frac{2}{3} = -0.176$, $\log\frac{1}{3} = -0.477$.

$$\text{play}: \underbrace{-0.125}_{P(\text{play})} + \underbrace{(-0.699)}_{\text{rainy}} + \underbrace{(-0.097)}_{\text{warm}} + \underbrace{(-0.398)}_{\text{normal}} = \boxed{-1.319}$$

$$\text{not play}: \underbrace{-0.602}_{P(\neg\text{play})} + \underbrace{(-0.176)}_{\text{rainy}} + \underbrace{(-0.477)}_{\text{warm}} + \underbrace{(-0.477)}_{\text{normal}} = \boxed{-1.732}$$

$-1.319 > -1.732$, so **predict play**.
If we want a probability, undo the logs and normalise. Products: play $\frac{3}{4}\cdot\frac{1}{5}\cdot\frac{4}{5}\cdot\frac{2}{5} = 0.0480$, not play $\frac{1}{4}\cdot\frac{2}{3}\cdot\frac{1}{3}\cdot\frac{1}{3} = 0.0185$. Their sum is $P(X = \mathbf{x})$ (up to the assumption), so $P(\text{play} \mid \mathbf{x}) = \frac{0.0480}{0.0480+0.0185} = 0.722$. This is the one place $P(X = \mathbf{x})$ is needed: only when you want a probability rather than a decision.

Mental model

Each feature casts a vote: $\log P(x_j \mid \text{play}) - \log P(x_j \mid \neg\text{play})$. Rainy votes against playing ($\log\frac{1/5}{2/3} < 0$); warm and normal vote for it. The prior starts the tally. Because the total is a sum of per-feature votes, Naive Bayes with discrete features is a *linear* classifier in disguise, the same family as the perceptron and logistic regression, fitted by counting instead of optimisation.

In code

```
import numpy as np
from collections import Counter
rows = [("sunny","warm","normal","yes"), ("sunny","warm","high","yes
    "),
         ("rainy","cold","high","no"),     ("sunny","warm","high","yes
            ")]
values = [{"sunny","rainy"}, {"warm","cold"}, {"normal","high"}]
classes = Counter(r[-1] for r in rows)                          # {'yes
```

```
    ': 3, 'no': 1}

def p(j, v, c):                                          # Laplace-smoothed P(X_j = v | c)
    count = sum(1 for r in rows if r[-1] == c and r[j] == v)
    return (count + 1) / (classes[c] + len(values[j]))

x = ("rainy", "warm", "normal")
score = {c: np.log10(classes[c] / len(rows)) + sum(np.log10(p(j, v, c)) for j, v in enumerate(x))
         for c in classes}
print(score)                              # {'yes': -1.319, 'no': -1.732} -> predict 'yes'
```

23.8 The algorithm, and continuous features

Training: for each class, estimate $P(Y = y_k)$; for each feature value, estimate $P(X_j = v \mid Y = y_k)$ with smoothing. **Testing**: add up the logs, pick the largest.

For real-valued features, model each $P(X_j \mid Y = y_k)$ as a normal distribution whose mean and variance are the sample mean and variance of feature j within class k (Gaussian Naive Bayes). Same algorithm, density instead of a count.

23.9 From the textbook: Naive Bayes is a linear classifier (Cornell CS4780)

Gaussian Naive Bayes with shared variances gives a straight boundary

Two classes ± 1 with priors $\pi_\pm$. For each feature, $X_j \mid Y = c \sim \mathcal{N}(\mu_{jc}, \sigma_j^2)$, with the variance σ_j^2 the same for both classes. Predict $+1$ when the log-ratio of the two scores is positive:

$$\ln \frac{\pi_+ \prod_j p(x_j \mid +)}{\pi_- \prod_j p(x_j \mid -)} = \ln \frac{\pi_+}{\pi_-} + \sum_j \left[\frac{(x_j - \mu_{j-})^2 - (x_j - \mu_{j+})^2}{2\sigma_j^2} \right]$$

(the Gaussian constants cancel because the variances match). Expanding each bracket, the x_j^2 terms cancel exactly as in the centroid derivation, leaving

$$\sum_j \underbrace{\frac{\mu_{j+} - \mu_{j-}}{\sigma_j^2}}_{w_j} x_j + \underbrace{\ln \frac{\pi_+}{\pi_-} + \sum_j \frac{\mu_{j-}^2 - \mu_{j+}^2}{2\sigma_j^2}}_{b} = \mathbf{w}^\top \mathbf{x} + b.$$

A straight boundary. For discrete features the same happens with $w_j = \log \theta_{j+} - \log \theta_{j-}$ on one-hot features. So Naive Bayes, logistic regression and the perceptron all draw lines; they differ in *how they choose* the line.

23.10 Summary

Advantages: fast training (one counting pass); fast classification; little affected by irrelevant features (their votes are near zero); handles real-valued data. **Disadvantage**: assumes the features are independent given the class, so its probabilities are often overconfident even when its decisions are good.

Naive Bayes	Perceptron
predicts using $P(Y = y_k \mid X = \mathbf{x})$, a probability	gives only a sign, no probability
generative: models $P(\mathbf{x} \mid y)$ and $P(y)$	finds $\boldsymbol{\theta}$, classifies by the sign of $\boldsymbol{\theta}^\top \mathbf{x}$

Where this shows up

Naive Bayes powered the first effective spam filters (Paul Graham, "A Plan for Spam", 2002) and remains a strong baseline for text: with tens of thousands of word features, counting does not overfit the way optimisation can.

Check yourself

Q. Why can Naive Bayes ignore $P(X = \mathbf{x})$?
A. It is the same for all classes, so it does not change the arg max.

Q. A value seen 0 times in a class of 9 examples, 3 possible values: smoothed probability?
A. $\frac{0+1}{9+3} = \frac{1}{12}$.

Q. Generative or discriminative?
A. Generative. Logistic regression is discriminative.

Go deeper

Read. Cornell CS4780 Lecture note 5, "Bayes Classifier and Naive Bayes". Includes the proof that Gaussian NB gives a linear boundary.

Read. Mitchell, *Machine Learning*, ch. 6. The PlayTennis example the slides are built on.

Watch. Kilian Weinberger, Cornell CS4780, *Lecture 9: Naive Bayes continued*

Exercises

Pencil first, then click the ⇝ to jump to the worked answer in Appendix D. The answer links back to the question.

23.1 ⇝ Using the smoothed tables, classify $\mathbf{x} = (\text{sunny}, \text{cold}, \text{high})$. Show both log-scores.

23.2 ⇝ Recompute $P(\text{high} \mid \neg\text{play})$ and $P(\text{rainy} \mid \text{play})$ if you use add-α smoothing with $\alpha = 0.5$.

23.3 ⇝ With 20 binary features and 3 classes, how many parameters does the full joint class-conditional model need, and how many does Naive Bayes need (count the class prior too)?

Lecture slides for reference

Naïve Bayes Classifier

- Idea: Use the training data to estimate $P(X|Y)$ and $P(Y)$.
- Use the Bayes' rule to infer $P(Y|X_{new})$ for new data.

$$P(Y = y_k|X = \boldsymbol{x}_i) = \frac{P(Y = y_k)P(X_1 = \boldsymbol{x}_{i,1} \cap \cdots \cap X_d = \boldsymbol{x}_{i,d}|Y = y_k)}{P(X_1 = \boldsymbol{x}_{i,1} \cap \cdots \cap X_d = \boldsymbol{x}_{i,d})}$$

- $P(Y = y_k)$: easy to estimate from the data
- $P(X_1 = \boldsymbol{x}_{i,1} \cap \cdots \cap X_d = \boldsymbol{x}_{i,d})$: Determination is not necessary
- $P(X_1 = \boldsymbol{x}_{i,1} \cap \cdots \cap X_d = \boldsymbol{x}_{i,d}|Y = y_k) = P(X_1, X_2, \ldots, X_d|Y)$: Estimation is impractical but necessary

Sir's lecture notes, slide 064. What is easy, unnecessary, impractical.

Naïve Bayes Classifier

- Issue: Estimating the joint PD is not practical
- If we assume that the features are independent given the class labels

$$P(X_1, X_2, \ldots, X_d|Y) = \prod_{j=1}^{d} P(X_j|Y)$$

- We assume all the features are conditionally independent given Y
- This is often violated in practice

Sir's lecture notes, slide 065. The naive assumption.

Laplace Smoothing

- Some probabilities estimated by counting process might be zero– possible case of overfitting!
- Regularize by Laplace smoothing
  - Add 1 to each count

$$P(X_j = v|Y = y_k) = \frac{c_v + 1}{\sum_{v' \in values(X_j)} c_{v'} + |values(X_j)|}$$

  - c_v is the count of training examples with a value of v for feature j and class label y_k
  - $\sum_{v' \in values(X_j)} c_{v'}$ = Count of values in that class
  - $|values(X_j)|$ is the number of values X_j can take on

Sir's lecture notes, slide 077. Laplace smoothing.

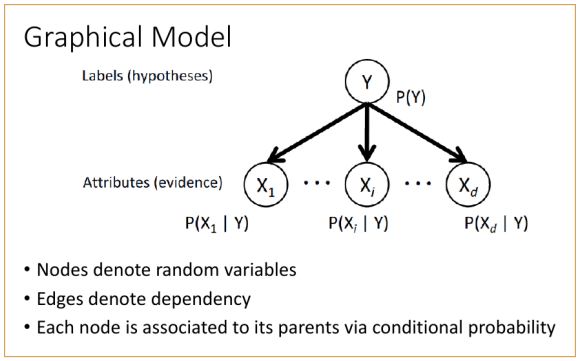
The Naïve Bayes Classifier Algorithm

- For each class y_k
  - Estimate $P(Y = y_k)$ from the data
  - For each value $x_{i,j}$ of each feature X_i -- Estimate $P(X_i = x_{i,j}|Y = y_k)$
- Classify a new point using:

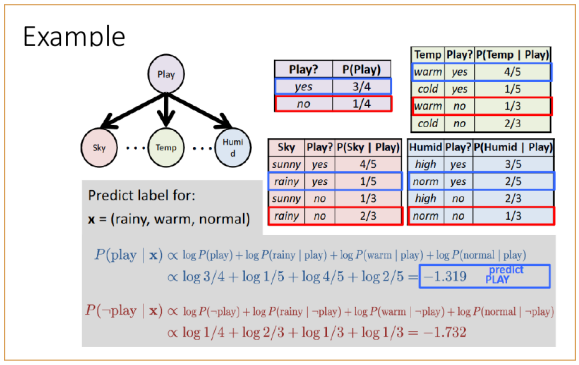
$$h(x) = \arg\max_{y_k} \log P(Y = y_k) + \sum_{j=1}^{d} \log P(X_j = \boldsymbol{x}_j|Y = y_k)$$

- Although the independence assumption does not often hold true, Naïve Bayes Classifier performs quite well.

Sir's lecture notes, slide 083. The algorithm.



Sir's lecture notes, slide 084. Graphical model.



Sir's lecture notes, slide 095. The worked prediction.

24

Linear Classifiers and the Perceptron

The distance-from-means classifier ended with a straight-line boundary it never asked for. This chapter asks for one directly: *find a line (hyperplane) that puts every training point on its correct side.* The perceptron does it with an update rule simple enough to run by hand, and it comes with the first convergence proof in machine learning. We first build the geometry of lines and distances from the dot product, then the algorithm, then the proof, each step explained. Sir's slides are at the end.

24.1 Geometry you need: the dot product

For vectors $\boldsymbol{\theta}, \mathbf{x} \in \mathbb{R}^d$, $\boldsymbol{\theta}^\top \mathbf{x} = \sum_j \theta_j x_j = \|\boldsymbol{\theta}\| \, \|\mathbf{x}\| \cos\phi$, where ϕ is the angle between them. So:

- $\boldsymbol{\theta}^\top \mathbf{x} > 0$: the angle is under $90°$ (they point roughly the same way).
- $\boldsymbol{\theta}^\top \mathbf{x} = 0$: they are perpendicular.
- $\boldsymbol{\theta}^\top \mathbf{x} < 0$: the angle is over $90°$.

$\frac{\boldsymbol{\theta}}{\|\boldsymbol{\theta}\|}$ is a **unit vector** (length 1) pointing the same way as $\boldsymbol{\theta}$, and $\frac{\boldsymbol{\theta}^\top \mathbf{x}}{\|\boldsymbol{\theta}\|}$ is the length of the shadow of $\mathbf{x}$ along that direction.

24.2 Hyperplanes and the linear classifier

A **hyperplane** is the set $\{\mathbf{x} : \boldsymbol{\theta}^\top \mathbf{x} + b = 0\}$: a line in 2-D, a plane in 3-D. It splits space into two half-spaces, $\boldsymbol{\theta}^\top \mathbf{x} + b > 0$ and < 0. With labels $y \in \{-1, +1\}$ the classifier is

$$h(\mathbf{x}) = \text{sign}(\boldsymbol{\theta}^\top \mathbf{x} + b), \qquad \text{sign}(z) = \begin{cases} +1 & z \geq 0 \\ -1 & z < 0. \end{cases}$$

Why θ is perpendicular to the hyperplane

Take two points $\mathbf{x}_a, \mathbf{x}_b$ on the hyperplane: $\boldsymbol{\theta}^\top \mathbf{x}_a + b = 0$ and $\boldsymbol{\theta}^\top \mathbf{x}_b + b = 0$. Subtract the equations: $\boldsymbol{\theta}^\top (\mathbf{x}_a - \mathbf{x}_b) = 0$. The vector $\mathbf{x}_a - \mathbf{x}_b$ runs along the hyperplane, and its dot product with $\boldsymbol{\theta}$ is zero, so $\boldsymbol{\theta}$ is perpendicular to every direction inside the hyperplane. We call $\boldsymbol{\theta}$ the **normal vector**. It points towards the $+1$ side.

The bias trick

Carrying b separately is a nuisance. Add a constant feature $x_0 = 1$ to every point and rename $b = \theta_0$:

$$\boldsymbol{\theta} = (\theta_0, \theta_1, \ldots, \theta_d), \quad \mathbf{x} = (1, x_1, \ldots, x_d) \quad \Longrightarrow \quad \boldsymbol{\theta}^\top \mathbf{x} = \theta_0 + \theta_1 x_1 + \cdots + \theta_d x_d.$$

Now the classifier is just $h(\mathbf{x}) = \operatorname{sign}(\boldsymbol{\theta}^\top \mathbf{x})$, and in the enlarged space the hyperplane passes through the origin. A key observation (Weinberger's notes):

$$y\, \boldsymbol{\theta}^\top \mathbf{x} > 0 \iff \mathbf{x} \text{ is classified correctly.}$$

If $y = +1$ we need a positive score; if $y = -1$ a negative one; multiplying by y turns both into "positive". This only works because the labels are ± 1, not $0/1$.

24.3 Margins: how far is a point from the boundary?

Derivation: the signed distance to the hyperplane

Let $\mathbf{x}$ be any point and $\mathbf{x}_0$ the point on the hyperplane closest to it. The segment from $\mathbf{x}_0$ to $\mathbf{x}$ is perpendicular to the hyperplane, so it runs along the unit normal:

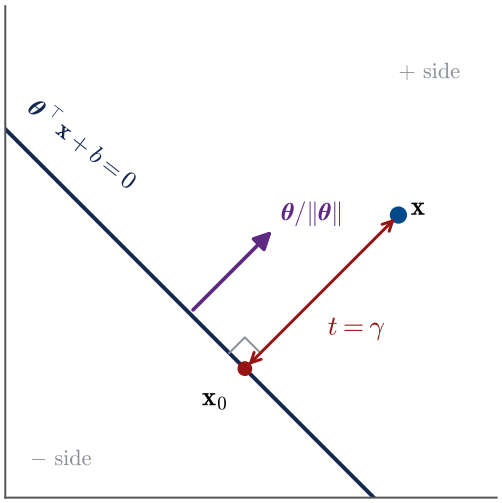
$$\mathbf{x} = \mathbf{x}_0 + t\, \frac{\boldsymbol{\theta}}{\|\boldsymbol{\theta}\|}, \qquad t = \text{signed distance.}$$

Compute $\boldsymbol{\theta}^\top \mathbf{x} + b$ by substituting:

$$\begin{aligned} \boldsymbol{\theta}^\top \mathbf{x} + b &= \boldsymbol{\theta}^\top \mathbf{x}_0 + t\, \frac{\boldsymbol{\theta}^\top \boldsymbol{\theta}}{\|\boldsymbol{\theta}\|} + b && \text{distribute} \\ &= \underbrace{(\boldsymbol{\theta}^\top \mathbf{x}_0 + b)}_{=0,\ \mathbf{x}_0 \text{ is on the plane}} + t\, \frac{\|\boldsymbol{\theta}\|^2}{\|\boldsymbol{\theta}\|} && \boldsymbol{\theta}^\top \boldsymbol{\theta} = \|\boldsymbol{\theta}\|^2 \\ &= t\, \|\boldsymbol{\theta}\|. \end{aligned}$$

Divide by $\|\boldsymbol{\theta}\|$:

$$\boxed{\gamma = t = \frac{\boldsymbol{\theta}^\top \mathbf{x} + b}{\|\boldsymbol{\theta}\|}}$$



The derivation in one picture: walk from $\mathbf{x}_0$ along the unit normal for a distance t.

- **Geometric margin** of a labelled example: $\gamma^{(i)} = \dfrac{y^{(i)}(\boldsymbol{\theta}^\top \mathbf{x}^{(i)} + b)}{\|\boldsymbol{\theta}\|}$. Positive if correctly classified, and equal to the true distance from the boundary.
- **Margin of a dataset**: the smallest one, $\gamma = \min_i |\gamma^{(i)}|$ (the closest point to the boundary).
- **Functional margin**: $y^{(i)}(\boldsymbol{\theta}^\top \mathbf{x}^{(i)} + b)$, without dividing. Positive for a correct prediction, negative for a wrong one, but its size depends on how $\boldsymbol{\theta}$ is scaled.

Sir's one-dimensional example

Boundary $2x - 6 = 0$, i.e. $x = 3$. Point $x = 5$. True distance: $|5 - 3| = 2$. Formula: $\gamma = \frac{2(5)-6}{|2|} = \frac{4}{2} = 2$. Now write the *same* boundary as $20x - 60 = 0$. Functional value at $x = 5$: $20(5) - 60 = 40$ (ten times bigger). Geometric margin: $\frac{40}{20} = 2$ (unchanged). Lesson: the geometric margin measures the real distance; the functional margin can be inflated by rescaling $\boldsymbol{\theta}$.

24.4 The perceptron

From a loss to an update

We want a cost that is zero for correct points and positive for wrong ones:

$$J(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^{n} \max\big(0,\ -y^{(i)} \boldsymbol{\theta}^\top \mathbf{x}^{(i)}\big).$$

If $y^{(i)} \boldsymbol{\theta}^\top \mathbf{x}^{(i)} \geq 0$ (correct), the term is $\max(0, \text{negative}) = 0$. If it is wrong, the term is $-y^{(i)} \boldsymbol{\theta}^\top \mathbf{x}^{(i)} > 0$, and its gradient with respect to $\boldsymbol{\theta}$ is $-y^{(i)} \mathbf{x}^{(i)}$. One stochastic gradient step on a wrong example, with learning rate α:

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha\big(-y^{(i)} \mathbf{x}^{(i)}\big) = \boldsymbol{\theta} + \alpha\, y^{(i)} \mathbf{x}^{(i)}.$$

Sir's version of the same update

The slide writes $\theta_j \leftarrow \theta_j - \frac{\alpha}{2}\big(h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) - y^{(i)}\big)x_j^{(i)}$. Check both cases: correct prediction, $h = y$, so the bracket is 0 and nothing changes. Wrong prediction, $h = -y$ (sir's margin note), so $h - y = -2y$ and the step is $\theta_j - \frac{\alpha}{2}(-2y)x_j = \theta_j + \alpha\, y\, x_j$. Same rule.

Starting from $\boldsymbol{\theta} = \mathbf{0}$, the learning rate only rescales $\boldsymbol{\theta}$, and rescaling never changes any sign. So we take $\alpha = 1$:

If $y^{(i)}\, \boldsymbol{\theta}^\top \mathbf{x}^{(i)} \leq 0$ (a mistake), set $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + y^{(i)}\mathbf{x}^{(i)}$. Otherwise do nothing.

Watch out

The slides test "< 0". From $\boldsymbol{\theta} = \mathbf{0}$ every score is exactly 0, and with < 0 nothing would ever update. Use ≤ 0: a point exactly on the boundary counts as a mistake.

Online and batch versions

Online: process examples one at a time and update immediately after each mistake. **Batch**: go through all n examples, add up $\boldsymbol{\Delta} = \sum_{\text{mistakes}} y^{(i)}\mathbf{x}^{(i)}$, average it ($\boldsymbol{\Delta}/n$), then take one step $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha\boldsymbol{\Delta}$; stop when $\|\boldsymbol{\Delta}\|$ is tiny.

Why one update helps

Derivation

Suppose $\mathbf{x}$ has $y = +1$ but the perceptron scored it negative: $\boldsymbol{\theta}_{\text{old}}^\top \mathbf{x} < 0$. After the update $\boldsymbol{\theta}_{\text{new}} = \boldsymbol{\theta}_{\text{old}} + \mathbf{x}$:

$$\boldsymbol{\theta}_{\text{new}}^\top \mathbf{x} = (\boldsymbol{\theta}_{\text{old}} + \mathbf{x})^\top \mathbf{x} = \boldsymbol{\theta}_{\text{old}}^\top \mathbf{x} + \mathbf{x}^\top \mathbf{x} = \boldsymbol{\theta}_{\text{old}}^\top \mathbf{x} + \|\mathbf{x}\|^2.$$

The score went up by $\|\mathbf{x}\|^2 > 0$: less negative, closer to correct. (For $y = -1$ it goes down by $\|\mathbf{x}\|^2$.) One update may not fix the point completely, but it always moves in the right direction. Geometrically, $\boldsymbol{\theta}$ rotates towards $\mathbf{x}$, and the boundary (perpendicular to $\boldsymbol{\theta}$) rotates with it.

A full run by hand

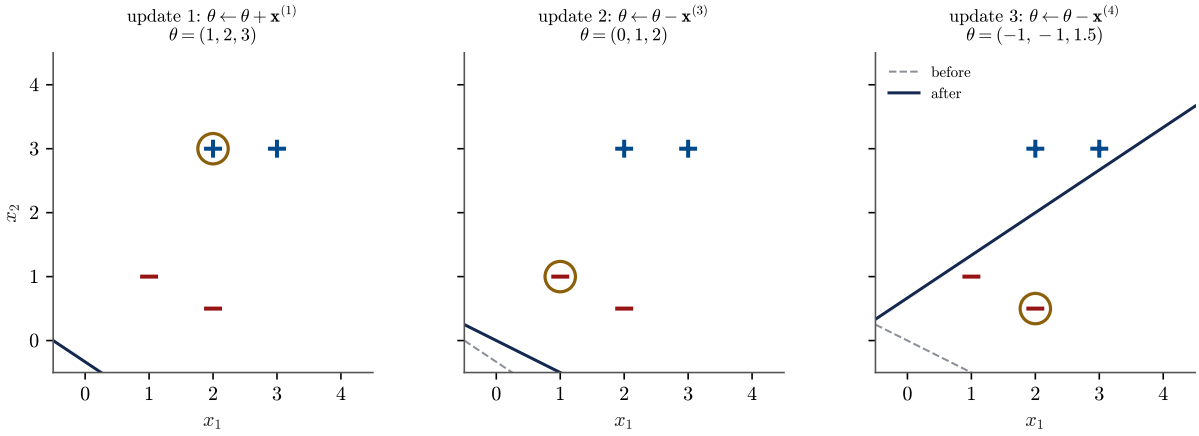
Points with the bias feature first: $\mathbf{x}^{(1)} = (1, 2, 3)$ and $\mathbf{x}^{(2)} = (1, 3, 3)$ are $+1$; $\mathbf{x}^{(3)} = (1, 1, 1)$ and $\mathbf{x}^{(4)} = (1, 2, 0.5)$ are -1. Start $\boldsymbol{\theta} = (0, 0, 0)$.

Epoch 1.

- $\mathbf{x}^{(1)}$: score 0, so $y \cdot 0 = 0 \leq 0$: mistake. $\boldsymbol{\theta} = (0, 0, 0) + (1, 2, 3) = (1, 2, 3)$.
- $\mathbf{x}^{(2)}$: score $1 + 6 + 9 = 16$; $y \cdot 16 > 0$: correct.
- $\mathbf{x}^{(3)}$: score $1{+}2{+}3 = 6$; $y = -1$ gives $-6 \leq 0$: mistake. $\boldsymbol{\theta} = (1, 2, 3) - (1, 1, 1) = (0, 1, 2)$.
- $\mathbf{x}^{(4)}$: score $0 + 2 + 1 = 3$; $-3 \leq 0$: mistake. $\boldsymbol{\theta} = (0, 1, 2) - (1, 2, 0.5) = (-1, -1, 1.5)$.

Epoch 2. Scores with $\boldsymbol{\theta} = (-1, -1, 1.5)$: $\mathbf{x}^{(1)}$: $-1 - 2 + 4.5 = 1.5$ (positive, correct); $\mathbf{x}^{(2)}$: 0.5 (correct); $\mathbf{x}^{(3)}$: -0.5 (negative, correct); $\mathbf{x}^{(4)}$: -2.25 (correct). **No mistakes,**

so it has converged after 3 updates. Boundary: $-1 - x_1 + 1.5x_2 = 0$, i.e. $x_2 = \frac{1+x_1}{1.5}$.



The three updates. Gold ring: the point that triggered the update. Dashed: boundary before; solid: after.

24.5 The convergence theorem (Block and Novikoff)

Assume (1) some unit vector $\boldsymbol{\theta}_*$ ($\|\boldsymbol{\theta}_*\| = 1$) separates the data with margin $\gamma > 0$, meaning $y^{(i)}\boldsymbol{\theta}_*^\top \mathbf{x}^{(i)} \geq \gamma$ for every i; and (2) every point has length $\|\mathbf{x}^{(i)}\| \leq R$. Then the perceptron makes at most R^2/γ^2 mistakes.

The proof watches two numbers grow after each mistake. Let $\boldsymbol{\theta}_k$ be the weights after k mistakes, $\boldsymbol{\theta}_0 = \mathbf{0}$, and suppose mistake k happens on $(\mathbf{x}, y)$, so $y\,\boldsymbol{\theta}_{k-1}^\top \mathbf{x} \leq 0$ and $\boldsymbol{\theta}_k = \boldsymbol{\theta}_{k-1} + y\mathbf{x}$.

Step 1: alignment with the right answer grows by at least γ per mistake

$$\boldsymbol{\theta}_k^\top \boldsymbol{\theta}_* = (\boldsymbol{\theta}_{k-1} + y\mathbf{x})^\top \boldsymbol{\theta}_* = \boldsymbol{\theta}_{k-1}^\top \boldsymbol{\theta}_* + \underbrace{y\,\mathbf{x}^\top \boldsymbol{\theta}_*}_{\geq \gamma \text{ by assumption (1)}} \geq \boldsymbol{\theta}_{k-1}^\top \boldsymbol{\theta}_* + \gamma.$$

Starting from 0 and adding at least γ each time: $\boldsymbol{\theta}_k^\top \boldsymbol{\theta}_* \geq k\gamma$.

Step 2: the length grows slowly

$$\|\boldsymbol{\theta}_k\|^2 = (\boldsymbol{\theta}_{k-1} + y\mathbf{x})^\top (\boldsymbol{\theta}_{k-1} + y\mathbf{x})$$
$$= \|\boldsymbol{\theta}_{k-1}\|^2 + \underbrace{2y\,\boldsymbol{\theta}_{k-1}^\top \mathbf{x}}_{\leq 0 \text{ (it was a mistake)}} + \underbrace{y^2\|\mathbf{x}\|^2}_{\leq R^2 \ (y^2=1)} \leq \|\boldsymbol{\theta}_{k-1}\|^2 + R^2.$$

Adding at most R^2 each time: $\|\boldsymbol{\theta}_k\|^2 \leq kR^2$, so $\|\boldsymbol{\theta}_k\| \leq \sqrt{k}\,R$.

Step 3: squeeze

By Cauchy–Schwarz, a dot product is at most the product of the lengths: $\boldsymbol{\theta}_k^\top \boldsymbol{\theta}_* \leq \|\boldsymbol{\theta}_k\|\,\|\boldsymbol{\theta}_*\| = \|\boldsymbol{\theta}_k\|$ (since $\|\boldsymbol{\theta}_*\| = 1$). Chain the three facts:

$$k\gamma \leq \boldsymbol{\theta}_k^\top \boldsymbol{\theta}_* \leq \|\boldsymbol{\theta}_k\| \leq \sqrt{k}\,R \implies k\gamma \leq \sqrt{k}\,R \implies \sqrt{k} \leq \frac{R}{\gamma} \implies \boxed{k \leq \frac{R^2}{\gamma^2}}.$$

Mental model

Every mistake pushes $\boldsymbol{\theta}$ a fixed amount (at least γ) along the right direction, but lengthens it only a little (at most R, in quadrature). Progress grows like k, length like $\sqrt{k}$, and since a cosine can never exceed 1, the mistakes must stop. Sir's notes: large margin γ means faster convergence; large R means slower. Notably the bound does not mention n or d.

Watch out

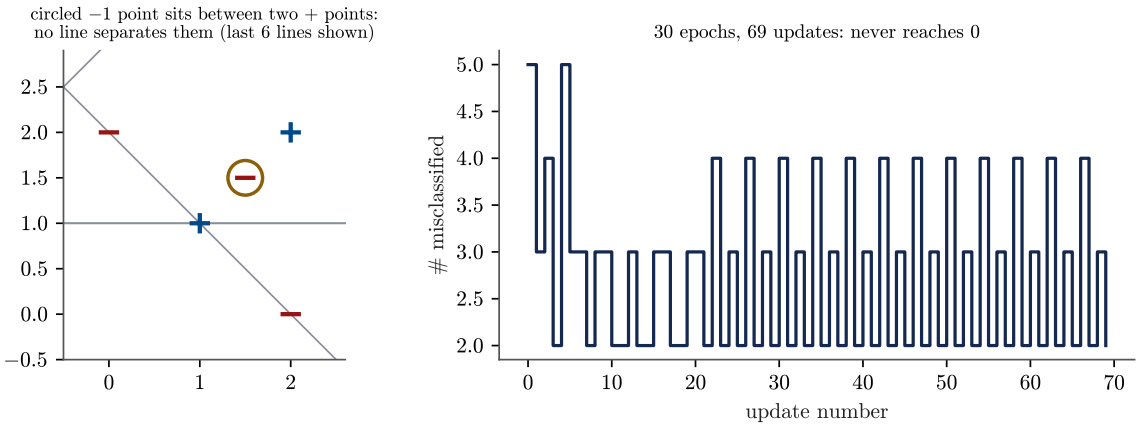
The bound uses the margin of the *best* separator, not of the one the perceptron ends with. In the hand run, $R = \|(1,3,3)\| = 4.36$ and the final boundary's margin is only 0.24, which would give a useless bound of 323; the run finished after 3 mistakes because a much wider separator exists. The theorem guarantees termination, not a good margin. Finding the widest margin is the job of the support vector machine.

24.6 When no line exists

Sir's five points

$(1,1), (2,2)$ are $+1$; $(2,0), (0,2), (1.5,1.5)$ are -1. The point $(1.5,1.5)$ lies on the segment between the two positive points. Any half-plane that contains both positive points contains the whole segment between them (a half-plane is convex), so it contains $(1.5,1.5)$ as well. Therefore **no line separates the data**.

What the perceptron does (sir's notes): it makes a mistake, updates, fixes that point, makes a mistake on another, and the cycle continues forever. Running it for 30 epochs: 69 updates, never a clean epoch, the weights cycling through $(-2,0,2)$, $(-3,0,0)$, $(-2,1,1)$, $(-3,-1,1)$ and back.



Left: the five points and the last six boundaries tried. Right: the number of misclassified points never reaches zero.

Fixes: stop after a fixed number of epochs and keep the best weights seen (the pocket algorithm); average the weights; switch to a loss that has a minimum even with overlap (logistic regression); or add features so that a separating line exists.

24.7 From the textbook: the weight-space view (Shewchuk, Berkeley CS189)

Shewchuk offers a second picture that makes the perceptron feel inevitable.

- In **x-space** (where the data live), a weight vector $\boldsymbol{\theta}$ is a hyperplane and each training point is a dot.
- In **weight space** (where $\boldsymbol{\theta}$ lives), swap the roles: each training point $\mathbf{x}_i$ becomes a hyperplane $\{\boldsymbol{\theta} : \boldsymbol{\theta}^\top \mathbf{x}_i = 0\}$, and our classifier is a dot.

Point i is classified correctly exactly when $y_i \boldsymbol{\theta}^\top \mathbf{x}_i > 0$, i.e. when the dot $\boldsymbol{\theta}$ is on the correct side of the hyperplane belonging to $\mathbf{x}_i$. Every point therefore cuts weight space in half and keeps one half. The **feasible region** (all weight vectors that classify everything correctly) is the intersection of these half-spaces: a cone with its tip at the origin. The data are linearly separable exactly when this cone is non-empty.

The perceptron update $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + y_i \mathbf{x}_i$ moves the dot in the direction perpendicular to the violated hyperplane, towards its correct side. The convergence proof says that if the cone has some width (margin $\gamma > 0$), these steps reach it after at most R^2/γ^2 of them.

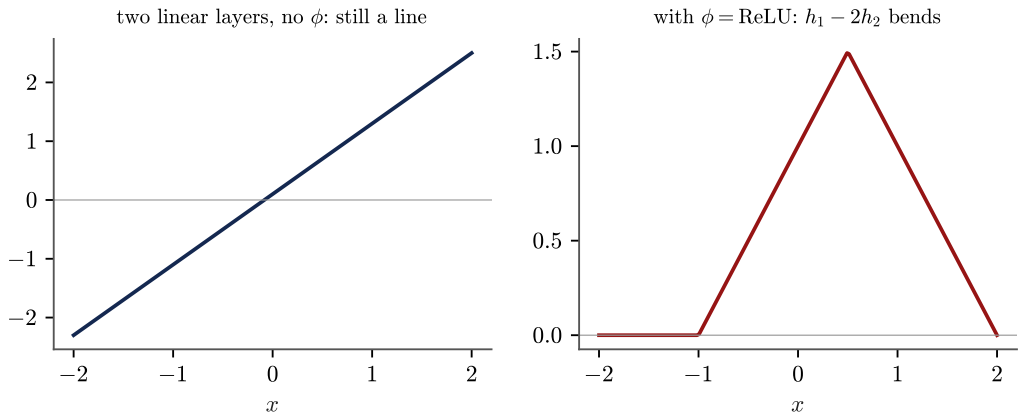
24.8 Stacking perceptrons: why nonlinearity matters

Derivation

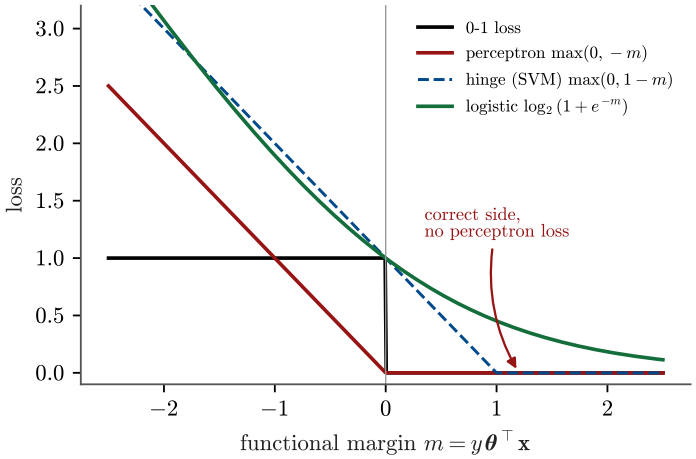
Chain two linear units with no activation: $\mathbf{h} = \boldsymbol{\Theta}_1 \mathbf{x} + \mathbf{b}_1$, then $\hat{y} = \boldsymbol{\Theta}_2 \mathbf{h} + \mathbf{b}_2$. Substitute:

$$\hat{y} = \boldsymbol{\Theta}_2(\boldsymbol{\Theta}_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = \underbrace{\boldsymbol{\Theta}_2 \boldsymbol{\Theta}_1}_{\boldsymbol{\Theta}} \mathbf{x} + \underbrace{\boldsymbol{\Theta}_2 \mathbf{b}_1 + \mathbf{b}_2}_{\mathbf{b}}.$$

Still one linear function: the second layer added nothing. Put a nonlinear function ϕ (a step, sigmoid, or ReLU) between them, $\hat{y} = \boldsymbol{\Theta}_2 \, \phi(\boldsymbol{\Theta}_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2$, and the collapse is impossible. That is why neural networks need activation functions.



Left: two linear layers, still a line. Right: two ReLU units already bend.



The perceptron loss beside its relatives, as functions of the functional margin $m = y\boldsymbol{\theta}^\top \mathbf{x}$: 0-1 loss, perceptron, hinge (SVM), logistic.

Check yourself

Q. Does the perceptron output a probability?
A. No, only a sign. Naive Bayes and logistic regression give probabilities.

Q. Does scaling $\boldsymbol{\theta}$ change predictions?
A. No: $\text{sign}(c\boldsymbol{\theta}^\top \mathbf{x}) = \text{sign}(\boldsymbol{\theta}^\top \mathbf{x})$ for $c > 0$.

Q. Why does the bound not depend on d?
A. The proof uses only dot products and lengths; γ and R already encode the geometry.

Go deeper

Read. Novikoff, "On convergence proofs for perceptrons" (1962). The original bound.

Read. Cornell CS4780 Lecture note 3, "The Perceptron". The same proof with $R = 1$.

Watch. Kilian Weinberger, Cornell CS4780, *Lecture 6: Perceptron Convergence Proof*

Exercises

Pencil first, then click the ⇝ to jump to the worked answer in Appendix D. The answer links back to the question.

24.1 ⇝ Run the perceptron (augmented with $x_0 = 1$, start at $\mathbf{0}$, visit in order) on $(0, 1) \to +1$, $(1, 0) \to -1$, $(2, 2) \to +1$, $(1, -1) \to -1$ until an epoch has no mistakes. Report every update.

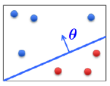
24.2 ⇝ Find the geometric margin of $\mathbf{x} = (4, 1)$ from the line $3x_1 + 4x_2 - 6 = 0$, and the functional margin if $y = -1$.

24.3 ⇝ A separable dataset has $R = 5$ and best margin $\gamma = 0.5$. Bound the number of perceptron mistakes. What happens to the bound if every $\mathbf{x}$ is multiplied by 10?

Lecture slides for reference

Linear Classifiers

- Represent decision boundary by hyperplane
- $\boldsymbol{\theta} = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_d \end{bmatrix}$ $\quad \boldsymbol{x}^T = [1 \quad x_1 \quad \cdots \quad x_d]$



- $h(\boldsymbol{x}) = sign(\boldsymbol{\theta}^T \boldsymbol{x})$ where $sign(z) = \begin{cases} 1 & if\ z \geq 0 \\ -1 & if\ z < 0 \end{cases}$
- $\boldsymbol{\theta}^T \boldsymbol{x} > 0 \Rightarrow y = +1$
- $\boldsymbol{\theta}^T \boldsymbol{x} < 0 \Rightarrow y = -1$

Sir's lecture notes, slide 099. Linear classifiers.

Notion of Margins

- Geometric Margin $\gamma^{(i)}$ of an example $\boldsymbol{x}^{(i)}$ is its signed distance from hyperplane

$$\gamma^{(i)} = \frac{\boldsymbol{\theta}^T \boldsymbol{x}^{(i)}}{\|\boldsymbol{\theta}\|}$$

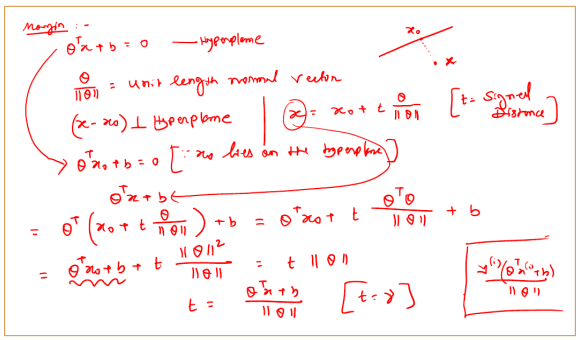


- $\gamma^{(i)}$ may be +ve or –ve based on which side of the plane $\boldsymbol{x}^{(i)}$ is
- Margin of a set $\{\boldsymbol{x}^{(1)}, \boldsymbol{x}^{(2)}, \ldots, \boldsymbol{x}^{(n)}\}$ with respect to $\boldsymbol{\theta}$ is the minimum absolute geometric margin

$$\gamma = \min_{1 \leq i \leq n} |\gamma^{(i)}| = \min_{1 \leq i \leq n} \frac{|\boldsymbol{\theta}^T \boldsymbol{x}^{(i)}|}{\|\boldsymbol{\theta}\|}$$

- Functional margin of $\boldsymbol{\theta}$ on a training example $(\boldsymbol{x}^{(i)}, y^{(i)})$: $y^{(i)}(\boldsymbol{\theta}^T \boldsymbol{x}^{(i)})$
  - +ve for correct prediction
  - -ve for incorrect prediction

Sir's lecture notes, slide 100. Margins.



Sir's lecture notes, slide 101. Margin derivation.

The Perceptron

- $h(\boldsymbol{x}) = sign(\boldsymbol{\theta}^T \boldsymbol{x})$ where $sign(z) = \begin{cases} 1 & if\ z \geq 0 \\ -1 & if\ z < 0 \end{cases}$

$h_\theta(x^{(i)}) = -y^{(i)}$

- The update rule for each training example $(\boldsymbol{x}^{(i)}, y^{(i)})$

$$\theta_j := \theta_j - \frac{\alpha}{2}(h_{\boldsymbol{\theta}}(\boldsymbol{x}^{(i)}) - y^{(i)}) x_j^{(i)}$$

  - For correct prediction, update will be stopped
  - For incorrect prediction update θ
- The update step can be re-written for incorrect prediction as

$$\theta_j := \theta_j + \alpha y^{(i)} x_j^{(i)}$$

- The perceptron rule: If $\boldsymbol{x}^{(i)}$ is misclassified, do $\boldsymbol{\theta} := \boldsymbol{\theta} + y^{(i)} \boldsymbol{x}^{(i)}$

Sir's lecture notes, slide 104. The perceptron update.

Why it works

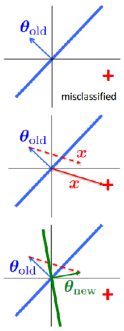
- Consider the misclassified positive example $y^{(i)} = +1$
- Perceptron wrongly predicts that $\boldsymbol{\theta}_{old}^T \boldsymbol{x} < 0$
- Update step:

$\boldsymbol{\theta}_{new} = \boldsymbol{\theta}_{old} + y\boldsymbol{x} = \boldsymbol{\theta}_{old} + \boldsymbol{x} \quad (\because y = +1)$

- Note

$$\boldsymbol{\theta}_{new}^T \boldsymbol{x} = (\boldsymbol{\theta}_{old} + \boldsymbol{x})^T \boldsymbol{x} = \boldsymbol{\theta}_{old}^T \boldsymbol{x} + \boldsymbol{x}^T \boldsymbol{x}$$

- $\boldsymbol{\theta}_{new}^T \boldsymbol{x}$ is less negative than $\boldsymbol{\theta}_{old}^T \boldsymbol{x}$
- So we are progressing towards correct prediction.

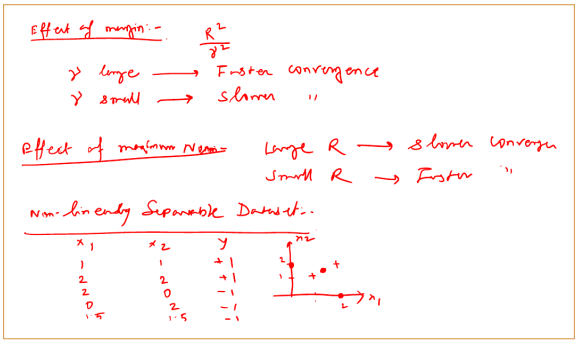


Sir's lecture notes, slide 107. Why it works.

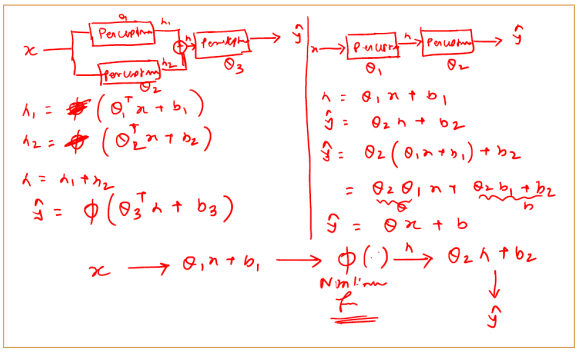
Convergence of Perceptron

- **Theorem (Block & Novikoff):** If the training data is linearly separable with margin γ by a unit norm hyperplane $\boldsymbol{\theta}_*$ ($\|\boldsymbol{\theta}_*\| = 1$), then perceptron converges after $\frac{R^2}{\gamma^2}$ mistakes during training (assuming $\|\boldsymbol{x}\| \leq R$ for all $\boldsymbol{x}$)
- Guaranteed to find a separating hyperplane if one exists
- Convergence rate does not depend on the number of training examples or the data dimensionality.
- Depends only on the margin.

Sir's lecture notes, slide 108. Convergence theorem.



Sir's lecture notes, slide 109. Effect of margin; non-separable data.



Sir's lecture notes, slide 111. Stacking perceptrons.

25

Logistic Regression, Newton's Method and Momentum

This chapter builds logistic regression from nothing. We start with a small medical table and a question a doctor might ask. We try the tool we already own (linear regression), watch exactly where it breaks, and then repair it one piece at a time. Every formula that appears is derived on the page, one line at a time, with the reason for each line written next to it. Sir's slides are collected at the end of the chapter for reference; you should be able to follow everything without them.

25.1 The problem: predicting a yes/no answer

Here are eight patients (the running example from the class decks):

blood pressure (mm Hg)	80	90	100	110	120	130	140	150
diabetic?	no	no	no	no	yes	yes	yes	yes

We code "yes" as $y = 1$ and "no" as $y = 0$. A new patient walks in with blood pressure 118. We want two things: a **decision** (diabetic or not) and, better, a **probability** (how sure are we?). This is **classification**: the output is a category, not a number on a continuous scale. Two categories is *binary* classification.

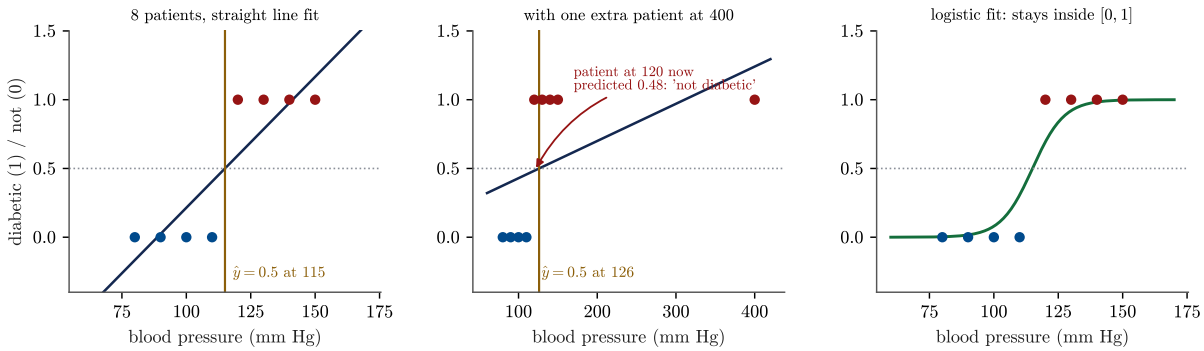
25.2 First attempt: just use linear regression

We already know how to fit $\hat{y} = w_0 + w_1 x$ by least squares. Fit it to the 0/1 labels, and say "diabetic" whenever $\hat{y} \geq 0.5$.

What the straight line does on our data

Least squares gives $\hat{y} = -1.690 + 0.0190\,x$. Setting $\hat{y} = 0.5$: $x = (0.5 + 1.690)/0.0190 = 115$. So the rule is "diabetic if BP ≥ 115". On these eight patients that is perfect.

Now add one more diabetic patient with an extreme reading of 400 (certainly diabetic). Refit: the line tilts to reach the new point, and the 0.5 crossing moves to 126. The patient at 120, who was correctly called diabetic, is now predicted $\hat{y} = 0.48$: **not diabetic**. One *correct*, extreme example made the classifier worse.



Left: least squares on the eight patients. Middle: one extra patient at 400 drags the line, and the threshold moves from 115 to 126. Right: the curve we will build in this chapter. It never leaves $[0, 1]$ *and is not dragged by far-away points.*

Three problems, all from the class slides:

1. **Outputs outside** $[0, 1]$**.** At BP $= 60$ the line predicts -0.55. A probability of -0.55 means nothing.
2. **Sensitivity to outliers.** Squared error punishes the far point at 400 heavily, so the line tilts to please it, even though that point was already on the correct side.
3. **Wrong idea of error.** A prediction of $\hat{y} = 3$ for a diabetic patient is "very confidently right", but squared error charges it $(3 - 1)^2 = 4$, more than a borderline mistake.

Mental model

For classification we do not want a line *through* the points. We want a curve that answers "what is the chance that $y = 1$ at this x?". A chance must live between 0 and 1, be near 0 far on one side, near 1 far on the other, and switch over somewhere in the middle. That is an **S-shaped** curve.

25.3 The sigmoid function, and its three properties

The S-curve we use is the **sigmoid** (also called the *logistic function*):

$$\sigma(a) = \frac{1}{1 + e^{-a}}.$$

Understand it by plugging in numbers before any theory.

a	-6	-2	-1	0	1	2	6
e^{-a}	403.4	7.389	2.718	1	0.368	0.135	0.0025
$\sigma(a)$	0.0025	0.119	0.269	**0.5**	0.731	0.881	0.9975

When a is very negative, e^{-a} is huge, so $\sigma(a) = \frac{1}{\text{huge}} \approx 0$. When a is very positive, $e^{-a} \approx 0$, so $\sigma(a) \approx 1$. At $a = 0$, $\sigma(0) = \frac{1}{1+1} = \frac{1}{2}$.

Property 1: bounded

$e^{-a} > 0$ for every real a, so the denominator $1 + e^{-a} > 1$, so $0 < \sigma(a) < 1$. The output can *always* be read as a probability.

Property 2: symmetric, $1-\sigma(a)=\sigma(-a)$

Derivation

$$
\begin{aligned}
1-\sigma(a) &= 1-\frac{1}{1+e^{-a}} && \text{definition}\\
&= \frac{(1+e^{-a})-1}{1+e^{-a}} = \frac{e^{-a}}{1+e^{-a}} && \text{common denominator}\\
&= \frac{e^{-a}\cdot e^{a}}{(1+e^{-a})\cdot e^{a}} = \frac{1}{e^{a}+1} && \text{multiply top and bottom by } e^{a}\\
&= \sigma(-a) && \text{since } \sigma(-a)=\tfrac{1}{1+e^{-(-a)}}=\tfrac{1}{1+e^{a}}.
\end{aligned}
$$

Check with the table: $1-\sigma(1)=1-0.731=0.269=\sigma(-1)$. In words: the probability of "no" at score a equals the probability of "yes" at score $-a$.

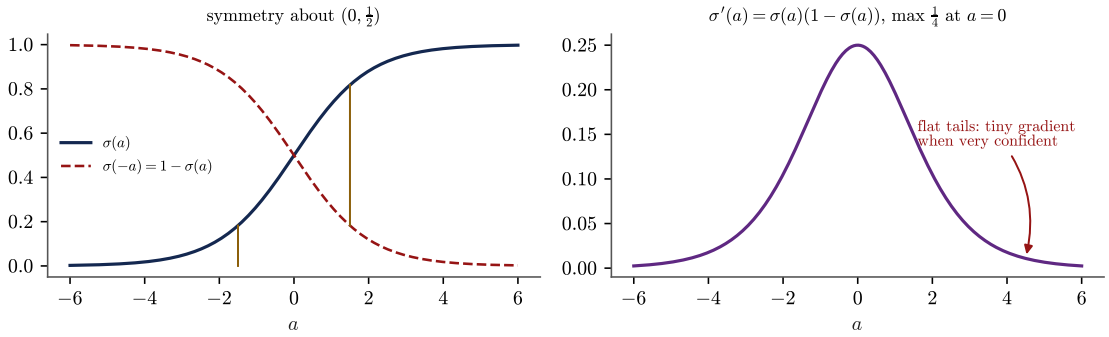
Property 3: the derivative is $\sigma'(a)=\sigma(a)\,(1-\sigma(a))$

Derivation

Write $\sigma(a)=(1+e^{-a})^{-1}$ and use the chain rule (power rule outside, e^{-a} inside):

$$
\begin{aligned}
\sigma'(a) &= -1\cdot(1+e^{-a})^{-2}\cdot\frac{d}{da}(1+e^{-a}) = -(1+e^{-a})^{-2}\cdot(-e^{-a}) = \frac{e^{-a}}{(1+e^{-a})^{2}}\\
&= \underbrace{\frac{1}{1+e^{-a}}}_{\sigma(a)}\cdot\underbrace{\frac{e^{-a}}{1+e^{-a}}}_{1-\sigma(a)\ \text{(Property 2)}} = \sigma(a)\,\big(1-\sigma(a)\big).
\end{aligned}
$$

This is the most useful fact in the chapter: the derivative of the sigmoid is built from the sigmoid itself. At $a=0$ it equals $\frac{1}{2}\cdot\frac{1}{2}=\frac{1}{4}$, its largest value.



Left: $\sigma(-a)$ is $\sigma(a)$ flipped; the gold segments show $\sigma(1.5)$ and $\sigma(-1.5)$ add to 1. Right: the derivative peaks at $\frac{1}{4}$ and is almost zero when $|a|$ is large.

Exam question: $\sigma(a)=\frac{1}{3}$; find $\sigma'(-a)$

By Property 3 applied at $-a$: $\sigma'(-a)=\sigma(-a)\,(1-\sigma(-a))$. By Property 2: $\sigma(-a)=1-\sigma(a)=\frac{2}{3}$, and $1-\sigma(-a)=\sigma(a)=\frac{1}{3}$. So $\sigma'(-a)=\frac{2}{3}\cdot\frac{1}{3}=\boxed{\frac{2}{9}}$. (Notice $\sigma'(-a)=\sigma'(a)$: the derivative is symmetric too.)

25.4 The logistic regression model

Take the same linear score as before, $z = w_0 + w_1x_1 + \cdots + w_dx_d = \mathbf{w}^\top\mathbf{x}$ (with $x_0 = 1$ so that w_0 is the bias), and push it through the sigmoid:

$$P(Y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top\mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w}^\top\mathbf{x}}},$$

$$P(Y = 0 \mid \mathbf{x}) = 1 - \sigma(\mathbf{w}^\top\mathbf{x}) = \sigma(-\mathbf{w}^\top\mathbf{x}) = \frac{1}{1 + e^{\mathbf{w}^\top\mathbf{x}}}.$$

Sir writes the same thing as $h_{\boldsymbol{\theta}}(\mathbf{x}) = g(\boldsymbol{\theta}^\top\mathbf{x})$ with $g = \sigma$ and $\boldsymbol{\theta} = \mathbf{w}$. The two probabilities add to 1, as they must.

Think of $z = \mathbf{w}^\top\mathbf{x}$ as a **score of evidence** for "yes". Positive score: more evidence for yes. Negative: more evidence for no. The sigmoid converts the score into a probability. The weights say how much each feature adds to the score.

Multiply top and bottom of $\frac{1}{1+e^{-z}}$ by e^z and you get the other form sir uses, $\sigma(z) = \frac{e^z}{1+e^z}$. Same number; use whichever makes the algebra shorter.

Exam question: compute a probability

$\boldsymbol{\theta} = (3, 4, 1)$, test point $\mathbf{x} = (1, 1, 1)$ (the first 1 is the bias feature). Score: $z = 3+4+1 = 8$. Probability: $\sigma(8) = \frac{1}{1+e^{-8}} = \frac{1}{1.000335} = \boxed{0.99966}$.

Exam question: read the output of $h(x) = \sigma(1 - x)$ at $x = 2$

$z = 1 - 2 = -1$, so $P(y = 1 \mid x) = \sigma(-1) = 0.269 \approx 0.27$, and $P(y = 0 \mid x) = 0.73$. The two true statements are "$P(y = 1) \approx 0.27$" and "$P(y = 0) \approx 0.73$".

25.5 From probability to decision: the threshold and the boundary

To turn a probability into a class we pick a threshold, by default $T = 0.5$: predict $Y = 1$ if $P(Y = 1 \mid \mathbf{x}) \geq 0.5$, else $Y = 0$.

Derivation: where is the decision boundary?

We predict 1 exactly when

$$\begin{aligned}
\frac{1}{1 + e^{-z}} \geq \frac{1}{2} &\iff 1 + e^{-z} \leq 2 && \text{flip both positive fractions} \\
&\iff e^{-z} \leq 1 && \text{subtract 1} \\
&\iff -z \leq 0 && \ln \text{ is increasing; } \ln 1 = 0 \\
&\iff z = \mathbf{w}^\top\mathbf{x} \geq 0.
\end{aligned}$$

So "probability above one half" is *exactly* "score above zero". The **decision boundary** is the set where the score is zero, $\mathbf{w}^\top\mathbf{x} = 0$.

- **One feature:** $w_0 + w_1x = 0$, so the boundary is the single point $x = -w_0/w_1$.
- **Two features:** $w_0 + w_1x_1 + w_2x_2 = 0$ is a **straight line** in the (x_1, x_2) plane.

- d **features:** a flat hyperplane. Logistic regression is a *linear classifier*: the sigmoid curves the probabilities, not the boundary.

Exam question: which picture is the boundary for $\theta = (6, 0, -1)$?

Score $z = 6+0\cdot x_1-x_2 = 6-x_2$. Predict 1 when $x_2 \le 6$. The boundary is the **horizontal line** $x_2 = 6$, with $y = 1$ *below* it. ($\theta_1 = 0$ means x_1 is irrelevant, which is why the line is horizontal.)

Exam question: $w_1 = 3,\ w_2 = 5,\ w_0 = -15$

Boundary: $3x_1+5x_2 = 15$. Two points: $x_1 = 0 \Rightarrow x_2 = 3$; $x_2 = 0 \Rightarrow x_1 = 5$. Draw the line through $(0, 3)$ and $(5, 0)$. At the origin the score is $-15 < 0$, so the origin's side is $Y = 0$ and the far side is $Y = 1$. The model's probability is $P(Y = 1 \mid x_1, x_2) = \sigma(3x_1+5x_2-15)$.

Exam question: thresholds change accuracy

True labels and predicted $P(Y = 1)$: $(1, 0.9), (1, 0.6), (0, 0.6), (0, 0.55), (1, 0.4), (0, 0.3)$.

threshold	predictions	accuracy
$T = 0.25$	$1, 1, 1, 1, 1, 1$	$3/6$
$T = 0.5$	$1, 1, 1, 1, 0, 0$	$3/6$
$T = 0.75$	$1, 0, 0, 0, 0, 0$	$\mathbf{4/6}$

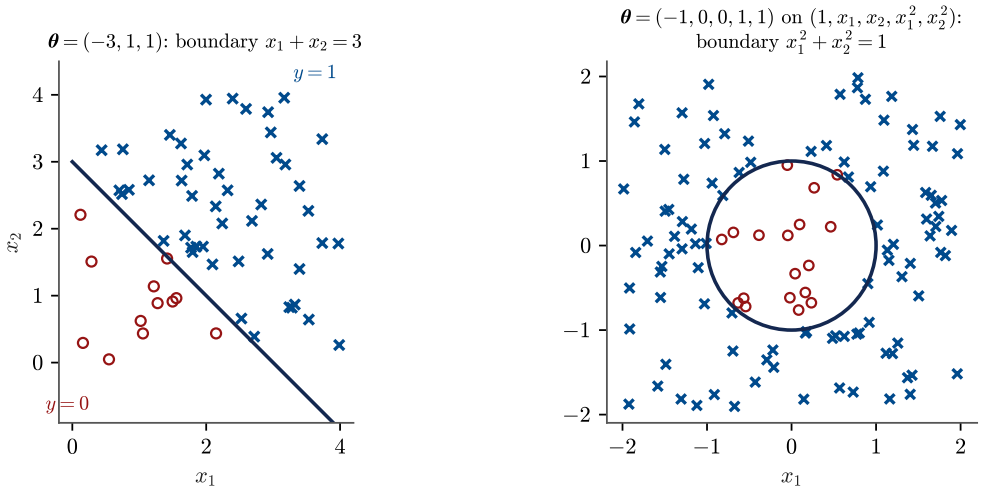
Model C ($T = 0.75$) is most accurate here. The threshold is a separate choice from the weights; 0.5 is a default, not a law. (In medicine you often lower it to catch more sick patients.)

25.5.1 Curved boundaries with extra features

If the classes are separated by a circle, no straight boundary can work. Give the model new features, exactly as in the basis-function chapter.

Sir's circle example

Use features $(1,\ x_1,\ x_2,\ x_1^2,\ x_2^2)$ and weights $\theta = (-1,\ 0,\ 0,\ 1,\ 1)$. Score: $z = -1+x_1^2+x_2^2$. Predict 1 when $x_1^2 + x_2^2 \ge 1$: outside the unit circle. The model is still linear in θ (a weighted sum), but the boundary is a circle in the original (x_1, x_2) space.



Left: a straight boundary $x_1 + x_2 = 3$ from $\theta = (-3, 1, 1)$. Right: a circular boundary from squared features.

25.6 Odds and log-odds: what the weights mean

The **odds** of an event with probability p is $\frac{p}{1-p}$ ("3 to 1" means $p = 0.75$).

Derivation: the log-odds are linear

$$\frac{P(Y=1 \mid \mathbf{x})}{P(Y=0 \mid \mathbf{x})} = \frac{e^z/(1+e^z)}{1/(1+e^z)} \qquad \text{use the } \tfrac{e^z}{1+e^z} \text{ form for } P(Y=1)$$

$$= e^z \qquad \text{the } (1+e^z) \text{ cancels}$$

$$\ln \frac{P(Y=1 \mid \mathbf{x})}{P(Y=0 \mid \mathbf{x})} = z = \mathbf{w}^\top \mathbf{x}.$$

Mental model

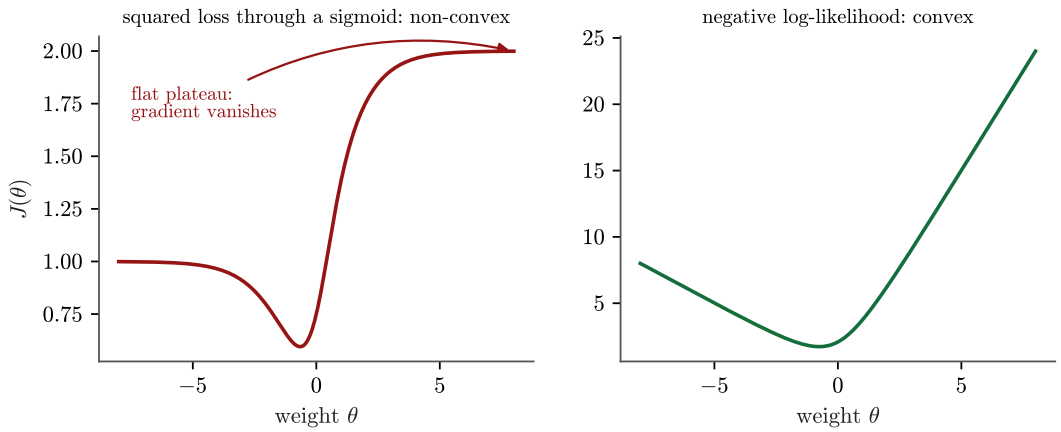
Logistic regression says: *the log of the odds is a straight-line function of the features.* Increase x_j by one unit and the log-odds go up by w_j, so the odds get **multiplied** by e^{w_j}. A weight of 0.7 roughly doubles the odds ($e^{0.7} = 2.01$). A weight of 0 means the feature does not matter.

Watch out

Exam trap: "logistic regression models the odds $\frac{p}{1-p}$ as a linear function" is **false**. It models the *log*-odds as linear. The odds themselves are $e^{\mathbf{w}^\top \mathbf{x}}$.

25.7 Choosing the weights, part 1: why not squared error?

The obvious recipe, minimise $\sum_i (y_i - \sigma(\mathbf{w}^\top \mathbf{x}_i))^2$, has a hidden problem: once the sigmoid sits inside the square, the cost is **not convex**. It has flat plateaus and can have several dips, so gradient descent may stall or stop in the wrong valley. We want a cost shaped like a single bowl (sir's "desired function").



One weight, three data points. Left: squared error through the sigmoid has a flat plateau. Right: the cost we derive next is a single smooth bowl.

25.8 Choosing the weights, part 2: maximum likelihood, step by step

Recap: what likelihood means

Fix the data. For a candidate parameter value, the **likelihood** is the probability the model gives to the data we actually observed. Good parameters make the observed data likely. **Maximum likelihood estimation** (MLE) picks the parameter that makes the observed data *most* likely. The class deck did this for house prices with a Gaussian; here we do it for 0/1 labels.

Step 1: one label, one formula

For example i, write $p_i = \sigma(\mathbf{w}^\top \mathbf{x}_i)$ for the model's $P(Y = 1 \mid \mathbf{x}_i)$. The probability of the label that actually occurred is

$$P(y_i \mid \mathbf{x}_i) = \begin{cases} p_i & \text{if } y_i = 1 \\ 1 - p_i & \text{if } y_i = 0 \end{cases} \qquad = \qquad p_i^{\,y_i}\,(1 - p_i)^{\,1 - y_i}.$$

Check: if $y_i = 1$ the single formula gives $p_i^1(1-p_i)^0 = p_i$; if $y_i = 0$ it gives $p_i^0(1-p_i)^1 = 1 - p_i$. This is the **Bernoulli** distribution, one coin flip with heads-probability p_i.

Step 2: all n labels

Assume the examples are independent. The joint probability is the product:

$$L(\mathbf{w}) = \prod_{i=1}^{n} p_i^{\,y_i}(1 - p_i)^{\,1 - y_i}.$$

Step 3: take the logarithm

A product of many numbers below 1 is tiny and awkward to differentiate. The log turns products into sums ($\ln(ab) = \ln a + \ln b$) and powers into multipliers ($\ln a^k = k \ln a$), and it is increasing,

so the same $\mathbf{w}$ maximises both:

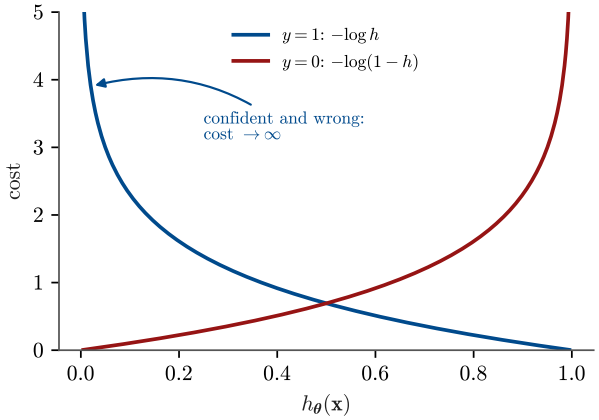
$$\ell(\mathbf{w}) = \ln L(\mathbf{w}) = \sum_{i=1}^{n} \Big[y_i \ln p_i + (1 - y_i) \ln(1 - p_i) \Big].$$

Step 4: flip the sign to get a cost

Maximising ℓ is minimising $-\ell$, the **cross-entropy loss** (log loss, negative log-likelihood):

$$\boxed{J(\mathbf{w}) = -\sum_{i=1}^{n} \Big[y_i \ln p_i + (1 - y_i) \ln(1 - p_i) \Big], \qquad p_i = \sigma(\mathbf{w}^\top \mathbf{x}_i).}$$

For one example only one term is alive: cost $= -\ln p$ if $y = 1$, and $-\ln(1 - p)$ if $y = 0$.



True label 1 (blue): predicting p near 1 costs almost nothing, predicting p near 0 costs without limit. Red is the mirror image for $y = 0$.

The cost of a prediction is how surprised the model is by the truth. Said 99% yes, answer was yes: surprise $-\ln 0.99 = 0.01$. Said 1% yes, answer was yes: surprise $-\ln 0.01 = 4.6$. Confident mistakes are punished hard; that is exactly the behaviour linear regression lacked.

Exam question: mean cross-entropy on three rows

Rows $(y, \hat{y})$: $(0,\ 0.45)$, $(0,\ 0.19)$, $(1,\ 0.80)$. For $y = 0$ the cost is $-\ln(1 - \hat{y})$: $-\ln 0.55$, $-\ln 0.81$. For $y = 1$ it is $-\ln 0.80$. Mean: $-\frac{1}{3}\big(\ln 0.55 + \ln 0.81 + \ln 0.80\big)$, the fourth option; numerically 0.344.

25.9 The gradient of the cost, every line

Route A: simplify first (sir's route)

Derivation

Write $z_i = \mathbf{w}^\top \mathbf{x}_i$. From the two forms of the sigmoid,

$$\ln p_i = -\ln(1 + e^{-z_i}), \qquad \ln(1 - p_i) = \ln \frac{e^{-z_i}}{1 + e^{-z_i}} = -z_i - \ln(1 + e^{-z_i}).$$

Substitute into one term of J (drop the index i):

$$\begin{aligned}
&-\big[y\ln p+(1-y)\ln(1-p)\big] \\
&= y\ln(1+e^{-z})+(1-y)\big(z+\ln(1+e^{-z})\big) && \text{substitute} \\
&= \ln(1+e^{-z})+(1-y)z && \text{combine the logs} \\
&= \big(\ln(1+e^{z})-z\big)+z-yz && \text{(see below)} \\
&= \ln(1+e^{z})-y\,z.
\end{aligned}$$

The step marked "see below" uses $\ln(1+e^{-z}) = \ln\frac{e^z+1}{e^z} = \ln(1+e^z) - z$. So the whole cost is

$$J(\mathbf{w}) = \sum_{i=1}^{n}\Big[\ln\big(1+e^{\mathbf{w}^\top\mathbf{x}_i}\big) - y_i\,\mathbf{w}^\top\mathbf{x}_i\Big].$$

Differentiate each piece. Chain rule: $\frac{\partial}{\partial\mathbf{w}}\ln(1+e^{\mathbf{w}^\top\mathbf{x}}) = \frac{e^{\mathbf{w}^\top\mathbf{x}}}{1+e^{\mathbf{w}^\top\mathbf{x}}}\mathbf{x} = \sigma(\mathbf{w}^\top\mathbf{x})\,\mathbf{x}$, and $\frac{\partial}{\partial\mathbf{w}}(y\,\mathbf{w}^\top\mathbf{x}) = y\,\mathbf{x}$. Hence

$$\boxed{\nabla J(\mathbf{w}) = \sum_{i=1}^{n}\big(\sigma(\mathbf{w}^\top\mathbf{x}_i) - y_i\big)\,\mathbf{x}_i = \mathbf{X}^\top(\mathbf{p}-\mathbf{y})}$$

where $\mathbf{X}$ stacks the $\mathbf{x}_i^\top$ as rows and $\mathbf{p},\mathbf{y}$ are the vectors of p_i, y_i.

Route B: chain rule with $\sigma' = \sigma(1-\sigma)$

Derivation

For one example, cost $= -y\ln\sigma(z) - (1-y)\ln(1-\sigma(z))$ and $\partial z/\partial\mathbf{w} = \mathbf{x}$:

$$\begin{aligned}
\frac{\partial\,\text{cost}}{\partial z} &= -y\,\frac{\sigma'(z)}{\sigma(z)} + (1-y)\,\frac{\sigma'(z)}{1-\sigma(z)} && (\ln u)' = u'/u \\
&= -y\,(1-\sigma) + (1-y)\,\sigma && \text{Property 3, then cancel} \\
&= -y + y\sigma + \sigma - y\sigma = \sigma - y.
\end{aligned}$$

So $\partial\,\text{cost}/\partial\mathbf{w} = (\sigma(z)-y)\,\mathbf{x}$. Same answer.

Sir's board writes it per coordinate, with residual $\epsilon_i = y_i - \sigma_i$: $\frac{\partial J}{\partial w_0} = \sum_i(\sigma_i - y_i) = -\sum_i \epsilon_i$ and $\frac{\partial J}{\partial w_1} = \sum_i(\sigma_i - y_i)\,x_{i1} = -\sum_i \epsilon_i\,x_{i1}$. The bias gets the plain sum of residuals because its "feature" is the constant 1.

Mental model

The gradient is *(prediction − truth) × input*, summed over examples: the same shape as the least-squares gradient $\mathbf{X}^\top(\mathbf{X}\mathbf{w}-\mathbf{y})$. An example already predicted well contributes almost nothing; a badly wrong one pulls hard in the direction of its own features.

Why there is no closed form

Setting the gradient to zero gives $\mathbf{X}^\top\big(\sigma(\mathbf{X}\mathbf{w})-\mathbf{y}\big)=\mathbf{0}$. In linear regression $\mathbf{w}$ appeared linearly and a matrix inverse solved it. Here it is trapped inside the nonlinear σ, so no rearrangement isolates it. We iterate.

25.10 Gradient descent for logistic regression

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha\,\nabla J(\mathbf{w}) = \mathbf{w} - \alpha\sum_{i=1}^{n}\big(\sigma(\mathbf{w}^\top\mathbf{x}_i) - y_i\big)\mathbf{x}_i,$$

all coordinates updated at once, repeated until the change is tiny. The cost is convex (proved in the Newton section), so there is one valley and the starting point does not change where you end up.

One gradient step by hand

Two examples with the bias feature first: $\mathbf{x}_1=(1,\ 2)$, $y_1=1$; $\mathbf{x}_2=(1,\ -1)$, $y_2=0$. Start at $\mathbf{w}=(0,0)$, $\alpha=0.5$. Every score is 0, so every $\sigma=0.5$. Errors: -0.5 and $+0.5$. Gradient: $-0.5\,(1,2)+0.5\,(1,-1)=(0,\ -1.5)$. Update: $\mathbf{w}=(0,\ 0.75)$. The weight on x_1 became positive, pushing the positive example (with $x_1=2$) towards 1 and the negative one towards 0.

Exam question: SGD with a penalty

Model $f_\theta=\sigma(\theta-2)$, loss $\ell=-y\ln f-(1-y)\ln(1-f)+\frac{1}{2}\theta^2$, learning rate 1. Route B with $z=\theta-2$, $\partial z/\partial\theta=1$, gives $\sigma(\theta-2)-y$; the penalty gives θ. So $\theta^{(t+1)}=\theta^{(t)}-\big[\sigma(\theta^{(t)}-2)-y+\theta^{(t)}\big]=\boxed{y-\sigma(\theta^{(t)}-2)}$.

25.11 Regularization, and the MAP view

Why regularize

If the training data are **linearly separable**, the MLE does not exist. Take a separating line and double all its weights: every score doubles, every probability moves closer to the right answer, and the cost drops. Repeat forever; the weights run off to infinity. (Exam: "with no regularization and separable data, the optimal parameters tend to $\pm\infty$" is **true**.) The 8-patient table is separable, which is why the logistic curve in the first figure needed a little regularization.

The regularized cost

$$J_{\text{reg}}(\mathbf{w}) = -\sum_{i=1}^{n}\Big[y_i\ln p_i + (1-y_i)\ln(1-p_i)\Big] + \lambda\sum_{j=1}^{d} w_j^2.$$

The extra term charges for large weights, like ridge. The bias w_0 is not penalized. The gradient gains $2\lambda w_j$, so $w_j \leftarrow w_j - \alpha\big[\sum_i (p_i-y_i)x_{ij} + 2\lambda w_j\big]$: every weight shrinks a little each step ("weight decay").

Where the penalty comes from: MAP

Derivation

MLE trusts only the data. **MAP** (maximum a posteriori) also uses a prior belief that weights are small. Prior: $p(\mathbf{w}) \propto \exp(-\lambda\,\mathbf{w}^\top\mathbf{w})$, a Gaussian centred at zero. Bayes' rule says posterior $\propto$ likelihood $\times$ prior, so

$$\ln p(\mathbf{w} \mid \text{data}) = \ell(\mathbf{w}) + \ln p(\mathbf{w}) + \text{const} = \ell(\mathbf{w}) - \lambda\,\mathbf{w}^\top\mathbf{w} + \text{const}.$$

Maximising this is minimising $-\ell(\mathbf{w}) + \lambda\,\mathbf{w}^\top\mathbf{w}$, the regularized cost. *The $\lambda\|\mathbf{w}\|^2$ penalty is a Gaussian prior in disguise.*

Watch out

Two slips in the slides. The prior $\mathcal{N}(\mathbf{0}, \lambda^{-1}\mathbf{I})$ has log-density $-\frac{\lambda}{2}\mathbf{w}^\top\mathbf{w}$, not $-\lambda\mathbf{w}^\top\mathbf{w}$; the factor 2 only relabels λ. And slide 132 writes "$-\frac{\lambda}{n}\theta_j$" inside a bracket that is then subtracted, which would *grow* the weights. The penalty must shrink them, as written above.

Exam question: credit-card default

$\hat{\theta} = (-1.30,\ 0.08,\ -0.08,\ 0.001)$ for (intercept, education, marriage, age). A 50-year-old, married (code 1), high-school (code 3) customer: $z = -1.30 + 0.08(3) - 0.08(1) + 0.001(50) = -1.09$, so $P(\text{fail}) = \sigma(-1.09) = 0.252$.

- They paid ($y = 0$): cross-entropy $-\ln(1 - \sigma(-1.09)) = -\ln\sigma(1.09) = 0.290$.
- One more year of age adds 0.001 to the log-odds.
- Married, high school: $z = -1.14 + 0.001\,\text{age} > 0$ needs age > 1140. No realistic customer in that group is more likely to fail than pay.
- Threshold $T = 0.8$: $\sigma(z) \geq 0.8 \iff z \geq \ln\frac{0.8}{0.2} = \ln 4$. Boundary: $A = 0.08$, $B = -0.08$, $C = 0.001$, $D = -1.30 - \ln 4$.
- 100% training accuracy with this boundary: yes, the data were linearly separable in that feature space, because a hyperplane separated them.

25.12 Newton's method: using curvature

Gradient descent only knows the slope. On a long narrow valley it zig-zags. Newton's method also uses the *curvature* to choose the step.

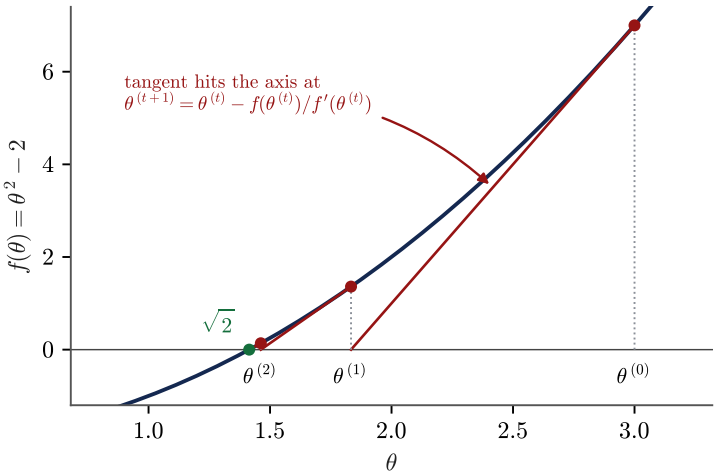
In one dimension: the tangent-line idea

Derivation

To solve $f(\theta) = 0$: at the current guess $\theta^{(0)}$ draw the tangent line, $y = f(\theta^{(0)}) + f'(\theta^{(0)})(\theta - \theta^{(0)})$. Set $y = 0$ and solve for θ:

$$\theta^{(1)} = \theta^{(0)} - \frac{f(\theta^{(0)})}{f'(\theta^{(0)})}.$$

Repeat from the new guess. To **minimise** a cost ℓ, find where its slope is zero: apply the same rule to $f = \ell'$, giving $\theta^{(t+1)} = \theta^{(t)} - \ell'(\theta^{(t)})/\ell''(\theta^{(t)})$.



Newton on $f(\theta) = \theta^2 - 2$ from $\theta^{(0)} = 3$: $3 \to 1.833 \to 1.462 \to 1.4150 \to 1.41421 = \sqrt{2}$. Each step follows a tangent down to the axis.

Newton for $\sqrt{2}$ from $\theta^{(0)} = 1$

$\theta^{(t+1)} = \theta^{(t)} - \frac{(\theta^{(t)})^2 - 2}{2\theta^{(t)}}$: $1 \to 1.5 \to 1.41667 \to 1.41422 \to 1.4142136$. The number of correct digits roughly doubles every step.

In many dimensions

The second derivative becomes the **Hessian H**, $H_{jk} = \frac{\partial^2 J}{\partial w_j \partial w_k}$, and "divide by ℓ''" becomes "multiply by $\mathbf{H}^{-1}$": $\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \mathbf{H}^{-1}\nabla J(\mathbf{w}^{(t)})$.

The Hessian of logistic regression

Derivation

Differentiate $\nabla J = \sum_i (\sigma_i - y_i)\mathbf{x}_i$ once more. Only $\sigma_i = \sigma(\mathbf{w}^\top \mathbf{x}_i)$ depends on $\mathbf{w}$, and $\partial\sigma_i/\partial\mathbf{w} = \sigma_i(1-\sigma_i)\,\mathbf{x}_i$ (Property 3 and the chain rule). So

$$\mathbf{H} = \sum_{i=1}^{n} \sigma_i(1-\sigma_i)\,\mathbf{x}_i\mathbf{x}_i^\top = \mathbf{X}^\top \mathbf{S}\,\mathbf{X}, \qquad \mathbf{S} = \operatorname{diag}\big(\sigma_i(1-\sigma_i)\big).$$

Convexity, proved. For any direction $\mathbf{v}$, $\mathbf{v}^\top \mathbf{H}\mathbf{v} = \sum_i \sigma_i(1-\sigma_i)\,(\mathbf{x}_i^\top \mathbf{v})^2 \geq 0$: positive weights times squares. A positive semi-definite Hessian everywhere means a convex cost: one bowl, no false valleys.

Newton as weighted least squares (IRLS)

Derivation

$$
\begin{aligned}
\mathbf{w}^{(t+1)} &= \mathbf{w}^{(t)} - (\mathbf{X}^\top \mathbf{S}\mathbf{X})^{-1}\mathbf{X}^\top(\boldsymbol{\sigma} - \mathbf{y}) \\
&= (\mathbf{X}^\top \mathbf{S}\mathbf{X})^{-1}\Big[(\mathbf{X}^\top \mathbf{S}\mathbf{X})\mathbf{w}^{(t)} + \mathbf{X}^\top(\mathbf{y} - \boldsymbol{\sigma})\Big] && \text{factor out } (\mathbf{X}^\top \mathbf{S}\mathbf{X})^{-1} \\
&= (\mathbf{X}^\top \mathbf{S}\mathbf{X})^{-1}\mathbf{X}^\top \mathbf{S}\Big[\mathbf{X}\mathbf{w}^{(t)} + \mathbf{S}^{-1}(\mathbf{y} - \boldsymbol{\sigma})\Big] && \text{pull out } \mathbf{X}^\top \mathbf{S} \\
&= (\mathbf{X}^\top \mathbf{S}\mathbf{X})^{-1}\mathbf{X}^\top \mathbf{S}\,\mathbf{z}^{(t)}.
\end{aligned}
$$

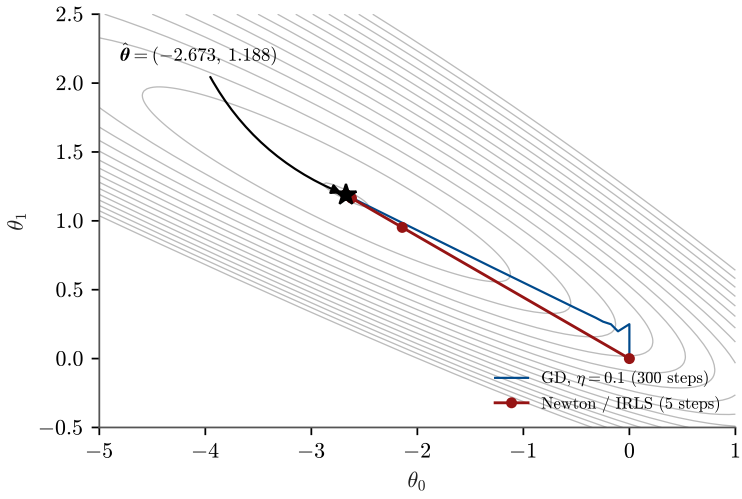
That is the weighted least-squares formula with weights $\mathbf{S}$ and adjusted target $\mathbf{z}^{(t)} = \mathbf{X}\mathbf{w}^{(t)} + \mathbf{S}^{-1}(\mathbf{y} - \boldsymbol{\sigma})$. The weights change every iteration: **iteratively reweighted least squares** (IRLS).

Newton vs gradient descent on 8 points

$x = (0.5, 1, 1.5, 2, 2.5, 3, 3.5, 4)$, $y = (0, 0, 1, 0, 1, 0, 1, 1)$ (overlapping, so the MLE exists), model $\sigma(w_0 + w_1 x)$, start $(0, 0)$.

iteration	$\mathbf{w}$	cost J	$\lVert\nabla J\rVert$
0	$(0,\ 0)$	5.5452	2.5
1	$(-2.143,\ 0.952)$	4.2649	0.35
2	$(-2.622,\ 1.165)$	4.2251	0.031
3	$(-2.673,\ 1.188)$	4.2248	0.00033
4	$(-2.6734,\ 1.1882)$	4.2248	0.00000004

Four Newton steps reach machine precision (and match scikit-learn). Gradient descent with $\alpha = 0.1$ needs 652 steps to get $\lVert\nabla J\rVert$ below 10^{-6}.



The two paths on the cost contours. GD zig-zags along the narrow valley; Newton walks almost straight in.

Newton: advantages	Newton: disadvantages
step size chosen automatically by $\mathbf{H}^{-1}$	builds and inverts a $d \times d$ matrix every step
very few iterations near the optimum	too expensive when d is large (deep networks)

25.13 Gradient descent with momentum

The problem momentum solves

On a long narrow valley the gradient points mostly *across* the valley (steep walls) and only a little *along* it (gentle floor). Plain GD bounces wall to wall and creeps along the floor. Sir's picture: reduce the vertical oscillation, increase the horizontal progress.

The update

$$\mathbf{v}_{t+1} = \beta\,\mathbf{v}_t - \eta\,\nabla J(\mathbf{w}_t), \qquad \mathbf{w}_{t+1} = \mathbf{w}_t + \mathbf{v}_{t+1}, \qquad \mathbf{v}_0 = \mathbf{0}.$$

η is the learning rate; $\beta \in [0, 1)$ is the damping (friction) coefficient, typically 0.9.

Derivation: velocity is an average of past gradients

Write $\mathbf{g}_t = \nabla J(\mathbf{w}_t)$ and unroll from $\mathbf{v}_0 = \mathbf{0}$:

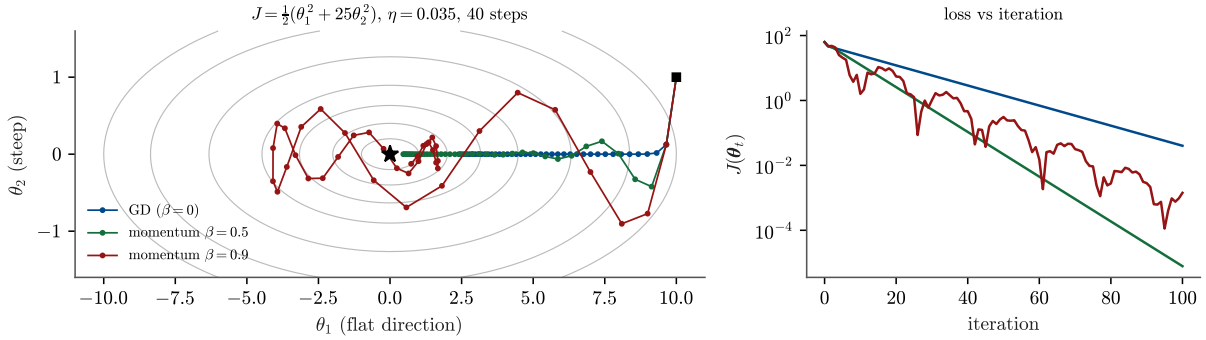
$$\mathbf{v}_1 = -\eta\,\mathbf{g}_0, \qquad \mathbf{v}_2 = \beta\mathbf{v}_1 - \eta\,\mathbf{g}_1 = -\eta(\mathbf{g}_1 + \beta\,\mathbf{g}_0),$$

$$\mathbf{v}_3 = \beta\mathbf{v}_2 - \eta\,\mathbf{g}_2 = -\eta(\mathbf{g}_2 + \beta\,\mathbf{g}_1 + \beta^2\mathbf{g}_0), \qquad \dots \qquad \mathbf{v}_t = -\eta \sum_{k=0}^{t-1} \beta^k\,\mathbf{g}_{t-1-k}.$$

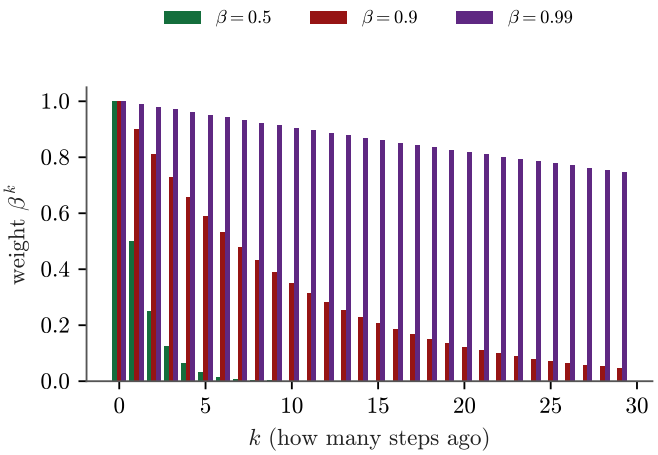
Newest gradient weight 1, the one before β, then β^2: an **exponentially weighted moving average**. With $\beta = 0.9$ the effective memory is about $1/(1-\beta) = 10$ steps.

Mental model

A ball rolling in the valley. Gradient components with the same sign step after step (along the floor) add up, so the ball speeds up, up to $\frac{1}{1-\beta}$ times. Components that flip sign (across the valley) cancel in the average, so the bouncing dies down. $\beta = 0$: plain GD. $\beta \to 1$: almost no friction, long oscillation. $\beta = 0.9$: the usual compromise.



A narrow bowl $J = \frac{1}{2}(w_1^2 + 25w_2^2)$, $\eta = 0.035$, start $(10, 1)$. Plain GD (blue) crawls: cost 0.040 after 100 steps. $\beta = 0.5$ (green) reaches 8×10^{-6}. $\beta = 0.9$ (red) overshoots and spirals: too little friction for this bowl.



The weight β^k of the gradient from k steps ago.

25.14 More than two classes: softmax regression

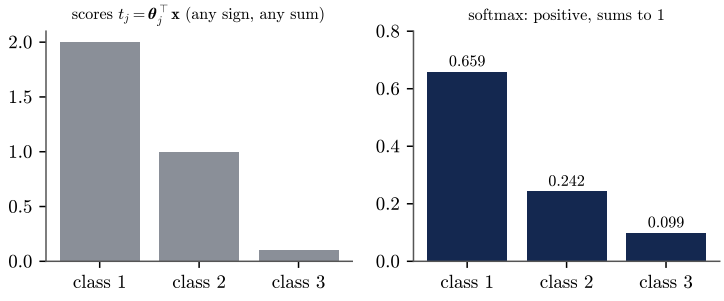
With K classes, give each class its own weight vector and score $t_j = \mathbf{w}_j^\top \mathbf{x}$. Scores can be any real numbers and need not add to 1. Exponentiate (makes them positive), then divide by the total (makes them add to 1):

$$P(Y = j \mid \mathbf{x}) = \frac{e^{t_j}}{\sum_{k=1}^{K} e^{t_k}}.$$

This is a **multinomial** distribution over the K labels.

Softmax by hand

Scores (2, 1, 0.1): $e^2 = 7.389$, $e^1 = 2.718$, $e^{0.1} = 1.105$; total 11.212. Probabilities 0.659, 0.242, 0.099, adding to 1.



Raw scores become probabilities.

Derivation: two-class softmax is the sigmoid

$P(Y = 1) = \dfrac{e^{t_1}}{e^{t_1} + e^{t_2}} = \dfrac{1}{1 + e^{-(t_1 - t_2)}} = \sigma(t_1 - t_2)$ (divide top and bottom by e^{t_1}). Only the difference of scores matters, so set $\mathbf{w}_2 = \mathbf{0}$ and you have logistic regression. The same division shows that adding a constant to every score leaves softmax unchanged.

25.15 From the textbook: where the sigmoid comes from

So far we *chose* the sigmoid because it is S-shaped. CS229 (Gaussian discriminant analysis) and Bishop (§4.2) show something deeper: if each class produces its feature from a bell curve, the sigmoid is not a choice, it is forced.

Two Gaussian classes give a logistic posterior

Suppose non-diabetic blood pressures follow $\mathcal{N}(\mu_0, \sigma^2)$, diabetic ones $\mathcal{N}(\mu_1, \sigma^2)$ (same spread), with prior $\pi = P(Y = 1)$. Bayes' rule:

$$P(Y = 1 \mid x) = \frac{\pi\, p(x \mid 1)}{\pi\, p(x \mid 1) + (1 - \pi)\, p(x \mid 0)} = \frac{1}{1 + \dfrac{(1 - \pi)\, p(x \mid 0)}{\pi\, p(x \mid 1)}} = \frac{1}{1 + e^{-a}},$$

$$\text{where} \quad a = \ln \frac{\pi\, p(x \mid 1)}{(1 - \pi)\, p(x \mid 0)}.$$

(Divide top and bottom by the numerator; then write the ratio as e^{-a}.) So the posterior is *always* a sigmoid of the log-ratio a. Now compute a for Gaussians. The normalising constants $\frac{1}{\sqrt{2\pi}\sigma}$ are equal and cancel:

$$\begin{aligned} a &= \ln \frac{\pi}{1 - \pi} - \frac{(x - \mu_1)^2}{2\sigma^2} + \frac{(x - \mu_0)^2}{2\sigma^2} && \ln e^u = u \\ &= \ln \frac{\pi}{1 - \pi} + \frac{-x^2 + 2\mu_1 x - \mu_1^2 + x^2 - 2\mu_0 x + \mu_0^2}{2\sigma^2} && \text{expand both squares} \\ &= \underbrace{\frac{\mu_1 - \mu_0}{\sigma^2}}_{w_1} x + \underbrace{\frac{\mu_0^2 - \mu_1^2}{2\sigma^2} + \ln \frac{\pi}{1 - \pi}}_{w_0} && \text{the } x^2 \text{ terms cancel.} \end{aligned}$$

$a = w_0 + w_1 x$ is **linear**, so $P(Y = 1 \mid x) = \sigma(w_0 + w_1 x)$: exactly logistic regression.

The blood-pressure classes as Gaussians

Take $\mu_0 = 100$, $\mu_1 = 130$, $\sigma = 10$, $\pi = \frac{1}{2}$. Then $w_1 = \frac{30}{100} = 0.3$ and $w_0 = \frac{100^2 - 130^2}{200} + \ln 1 = \frac{-6900}{200} = -34.5$. Boundary: $w_0 + w_1 x = 0 \Rightarrow x = 115$, the same threshold the eight patients gave. A patient at 118: $a = -34.5 + 0.3(118) = 0.9$, so $P(\text{diabetic}) = \sigma(0.9) = 0.711$ (checked by plugging the two Gaussian densities straight into Bayes' rule).

Mental model

Gaussian classes with equal spread ⇒ logistic posterior. The reverse is not true: logistic regression only assumes the *posterior* is a sigmoid, which also holds for many non-Gaussian classes. That is why the discriminative model is more robust when its generative cousin's assumptions are wrong, and why the generative model wins when they are right and data are scarce (Ng and Jordan, 2001).

25.16 From the textbook: the credit-card Default data (ISLR §4.3)

ISLR fits logistic regression to 10 000 credit-card customers, predicting default from the monthly balance: $\hat{\beta}_0 = -10.6513$, $\hat{\beta}_1 = 0.0055$.

Reading the fitted model

Balance \$1000: $z = -10.6513 + 5.5 = -5.151$, $\hat{p} = \sigma(-5.151) = 0.00576$ (under 1%). Balance \$2000: $z = 0.349$, $\hat{p} = 0.586$. Each extra dollar adds 0.0055 to the log-odds, so each extra \$100 multiplies the odds of default by $e^{0.55} = 1.73$. Notice the probability does not rise by a fixed amount per dollar: the change is tiny at low balance and large near the middle of the S.

A coefficient that changes sign (confounding)

Student status alone: $\hat{\beta}_0 = -3.5041$, $\hat{\beta}_{\text{student}} = 0.4049$. So $P(\text{default} \mid \text{student}) = \sigma(-3.0992) = 0.0431$ and $P(\text{default} \mid \text{non-student}) = \sigma(-3.5041) = 0.0292$: students look riskier. Add balance and income to the model and the student coefficient becomes -0.6468: *at the same balance*, a student is *less* likely to default. Both are true. Students carry larger balances, and large balances drive default. A coefficient in multiple logistic regression means "the effect of this feature with the others held fixed", which can differ even in sign from its effect alone.

25.17 From the textbook: judging a classifier (ISLR §4.4.3)

Accuracy is one number. For a yes/no classifier there are four kinds of outcome, arranged in a **confusion matrix**:

	predicted 1	predicted 0
actual 1	TP (true positive)	FN (false negative)
actual 0	FP (false positive)	TN (true negative)

Sensitivity (recall, true positive rate) = $\frac{TP}{TP+FN}$: of the real positives, how many did we catch?
Specificity (true negative rate) = $\frac{TN}{TN+FP}$: of the real negatives, how many did we correctly clear?
Precision = $\frac{TP}{TP+FP}$: of the ones we flagged, how many were real?

The deck's six predictions, at three thresholds

Labels $y = (1, 1, 0, 0, 1, 0)$, probabilities $(0.9,\ 0.6,\ 0.6,\ 0.55,\ 0.4,\ 0.3)$.

T	TP	FP	FN	TN	accuracy	sensitivity	specificity	precision
0.25	3	3	0	0	3/6	1	0	0.5
0.5	2	2	1	1	3/6	0.67	0.33	0.5
0.75	1	0	2	3	4/6	0.33	1	1

Lowering T catches more positives (sensitivity up) at the cost of more false alarms (specificity down). Which threshold is "best" depends on which mistake is more expensive: missing a diabetic patient is worse than an extra test.

The **ROC curve** plots sensitivity against 1−specificity as the threshold sweeps from 1 down to 0. The **AUC** (area under it) summarises all thresholds at once. It has a clean meaning: *the probability that a randomly chosen positive gets a higher score than a randomly chosen negative* (ties count half). AUC = 1 is perfect ranking; 0.5 is coin-flipping.

AUC by counting pairs

Positive scores $\{0.9,\ 0.6,\ 0.4\}$, negative scores $\{0.6,\ 0.55,\ 0.3\}$: $3 \times 3 = 9$ pairs. 0.9 beats all 3. 0.6 beats 0.55 and 0.3, ties 0.6 (half): 2.5. 0.4 beats only 0.3: 1. Total $6.5/9 = \boxed{0.722}$.

25.18 Generative vs discriminative

Generative (e.g. Naive Bayes)	**Discriminative** (e.g. logistic regression)
models how each class produces features, $P(\mathbf{x} \mid Y)$ and $P(Y)$	models the answer directly, $P(Y \mid \mathbf{x})$
uses Bayes' rule to get $P(Y \mid \mathbf{x})$	no Bayes' rule needed
fitted by counting (closed form)	fitted by optimisation (no closed form)
learns fast from little data	usually more accurate with plenty of data

25.19 Exam drill from the class decks

1. **Where can logistic regression be used?** Spam or not, disease or not: yes. Rainfall depth, house price: no (continuous outputs).
2. **Purpose of the sigmoid?** It transforms the score into a probability.
3. $\mathbf{x}^{*\top}\theta > 0$ **for a tumour model.** The tumour is *more likely* malignant, not certainly.
4. **When does regularization help?** High-dimensional data. Not when training error is already too high (that is underfitting).
5. **Odds ratio** $\frac{P(Y=1|X)}{P(Y=0|X)} = \exp(X^\top\beta)$; $P(Y=0 \mid X) = \sigma(-X^\top\beta)$.
6. $P(Y=1 \mid X) = \sigma(-2-3X)$**; when is** $P(Y=0) \geq \frac{3}{4}$**?** Need $\sigma(-2-3X) \leq \frac{1}{4}$, i.e. $-2-3X \leq \ln\frac{1/4}{3/4} = -1.099$, so $X \geq -0.300$.
7. **Always / sometimes / never**, $P(Y=1 \mid X) = \sigma(X^\top\beta)$: $P > X^\top\beta$ sometimes; $P(Y=1 \mid X) = P(Y=0 \mid -X)$ always (symmetry); $P < 1$ always; $\sigma(X^\top\beta) \leq \sigma(2X^\top\beta)$ sometimes (only when $X^\top\beta \geq 0$).
8. **Logistic regression...** minimises cross-entropy (yes), has a closed form (no), models log-odds as linear (yes), estimates class posteriors (yes).
9. **Different initialisations reach different optima?** False: the cost is convex.
10. **Spam words** "free", "order" raise spam probability and "homework" lowers it: $w_1, w_2 > 0$, $w_3 < 0$.

11. **No-bias model and the point $(0,0)$.** Without a bias the score at the origin is always 0, so its probability is stuck at 0.5 and its label cannot change $\mathbf{w}$. Changing that label affects only the model with a bias.

12. $y_i \sim \mathcal{N}(e^{wx_i}, 1)$**.** Non-linear regression. MLE minimises $\sum_i (y_i - e^{wx_i})^2$; setting the derivative $-2\sum_i (y_i - e^{wx_i}) x_i e^{wx_i}$ to zero gives $\sum_i x_i e^{2wx_i} = \sum_i x_i y_i e^{wx_i}$ (option B).

> **Go deeper**
>
> **Read.** Stanford CS229 main notes, Part II and §3. Logistic regression, Newton's method, softmax.
>
> **Read.** *Elements of Statistical Learning*, §4.4. Fitting by IRLS.
>
> **Read.** Goh, "Why Momentum Really Works", *Distill* (2017). Interactive picture of momentum.
>
> **Watch.** Kilian Weinberger, Cornell CS4780, *Lecture 11: Logistic Regression*
>
> **Watch.** Kilian Weinberger, Cornell CS4780, *Lecture 12: Gradient Descent / Newton's Method*

Exercises

Pencil first, then click the ⇝ to jump to the worked answer in Appendix D. The answer links back to the question.

25.1 ⇝ With $\boldsymbol{\theta} = (-4,\ 1,\ 2)$ and $\mathbf{x} = (1,\ 1,\ 1.5)$ (first entry the bias feature), compute z, h, the predicted class, and the cost if the true label is $y = 0$.

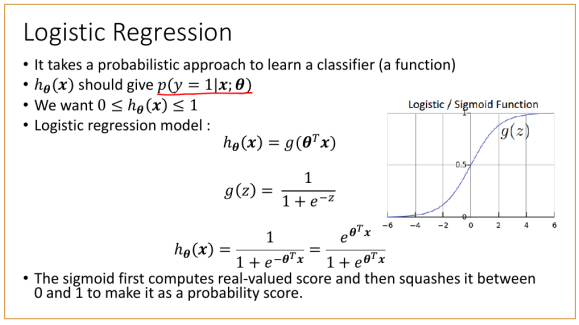
25.2 ⇝ One gradient-descent step from $\boldsymbol{\theta} = \mathbf{0}$ with $\alpha = 0.5$ on the two points $(1,\ 2) \to 1$ and $(1,\ -1) \to 0$ (first entry the bias feature). Give the new $\boldsymbol{\theta}$.

25.3 ⇝ Apply two Newton steps to minimize $\ell(\theta) = \theta^4 - 3\theta$ from $\theta^{(0)} = 1$. What is the exact minimizer?

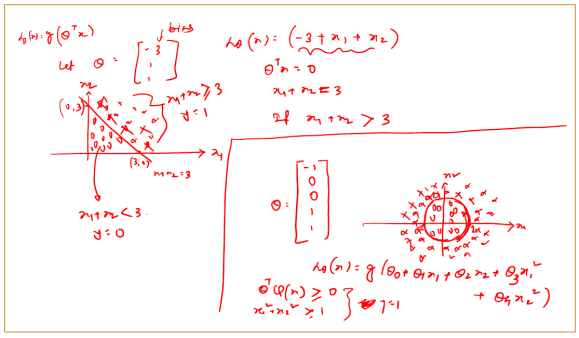
25.4 ⇝ Compute the softmax of (1, 1, 1) and of (3, 0, 0). What happens to the softmax if a constant c is added to every score?

Lecture slides for reference

Sir's slides on this topic, kept for revision. Everything on them is explained above.



Sir's lecture notes, slide 114. The model.



Sir's lecture notes, slide 118. Linear and circular boundaries.

Finding the Cost Function via MLE

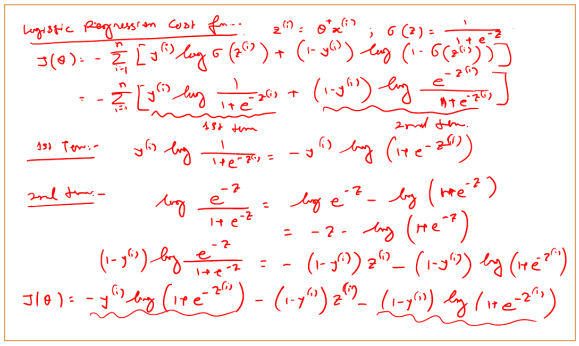
- Each label $y^{(i)}$ is binary with probability $h_\theta(\boldsymbol{x}^{(i)})$
- Assume Bernoulli likelihood

$$p(\boldsymbol{y}|\boldsymbol{X},\boldsymbol{\theta}) = \prod_{i=1}^{n} p(y^{(i)}|\boldsymbol{x}^{(i)};\boldsymbol{\theta})$$
$$= \prod_{i=1}^{n} h_\theta(\boldsymbol{x}^{(i)})^{y^{(i)}} \left(1 - h_\theta(\boldsymbol{x}^{(i)})\right)^{1-y^{(i)}}$$

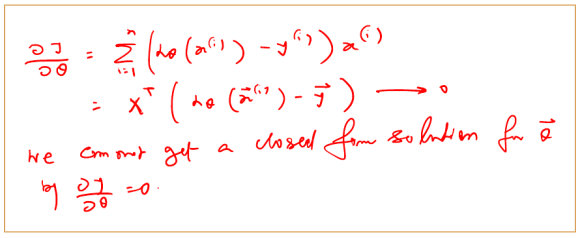
- The log-likelihood

$$l(\boldsymbol{\theta}) = \sum_{i=1}^{n} y^{(i)} \log h_\theta(\boldsymbol{x}^{(i)}) + (1 - y^{(i)}) \log \left(1 - h_\theta(\boldsymbol{x}^{(i)})\right)$$

Sir's lecture notes, slide 121. Bernoulli likelihood.

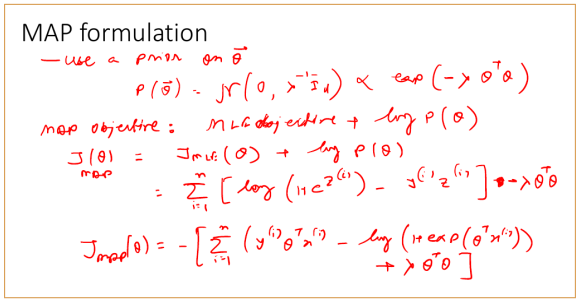


Sir's lecture notes, slide 125. Simplifying the cost.

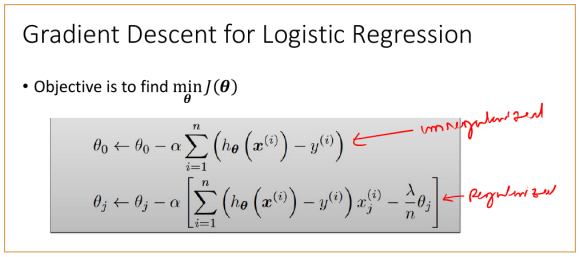


Sir's lecture notes, slide 127. The gradient; no closed form.

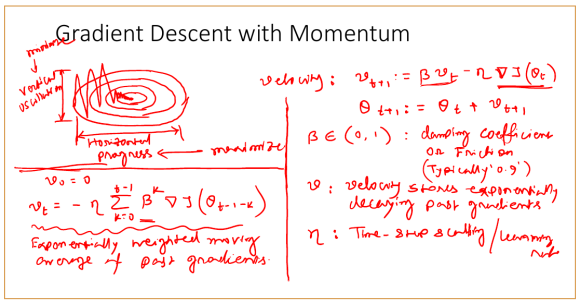
MAP formulation



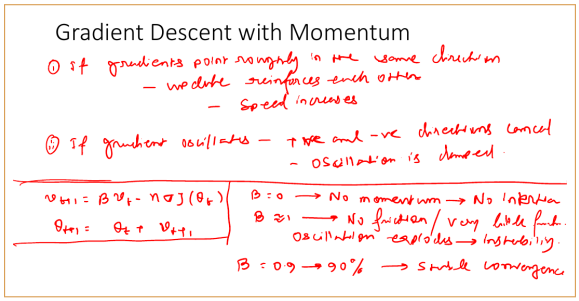
Sir's lecture notes, slide 129. MAP.



Sir's lecture notes, slide 132. GD updates.



Sir's lecture notes, slide 134. Momentum.



Sir's lecture notes, slide 135. Effect of β.

26

Insem 2: The Map, the Traps, the Drill

One chapter to revise from. First a map of every page of sir's 135 slides and your 16 pages of notes to the place in this part that teaches it, so nothing is missing. Then the corrections, collected. Then one-line answers to the questions most likely to be asked.

26.1 Slide-by-slide map

Slides	Content	In this part
1–5	capacity, hypothesis space, polynomial fits, appropriate capacity, reducing overfitting	Ch. 19
6	effective number of parameters $d(\mathbf{S}) = \operatorname{tr}\mathbf{S}$	Ch. 19
7	U-shaped generalization error	Ch. 19
8–9, 15	LASSO, orthonormal derivation, soft thresholding; ridge vs LASSO picture	Ch. 20
10–13, 16–19, 25–26	train/test, k-fold CV, CV formula, examples $k = 2, 3$	Ch. 21
14	section title	—
20–23	bias of an estimator; DC level; LOOCV estimator and error	Ch. 19; Ch. 21
24	choosing k	Ch. 21
27–31	bootstrap: motivation, method, diagram, Fox example	Ch. 21
32, 34	measuring bias and variance with the bootstrap	Ch. 19 (with correction)
33, 35–37	bias–variance decomposition, full derivation	Ch. 19
38–40	regularization; Bishop's illustration	Ch. 19 (regenerated)
41–46	distance-from-means classifier, decision rule, features	Ch. 22
47–50	1-NN, its features, Voronoi cells, decision boundary	Ch. 22
51–57	K-NN, algorithm, $K = 1..5$, MNIST, choosing K, regression	Ch. 22
58–59	distance functions; p-norm balls	Ch. 22; Ch. 20
60	resolving ties	Ch. 22

Slides	Content	In this part
61	K-NN density estimate, $P(C_k \mid \mathbf{x}) = K_k/K$	Ch. 22
62–83	Bayes' rule, naive assumption, training by counting, Laplace smoothing, classifier	Ch. 23
84–95	graphical model, worked PlayTennis prediction	Ch. 23
96, 112	NB summary; NB vs perceptron	Ch. 23
97–102	hyperplanes, margins, margin derivation and example	Ch. 24
103–107	perceptron loss, rule, online and batch, why it works	Ch. 24
108–110	convergence theorem, effect of γ and R, non-separable data	Ch. 24
111	stacked perceptrons and nonlinearity	Ch. 24
113–118	logistic model, log-odds, threshold, nonlinear boundary	Ch. 25
119–127	cost function via MLE, intuition, gradient derivation	Ch. 25
128–132	MAP, regularized LR, GD updates	Ch. 25
133–135	GD issues, momentum	Ch. 25

Your notes, beyond the slides. Newton's method in 1-D and for logistic regression, the Hessian $\mathbf{X}^\top\mathbf{S}\mathbf{X}$ and IRLS (Ch. 25); softmax and multi-class (Ch. 25); K-NN pros, cons and "no extrapolation" (Ch. 22 and §6); "5- or 10-fold in papers" (Ch. 21).

The class reference decks. The seven practice decks (Logistic Regression 1–3, Resampling 1–2, KNN 1–2) are covered as follows: the blood-pressure example, sigmoid properties, decision-boundary derivations, likelihood recap, cross-entropy and every multiple-choice question are worked in Ch. 25 ("Exam drill" and the worked boxes); the validation-set approach, LOOCV for regression and classification, k-fold, cost and the polynomial-degree story are in Ch. 21; the KNN algorithm, the $K = 3$ regression example, LOOCV on the ten-point grid, the $[4a, 3b]$ question, normalization and computational cost are in Ch. 22.

26.2 Corrections, all in one place

Each is explained, with numbers, where it occurs.

1. **LASSO threshold.** Zero iff $|\theta_j^0| \le \lambda/2$, decided on the OLS value, not "negative values become zero" (Ch. 20).
2. **Bootstrap "Bias^2".** The slide's formula is the variance; bias is $\overline{S^*} - S(\mathbf{Z})$ (Ch. 19).
3. **LOOCV variance.** Not always higher than 10-fold; for stable learners it was not (Ch. 21).
4. **KL divergence.** No leading minus sign; and it is not symmetric, so not a true distance (Ch. 22).
5. **Perceptron test.** Use $y\,\boldsymbol{\theta}^\top\mathbf{x} \le 0$, otherwise nothing updates from $\boldsymbol{\theta} = \mathbf{0}$ (Ch. 24).
6. **Regularized LR update.** The λ term must shrink θ_j, giving $(1 - \alpha\lambda)\theta_j$ (Ch. 25).

7. **Momentum with $\beta \approx 1$.** Long oscillation, not literal explosion; instability starts at $\beta \geq 1$ or too large η (Ch. 25).

26.3 The one-line drill

Q. Bias–variance decomposition?
A. $\mathbb{E}(y-h)^2 = \sigma^2 + (f-\bar{h})^2 + \mathbb{E}(h-\bar{h})^2$; cross terms vanish because the noise is independent with mean 0 and $\mathbb{E}[h-\bar{h}] = 0$.

Q. Effective parameters of ridge?
A. $\sum_j d_j^2/(d_j^2+\lambda)$, from $\operatorname{tr}\mathbf{X}(\mathbf{X}^\top\mathbf{X}+\lambda\mathbf{I})^{-1}\mathbf{X}^\top$.

Q. LASSO, orthonormal X?
A. $\hat{\theta}_j = \operatorname{sign}(\theta_j^0)\max(|\theta_j^0| - \lambda/2, 0)$, $\theta_j^0 = \mathbf{y}^\top\mathbf{x}_j$.

Q. LOOCV error variance on the DC level?
A. $\frac{n}{n-1}\sigma^2$.

Q. Bootstrap variance estimate?
A. $\frac{1}{B-1}\sum_b (S(\mathbf{Z}^{*b}) - \overline{S^*})^2$; each resample omits $\approx 36.8\%$ of the points.

Q. Distance-from-means rule?
A. $\operatorname{sign}(\mathbf{w}^\top\mathbf{x}+b)$, $\mathbf{w} = 2(\boldsymbol{\mu}_+ - \boldsymbol{\mu}_-)$, $b = \|\boldsymbol{\mu}_-\|^2 - \|\boldsymbol{\mu}_+\|^2$.

Q. K-NN posterior?
A. K_k/K, from $p(\mathbf{x}) \approx K/(NV)$ and Bayes' rule.

Q. 1-NN asymptotic error?
A. At most twice the Bayes error (Cover and Hart, 1967).

Q. Naive Bayes prediction?
A. $\arg\max_k \log P(y_k) + \sum_j \log P(x_j \mid y_k)$, Laplace: $(c_v+1)/(\sum c + |\text{values}|)$.

Q. Geometric margin?
A. $y(\boldsymbol{\theta}^\top\mathbf{x}+b)/\|\boldsymbol{\theta}\|$, scale-free; the functional margin $y(\boldsymbol{\theta}^\top\mathbf{x}+b)$ is not.

Q. Perceptron mistake bound?
A. R^2/γ^2, independent of n and d.

Q. Logistic gradient and Hessian?
A. $\mathbf{X}^\top(\mathbf{h}-\mathbf{y})$ and $\mathbf{X}^\top\mathbf{S}\mathbf{X}$, $S_{ii} = h_i(1-h_i)$.

Q. Momentum unrolled?
A. $\mathbf{v}_t = -\eta\sum_k \beta^k \nabla J(\boldsymbol{\theta}_{t-1-k})$, an exponentially weighted moving average.

26.4 Python you should be able to write cold

In code

```python
import numpy as np
soft = lambda t, lam: np.sign(t) * np.maximum(np.abs(t) - lam/2, 0)
          # LASSO, orthonormal X

def knn_predict(Xtr, ytr, x, k):
                                                  # K-NN classification
    d = np.sqrt(((Xtr - x) ** 2).sum(1))
    nn = np.argsort(d)[:k]
    return np.bincount(ytr[nn]).argmax()

def perceptron(X, y, epochs=100):
                                                 # y in {-1,+1}, X has a
    1-column
    th = np.zeros(X.shape[1])
    for _ in range(epochs):
        mistakes = 0
        for xi, yi in zip(X, y):
            if yi * (th @ xi) <= 0:
                th += yi * xi; mistakes += 1
        if mistakes == 0: break
    return th

sigmoid = lambda z: 1 / (1 + np.exp(-z))
def logistic_newton(X, y, iters=10):
                                              # y in {0,1}
    th = np.zeros(X.shape[1])
    for _ in range(iters):
        h = sigmoid(X @ th)
        H = X.T @ ((h * (1 - h))[:, None] * X)
        th -= np.linalg.solve(H, X.T @ (h - y))
    return th

def gd_momentum(grad, th, eta=0.01, beta=0.9, steps=500):
    v = np.zeros_like(th)
    for _ in range(steps):
        v = beta * v - eta * grad(th); th = th + v
    return th
```

Every function here was run on the worked examples in this part; the full script with assertions is `notebooks/theory/insem2_numbers.py`.

Go deeper

Watch, in order. Kilian Weinberger's Cornell CS4780 lectures map almost one-to-one onto this part: L3 k-NN, L5 Perceptron, L6 convergence proof, L8 and L9 Naive Bayes, L11 logistic regression, L12 GD and Newton, L13 ridge, L19 bias–variance, L20 model selection.

Part VII

Lab kits — the pocket cards

27

Feature Scaling, Conditioning, and Ridge — Lab 3 Kit

Lab 3 of the course is this chapter in assignment form. We keep the same promise as the rest of the book: plain words, then algebra, then a picture.

27.1 Scaling is a change of units

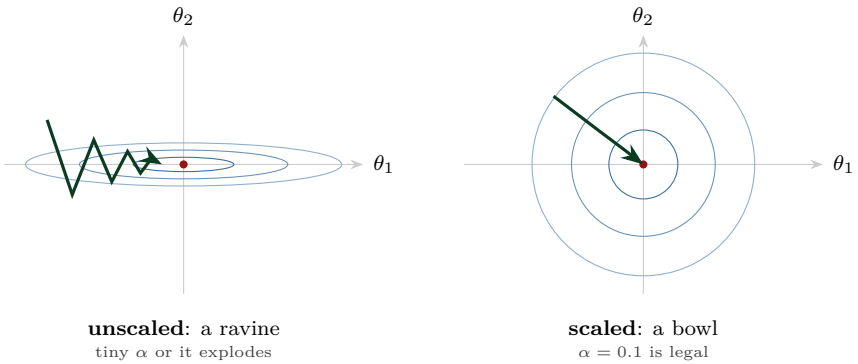
A column of Population (thousands of people) and a column of AveBedrms (a number near 1) cannot share one learning rate. Standardise with the *training* mean and std,

$$x_j \leftarrow \frac{x_j - \mu_j}{\sigma_j},$$

then apply those frozen μ_j, σ_j to val/test. Do not scale the intercept column. After this, $X^\top X$ is better conditioned: the loss bowl is round, and $\alpha = 0.1$ is a legal step. Unscaled, α may have to be 10^{-7} or the iterate explodes.

Scaling does not add information. It stops the loudest units from bullying the quiet ones during gradient descent, and it makes a ridge penalty $\lambda\|\theta\|^2$ mean the same thing for every feature.

The reason is geometric, and one picture settles it. Before scaling, the cost contours are a long thin ravine, because moving $\theta_{\text{population}}$ by 0.001 changes predictions more than moving θ_{bedrooms} by 1. Gradient descent bounces across the narrow direction and inches along the long one. After scaling, the same cost is a near-circular bowl and a sane α works in every direction.



The same data, the same J, two choices of units. On the left the gradient mostly points across the

ravine instead of along it, so the path zig-zags and every step must be tiny. On the right the gradient points at the answer. Standardising is not cosmetic — it changes the eigenvalues of $X^\top X$, which is the condition number of Chapter 11.

27.2 Ridge: a tax on slopes, not on the intercept

$$J(\theta) = \frac{1}{2n}\Big(\|X\theta - y\|_2^2 + \lambda \sum_{j=1}^{d} \theta_j^2\Big).$$

θ_0 is omitted from the tax — it is the prediction at mean features, not a slope. The gradient is $\nabla J = \frac{1}{n}X^\top(X\theta - y) + \frac{\lambda}{n}R\theta$ with $R = \mathrm{diag}(0, 1, \ldots, 1)$. Setting it to zero gives the regularized normal equation $\theta = (X^\top X + \lambda R)^{-1} X^\top y$. Gradient descent walks to the same point; the solve jumps there.

Derivation

From $J = \frac{1}{2n}(X\theta - y)^\top(X\theta - y) + \frac{\lambda}{2n}\theta^\top R\theta$,

$$\nabla J = \frac{1}{n}X^\top(X\theta - y) + \frac{\lambda}{n}R\theta = 0 \;\Rightarrow\; (X^\top X + \lambda R)\theta = X^\top y.$$

Ridge is the uncle at a farsan counter who will not let one huge past order dominate the recipe. Every fancy coefficient is nudged toward zero unless the data really insists. The fact that people eat something (the intercept) is never nudged.

In code

```
R = np.diag(np.r_[0.0, np.ones(X.shape[1]-1)])
theta = np.linalg.solve(X.T @ X + lam * R, X.T @ y)
```

Same J as batch GD with the R-masked extra term. $\lambda = 0$ *is* least squares.

Go deeper

`notebooks/theory/ch_scaling.py` and `notebooks/theory/ch_ridge_path.py` reproduce every number in this lab from scratch in about forty lines of NumPy each. Run them, then change one thing — the outlier, the λ grid, the split — and see which conclusions survive.

Read. Hastie, Tibshirani & Friedman, *Elements of Statistical Learning*, §3.4.1. Ridge with the standardisation convention spelled out, including why the intercept is left alone. `https://hastie.su.domains/ElemStatLearn/`

Exercises

Pencil first, then click the ⇝ to jump to the worked answer in Appendix D. The answer links back to the question.

27.1 ⇝ Why must you scale before you apply ridge? Answer in terms of what λ is charging for. Then write the regularized normal equation with $R_{00} = 0$.

27.2 ⇝ You standardise using μ, σ computed on the training rows only. Write the two lines of code that transform validation and test. What is the name of the bug you avoided?

27.3 ⇝ This page is a pocket card. Recite it from memory, then return to Chapters 8 and 9 and locate the matching contour picture and the shrinkage factor $1 - \alpha\lambda/n$.

The short version above is the Lab 3 kit. Lectures 10 and 11 of the course write the same story with every contour, every λ convention, and the shrinkage factor $1 - \alpha\lambda/n$ — that full treatment is Chapters 8 and 9, which you just read. This page is the pocket card you take to the lab.

28

Basis Functions — Lab 4 Kit

Lab 4 of the course is this chapter in assignment form. Chapter 17 built the theory; this page is the pocket card you take to the lab. The whole lab is one idea: keep the linear-regression fit from Labs 1–3 exactly as it is, and change only the *features* — replace the raw x with a bank of fixed nonlinear functions of it.

28.1 One fit, three alphabets

A basis-function model is

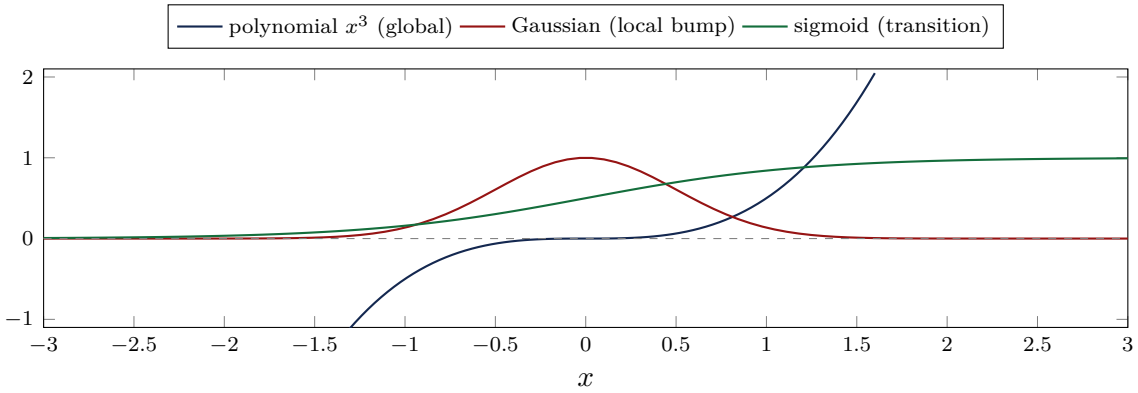
$$h_\theta(x) = \sum_{j=0}^{M} \theta_j \, \varphi_j(x), \qquad \varphi_0(x) = 1.$$

It is nonlinear in x but **linear in** θ, so if we stack the basis evaluations into a design matrix Φ with $\Phi_{ij} = \varphi_j(x^{(i)})$, every formula from the earlier labs holds with X replaced by Φ:

$$\hat{\theta} = (\Phi^\top \Phi)^{-1} \Phi^\top y \quad \text{(OLS)}, \qquad \hat{\theta} = (\Phi^\top \Phi + \lambda R)^{-1} \Phi^\top y \quad \text{(ridge)}.$$

That is the entire mechanical content of the lab. There is no new optimiser and no new loss — only a new choice of columns. The skill the lab is testing is *which shapes to put in the bank*, and that is a judgement about the data, not the algebra.

The three shapes on offer differ in one property: the *reach* of a single coefficient.



A polynomial term never settles — move its coefficient and the whole curve moves (global). A Gaussian is a hill that is essentially zero more than a few s from its centre, so its coefficient edits the curve only locally. A sigmoid climbs once and then saturates — a one-way transition, not a bump.

28.2 Choosing the alphabet

- **Polynomial** $\varphi_j(x) = x^j$: one smooth *global* trend. Cheap, but a few extra degrees start fitting noise and oscillate near the edges (Runge).
- **Gaussian** $\varphi_j(x) = e^{-(x-\mu_j)^2/2s^2}$: *localized* features — an isolated bump lives where its centre is. Width s is the reach: too small leaves gaps and chases noise, too large blurs neighbouring bumps into one.
- **Sigmoid** $\varphi_j(x) = \sigma\big((x-\mu_j)/s\big)$: a bounded *transition / saturation*. Slope s sets how sharp the climb is; a polynomial cannot saturate (it must run to $\pm\infty$).

Derivation

Fitting is unchanged. With Φ the design matrix and $R = \mathrm{diag}(0, 1, \ldots, 1)$, minimising $J = \frac{1}{2n}\big(\|\Phi\theta - y\|^2 + \lambda\theta^\top R\theta\big)$ gives $\nabla J = \frac{1}{n}\Phi^\top(\Phi\theta - y) + \frac{\lambda}{n}R\theta = 0$, i.e. $(\Phi^\top\Phi + \lambda R)\theta = \Phi^\top y$ — the same normal equation as Chapter 27, now on transformed columns.

In code

```
# build the columns by hand -- no PolynomialFeatures / kernel utilities
def poly(x, deg):  return np.column_stack([x**k for k in range(1, deg+1)])
def gauss(x, c, s):return np.column_stack([np.exp(-(x-ci)**2/(2*s**2)) for ci in c])
def sig(x, c, s):  return np.column_stack([1/(1+np.exp(-(x-ci)/s)) for ci in c])

Phi = np.column_stack([np.ones(len(x)), gauss(x, centers, s)])    # ones = intercept
theta = np.linalg.solve(Phi.T@Phi + lam*R, Phi.T@y)               # same fit as Lab 3
```

Standardise the columns (or keep degrees modest): raw powers of x blow up and make $\Phi^\top\Phi$ ill-conditioned — the Chapter 27 conditioning lesson does not disappear just because the features are now x^j.

The four families are four alphabets for writing curves. A straight line has two letters and can only spell trends; Gaussians spell isolated words (bumps); sigmoids spell "and then it levelled off". You do not pick the fanciest alphabet — you pick the one whose letters are the shapes already in the data.

Watch out

Choose the degree or width s on the *validation* set and report on *test*. Selecting a hyperparameter by test MSE leaks the test set and reports an optimistic error — the same discipline as choosing λ in Chapter 27.

Go deeper

Read. Bishop, *Pattern Recognition and Machine Learning*, §3.1 (Linear Basis Function Models). Polynomial, Gaussian and sigmoidal bases, and why the model stays linear in the parameters. https://www.microsoft.com/en-us/research/publication/pattern-recognition-machine-learning/

Read. Deisenroth, Faisal & Ong, *Mathematics for Machine Learning*, §9. Feature maps and the

> design matrix. https://mml-book.github.io/book/mml-book.pdf

Exercises

Pencil first, then click the ⇝ to jump to the worked answer in Appendix D. The answer links back to the question.

28.1 ⇝ A degree-5 polynomial fit "bends" but you still call it *linear* regression. Explain in one sentence why, and write the design matrix Φ for the single-feature degree-3 case.

28.2 ⇝ At $x_0 = 0.3$ a small step $\delta = 0.1$ barely changes x^5 but strongly changes the Gaussian centred at $\mu = 0$. What property of each basis does this demonstrate, and which of the two would you add to model an isolated spike in the data?

28.3 ⇝ The target is flat, then climbs once to a higher flat level, and stays there. Which basis family matches this most naturally, and why is a high-degree polynomial a poor choice here?

The short version above is the Lab 4 kit. Chapter 17 writes the same story with the full generalisation and capacity discussion; this page is what you carry into the lab.

Part VIII

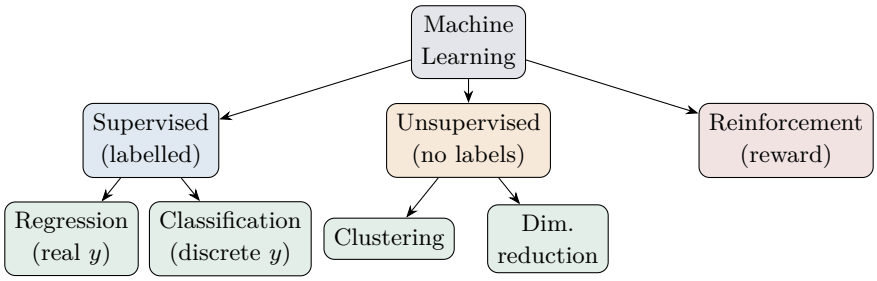
The rest of the map — uncut

29

Learning Paradigms

There is not one kind of machine learning — there are a few families, sorted by *what kind of data you have* and *what you want out*. This chapter is the course's full map, **uncut**: unsupervised learning, clustering, dimensionality reduction, topic modelling, reinforcement learning, and the other flavours. It sits *after* the linear learner so those words have a concrete example to hang on. Nothing from the class survey is omitted.

29.1 The map of machine learning



The three main families. Supervised learning splits by output type (real → regression, discrete → classification). Online, transfer and adversarial learning are extra flavours discussed at the end.

*Think of **three ways a child learns**. Supervised: a parent points and says "that's a dog, that's a cat" (labelled examples). Unsupervised: the child sorts a toy box into piles by themselves, with no one naming the piles (structure without labels). Reinforcement: the child touches a hot cup, gets "ouch" (a reward signal), and learns by trial and error.*

29.2 The notation we will use

The course fixes this notation — worth memorising, because every later chapter uses it.

Symbol	Meaning
a, b, c	a scalar (a single number)
$\mathbf{x}, \mathbf{y}$	a vector (a list of numbers)
$\mathbf{A}, \mathbf{B}$	a matrix (a grid of numbers)
X, Y	a random variable
$a \in \mathcal{A}$	"a is a member of set $\mathcal{A}$"
$\lvert\mathcal{A}\rvert$	cardinality — how many items are in $\mathcal{A}$
$\lVert\mathbf{v}\rVert$	the norm (length) of vector $\mathbf{v}$
$\mathbf{u} \cdot \mathbf{v}$ or $\langle \mathbf{u}, \mathbf{v} \rangle$	dot product
$\mathbb{R}$, $\mathbb{R}^n$	the real numbers; n-dimensional real space
$f : \mathbb{R}^n \to \mathbb{R}$	a function taking an n-vector to one number
$x^{(i)}, y^{(i)}$	the i-th training input / output

The little superscript (i) always means "example number i", *not* a power. So $x^{(3)}$ is the 3rd data point, while x^3 is x cubed.

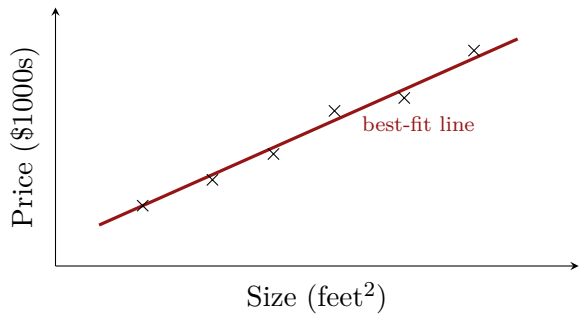
29.3 Supervised learning

Supervised learning is learning *with an answer key.* You are given pairs of (input, correct output), and you learn a function that reproduces the outputs — so that when a *new* input arrives, you can predict its output.

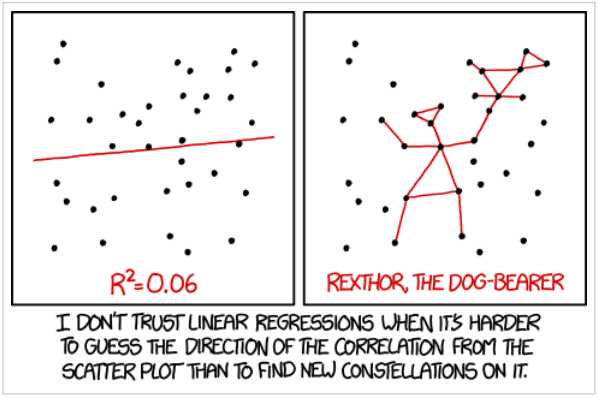
> Given labelled data $\{(x^{(1)}, y^{(1)}), \ldots, (x^{(n)}, y^{(n)})\}$, learn a mapping $h : x \mapsto y$ so that the prediction $\hat{y} = h_\theta(x)$ is close to the true y. The output y can be either **real-valued** (a number) or **discrete** (a label) — and that single choice splits supervised learning into two:

29.3.1 Regression — when the output is a number

If the answer is a *quantity* — a price, a temperature, a house value — the task is **regression**. Regression = fitting a line (or a smooth curve) through the data so you can read off predictions.



The course's housing example: each × is a house (size, price). Regression fits a line so you can predict the price of a new house from its size.

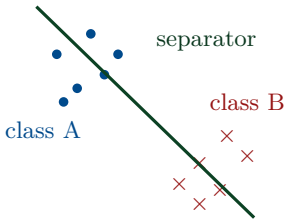


xkcd #1725 "Linear Regression" by Randall Munroe — https://xkcd.com/1725/ (CC BY-NC 2.5).

The joke: eyeball a cloud of points and draw a confident line. Regression is the honest, mathematical version of exactly that — and Chapter 2 shows precisely how the "best" line is chosen.

29.3.2 Classification — when the output is a label

If the answer is a *category* — spam / not-spam, digit 0–9, tumour benign / malignant — the task is **classification**. Instead of a line to *sit on*, we look for a line (or curve) that *separates* the categories.



Classification finds a boundary that separates the classes (benign vs malignant, cat vs dog, digit 3 vs 8). The boundary can be a straight line or a curve.

> **Regression vs classification in one line:** regression outputs a *continuous number* and fits a curve; classification outputs a *discrete label* and fits a separator. Same supervised setup, different kind of answer.

In code

A linear classifier is just "compute a score, then check its sign". Points on one side of the boundary get label 1, the other side 0:

```
import numpy as np
def classify(X, w, b):
    score = X @ w + b              # a number for each point
    return np.where(score >= 0, 1, 0)      # sign -> class label

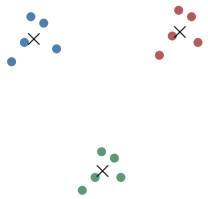
X = np.array([[1., 3], [2, 2.5], [3, 1], [3.5, 0.5]])
print(classify(X, w=np.array([1., -1]), b=-0.5))    # [0 0 1 1]
```

The line $w \cdot x + b = 0$ is the separator; `np.where` turns the score's sign into a 0/1 label. (Learning w, b is what logistic regression / SVMs do later in the course.)

29.4 Unsupervised learning

Unsupervised learning has *no answer key* — only inputs $\{x^{(1)}, \ldots, x^{(n)}\}$, no y's. The goal is to discover *hidden structure*: which points group together, or what few directions explain most of the data.

29.4.1 Clustering — finding natural groups



three clusters, × = centre of each

Clustering (e.g. K-Means) sorts unlabelled points into groups by closeness — no one told it the groups; it found them. Used to cluster images or documents.

In code

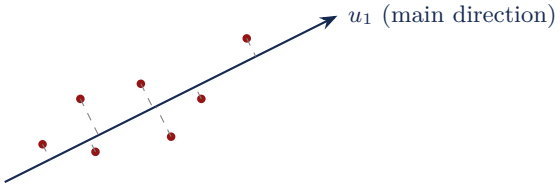
The heart of K-Means is "assign each point to its nearest centre". One assignment step:

```
import numpy as np
pts     = np.array([[0., 0], [0.2, 0.1], [5, 5], [5.1, 4.9]])
centres = np.array([[0., 0], [5., 5]])
# distance from every point to every centre, then pick the closest
dist    = np.linalg.norm(pts[:, None, :] - centres[None, :, :], axis
    =2)
labels = dist.argmin(axis=1)
print(labels)           # [0 0 1 1]  -> first two join centre 0, last
     two centre 1
```

Full K-Means just repeats: assign points to nearest centre, then move each centre to the average of its points, until nothing changes.

29.4.2 Dimensionality reduction — squashing without losing much

Sometimes data has many columns but really only varies along a few directions. Dimensionality reduction (like PCA) finds those directions and *projects* the data down — fewer numbers, almost the same information.



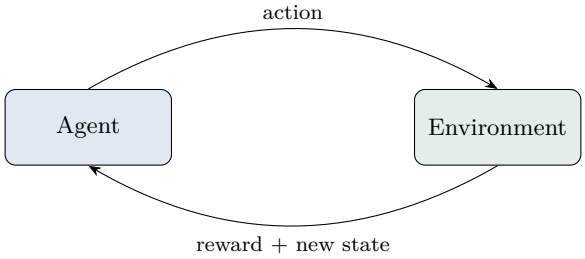
Every point is dropped (dashed) onto the main direction u_1. Keeping just its position along u_1 replaces two numbers with one, losing little.

29.4.3 A peek at topic modelling

> Given many documents with no labels, *topic modelling* (LDA) discovers themes on its own. In the course example, three sentences about matches, elections and teams get sorted into two topics — "Sports" {game, players, score, match} and "Politics" {election, results, policy} — and each document becomes a *mixture*, e.g. 80% Sports / 20% Politics. Two ideas: each document is a mixture of topics, and each topic is a probability distribution over words.

29.5 Reinforcement learning

Reinforcement learning has no fixed dataset at all. An **agent** acts in an **environment**, receives a **reward** (good or bad), and learns by *trial and error* which actions earn the most reward over time.



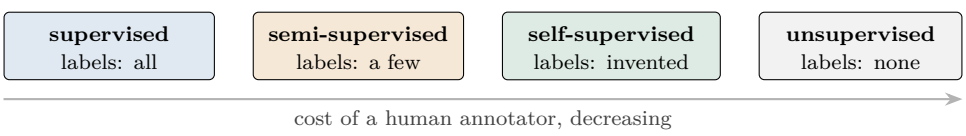
The reinforcement loop: the agent takes an action, the environment returns a reward and a new state, and the agent adjusts to earn more reward. Like a baby learning to walk: try, wobble, reward, improve.

29.6 Semi-supervised and self-supervised learning

The three-way split above is clean, and reality is not. It rests on an assumption that labels either exist or do not, when in practice you almost always have a little of one and an ocean of the other: labelling is expensive because a human has to do it, while raw data accumulates for free.

Semi-supervised learning uses a small labelled set together with a much larger unlabelled one. The unlabelled data cannot tell you what the answer is, but it can tell you where the data *lives* — which regions of input space are dense, where the natural gaps are — and a decision boundary that cuts through a dense region is probably wrong even if the handful of labels you have does not say so.

Self-supervised learning is the sharper trick: manufacture labels out of the structure of the data itself, so that an unlabelled problem becomes a supervised one with no human involved. Hide a word in a sentence and predict it from its neighbours. Cut a patch out of an image and predict what was there. Take an audio clip and predict the next second. Each of these is an ordinary supervised problem with an ordinary loss, and the labels cost nothing because they were already in the data — you simply deleted them on purpose.



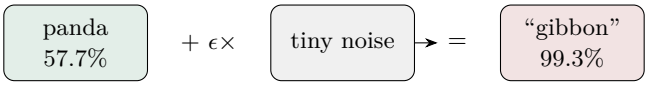
The labelled/unlabelled boundary is a spectrum, not a wall. Self-supervision sits oddly on it: mechanically it is supervised learning, with a genuine y and a genuine loss, but no person ever wrote a label down.

Where this shows up

This is not a footnote — it is why the last decade happened. Every large language model is trained self-supervised on the "predict the next token" task, which is a supervised problem whose labels are just the text shifted by one position. The entire internet becomes a labelled dataset at zero annotation cost. The supervised fine-tuning that follows, with real human labels, is a rounding error in comparison. When people say modern models are trained on "unlabelled" data, this is what they mean, and it is worth being precise: the data is unlabelled, the *task* is supervised.

29.7 Other flavours (briefly)

- **Online learning** — data arrives one example (or a small batch) at a time, and the model updates on the fly instead of from a fixed dataset.
- **Transfer learning** — knowledge learned on one task boosts a related task (knowing how to ride a bicycle helps you learn a motorbike).
- **Adversarial ML** — a tiny, cleverly-chosen change to the input can fool a model. The famous course example: adding an imperceptible noise pattern to a *panda* photo makes a classifier call it a *gibbon* with 99% confidence.



Adversarial example: a panda plus a noise pattern too faint for us to see flips the model's answer to "gibbon". A reminder that models can be brittle.

Go deeper

Watch. Cornell CS4780 (Kilian Weinberger), *the full lecture series*
Lecture 1 sets up the supervised/unsupervised distinction with more care than most courses bother with, including what the i.i.d. assumption is actually buying you.

Read. Deisenroth, Faisal & Ong, *Mathematics for Machine Learning*, Ch. 8. "When models meet data" — the taxonomy above, stated precisely enough to argue with. `https://mml-book.github.io/book/mml-book.pdf`

Exercises

Pencil first, then click the ⇝ to jump to the worked answer in Appendix D. The answer links back to the question.

29.1 ⇝ Supervised versus unsupervised, stated in terms of whether y is present in the training data. Give one example of each from the gallery above.

29.2 ⇝ Regression versus classification. Which one was the line in Chapter 2? Now invent a classification problem on the same houses, and say what h_θ would have to output instead of a price.

29.3 ⇝ Clustering: what is K in K-means, and what goes wrong if you choose K by looking at test labels?

29.4 ⇝ Reinforcement learning has no fixed labelled set. What plays the role of P in Mitchell's test from Chapter 1?

29.5 ⇝ What does the superscript in $x^{(i)}$ mean, and what does it emphatically not mean? Write $x_j^{(i)}$ in words.

30

One Story, Told Three Times

Here is something worth noticing before you close the book. Across thirteen lectures you solved exactly one problem — fit a linear model to data — and you solved it three separate times, from three starting points that have nothing obvious to do with each other. Each time you got the same answer.

That is not a coincidence, and it is not a curriculum design choice. It is the actual content of the course.

30.1 The three routes

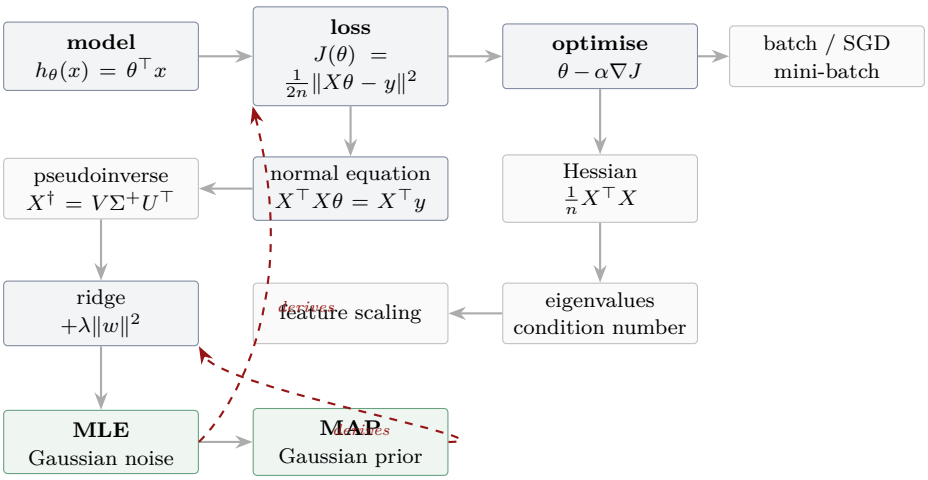
Route	What you assume	Where you land
Iterative	Squared error is the thing to minimise, and you can only feel the slope under your feet	$\theta \leftarrow \theta - \alpha \nabla J(\theta)$, which converges to the minimiser because the bowl is convex
Algebraic	Squared error is the thing to minimise, and you are allowed to use calculus on the whole surface at once	$X^\top X\theta = X^\top y$, so $\hat{\theta} = (X^\top X)^{-1} X^\top y$; and when that inverse does not exist, $\hat{\theta} = X^\dagger y$
Probabilistic	Nothing about squared error. Only: the noise is Gaussian, and optionally the weights are drawn from a Gaussian centred at zero	Maximum likelihood *produces* squared error; adding the prior *produces* ridge, with $\lambda = \sigma^2/\tau^2$

The third row is the one that should bother you in a productive way. The first two rows both start by assuming the loss. The third does not: it assumes something about the world (measurement noise is Gaussian) and the loss falls out as a consequence. That is the difference between a convention and a derivation, and it is why Chapter 14 is the last chapter rather than an aside.

> **Why anyone outside a classroom cares**
>
> The equivalence is load-bearing in practice. If your residuals are clearly not Gaussian — say they have heavy tails because a sensor occasionally reports garbage — then the third route tells you that squared error is now the *wrong* loss, and tells you what to use instead. Laplace noise gives absolute error. Student-t noise gives one of the robust losses. You are not guessing; you are reading a consequence off an assumption you can check with a histogram of the residuals.

30.2 The dependency map

Chapters do not sit in a list. They sit in a graph, and the graph has one trunk.



Solid grey arrows are "needs". The two dashed red arrows, which run backwards, are the payoff: the probabilistic chapters do not add a new technique — they re-derive two things you already had, from assumptions that never mention squared error. Every arrow in this diagram is a derivation you were asked to reproduce.

30.3 The checklist

These six are the exam, whatever the exam actually asks. Say each one out loud, with a pen, at a blank page. If you stall, the chapter reference tells you where to go; the pointer after it tells you who else explains it.

1. Derive $\nabla_\theta J(\theta)$ for plain least squares **twice**: once coordinate-by-coordinate with the chain rule, once in matrix form using $\nabla_\theta(a^\top\theta) = a$ and $\nabla_\theta(\theta^\top A\theta) = 2A\theta$. Check the two agree. Ch. 4, Ch. 6

2. Explain, with the geometric series $e^{(t)} = (1 - 2\alpha)^t e^{(0)}$, exactly why a large α diverges — and state the threshold in terms of the curvature a rather than quoting a number. Ch. 5

3. State the difference between batch, mini-batch and stochastic gradient descent along both axes at once: cost per step, and noise in the direction. Then say what SGD does near the optimum that batch does not, and why. Ch. 7

4. Connect the eigenvalues of $X^\top X$ to feature scaling. The chain has four links: units → eigenvalue spread → condition number → zig-zag. Reproduce all four. Ch. 11, Ch. 8

5. Derive the ridge gradient and the regularised normal equation, then explain why $\lambda > 0$ makes $X^\top X + \lambda I$ invertible even when $X^\top X$ is not. The eigenvalue argument is one line; give it. Ch. 9, Ch. 10

6. Re-derive least squares as MLE under Gaussian noise, and ridge as MAP under a Gaussian prior, ending at $\lambda = \sigma^2/\tau^2$. Then say what $\tau^2 \to \infty$ means in words. Ch. 3, Ch. 14

If you can walk through all six without notes, you have the course — not the formulas, the course. The formulas are the residue.

Part IX

Toolkit

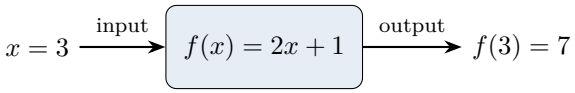
A

The Math You Need, From Zero

Not a course. A drawer. Open the section you need when a symbol in the main text is opaque; close it again. If you already know slopes and $\sum$, skip the drawer entirely.

A.1 Numbers, variables and functions

A **variable** is just a name for a number we do not want to fix yet — like a labelled empty box. If I say "let x be your age", then x stands for whatever number that is. A **function** is a machine: put a number in, get a number out, by a fixed rule.

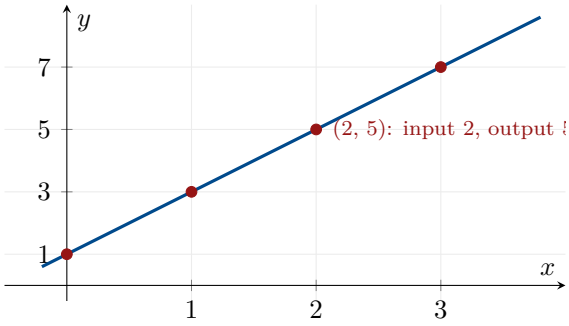


A function is a machine: it takes an input number, applies a rule, and returns an output number.

> We write a function as $f(x) =$ (some rule using x). The symbol $f(3)$ means "run the machine on input 3". For $f(x) = 2x + 1$: $f(3) = 2 \cdot 3 + 1 = 7$. The letter f is just a name; we could call it g, h, or — in machine learning — the *hypothesis* h.

A.2 Drawing a function: the graph

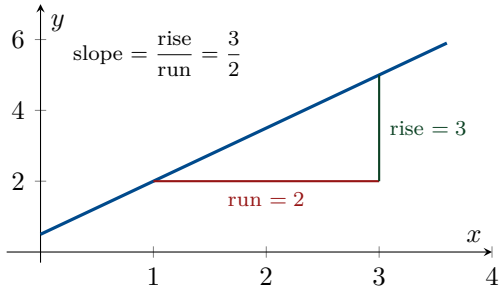
To *see* a function, we draw it. We make a flat sheet with a horizontal line (the x-axis, inputs) and a vertical line (the y-axis, outputs). For each input x we go right by x and up by $f(x)$, and mark a dot. Join the dots — that curve *is* the function.



The graph of $f(x) = 2x + 1$. Each red dot is one (input, output) pair; the line is the whole function at a glance.

A.3 Slope: how steep is a line?

The **slope** of a straight line is how much it rises when you walk one step to the right. Big slope = steep climb; zero slope = flat; negative slope = going downhill. It is "rise over run": how far up divided by how far across.



Slope = rise ÷ run. Walk 2 across, climb 3 up ⇒ slope = 1.5. This is the single most important idea in this book.

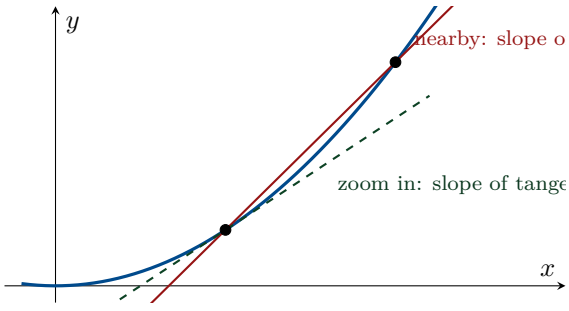
For a line through two points (x_1, y_1) and (x_2, y_2),

$$\text{slope} = \frac{y_2 - y_1}{x_2 - x_1} = \frac{\text{change in } y}{\text{change in } x}.$$

A line's equation is $y = mx + c$, where m is the slope and c is where it crosses the y-axis (the "intercept"). In machine learning this exact line becomes our first model.

A.4 The derivative: slope of a curve

A curve is not equally steep everywhere. The **derivative** is the slope *at a single point* — the steepness you would feel if you stood right there. We find it by zooming in: pick the point, pick a second point very close by, take rise-over-run, then slide the second point in until they almost touch.



Slide the far point toward the near one: the red "secant" rotates into the green "tangent". That tangent's slope is the derivative at the point.

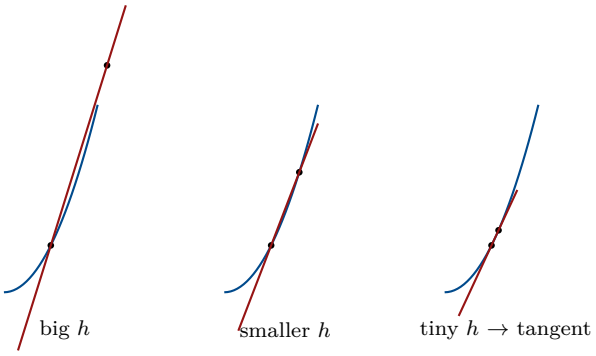
Derivation

Derivative of $f(x) = x^2$, from scratch. Take a point x and a tiny step h:

$$\begin{aligned}\text{slope near } x &= \frac{f(x+h) - f(x)}{h} = \frac{(x+h)^2 - x^2}{h} \\ &= \frac{x^2 + 2xh + h^2 - x^2}{h} && \text{(expand the square)} \\ &= \frac{2xh + h^2}{h} = 2x + h. && \text{(cancel } x^2\text{, divide by } h\text{)}\end{aligned}$$

Now let the step h shrink to 0: the leftover h vanishes, leaving the slope $2x$. So the derivative of x^2 is $\boxed{2x}$. (We write it $f'(x) = 2x$ or $\frac{df}{dx} = 2x$.)

The power rule generalises this: $\frac{d}{dx}x^n = n\,x^{n-1}$. A constant multiplier rides along ($\frac{d}{dx}(c\,x^n) = c\,n\,x^{n-1}$), you differentiate a sum term by term, and a lone constant has slope 0 (a flat line). These few rules cover everything in this course.

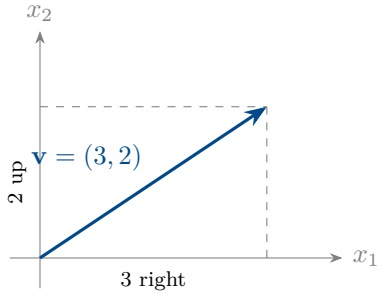


Zooming in: as the step h shrinks, the red secant line through the two dots rotates until it becomes the tangent — its slope is the derivative. This is what "$\lim_{h\to 0}$" means, drawn.

*The derivative is your car's **speedometer**. The road-position over time is the function; the speedometer shows the slope right now — how fast position is changing at this instant. Speeding up, slowing down, or cruising flat are positive, negative, and zero derivatives.*

A.5 Vectors: numbers with direction

A **vector** is just a list of numbers, e.g. $(3, 2)$. You can picture it as an *arrow*: start at the origin, go 3 right and 2 up. In machine learning, one data point (say a house's features) is a vector — a list of its measurements.



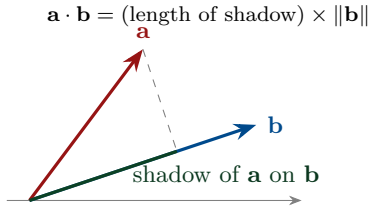
A vector as an arrow: $(3, 2)$ means "3 along, 2 up". A list of numbers and an arrow are the same thing.



addition: tip-to-tail scaling: same direction, twice as long

Add vectors by placing them tip-to-tail; scale a vector by stretching it (the direction stays, the length changes).

Dot product. For $\mathbf{a} = (a_1, a_2)$ and $\mathbf{b} = (b_1, b_2)$, the dot product is $\mathbf{a} \cdot \mathbf{b} = a_1 b_1 + a_2 b_2$ — multiply matching entries and add. It gives a single number that measures how much the two arrows point the same way. In ML, a prediction is often a dot product of a weight vector with a feature vector.



The dot product, geometrically: drop **a***'s shadow onto* **b***. Aligned arrows ⇒ big positive; perpendicular ⇒ 0; opposite ⇒ negative.*

Derivation

Worked numbers. For $\mathbf{a} = (1, 2)$, $\mathbf{b} = (3, 4)$: $\mathbf{a} \cdot \mathbf{b} = 1 \cdot 3 + 2 \cdot 4 = 11$. A **length** is the dot product of a vector with itself, square-rooted: for $x = (2, 3, 4)$,

$$\|x\| = \sqrt{x \cdot x} = \sqrt{2^2 + 3^2 + 4^2} = \sqrt{29} \approx 5.39.$$

(This exact $\sqrt{29}$ appears in the class notes — it measures "how far did θ move?" in the stopping rules of Chapter 4.)

In code

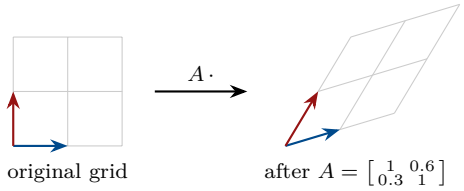
Every vector operation is one line of NumPy:

```
import numpy as np
a = np.array([1, 2]);  b = np.array([3, 4])
print(a + b)                # [4 6]    addition (tip-to-tail)
print(2 * a)                # [2 4]    scaling
print(a @ b)                # 11       dot product = 1*3 + 2*4
x = np.array([2, 3, 4])
print(np.linalg.norm(x))    # 5.385... = sqrt(29), the length of x
```

The `@` operator *is* the dot product; `np.linalg.norm` gives $\|x\|$.

A.6 Matrices: tables that transform vectors

A **matrix** is a grid of numbers (rows and columns). One use: it stores a whole dataset — one row per example, one column per feature. Another use: multiplying a matrix by a vector transforms that vector (rotates/stretches the arrow). For us, matrices let us write "do this to every data point at once" in one short symbol.



A matrix transforms space: it takes the square grid (and the basis arrows) and stretches/shears it into a new grid. Multiplying Ax moves the point x to its new spot.

$$\underbrace{\begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ 1 & x_3 \end{bmatrix}}_{X\ (3\times 2)} \underbrace{\begin{bmatrix} \theta_0 \\ \theta_1 \end{bmatrix}}_{\theta} = \begin{bmatrix} \theta_0 + \theta_1 x_1 \\ \theta_0 + \theta_1 x_2 \\ \theta_0 + \theta_1 x_3 \end{bmatrix}$$

Matrix × vector: each row of X dotted with θ gives one prediction. One symbol, all examples at once — this is exactly how we will predict house prices.

To multiply a matrix by a vector, take the **dot product of each row** with the vector. An $(m \times n)$ matrix times an $(n \times 1)$ vector gives an $(m \times 1)$ vector — the inner sizes (n) must match. The transpose $X^\top$ flips rows into columns; it appears whenever we "fold errors back onto parameters" (Chapter 6).

Derivation

Worked matrix–vector multiply. Take the design matrix and a parameter vector:

$$X = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix}, \quad \theta = \begin{bmatrix} 0.5 \\ 2 \end{bmatrix} \Rightarrow X\theta = \begin{bmatrix} 1(0.5) + 1(2) \\ 1(0.5) + 2(2) \\ 1(0.5) + 3(2) \end{bmatrix} = \begin{bmatrix} 2.5 \\ 4.5 \\ 6.5 \end{bmatrix}.$$

Each row of X is dotted with θ — so one matrix multiply produces *all three* predictions at once. That is the whole reason we use matrices.

In code

The same multiply, in NumPy:

```
import numpy as np
X = np.array([[1, 1],
              [1, 2],
              [1, 3]])          # rows = examples; first column of 1s = bias
theta = np.array([0.5, 2.0])    # intercept 0.5, slope 2.0
print(X @ theta)                # [2.5 4.5 6.5]  one prediction per row
```

`X @ theta` dots each row with `theta` — exactly $X\theta$.

A.7 Summation notation $\sum$

The big Greek Σ ("sigma") just means **add up a list**. It is shorthand so we do not write "+" a hundred times.

$$\sum_{i=1}^{4} a_i \xrightarrow{\text{unroll}} = a_1 + a_2 + a_3 + a_4$$

$\sum_{i=1}^{4} a_i$ reads "add a_i for i from 1 to 4". The bottom is where to start, the top where to stop.

The **mean** (average) of m numbers uses it: $\bar{a} = \frac{1}{m}\sum_{i=1}^{m} a_i$ — add them all, divide by how many. Our main error measure, the Mean Squared Error, is just the mean of squared mistakes: $\frac{1}{m}\sum_{i=1}^{m}(\text{mistake}_i)^2$.

Go deeper

Watch. 3Blue1Brown, *Essence of Linear Algebra (full series)*
Fifteen short videos. If you do nothing else on this list, do this one, and do it before Chapter 6. Matrices as transformations rather than grids of numbers is the single change of perspective that makes the rest of the book easy.

Watch. 3Blue1Brown, *Essence of Calculus*
The same treatment for derivatives. Chapters 2–4 are enough for our purposes.

Read. Boyd & Vandenberghe, *Introduction to Applied Linear Algebra* (VMLS), Ch. 1–5. Free, and unusually good at keeping the notation honest: everything is a vector of numbers you could actually type in. Chapter 12 does least squares from this foundation. `https://web.stanford.edu/~boyd/vmls/vmls.pdf`

Read. Deisenroth, Faisal & Ong, *Mathematics for Machine Learning*, Ch. 2–5. The reference to reach for when you want a statement proved rather than motivated. `https://mml-book.github.io/book/mml-book.pdf`

Exercises

Pencil first, then click the ⇝ to jump to the worked answer in Appendix D. The answer links back to the question.

A.1 ⇝ $f(x) = 3x - 1$. What is $f(2)$? What is the slope of the graph?

A.2 ⇝ Slope through $(0, 1)$ and $(2, 5)$. Derivative of x^3 by the power rule.

A.3 ⇝ Dot product $(1, 2) \cdot (3, 4)$. What is $\|(3, 4)\|$?

A.4 ⇝ Write all predictions of a linear model as one product $X\theta$. What is the shape of X if you have n houses and d features plus an intercept?

B

What Stanford, Berkeley, and Princeton Do Next

This book is FOML, not CS229. The linear learner is complete. The courses we studied after the class notes — Stanford CS229, Berkeley CS189, Princeton COS 324, UCLA CS260, Caltech LFD — spend the *rest* of their semester on the list below. One honest page beats twenty vague ones. Nothing here replaces a class derivation; it is a map of doors.

Logistic regression.
Same $X\theta$, new link $\sigma(z) = 1/(1+e^{-z})$, new loss $= -\sum \big[y \log h + (1-y)\log(1-h)\big]$. The gradient is $X^\top(h-y)$ — the linear-regression gradient with h no longer $\theta^\top x$. CS229 notes 1 §2; COS 324 Ch. 4. FOML stops before this chapter on purpose.

Locally weighted / feature maps.
CS229's last linear-regression twist: re-fit θ at each query x, weighting nearby rows. Polynomial features are the same idea with a fixed map $\phi(x)$.

GDA, naive Bayes, SVMs, kernels.
CS229 / CS189 classification toolkit. The geometry of a hyperplane is the Hessian chapter's $w^\top x + b$ with a different loss.

Bias–variance, double descent.
CS189 Regression II; Nakkiran et al. already cited in Chapter 14. The class under/overfit trio is the classical U; modern interpolating models can descend again.

Neural nets / backprop.
COS 324 Ch. 10–12; CS229 later notes. Every layer is $W^\top h$ plus a nonlinearity; the chain rule is Appendix A's derivative, stacked. GD does not change.

Unsupervised / RL.
Already in Chapter 29 at FOML depth. COS 324 spends weeks here; we do not fake those weeks.

If you can write GD, the normal equation, scaling, ridge, $X^\dagger$, Bernoulli MLE, and Beta–MAP from memory, you have this course. The Ivy notes are linked in the closing bibliography; cite them, do not paste them.

In code

Every formula in this book that can be checked by machine, is. Run the lot from the repo root:

```
python notebooks/theory/run_all.py
```

Script	What it refuses to let you get wrong
`ch_linreg_mle.py`	$\nabla \frac{1}{2}\|X\theta - y\|^2 = X^\top(X\theta - y)$; normal equation
`ch_gd_bowl.py`	the class bowl, and $(1-2\alpha)^k$ in closed form
`ch_scaling.py`	condition number before/after; frozen μ, σ
`ch_ridge_path.py`	$\lambda = 0$ is OLS; $\|w\|$ shrinks; R spares θ_0
`ch_svd_pinv.py`	$X = U\Sigma V^\top$; Penrose conditions; minimum norm
`ch_mle_map.py`	Bernoulli MLE and Beta–Bernoulli MAP

Each file asserts a class formula against a finite difference, a closed form, or a brute-force search. If an assert fails, the algebra in the matching chapter is the debug target — not Stack Overflow.

Go deeper

Appendix C is the long version of this chapter: every course, book and lecture series referenced anywhere in the text, sorted by what it is actually good at rather than by prestige. Start there rather than with a search engine.

Exercises

Pencil first, then click the ⇝ to jump to the worked answer in Appendix D. The answer links back to the question.

B.1 ⇝ Logistic regression uses the same $X\theta$ and a different h. Write h and the loss. (You do not have to derive the gradient — looking it up in CS229 notes 1 after you have tried is allowed.)

B.2 ⇝ Name one idea in this book that is still the whole of a later course (GD, SVD, λ). Then name one that this book only points at.

C

Where to Learn This From Someone Else

No single explanation lands for everyone. A derivation that reads as obvious on Tuesday can read as gibberish on Wednesday, and the fastest repair is usually not re-reading the same paragraph harder — it is hearing a different person set the same idea up differently.

This appendix is that second opinion, organised by topic rather than by author. Everything listed is free, everything was reachable in August 2026, and every video link points at a specific lecture rather than at a channel, so you are never left scrolling. Links are live in the PDF: click them.

The list is deliberately not the first page of search results. Andrew Ng and StatQuest are excellent and you will find them without help. What follows is the layer underneath: people who derive rather than summarise.

A note on the figures in this book

Every diagram in these chapters was drawn from scratch in Ti*k*Z. None of them are scans or screenshots of the course notes cited below. That is a deliberate constraint, and it costs something: the figure in CS229's notes for a particular idea may well be better than mine. So where a source has a picture worth seeing, the pointer below tells you which document and roughly where, and you should go look at it in the original. Reading a good figure at its source is not a detour; it is the whole point of a citation.

C.1 Readings, by what they are good at

Stanford CS229, main notes — ***the reference derivation of everything in Part I.***
Andrew Ng's notes open with linear regression, LMS, the normal equation, and then the probabilistic interpretation, in exactly the order this book uses. If you read one outside document, read the first twenty pages of this one. https://cs229.stanford.edu/main_notes.pdf

Caltech, *Learning From Data* (Abu-Mostafa) — ***the best account of why regularisation is not a hack.***
Lecture 12 develops weight decay by starting from a hard constraint $w^\top w \le C$ and letting the Lagrange multiplier appear, which is the argument reconstructed in Chapter 10. Slides and recordings are on the course page. https://work.caltech.edu/telecourse.html

Boyd & Vandenberghe, *Introduction to Applied Linear Algebra* (VMLS) — *least squares without the hand-waving.*
Chapters 12–13 do least squares and its variants at exactly the level this course needs, with no measure theory and no apologies. Free PDF: https://web.stanford.edu/~boyd/vmls/vmls.pdf

Deisenroth, Faisal & Ong, *Mathematics for Machine Learning* — *the appendix you wish this book had.*
When a symbol in Appendix A is still opaque, this is where to go. Chapter 9 rebuilds linear regression from the linear algebra up, including the MLE and MAP treatment. https://mml-book.github.io/book/mml-book.pdf

Brunton & Kutz, *Data-Driven Science and Engineering* — *the SVD chapter.*
Chapter 1 is the clearest written treatment of the pseudoinverse as "invert what you can, ignore what you cannot" that exists at this level, and it matches Chapter 12 closely. http://databookuw.com/databook.pdf

Cornell CS4780 lecture notes (Weinberger) — *terse, and written by someone who is bored of hand-waving.*
Useful as a sanity check: if you can read his one-page version of a topic and agree with it, you understand the topic. https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/

Tübingen, *Mathematics for Machine Learning* (von Luxburg) — *the maths, done properly, on video.*
Course page with notes split into Linear Algebra, Calculus, Probability and Statistics tracks; the video list below matches it. https://www.tml.cs.uni-tuebingen.de/teaching/2020_maths_for_ml/

Petersen & Pedersen, *The Matrix Cookbook* — *look up, do not read.*
Section 2 has every gradient identity used in Chapters 6 and 11, including $\nabla_A \det A$ and $\nabla_A \log\det A$. Treat it as a table, not a textbook. https://www.math.uwaterloo.ca/~hwolkowi/matrixcookbook.pdf

van Wieringen, *Lecture notes on ridge regression* — *far more about ridge than a course needs, which is the point.*
The bias–variance decomposition of the ridge estimator is done explicitly, which is the calculation Chapter 9 states and does not prove. https://arxiv.org/abs/1509.09169

C.2 Video, by topic

Linear regression, and why "linear" is a bigger word than it looks

mathematicalmonk (Jeff Miller), *ML 9.1 — Linear regression: nonlinearity via basis functions.*
Fifteen minutes, no slides, just a pen. He starts with the point Section 2.4 makes — that the model is linear in θ and need not be linear in x — and shows what basis functions buy you. Watch

MIT 18.06 (Strang), *Lecture 16 — Projection matrices and least squares.*
The geometric reading of the normal equation: $X^\top(X\theta - y) = 0$ says the residual is orthogonal to the column space. Strang derives $X^\top X$ invertibility from independent columns on the board. Watch

Gradient descent

3Blue1Brown, *Gradient descent, how neural networks learn.*
The canonical animation of a ball on a surface, and — more useful — of the gradient as the vector that says how much each coordinate wants to change. Watch

Tübingen ML, *Mathematics for Machine Learning* (von Luxburg, 2020/21).
The calculus track (videos labelled C) covers multivariate derivatives, gradients and Hessians at the level Chapter 6 assumes. Playlist

Eigenvalues, and the shape of the bowl

3Blue1Brown, *Eigenvectors and eigenvalues* (Essence of Linear Algebra, ch. 14).
Watch this before Chapter 11. The single idea — an eigenvector is a direction the matrix does not rotate — is what makes the Rayleigh quotient argument feel like a description of a picture rather than a trick. Watch

3Blue1Brown, *A quick trick for computing eigenvalues* (ch. 15).
Mean and product: $\lambda = m \pm \sqrt{m^2 - p}$. Genuinely useful in an exam for 2×2 matrices. Watch

The pseudoinverse and the SVD

Steve Brunton, *Singular Value Decomposition: mathematical overview.*
Thirteen minutes for the rotate–stretch–rotate picture that Chapter 12 draws. Watch

Steve Brunton, the full SVD series.
Later videos in the playlist do least squares and the pseudoinverse directly, with both MATLAB and Python code on screen. Playlist

MLE, MAP, and the claim that ridge is a prior

This is the part of the course where a second voice helps most, because the argument is short but the notation is heavy.

mathematicalmonk, *ML 4.1 and 4.2 — Maximum likelihood estimation.*
Definition, then the honest list of what MLE is bad at. Fifteen minutes each. Part 1, Part 2

mathematicalmonk, *ML 6.1 — Maximum a posteriori (MAP) estimation.*
He states, in passing, exactly the equivalence Chapter 14 derives: MAP with a prior is regularisation, and the two words describe one operation. Watch

mathematicalmonk, *ML 6.2 — MAP for a univariate Gaussian mean*, and *ML 6.3 — MAP as a convex combination.*
6.3 is the one to watch twice. The MAP estimate turns out to be a weighted average of the

prior mean and the MLE, with the weights set by how much data you have — which is why $\lambda \to 0$ as n grows. 6.2, 6.3

Ali Ghodsi (Waterloo), *Deep Learning, Lec 2.1 — Regularization.*
About twenty minutes in he does the L_2-penalty-equals-Gaussian-prior argument on the board, then notes that an L_1 penalty is a Laplace prior. This is the derivation of Chapter 14 delivered by someone who clearly finds it delightful. Watch

Whole courses, if you want to keep going

mathematicalmonk, the Machine Learning playlist.
Around 160 short videos. Nothing is animated, nothing is produced, and it is still one of the best things on the internet for this material. Playlist

Cornell CS4780 (Kilian Weinberger).
Recorded lectures with a live audience, and a lecturer who argues with the material. Pairs with the written notes linked above. Playlist

Ali Ghodsi (Waterloo), the full index of courses.
Statistical Learning, Deep Learning and Data Visualization, with slides beside every video. His lectures assume you want the derivation, which is unusual and welcome. `https://uwaterloo.ca/data-analytics/videos-and-slides-courses`

C.3 How to actually use this

Do not watch a video before you have tried the derivation. The videos above are all good enough that watching one produces a strong feeling of understanding, and that feeling is not the same thing as being able to reproduce the argument on blank paper. The order that works:

1. Read the chapter and attempt the derivation with the book shut.
2. Get stuck. Note precisely which line you cannot produce.
3. Watch the relevant ten minutes above, looking for that line.
4. Shut everything and derive it again from the top.

Step 4 is the one people skip, and it is the only one that does anything.

D

Answers

Every exercise in the book is answered here. The bold number beginning each answer is a link back to the question it belongs to, so you can bounce between the two without hunting.

Two rules before you look. If your answer disagrees by algebra, trust the definition of J in Chapter 1 and recompute rather than reading on. If it disagrees by exactly a factor of two, you either dropped the $\frac{1}{2}$ or you kept it when the formula did not.

Answers are terse on purpose, and several of them refuse to be a single sentence because the question genuinely was not one. A worked solution you read is worth less than a wrong one you fixed.

Chapter 1 — What is Machine Learning?

1.1 Houses: T is predicting price from living area; P is test J (or mean absolute error); E is more labelled houses. Spam: T is labelling a message; P is error rate, or precision and recall if the classes are lopsided; E is messages a human already labelled. If you cannot name all three, you have a wish, not a learning problem.

1.2 Add *bigram* features: one knob per adjacent pair of words, so `not_good` is its own feature with its own θ, free to be strongly negative while θ_{good} stays positive. The cost is combinatorial. With $|V| \approx 10^5$ words there are $|V|^2 = 10^{10}$ possible pairs, so you cannot allocate a knob to each. In practice you keep only bigrams that actually occur some minimum number of times, which brings it back to perhaps 10^6 — still ten times the unigram model. This is the first appearance of a trade-off the whole book is about: more expressive $\mathcal{H}$, more parameters, more ways to overfit.

1.3 Residuals $1 - 0 = 1$, $2 - 2 = 0$, $3 - 5 = -2$. Squares $1, 0, 4$, summing to 5. With $n = 3$, $J = \frac{1}{2 \cdot 3} \cdot 5 = \frac{5}{6} \approx 0.833$.

1.4 Because $J = 0$ on the training set measures recall of things you were shown, not skill on things you were not. Story: a model with one parameter per training house stores each price in its own knob and returns it on an exact match, achieving $J = 0$; shown a house whose area differs by one square foot, it has nothing to return and predicts garbage.

1.5 They have measured how well the model reproduces the data it was fitted on — training error, which is a measure of memorisation capacity plus fit, not of generalisation. It is guaranteed to be optimistic, and the more flexible the model the more optimistic it is. They should have split the data first, fitted on the training portion only, frozen θ, and evaluated once on the untouched test portion.

1.6 $\theta^{(t+1)} = \theta^{(t)} - \alpha \nabla J(\theta^{(t)})$. Here θ is a vector (one entry per knob), ∇J is a vector of the same shape, and α is a positive scalar. ∇J points in the direction of steepest *increase*, which is why it is subtracted rather than added.

1.7 (a) The hypothesis class changed: cubics instead of lines. Loss and optimiser untouched. (b) The loss changed: squared error plus a penalty. Class and optimiser untouched — note the class is still all linear functions, since ridge does not forbid any θ, it merely charges for some. (c) The optimiser changed: one random example per step instead of the full average. Class and loss untouched. (d) The optimiser changed: a direct linear solve instead of an iterative walk. Class and loss untouched, and this is why it lands on the same θ.

1.8 For example

$$h(x) = \begin{cases} y^{(i)} & \text{if } x = x^{(i)} \text{ for some training index } i, \\ -10^6 & \text{otherwise.} \end{cases}$$

This is a perfectly well-defined function $\mathbb{R} \to \mathbb{R}$, so if $\mathcal{H}$ is unrestricted it is a member, and it attains $J = 0$, the global minimum. A better optimiser cannot help, because the optimiser's job is to find the minimiser of J over $\mathcal{H}$ and this *is* a minimiser. The defect is in the choice of $\mathcal{H}$, and no amount of work anywhere else in the pipeline can repair it.

Chapter 2 — Linear Regression

2.1 $J(0) = \frac{1}{6}(1^2 + 2^2 + 3^2) = \frac{14}{6} = \frac{7}{3}$. For $\theta_1 = 2$ the residuals are $1, 2, 3$ again, so $J(2) = \frac{7}{3}$ as well. Both sit one unit either side of the minimum at $\theta_1 = 1$, and the parabola is symmetric.

2.2

$$J(\theta_0, \theta_1) = \tfrac{1}{6}\Big[(\theta_0 + \theta_1 - 1)^2 + (\theta_0 + 2\theta_1 - 2)^2 + (\theta_0 + 3\theta_1 - 3)^2\Big].$$

Every bracket is (prediction − truth) for one house. By inspection $\theta_0 = 0$, $\theta_1 = 1$ makes all three vanish, so the minimum is 0 — but you were asked not to minimise, and the point of writing it out is to see that J is a sum of squares of *linear* expressions in θ, which is exactly what makes it a paraboloid.

2.3 $X = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix}$. Then $X(0,1)^\top = (1,2,3)^\top$, the line $y = x$; and $X(1,0)^\top = (1,1,1)^\top$, the horizontal line $y = 1$. The first column of ones is what lets the intercept ride inside the same matrix product.

2.4 Square metres are square feet times a fixed constant (≈ 0.0929), so the two columns are scalar multiples of each other: the matrix has a redundant direction. $X^\top X$ is then singular and cannot be inverted, and the reason is not numerical bad luck but the actual structure of the problem — infinitely many $(\theta_{\text{ft}}, \theta_{\text{m}})$ pairs give literally identical predictions, so the data cannot possibly single one out. There is no unique answer to find.

2.5 Because y is the quantity we are asked to predict and x is handed to us. The loss should measure how wrong the *prediction* was, and the prediction lives on the vertical axis. Perpendicular distance would mix an error in y with a fictitious error in x, and it would also change answer if you rescaled the x-axis — which is a strong hint it is measuring the wrong

thing. (Perpendicular distance is a real method, total least squares, used when both variables are noisy measurements. It is not this course.)

2.6 The two agree to machine precision on the three points. On $n = 10^5$ the loop typically takes tens of milliseconds and the vectorised version tens of microseconds — a factor of roughly 100, and it grows with n. The reason is not that NumPy is a faster language; it is that `X @ theta` dispatches to a BLAS routine that does the whole thing in compiled code with contiguous memory and vector instructions, while the Python loop pays interpreter overhead on every single element.

2.7 X is 5×3: rows are examples, columns are 1, u, and u^2.

$$X = \begin{bmatrix} 1 & -2 & 4 \\ 1 & -1 & 1 \\ 1 & 0 & 0 \\ 1 & 1 & 1 \\ 1 & 2 & 4 \end{bmatrix}.$$

The third column carries the curvature: θ_2 is the only parameter that can bend the fit, since the first two columns can only produce straight lines.

2.8 (a) Yes: take $\phi(u) = (1, \sqrt{u})$, and it is linear regression in that feature. (b) No: θ_1 sits in the exponent, so h is not a weighted sum of fixed numbers. (Taking logs gives $\log y = \log \theta_0 + \theta_1 \log u$, which *is* linear in the new parameters — but that fits a different loss, squared error on $\log y$, and is not the same estimator.) (c) Yes: define a third feature $w = uv$ and use $\phi = (1, u, w)$. Products of measurements are fine; products of parameters are not. (d) No: θ is in the denominator. Note that $1/y$ is linear in θ, so again a transform of the *target* rescues the algebra while changing what you are minimising.

2.9 $A = \sum x_i^2 = 1 + 4 + 9 = 14$ and $B = \sum x_i y_i = 1(2) + 2(3) + 3(7) = 2 + 6 + 21 = 29$. So $\theta_1^\star = 29/14 \approx 2.0714$. Residuals there are $2.071 - 2 = 0.071$, $4.143 - 3 = 1.143$, $6.214 - 7 = -0.786$, giving SSE ≈ 1.929 and $J = \text{SSE}/6 \approx 0.3214$. At $\theta_1 = 2.1714$ and 1.9714 you should get $J \approx 0.3238$ both times — larger, symmetrically, which is the parabola again.

Chapter 3 — Least Squares as Gaussian MLE

3.1 With Laplace noise, $\log L = c - \frac{1}{\sigma} \sum_i |y^{(i)} - \theta^\top x^{(i)}|$. Maximising it minimises the sum of *absolute* residuals, not squares. Squaring was never neutral; it was the Gaussian talking. (The resulting estimator is called least absolute deviations, and it is markedly more robust to outliers, for exactly the reason the squares picture in Chapter 2 predicts.)

3.2 In $\ell(\theta) = -n \log \sqrt{2\pi\sigma^2} - \frac{1}{2\sigma^2} \sum_i r_i^2$, the first term has no θ and the second is a *positive constant* times $\sum r_i^2$. Multiplying an objective by a positive constant cannot move its arg max, so σ is irrelevant to which θ wins. It returns the moment you also estimate it: setting $\partial \ell / \partial \sigma^2 = 0$ gives $\hat{\sigma}^2 = \frac{1}{n} \sum_i r_i^2$, the mean squared residual at the fitted θ.

3.3 A Gaussian prior multiplies the likelihood by $e^{-\|\theta\|^2/(2\tau^2)}$, so the negative log posterior picks up $\frac{1}{2\tau^2}\|\theta\|^2$. That is a ridge tax: Chapter 9, and the full derivation is Section 14.6, where the constant works out to $\lambda = \sigma^2/\tau^2$ once you also clear the $1/(2\sigma^2)$ from the likelihood. A tighter prior is a heavier tax.

3.4 With $n = 1$, $x = 1$, $y = 4$, $h = \theta$: the likelihood is $L(\theta) \propto \exp\big(-(4-\theta)^2/2\sigma^2\big)$, a bell curve peaking at $\theta = 4$, and $J(\theta) = \frac{1}{2}(\theta - 4)^2$, an upward parabola with its minimum at $\theta = 4$. Same location, opposite shape — L has a maximum exactly where J has a minimum, because J is (up to positive constants and an additive constant) the negative log of L. That single picture is the whole content of Chapter 3.

Chapter 4 — Gradient Descent

4.1 Chain rule on $\frac{1}{2n}\sum(h-y)^2$ brings the power 2 down; the inner derivative of $\sum_k \theta_k x_k^{(i)} - y^{(i)}$ with respect to θ_j is $x_j^{(i)}$, because every other term is constant. The 2 cancels the $\frac{1}{2}$, leaving $\frac{1}{n}\sum_i (h_\theta(x^{(i)}) - y^{(i)})x_j^{(i)}$.

4.2 $\nabla J = 2\theta$, so $\theta \leftarrow (1 - 2\alpha)\theta$ and $1 - 2\alpha = 0.5$. From $(3, 4)$: after one step $(1.5, 2)$, after two $(0.75, 1)$. The cost runs $29 \to 4 + 2.25 + 4 = 10.25 \to 4 + 0.5625 + 1 = 5.5625$, using $J = \theta_1^2 + \theta_2^2 + 4$.

4.3 ∇J is defined *at a point*. Taking one coordinate of the step at (θ_0, θ_1) and the next at (θ_0', θ_1) builds a vector that is not ∇J at either point, so the iteration is not $-\alpha\nabla J$ at all. It is a different map with different fixed points — not a rounding error.

4.4 The gradient test fires first, and it is the one lying to you. In a long thin valley the small eigenvalue direction has a nearly flat slope, so $\|\nabla J\|$ can be tiny while θ is still a long way from the bottom. The honest bound is $\|\theta - \theta^\star\| \le \|\nabla J\|/\lambda_{\min}$, and when $\lambda_{\min}$ is small that bound permits an enormous distance. Drifting parameters with a small gradient is the signature of poor conditioning; the cure is scaling (Chapter 8), not a smaller ε.

4.5 The wrong version overwrites in place:

```
th[0] = th[0] - alpha*grad(th)[0]
th[1] = th[1] - alpha*grad(th)[1]  # grad recomputed at the NEW th[0]
```

The right version computes the whole gradient first, then updates:

```
g = grad(th);  th = th - alpha*g
```

On $J = \theta_1^2 + \theta_2^2 + 4$ with $\alpha = 0.4$ from $(3, 4)$ the correct step gives $(1 - 0.8)(3, 4) = (0.6, 0.8)$. Here the two agree, because this J has no cross term and the coordinates genuinely do not interact — which is precisely why you should test the difference on a J containing $\theta_1\theta_2$, where they diverge immediately.

4.6 With $\alpha = 0.1$ the run converges to $\theta \approx (0, 1)$ and $J \to 0$. With $\alpha = 1.5$ it diverges: J grows every iteration and overflows to `inf` or `nan` within a few dozen steps. This is the fourth cartoon — the step is larger than the bowl's curvature permits, so each step lands further up the opposite wall than where it started.

Chapter 5 — The Walkthrough

5.1 $\nabla J = 2\theta$, so one step is $\theta \leftarrow (1 - 2\alpha)\theta$. With $\alpha = 0.1$ the factor is 0.8: $(3, 4) \to (2.4, 3.2) \to (1.92, 2.56)$, and $J = 29 \to 20 \to 14.24$. The iteration tables in that chapter are the rest of the answer; if a row of yours disagrees, you almost certainly used the new θ_1 inside the θ_2 step.

5.2 Each coordinate obeys $\theta_j^{(k+1)} = \theta_j^{(k)} - \alpha \cdot 2\theta_j^{(k)} = (1-2\alpha)\theta_j^{(k)}$, and the coordinates never mix because J has no cross term. Induction on k gives $\theta^{(k)} = (1-2\alpha)^k \theta^{(0)}$. It converges to 0 iff $|1-2\alpha| < 1$, i.e. $0 < \alpha < 1$. The boundary $|1-2\alpha| = 1$ occurs at $\alpha = 0$ (nothing moves) and $\alpha = 1$ (perpetual flip between $\theta^{(0)}$ and $-\theta^{(0)}$).

5.3 $\|(2,3,4)\| = \sqrt{4+9+16} = \sqrt{29} \approx 5.385$. The first stopping rule uses this kind of length, applied to the step $\Delta\theta = \theta^{(k+1)} - \theta^{(k)}$.

5.4 The directional derivative along a unit vector v is $\nabla J \cdot v = \|\nabla J\| \cos\vartheta$. It is maximised at $\vartheta = 0$, when v points along ∇J (steepest ascent), and minimised at $\vartheta = \pi$, when v points along $-\nabla J$ (steepest descent). At $\vartheta = \pi/2$ it is zero: moving along a contour changes nothing, which is what a contour *is*.

5.5 $1-2\alpha = -3$, so $\theta^{(k)} = (-3)^k(3,4)$: $(3,4)$, $(-9,-12)$, $(27,36)$, $(-81,-108)$, and so on, with J multiplied by 9 each step — 29, 229, 2029, 18229. Signs alternate and magnitude explodes. If your printout does not alternate, you have applied the update to J rather than to θ, or dropped the factor of 2 in $\nabla J = 2\theta$.

5.6 For $J = 25(\theta-2)^2$ we have $a = 25$, so the condition $0 < \alpha < 1/a$ gives $\alpha < 0.04$, and the one-step value is $\alpha = 1/(2a) = 0.02$. For $J = 0.01(\theta-2)^2$, $a = 0.01$ gives $\alpha < 100$ and a one-step value of 50. The two answers differ by a factor of 2500 for the same shape of problem: the legal step size is a property of the *curvature*, not of the algorithm, which is why a learning rate copied from someone else's problem is a guess and why scaling your features changes which guesses work.

5.7 $\alpha < 2/\lambda_{\max} = 2/50 = 0.04$. Just under that, the contraction factor along the λ_2 direction is $|1 - 0.04 \times 0.5| = 0.98$, so that component shrinks by only two percent per step. To shrink it by a factor of ten you need $0.98^k \le 0.1$, i.e. $k \ge \ln(0.1)/\ln(0.98) \approx 114$ steps. The condition number is 100, and the roughly hundredfold slowdown is not a coincidence — it is the condition number showing up as iteration count.

5.8 Your α is far too small. Tiny steps make $\|\Delta\theta\| = \alpha\|\nabla J\|$ small regardless of how large the gradient is, so the parameter test can fire while you are nowhere near a minimum. Trust the gradient: it is the actual first-order optimality condition and does not depend on your step size. The parameter test is only meaningful as a companion to it.

Chapter 6 — Gradients and Hessians

6.1 $\nabla(b^\top x) = b$, generalising $\frac{d}{dx}(ax) = a$. For symmetric A, $\nabla(x^\top A x) = 2Ax$, generalising $\frac{d}{dx}(ax^2) = 2ax$. The finite-difference check with $A = \left[\begin{smallmatrix} 2 & 1 \\ 1 & 3 \end{smallmatrix}\right]$ and $x_0 = (1,2)$ gives $2Ax_0 = (8,14)$.

6.2 Because mixed partials commute: $\partial^2 f/\partial x_i \partial x_j = \partial^2 f/\partial x_j \partial x_i$ for a twice continuously differentiable f (Clairaut / Schwarz). Hence $H_{ij} = H_{ji}$.

6.3 The gradient is a vector, and "the gradient of a vector" is not defined — gradients are for functions that return one number. What you actually do is differentiate each component of ∇f, and each of those gradients becomes one column of H. Written honestly: $H = \nabla(\nabla f)^\top$, with $f : \mathbb{R}^n \to \mathbb{R}$, $\nabla f \in \mathbb{R}^n$, $H \in \mathbb{R}^{n \times n}$.

6.4 Expand $\|Ax-b\|^2 = x^\top A^\top A x - 2b^\top A x + b^\top b$. The gradient is $2A^\top A x - 2A^\top b$; setting it to zero gives $A^\top A x = A^\top b$, so $\hat{x} = (A^\top A)^{-1} A^\top b$ when the inverse exists. In linear regression

A is the design matrix X and b is the target vector y, giving $\hat{\theta} = (X^\top X)^{-1} X^\top y$ — the same point gradient descent walks toward, computed in one shot.

6.5 $\nabla^2 J = \frac{1}{n} X^\top X$, which does not depend on θ. It is positive semi-definite because for any v, $v^\top X^\top X v = \|Xv\|^2 \geq 0$. A function whose Hessian is positive semi-definite everywhere is convex, so the surface has no saddle and no second valley, and any stationary point is a global minimum.

6.6 It is illegal exactly when $X^\top X$ is singular, which happens when the columns of X are linearly dependent. Two data situations: (i) a feature is recorded twice in different units, or one feature is a sum of others, so a column is redundant; (ii) you have more features than examples, $d + 1 > n$, so the n rows cannot possibly span $d + 1$ independent directions. Chapter 12 replaces the inverse with the pseudoinverse, which is defined in both cases and returns the minimum-norm least-squares solution.

6.7 $A^\top A = \left[\begin{smallmatrix} 3 & 6 \\ 6 & 14 \end{smallmatrix}\right]$ and $A^\top b = (6,\, 13)^\top$. Solving gives $\hat{x} = (1,\, 0.5)^\top$, so $A\hat{x} = (1.5,\, 2,\, 2.5)^\top$ and $r = b - A\hat{x} = (-0.5,\, 1,\, -0.5)^\top$. Then $A^\top r = (-0.5 + 1 - 0.5,\ -0.5 + 2 - 1.5) = (0, 0)^\top$. Both facts are expected: $r \neq 0$ because b genuinely does not lie in the plane spanned by the two columns, and $A^\top r = 0$ because the closest point in that plane is the one whose leftover is perpendicular to it. Zero residual would mean an exact fit; perpendicular residual means the best possible fit.

6.8 Take $X = \left[\begin{smallmatrix} 1 & 2 \\ 1 & 2 \\ 1 & 2 \end{smallmatrix}\right]$, whose second column is twice the first. With $v = (2, -1)^\top$, $Xv = 0$, hence $v^\top X^\top X v = \|Xv\|^2 = 0$. So the Hessian is positive semi-definite but not positive definite, and the cost surface is perfectly flat along v: moving θ by any multiple of v changes no prediction and therefore no cost. There is a whole line of minimisers rather than a point.

6.9 $J'(\theta) = \theta - 3$ and $H = J'' = 1$. One Newton step from $\theta^{(0)} = 100$ gives $\theta^{(1)} = 100 - H^{-1} J'(\theta^{(0)}) = 100 - \frac{1}{1}(100 - 3) = 3$ — exact, immediately, because J is quadratic so the local model is not an approximation. Gradient descent with $\alpha = 0.1$ gives $e^{(k)} = 0.9^k \cdot 97$, so reaching $|e| < 10^{-6}$ needs $k > \ln(10^{-6}/97)/\ln(0.9) \approx 175$ steps. Newton knows the curvature and can therefore compute exactly how far to go; gradient descent only knows the slope and has to be told how far by a constant you guessed.

6.10 A gradient costs about $2nd = 2 \times 10^6 \times 10^4 = 2 \times 10^{10}$ operations. Newton additionally needs $X^\top X$ at $nd^2 = 10^6 \times 10^8 = 10^{14}$ and its factorisation at $d^3/3 \approx 3 \times 10^{11}$, so roughly 10^{14} in total. The ratio is about 5×10^3: one Newton step costs several thousand gradient steps, and it also needs to hold $d^2 = 10^8$ numbers at once, which at eight bytes each is about 800 MB for the Hessian alone — survivable here, but it grows as d^2, so ten times the features is a hundred times the memory and you are at 80 GB. That is why nobody inverts a Hessian at scale.

Chapter 7 — Batch and Stochastic

7.1 Batch: $\theta_j \leftarrow \theta_j - \alpha \frac{1}{n} \sum_{i=1}^{n} (h_\theta(x^{(i)}) - y^{(i)}) x_j^{(i)}$. Stochastic, one example at a time: $\theta_j \leftarrow \theta_j + \alpha(y^{(i)} - h_\theta(x^{(i)})) x_j^{(i)}$. The plus sign is not a different algorithm — the notes wrote $y - h$ where the batch line wrote $h - y$, and $y - h = -(h - y)$.

7.2 Batch is steady and accurate per step but costs n evaluations for one step, so it is slow on large data. SGD costs one evaluation per step, so it is fast and gets moving immediately, but

each direction is noisy and it never truly settles. Mini-batch averages a small group (say 32): it inherits most of batch's steadiness, because averaging b samples cuts the noise by $\sqrt{b}$, and most of SGD's speed, because $b \ll n$.

7.3 One batch step costs 10^6 gradient evaluations. One SGD epoch also costs 10^6 — but it has taken 10^6 *steps* in that time, against batch's one. Batch can use a larger α, because its direction is the exact gradient and the only limit is curvature; SGD's noise forces a smaller step or a decaying schedule. You might still prefer SGD because progress per unit of computation is what you actually pay for, and the first pass through the data has already moved θ enormously while batch has taken a single step.

7.4 All three descend, and they differ in the *texture* of the curve, not the destination. Batch ($b = n$): smooth and monotone, every point below the last, but very few points because each one is expensive. SGD ($b = 1$): a downward trend buried in violent jitter, and near the optimum it stops improving and just rattles inside a noise ball whose radius is proportional to α. Mini-batch ($b = 32$): the same trend with roughly $\sqrt{32} \approx 5.7$ times less jitter than SGD, and a much smaller noise ball.

7.5 SGD is still moving. It never stops, because a single example's gradient is not zero at the point where the *average* gradient is zero — every row still wants the line to shift toward itself. The consequence for Chapter 4's stopping rules is that none of the three can be applied to raw SGD as written: you must either decay α toward zero, average the iterates, or test the rules on an epoch average rather than on a single step.

Chapter 8 — Vectorization and Scaling

8.1 $h = X\theta$ and $J = \frac{1}{2n}(X\theta - y)^\top(X\theta - y)$. Expand to $\frac{1}{2n}(\theta^\top X^\top X\theta - 2y^\top X\theta + y^\top y)$. The $y^\top y$ term has no θ, so it vanishes under the gradient. Using $\nabla(x^\top Ax) = 2Ax$ and $\nabla(b^\top x) = b$ gives $\frac{1}{n}(X^\top X\theta - X^\top y) = 0$, i.e. $X^\top X\theta = X^\top y$.

8.2 Use the pseudoinverse (`np.linalg.pinv` or `lstsq`); drop a redundant column, since a duplicated or linearly dependent feature is what caused it; or add ridge, since $X^\top X + \lambda I$ is invertible for any $\lambda > 0$. The first becomes Chapter 12.

8.3 $\partial J/\partial\theta_j$ carries a factor $x_j^{(i)}$. With area near 10^3 and bedrooms near 10^0, the area partial is about a thousand times larger, so a step size safe for θ_{area} leaves θ_{beds} frozen, and one large enough to move θ_{beds} makes θ_{area} diverge. The contours stretch from a circle into a long thin ravine. `ch_scaling.py` measures it: the condition number drops from roughly 7×10^7 to about 1.2.

8.4 Freeze μ_j and σ_j computed on the training rows only, and apply those frozen numbers to validation and test. You are forbidden to re-estimate them using test data; doing so lets information from the test set influence the transform, which is leakage, and your reported error becomes optimistic.

8.5 Suppose the training areas are all near 1000 with $\mu = 1000$, $\sigma = 50$, and the test set contains one house at 5000. Done correctly, that house is transformed to $(5000 - 1000)/50 = 80$ — a wild value, which is honest: the model has never seen anything like it and should be expected to do badly. Recompute μ, σ on train and test together and both shift, so the outlier

maps to something like $z \approx 3$ and every training point shifts too. The test point now looks unremarkable, the reported error drops, and the improvement is entirely an artefact of having looked at the test set. The damage is that a number you were going to publish is wrong in the flattering direction.

8.6 Min–max on $\{2, 5, 9, 14\}$: min $= 2$, max $= 14$, range 12, giving $\{0,\ 0.25,\ 0.583,\ 1\}$. For the z-score, $\mu = 7.5$ and (population) $\sigma = \sqrt{81/4} = 4.5$, giving $\{-1.222,\ -0.556,\ 0.333,\ 1.444\}$.

Now the trap: *neither* transform changes the spacing, and if you answered otherwise, check the gaps. Both are affine maps $x \mapsto (x - c)/s$, so all gap *ratios* are preserved exactly — the gaps go $3 : 4 : 5$ before, $0.25 : 0.333 : 0.417$ after min–max, and $0.667 : 0.889 : 1.111$ after the z-score, which are the same ratios three times. This is worth knowing because it kills a common misconception: scaling cannot fix a skewed distribution. If you want to change the shape you need a nonlinear transform such as $\log x$, and that is a different decision with different consequences.

8.7 Appending $x = 200$ gives, for min–max, min $= 2$, max $= 200$, range 198: the original four land at $0,\ 0.015,\ 0.035,\ 0.061$. For the z-score, $\mu = 46$ and $\sigma \approx 77.1$: they land at $-0.571,\ -0.532,\ -0.480,\ -0.415$, with the outlier at $+2.0$.

The lesson is about *absolute* scale, not relative spacing (which, as above, cannot change). Min–max is forced to put the outlier at exactly 1, so everything else is compressed into six percent of the usable range and effectively switched off when a shared learning rate is applied. The z-score is unbounded, so the ordinary points keep a spread of 0.16 and the outlier is thrown clear — and as n grows, σ dilutes the outlier's influence while the range never does. The one-sentence version: min–max lets a single point define your units, and the z-score makes it vote alongside everyone else.

8.8 The ones column has $\mu = 1$ and $\sigma = 0$, so the z-score formula asks you to compute $(1 - 1)/0$ — division by zero. If you instead handled it by subtracting the mean, the column would become all zeros, which is worse than useless: X then has a zero column, $X^\top X$ has a zero row and column and is singular, and the model has lost its intercept entirely, forcing every fitted line through the origin. The intercept is not a measurement and has no units, so there is nothing for scaling to do to it.

Chapter 9 — Regularization

9.1 $J(\theta) = \frac{1}{2n}\big(\|X\theta - y\|_2^2 + \lambda \sum_{j=1}^{d} \theta_j^2\big)$, with θ_0 outside the sum. For $j \geq 1$, $\theta_j \leftarrow \theta_j(1 - \alpha\lambda/n) - \frac{\alpha}{n}\sum_i (h^{(i)} - y^{(i)})x_j^{(i)}$; for $j = 0$, the same line without the shrink factor.

9.2 Add $\lambda_3\theta_3^2 + \lambda_4\theta_4^2$ to J and leave the other coefficients untaxed. With $\lambda_3 = \lambda_4 = 1000$ those two knobs are driven to essentially zero, and the fitted curve loses exactly the two high-order wiggles they controlled.

9.3 Because the update multiplies θ_j by $1 - \alpha\lambda/n$ before the data term is applied. With $\alpha, \lambda > 0$ and $n > 0$ that factor is strictly less than one, so in the absence of data pressure every coefficient decays geometrically toward zero.

9.4 All slopes go to 0 and $h \to \theta_0$, a flat line at the mean: maximal underfitting. `ch_ridge_path.py` confirms it — with a huge λ the slopes vanish and θ_0 lands exactly on $\bar{y}$.

9.5 $\hat{\theta} = (X^\top X + \lambda I)^{-1} X^\top y$. Adding λI adds λ to every eigenvalue of $X^\top X$, so a zero eigenvalue becomes $\lambda > 0$ and the matrix is invertible again — the singular case is repaired, not hidden. You must mask entry $(0,0)$, using $R = \mathrm{diag}(0, 1, \ldots, 1)$ instead of I, or you tax the intercept, which is a prediction level rather than a slope and has no business being shrunk toward zero.

9.6 Write the Lab 3 convention as $J_1 = \frac{1}{2n}\big(\mathrm{SSE} + \lambda_1 \|\theta\|^2\big)$ and the other as $J_2 = \mathrm{SSE} + \lambda_2 \|\theta\|^2$. Since $J_1 = \frac{1}{2n}\big(\mathrm{SSE} + \lambda_1 \|\theta\|^2\big)$ is a positive multiple of $\mathrm{SSE} + \lambda_1 \|\theta\|^2$, the two have the *same minimiser* exactly when $\lambda_1 = \lambda_2$. The *steps* differ, though, because $\nabla J_1 = \frac{1}{2n} \nabla\big[\mathrm{SSE} + \lambda \|\theta\|^2\big]$ while ∇J_2 is the bracket itself. For identical steps you need $\alpha_1 \cdot \frac{1}{2n} = \alpha_2$, i.e. $\alpha_1 = 2n\,\alpha_2$. So with $n = 500$ and $\alpha_2 = 10^{-4}$ you must use $\alpha_1 = 0.1$. Conflating the two conventions is why a λ copied from a textbook can appear to do nothing.

9.7 Training error is monotone because raising λ shrinks the feasible set: by the hard-constraint reading of Chapter 10, a larger λ corresponds to a smaller budget C in $w^\top w \le C$, and minimising the same SSE over a smaller set can only give a larger or equal minimum. Test error is U-shaped because shrinking the set costs you fit (bias rises) but also removes the model's freedom to chase noise (variance falls), and the sum of a rising and a falling curve has an interior minimum.

9.8 Either (i) the whole grid is too coarse and starts too high — if the useful λ is around 10^{-3}, every point you tried except 0 was effectively "very strong" and the search never looked in the right decade; or (ii) there is genuinely nothing to regularise, because $n \gg d$ and the model is not overfitting, in which case $\lambda = 0$ really is best and the flatness is real. Tell them apart by looking at the gap between training and validation error: a large gap means overfitting and points at explanation (i). In either case search a logarithmic grid, $\lambda \in \{10^{-6}, 10^{-5}, \ldots, 10^{2}\}$.

9.9 Before: $\kappa = 9/10^{-8} = 9 \times 10^8$, which in double precision leaves you roughly $16 - 9 = 7$ reliable digits, and squaring it via $X^\top X$ would leave none. After adding $\lambda = 0.01$ the eigenvalues become $\{9.01,\ 4.01,\ 0.01000001\}$ and $\kappa \approx 901$, losing about three digits. Statistics aside, the solve has gone from numerically desperate to routine — a 10^6-fold improvement in conditioning bought by a penalty that barely moves the two large eigenvalues at all.

Chapter 10 — From a Constraint to Lambda

10.1 Form $\mathcal{L} = \|Zw - y\|^2 + \lambda(w^\top w - C)$. The constant $-\lambda C$ does not depend on w, so drop it. Then $\nabla_w = 2Z^\top(Zw - y) + 2\lambda w = 0$, giving $Z^\top Z w + \lambda w = Z^\top y$, which factors as $(Z^\top Z + \lambda I)w = Z^\top y$. Geometrically this is the tangency condition: at the optimum the contour normal and the ball normal are parallel, and λ is the ratio between them.

10.2 As a coordinate that is not inside C. The constraint is $w^\top w \le C$ where $w = (\theta_1, \ldots, \theta_d)$ and θ_0 is simply not one of the variables being bounded. In the full (θ_0, w) space the feasible region is therefore not a ball but an infinite cylinder: a ball in the w coordinates, extended without limit along the θ_0 axis. Differentiating the Lagrangian with respect to θ_0 picks up no λ term, which is exactly the R-mask with $R_{00} = 0$.

Chapter 27 — The Lab 3 Kit

27.1 Because λ charges for coefficient *size*, and size is measured in whatever units the feature happens to use. Unscaled, a feature recorded in thousands gets a tiny coefficient and pays

almost no tax, while a feature near 1 gets a large coefficient and is punished for it — the penalty ends up ranking units rather than importance. The regularized normal equation is $\theta = (X^\top X + \lambda R)^{-1} X^\top y$ with $R = \mathrm{diag}(0, 1, \ldots, 1)$.

27.2 `mu = Xtr[:, 1:].mean(0); sd = Xtr[:, 1:].std(0)` on the training block, then `Xte[:, 1:] = (Xte[:, 1:] - mu) / sd` using those same frozen values. The bug avoided is data leakage.

27.3 No answer key — the point is the recitation. Score yourself on whether you produced, unprompted: the scaled design matrix with an unscaled ones column; μ, σ frozen on train; the regularized normal equation with R rather than I; the shrinkage factor $1 - \alpha\lambda/n$ attached only to $j \geq 1$; and the fact that λ is chosen on validation, never on test. Five items. If you got fewer than four, the gap is almost always the R-mask.

Chapter 28 — The Lab 4 Kit

28.1 It is linear regression because the model is a *linear combination of the parameters* θ_j — the nonlinearity lives entirely in the fixed features $\varphi_j(x)$, which carry no unknowns. For a single feature and degree 3 the design matrix is $\Phi = \begin{bmatrix} \mathbf{1} & x & x^2 & x^3 \end{bmatrix}$ (one row per sample, the first column all ones for the intercept), and we solve the usual $(\Phi^\top \Phi)\theta = \Phi^\top y$.

28.2 It demonstrates *global* vs. *local* support: x^5 influences (and is influenced across) the whole axis but is nearly flat near 0, whereas the Gaussian at $\mu = 0$ responds only within a few s of its centre and so reacts strongly to a step taken right there. To model an isolated spike you add the *Gaussian* — its coefficient edits the curve locally, without rippling the rest of it the way a polynomial term would.

28.3 The *sigmoid* basis: flat, one climb, then a higher flat level is exactly a bounded transition/saturation, which a sum of sigmoids builds directly (the slope s sets the sharpness of the climb). A high-degree polynomial is poor because it cannot saturate — it must eventually shoot to $\pm\infty$ — so it would have to oscillate to fake the two flat regions and would blow up outside the data range.

Chapter 11 — Determinants and Eigenvalues

11.1 $\nabla_A |A| = (\mathrm{adj}\, A)^\top$, which for invertible A equals $|A|(A^{-1})^\top$.

11.2 The diagonal entries of $A\,\mathrm{adj}(A)$ are Laplace expansions of $|A|$ along each row, so each equals $|A|$. The off-diagonal entries are Laplace expansions of a matrix with two identical rows, whose determinant is 0. Hence $A\,\mathrm{adj}(A) = |A| I$.

11.3 $\nabla_A \log|A| = (A^{-1})^\top$, which is just A^{-1} when A is symmetric. That is the case that matters, because a covariance matrix is symmetric, and $\log|A|$ is sitting inside every multivariate Gaussian density.

11.4 Maximise $x^\top A x$ subject to $\|x\|_2^2 = 1$. The Lagrangian is $\mathcal{L} = x^\top A x - \lambda(x^\top x - 1)$, and $\nabla_x \mathcal{L} = 2Ax - 2\lambda x = 0$ gives $Ax = \lambda x$. Substituting back, $x^\top A x = \lambda$, so the maximum value on the sphere is the largest eigenvalue, attained at its unit eigenvector.

11.5 $\|A\|_F = \sqrt{\sum_{i,j} a_{ij}^2}$. Flattening A into a long vector and taking the ordinary Euclidean length gives the same number, which is why it behaves like a vector norm rather than an operator norm.

11.6 The multivariate Gaussian density is $p(x) = (2\pi)^{-d/2}|\Sigma|^{-1/2}\exp\big(-\frac{1}{2}(x-\mu)^\top\Sigma^{-1}(x-\mu)\big)$, so the negative log-likelihood contains $\frac{1}{2}\log|\Sigma|$. It is the normalising constant: a wider covariance spreads the same unit of probability over more volume, so the density must fall, and $\log|\Sigma|$ measures that volume. Compare Chapter 3, where $\Sigma = \sigma^2 I$ and hence $\log|\Sigma| = d\log\sigma^2$ — a constant with no θ in it, which is precisely why σ dropped out of the MLE there. The moment you let Σ depend on parameters, it stops being a constant and $\nabla_A \log|A|$ starts earning its keep.

11.7 For A: $m = \frac{1}{2}(2+2) = 2$, $p = 4-1 = 3$, so $s = \sqrt{4-3} = 1$ and $\lambda = \{3, 1\}$, giving $\lambda_{\max} = 3$. Gradient descent on a cost with Hessian A is stable for $0 < \alpha < 2/\lambda_{\max} = 2/3$.

For B: $m = \frac{1}{2}(4+1) = \frac{5}{2}$, $p = 4-4 = 0$, so $s = \sqrt{25/4 - 0} = \frac{5}{2}$ and $\lambda = \{5, 0\}$, giving $\lambda_{\max} = 5$ and the bound $0 < \alpha < 2/5$.

B is the singular one — you can see it two ways, either from $\det B = 0$ or from the eigenvalue $\lambda = 0$ that came with it, and the row structure says why: row two is exactly half of row one, so B has rank 1. If a matrix like this appeared as $X^\top X$, the normal equations $X^\top X\theta = X^\top y$ would have no unique solution: along the eigenvector for $\lambda = 0$ the cost is perfectly flat, so you can slide θ in that direction forever without changing J. Infinitely many θ fit equally well. This is the situation Chapter 12 resolves with the pseudoinverse (pick the shortest such θ) and Chapter 9 resolves with ridge (shift every eigenvalue up by λ, so 0 becomes $\lambda > 0$ and the flat direction acquires curvature). Notice the collision of notation, and that it is not an accident of this exercise: $\lambda_{\max} = 5$ here is an eigenvalue, while the ridge λ is a knob you choose. The reason the same letter keeps appearing is that the knob acts *on* the spectrum.

Chapters 12–14 — Pseudoinverse, MLE, MAP

12.1 Write $X = U\Sigma V^\top$. Form Σ^+ by replacing each non-zero σ_i with $1/\sigma_i$ and leaving every zero as zero; then transpose the shape. So $X^\dagger = V\Sigma^+U^\top$. You cannot write $1/0$ because a zero singular value is a direction the matrix crushed to nothing — there is no information there to invert. Leaving the zero in place chooses the minimum-norm solution: among all θ achieving the same residual, the shortest one, with no weight in the null space.

14.1 $P(x;\theta)$ means "the distribution of x, parameterised by a fixed but unknown θ" — the frequentist reading, where θ is a knob, not a random variable. $P(x \mid \theta)$ means "the distribution of x conditioned on the random variable θ" — the Bayesian reading, which is what lets you put a prior on θ at all.

14.2 $\ell(\theta) = \sum_i [y_i \log\theta + (1-y_i)\log(1-\theta)] = s\log\theta + (n-s)\log(1-\theta)$ with $s = \sum y_i$. Then $\ell' = \frac{s}{\theta} - \frac{n-s}{1-\theta} = 0$ gives $s(1-\theta) = (n-s)\theta$, so $\hat\theta_{\text{MLE}} = s/n$. The obvious answer, earned.

14.3 $\hat\theta_{\text{MAP}} = \arg\max_\theta \big[\sum_i \log p(y_i \mid \theta) + \log p(\theta)\big] = \arg\min_\theta \big[\text{NLL}(\theta) - \log p(\theta)\big]$. The two are the same estimator when the prior is uniform, since $\log p(\theta)$ is then a constant and cannot move the arg min. For the coin, that prior is $\text{Beta}(1,1)$.

14.4 $\hat\theta_{\text{MAP}} = \dfrac{s+\alpha-1}{n+\alpha+\beta-2}$. The prior acts as $\alpha-1$ extra heads and $\beta-1$ extra tails, added

before the counting starts.

14.5 MLE gives $2/2 = 1$. MAP with $\alpha = \beta = 2$ gives $\frac{2+2-1}{2+2+2-2} = \frac{3}{4}$. MAP refuses to claim certainty, and the term doing it is the $\beta - 1 = 1$ imaginary tail in the denominator, which keeps a never-observed outcome from being assigned probability zero.

14.6 $\Gamma(n) = (n-1)!$ for positive integers, from the general definition $\Gamma(z) = \int_0^\infty t^{z-1}e^{-t}\,dt$. In the Beta density it is only a normalising prefactor: it carries no θ, so it disappears the moment you differentiate the log posterior.

12.2 $X = \left[\begin{smallmatrix}1&2\\2&4\\3&6\end{smallmatrix}\right] = ab^\top$ with $a = (1,2,3)^\top$ and $b = (1,2)^\top$, so $\|a\|^2 = 14$ and $\|b\|^2 = 5$. For a rank-one matrix the pseudoinverse is

$$X^\dagger = \frac{b\,a^\top}{\|a\|^2\|b\|^2} = \frac{1}{70}\begin{bmatrix}1&2&3\\2&4&6\end{bmatrix}.$$

Check: $X^\dagger X = \frac{1}{70}ba^\top ab^\top = \frac{14}{70}bb^\top = \frac{1}{5}\left[\begin{smallmatrix}1&2\\2&4\end{smallmatrix}\right]$, and then $XX^\dagger X = ab^\top \cdot \frac{1}{5}bb^\top = \frac{5}{5}ab^\top = X$. ✓ Note $X^\dagger X \neq I$ — it is the projection onto the one direction X can actually see, and that is the best any pseudoinverse can do here.

Chapter 29 — Learning Paradigms

29.1 Supervised learning has y in the training data and learns a map $x \mapsto y$; unsupervised learning has only x and looks for structure — groups, directions, topics. Supervised: predicting a house price. Unsupervised: clustering customers nobody has labelled.

29.2 Regression predicts a continuous number, classification a discrete label. Chapter 2 was regression. To make it classification, ask instead whether a house sells above the median price; h_θ must then output a class, or a probability of a class, rather than a rupee amount.

29.3 K is the number of clusters, and you choose it before the algorithm runs — it is not learned. Choosing K by looking at test labels destroys the premise: you have quietly turned an unsupervised problem into a supervised one using labels you claimed not to have, and any subsequent claim that the method "discovered" the structure is false. Legitimate alternatives use only x: the elbow of within-cluster variance against K, silhouette scores, or stability under resampling.

29.4 The reward signal, accumulated over time. There is no fixed labelled set, so P is expected total reward and E is interaction with the environment rather than a stack of labelled rows.

29.5 The superscript indexes the training example: $x^{(i)}$ is the i-th example, and it is emphatically *not* x raised to the power i. So $x_j^{(i)}$ reads "feature j of example i".

Appendix A — The Maths Camp

A.1 $f(2) = 3 \cdot 2 - 1 = 5$; the slope is 3, everywhere, because the function is a straight line.

A.2 Slope $= (5-1)/(2-0) = 2$. By the power rule, $\frac{d}{dx}x^3 = 3x^2$.

A.3 $(1,2)\cdot(3,4) = 1\cdot 3 + 2\cdot 4 = 11$, and $\|(3,4)\| = \sqrt{9+16} = 5$.

A.4 All predictions at once are $X\theta$. With n houses, d features and an intercept, X is $n \times (d+1)$ — one row per house, one column per parameter, and the extra column is the ones column carrying θ_0. Then θ is $(d+1) \times 1$ and $X\theta$ is $n \times 1$, one prediction per house, which is the shape check you should run every time.

Appendix B — Where Next

B.1 Same linear score $z = \theta^\top x$, squashed: $h_\theta(x) = \sigma(\theta^\top x) = 1/(1 + e^{-\theta^\top x})$, which lands in $(0, 1)$ and is read as $P(y = 1 \mid x)$. The loss is not squared error — it is the negative log-likelihood of the Bernoulli model,

$$J(\theta) = -\frac{1}{n} \sum_i \Big[y^{(i)} \log h_\theta(x^{(i)}) + (1 - y^{(i)}) \log \big(1 - h_\theta(x^{(i)})\big) \Big],$$

which is the same MLE move as Chapter 3, with a Bernoulli likelihood in place of a Gaussian one. There is no closed form, so gradient descent is not a convenience here but the only option — and the gradient turns out to be $\frac{1}{n} X^\top (h - y)$, identical in shape to the linear case.

B.2 Plenty of defensible answers. Gradient descent is an entire course in convex and non-convex optimisation. The SVD is an entire course in numerical linear algebra. λ is an entire course in statistical learning theory, where the bias–variance picture of Section 9.7 is replaced by actual generalisation bounds. Things this book only points at: classification and the logistic loss; anything non-convex, meaning every neural network; and the whole question of *why* a model that could memorise sometimes generalises anyway, which is where double descent and the modern literature begin.

Insem 2: capacity, LASSO, CV and the bootstrap

19.1 OLS: $\operatorname{tr} \mathbf{S}$ equals the number of columns, $1 + 3 = 4$, whatever n is. Ridge: $\frac{16}{20} + \frac{4}{8} + \frac{1}{5} = 0.8 + 0.5 + 0.2 = 1.5$ effective parameters out of 3.

19.2 $\mathbb{E}[\tilde{A}] = \frac{n}{n+1} A$, so bias $= -\frac{A}{n+1}$. $\operatorname{Var}(\tilde{A}) = \frac{n\sigma^2}{(n+1)^2}$. MSE $= \frac{A^2 + n\sigma^2}{(n+1)^2}$ versus $\frac{\sigma^2}{n}$ for $\hat{A}$. It is smaller exactly when $A^2 < \frac{(2n+1)\sigma^2}{n}$, roughly $A^2 < 2\sigma^2$: a biased estimator can win when the true value is small compared with the noise. Shrinkage (ridge) is this idea in many dimensions.

19.3 (a) variance down, bias unchanged. (b) bias down, variance up. (c) bias up, variance down. (d) the noise term σ^2 down (and with it the variance, since the fits see less noise).

20.1 $\lambda = 2$ (threshold 1): $(3,\ 0,\ 0.5,\ -1)$. $\lambda = 3.2$ (threshold 1.6): $(2.4,\ 0,\ 0,\ -0.4)$. Ridge $\lambda = 1$: halve everything, $(2,\ -0.3,\ 0.75,\ -1)$.

20.2 $g(\theta) - g(0) = \theta^2 - 2\theta^0\theta + \lambda|\theta| = \theta^2 + |\theta|\big(\lambda - 2\theta^0 \operatorname{sign}\theta\big)$. Since $|\theta^0| \le \lambda/2$, $2\theta^0 \operatorname{sign}\theta \le \lambda$, so the bracket is ≥ 0 and the whole expression is $\ge \theta^2 \ge 0$.

20.3 Every coefficient is zero iff $|\theta_j^0| \le \lambda/2$ for all j, i.e. $\lambda \ge 2 \max_j |\mathbf{x}_j^\top \mathbf{y}| = 2\|\mathbf{X}^\top \mathbf{y}\|_\infty$. (This $\lambda_{\max}$ is where regularization paths start.)

21.1 $\operatorname{Var}(\hat{A}) = 4/11 = 0.364$; $\operatorname{Var}(\hat{A}_{-\kappa(i)}) = 4/10 = 0.4$; $\operatorname{Var}(e^{(i)}) = \frac{11}{10} \cdot 4 = 4.4$.

21.2 $3^3 = 27$ ordered samples. Plug-in SD of $(1, 4, 7)$ is $\sqrt{6} = 2.449$, so the ideal bootstrap SE is $2.449/\sqrt{3} = 1.414$ (checked by enumerating all 27).

21.3 5-fold: 5 models, each on 80 points. LOOCV: 100 models, each on 99 points.

Insem 2: distances, Naive Bayes, perceptron, logistic regression

22.1 $\boldsymbol{\mu}_- = (1, 0)$, $\boldsymbol{\mu}_+ = (5, 3)$, $\mathbf{w} = (8, 6)$, $b = 1 - 34 = -33$. $f(3, 1) = 24 + 6 - 33 = -3 < 0$: class -1. (Distances: 5 to $\boldsymbol{\mu}_-$, 8 to $\boldsymbol{\mu}_+$, difference -3.)

22.2 Nearest to 5.4: 5 (0.4), 6 (0.6), 4.6 (0.8). $K = 1$: 3; $K = 2$: $(3 + 2)/2 = 2.5$; $K = 3$: $(3 + 2 + 4.5)/3 = 3.167$.

22.3 Manhattan 7, Euclidean 5, Chebyshev 4. Manhattan is always the largest because $\|\mathbf{v}\|_1 \geq \|\mathbf{v}\|_2 \geq \|\mathbf{v}\|_\infty$ for every vector.

23.1 play: $\log \frac{3}{4} + \log \frac{4}{5} + \log \frac{1}{5} + \log \frac{3}{5} = -1.143$. not: $\log \frac{1}{4} + \log \frac{1}{3} + \log \frac{2}{3} + \log \frac{2}{3} = -1.431$. Predict play.

23.2 $P(\text{high} \mid \neg\text{play}) = \frac{1+0.5}{1+2(0.5)} = 0.75$; $P(\text{rainy} \mid \text{play}) = \frac{0+0.5}{3+1} = 0.125$.

23.3 Full joint: $3(2^{20} - 1) = 3\,145\,725$ conditionals, plus 2 free prior parameters. Naive Bayes: $3 \times 20 = 60$ conditionals plus 2 for the prior, 62 in total.

24.1 Epoch 1: $(1, 0, 1)$, score $0 \Rightarrow \boldsymbol{\theta} = (1, 0, 1)$; $(1, 1, 0)$, score 1, $y = -1 \Rightarrow \boldsymbol{\theta} = (0, -1, 1)$; $(1, 2, 2)$, score $0 \Rightarrow \boldsymbol{\theta} = (1, 1, 3)$; $(1, 1, -1)$: score -1, correct. Epoch 2: $(1, 1, 0)$ scores 2 $\Rightarrow \boldsymbol{\theta} = (0, 0, 3)$. Epoch 3: $(1, 1, 0)$ scores $0 \Rightarrow \boldsymbol{\theta} = (-1, -1, 3)$. Epoch 4: no mistakes. Final $-1 - x_1 + 3x_2 = 0$, after 5 updates.

24.2 $\gamma = \frac{12+4-6}{5} = 2$ (unsigned distance). Functional margin with $y = -1$: -10, a misclassification.

24.3 $25/0.25 = 100$ mistakes. Scaling every $\mathbf{x}$ by 10 multiplies both R and γ by 10, so the bound is unchanged (as it must be: the perceptron's run is just rescaled).

25.1 $z = -4 + 1 + 3 = 0$, $h = 0.5$, predict 1 (ties go to the positive class). Cost for $y = 0$: $-\ln(1 - 0.5) = \ln 2 = 0.693$.

25.2 At $\boldsymbol{\theta} = \mathbf{0}$ every $h = 0.5$. $\nabla J = (0.5 - 1)(1, 2) + (0.5 - 0)(1, -1) = (0,\ -1.5)$. $\boldsymbol{\theta} \leftarrow \mathbf{0} - 0.5(0, -1.5) = (0,\ 0.75)$.

25.3 $\ell' = 4\theta^3 - 3$, $\ell'' = 12\theta^2$. $\theta^{(1)} = 1 - \frac{1}{12} = 0.91667$; $\theta^{(2)} = 0.90863$. Exact: $\theta^* = (3/4)^{1/3} = 0.90856$.

25.4 $(\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$; $(0.909,\ 0.045,\ 0.045)$. Adding c multiplies numerator and denominator by e^c: softmax is unchanged (which is why implementations subtract the maximum score for numerical safety).

E

Worked Solutions to Past-Exam Questions

These are full solutions to the IT582 mid-semester (06-10-2025) and end-semester (26-11-2025) questions that this book actually teaches — convexity, the Hessian, positive-semidefiniteness, least squares and the pseudoinverse, PCA/eigen-analysis, maximum likelihood, Gaussians and covariance, and information theory. The few questions on topics beyond the current chapters (SVM, K-means, GMM) get short, correct answers at the end.

Every numerical answer below was recomputed and checked. Where a result matches the marker's handwriting on the paper, that is a good sign both agree.

E.1 Mid-Semester Exam (06-10-2025)

Q1. Positive semidefinite $\Rightarrow$ eigenvalues ≥ 0

Derivation

A is positive semidefinite (PSD) means $x^\top Ax \geq 0$ for every x. Let (λ, v) be an eigenpair with $v \neq 0$, so $Av = \lambda v$. Left-multiply by $v^\top$:

$$v^\top Av = v^\top(\lambda v) = \lambda \|v\|^2.$$

The left side is ≥ 0 (PSD) and $\|v\|^2 > 0$, so $\lambda = \dfrac{v^\top Av}{\|v\|^2} \geq 0$. Every eigenvalue is non-negative. ■

Q2. Convexity of $J(x, y) = 5x^2 - 10xy + y^2$ via the Hessian

Derivation

(a) Quadratic form / Hessian. $J = \frac{1}{2} z^\top Hz$ with $z = [x, y]^\top$ and

$$H = \begin{bmatrix} \partial_{xx} J & \partial_{xy} J \\ \partial_{yx} J & \partial_{yy} J \end{bmatrix} = \begin{bmatrix} 10 & -10 \\ -10 & 2 \end{bmatrix}.$$

(b)–(c) Definiteness. A twice-differentiable function is convex iff its Hessian is PSD everywhere. Check H: the determinant is $\det H = (10)(2) - (-10)^2 = 20 - 100 = -80 < 0$. A 2×2 symmetric matrix with negative determinant has one positive and one negative eigenvalue (here $\lambda \approx 16.77$ and -4.77), so H is **indefinite**. Because H is not PSD, J **is non-convex** (it is a saddle). Reason: along $x = y$, $J = 5x^2 - 10x^2 + x^2 = -4x^2 < 0$ curves down, while along $y = 0$, $J = 5x^2 > 0$ curves up — opposite curvatures, the signature of a saddle.

Q3. Are affine functions $f(x) = a^\top x + b$ convex?

Derivation

(a) Yes — convex (and concave). (b) Technical reason. $\nabla f = a$ (constant), so the Hessian $\nabla^2 f = 0$, which is both PSD and NSD; hence f is convex and concave. Equivalently, straight from the definition, for any $\theta \in [0, 1]$,

$$f(\theta x + (1-\theta)y) = a^\top(\theta x + (1-\theta)y) + b = \theta f(x) + (1-\theta) f(y),$$

which holds with *equality* — and equality satisfies the convexity inequality $f(\theta x + (1-\theta)y) \le \theta f(x) + (1-\theta) f(y)$ (and the concave one). Affine functions are exactly the functions that are both convex and concave.

Q4. Quadratic least-squares fit $y = c_0 + c_1 x + c_2 x^2$ by pseudoinverse

Data: $x = [0, 1, 2, 3]$, $y = [3, 2, 4, 4]$.

Derivation

(a) $Y = XC$**.** With $C = [c_0, c_1, c_2]^\top$,

$$Y = \begin{bmatrix} 3 \\ 2 \\ 4 \\ 4 \end{bmatrix}, \qquad X = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 1 \\ 1 & 2 & 4 \\ 1 & 3 & 9 \end{bmatrix} \quad (\text{rows } [1,\ x_i,\ x_i^2]).$$

(b) Pseudoinverse form. X is 4×3 (tall), so the least-squares solution is

$$\hat{C} = X^\dagger Y = (X^\top X)^{-1} X^\top Y$$

— no need to compute it numerically; this is the normal-equation / Moore–Penrose form.
(c) Squared error at $C = (2.75, -0.25, 0.25)$**.** Predictions $\hat{y}_i = c_0 + c_1 x_i + c_2 x_i^2$:

$\hat{y} = [2.75,\ 2.75,\ 3.25,\ 4.25]$, $\quad r = y - \hat{y} = [0.25,\ -0.75,\ 0.75,\ -0.25]$, $\quad \text{SSE} = \sum r_i^2 = \boxed{1.25}$.

(d) New error with $c_0 = 2.7$ **(others fixed).** Each prediction drops by 0.05: $\hat{y} = [2.70, 2.70, 3.20, 4.20]$, $r = [0.30, -0.70, 0.80, -0.20]$, SSE $= 0.09 + 0.49 + 0.64 + 0.04 = \boxed{1.26}$. **(e) Decreased? No** — it increased, $1.25 \to 1.26$. **(f) Why.** $C = (2.75, -0.25, 0.25)$ is the least-squares (pseudoinverse) solution, which *globally min-*

imises the SSE of a convex quadratic. At the optimum the residuals satisfy the normal equation; in particular $\sum r_i = 0.25 - 0.75 + 0.75 - 0.25 = 0$. Moving c_0 off 2.75 breaks $\sum r_i = 0$, so the error can only go up. Any perturbation away from the LSQ optimum increases (or ties) the squared error.

E.2 End-Semester Exam (26-11-2025)

Q2. Kernel trick: $\phi(x) = [1,\ \sqrt{2}\,x,\ x^2]^\top$

Derivation

(a) Dot product in feature space. For scalars $x^{(i)}, x^{(j)}$,

$$\phi(x^{(i)})^\top \phi(x^{(j)}) = 1 + (\sqrt{2}x^{(i)})(\sqrt{2}x^{(j)}) + (x^{(i)})^2(x^{(j)})^2 = 1 + 2x^{(i)}x^{(j)} + (x^{(i)}x^{(j)})^2.$$

(b) Same via the kernel. Expand $K(x^{(i)}, x^{(j)}) = (1 + x^{(i)}x^{(j)})^2 = 1 + 2x^{(i)}x^{(j)} + (x^{(i)}x^{(j)})^2$ — identical, so the kernel computes the feature dot product without ever forming ϕ. **(c) Verify with 2 and 3.** Feature route: $\phi(2) = [1, 2\sqrt{2}, 4]$, $\phi(3) = [1, 3\sqrt{2}, 9]$, dot $= 1 + (2\sqrt{2})(3\sqrt{2}) + (4)(9) = 1 + 12 + 36 = \boxed{49}$. Kernel route: $(1 + 2 \cdot 3)^2 = 7^2 = \boxed{49}$. Match.

Q3. PCA on $X_1 = X_2 = [1, 2, 3, 4, 5, 6]$

Derivation

(a) Sample covariance (divide by $N - 1 = 5$). Mean $\bar{X} = (3.5, 3.5)$; deviations for both coordinates are $[-2.5, -1.5, -0.5, 0.5, 1.5, 2.5]$ with $\sum d^2 = 17.5$, so each variance is $17.5/5 = 3.5$, and since the two coordinates are identical the covariance is also 3.5:

$$\Sigma = \begin{bmatrix} 3.5 & 3.5 \\ 3.5 & 3.5 \end{bmatrix}.$$

(b) Eigenvalues / eigenvectors (unit norm). For $\begin{bmatrix} a & a \\ a & a \end{bmatrix}$ the eigenvalues are $2a$ and 0: $\boxed{\lambda_1 = 7,\ \lambda_2 = 0}$. Eigenvectors: $\lambda_1 = 7 \Rightarrow v_1 = \frac{1}{\sqrt{2}}[1, 1]^\top$; $\lambda_2 = 0 \Rightarrow v_2 = \frac{1}{\sqrt{2}}[1, -1]^\top$. **(c) First principal component.** $v_1 = \frac{1}{\sqrt{2}}[1, 1]^\top$ (the direction of maximum variance, along the line $X_1 = X_2$). **(d) Point $(2, 2)$ in both bases.** Standard basis: $(2, 2) = 2e_1 + 2e_2$. PCA basis: project onto v_1, v_2: $\langle (2, 2), v_1 \rangle = \frac{2+2}{\sqrt{2}} = 2\sqrt{2}$, $\langle (2, 2), v_2 \rangle = \frac{2-2}{\sqrt{2}} = 0$. So $(2, 2) = 2\sqrt{2}\, v_1 + 0 \cdot v_2$ — it lies entirely along v_1. **(e) Reconstruct from PC1 only.** Every data point has zero coordinate along v_2 (all points sit on the line $X_1 = X_2$), so projecting onto v_1 and mapping back returns each point *exactly*: reconstruction = original. **(f) MSE.** Since reconstruction is exact, $\boxed{\text{MSE} = 0}$. The data is genuinely one-dimensional; PC1 captures $\lambda_1/(\lambda_1 + \lambda_2) = 7/7 = 100\%$ of the variance.

Q5 (theory sheet). Regression MLE with non-identically-distributed Gaussian errors

Derivation

In class the objective was $\frac{1}{2\sigma^2}\sum_{i=1}^{m}\big(y^{(i)}-\theta^\top x^{(i)}\big)^2$, which came from i.i.d. errors $\varepsilon^{(i)}\sim\mathcal{N}(0,\sigma^2)$ (same σ^2). Now keep Gaussian and independent, but let each error have its *own* variance, $\varepsilon^{(i)}\sim\mathcal{N}(0,\sigma_i^2)$ (heteroscedastic). Then

$$p(y\mid X,\theta)=\prod_{i=1}^{m}\frac{1}{\sqrt{2\pi\sigma_i^2}}\exp\Big(-\frac{(y^{(i)}-\theta^\top x^{(i)})^2}{2\sigma_i^2}\Big).$$

Take $-\log$ and drop the θ-free constants ($\frac{1}{2}\log(2\pi\sigma_i^2)$):

$$\boxed{\min_\theta\ \frac{1}{2}\sum_{i=1}^{m}\frac{1}{\sigma_i^2}\big(y^{(i)}-\theta^\top x^{(i)}\big)^2}.$$

This is **weighted least squares** with weights $w_i=1/\sigma_i^2$: noisier points (large σ_i^2) are down-weighted. In closed form $\hat\theta=(X^\top WX)^{-1}X^\top Wy$ with $W=\mathrm{diag}(1/\sigma_1^2,\dots,1/\sigma_m^2)$. (Equal variances give $W\propto I$ and recover ordinary least squares.)

Q6 (theory sheet). Entropy, cross-entropy, logistic vs LDA, covariance

Derivation

(a) Binary entropy and binary cross-entropy. For a true probability p and a predicted q,

$$H(p)=-p\log p-(1-p)\log(1-p),\qquad H(p,q)=-p\log q-(1-p)\log(1-q).$$

(b) Why "cross". Entropy $H(p)$ uses *one* distribution with itself — the minimum average bits to encode samples from p. Cross-entropy $H(p,q)$ uses *two*: the true p and a *different* model q — the average bits to encode samples from p using a code built for q. The "cross" is the mismatch between the two. Always $H(p,q)\geq H(p)$, and the gap is the KL divergence $D_{\mathrm{KL}}(p\|q)=H(p,q)-H(p)$. (Training a classifier minimises cross-entropy, i.e. drives $q\to p$; the irreducible floor is $H(p)$.) **(c)** $P(Y{=}k\mid X{=}x)$ **for two classes.** *Logistic regression:* directly a sigmoid, $P(Y{=}1\mid x)=\sigma(w^\top x+b)=\dfrac{1}{1+e^{-(w^\top x+b)}}$. *LDA (2 classes, shared covariance Σ):* from Bayes with Gaussian class-conditionals,

$$P(Y{=}k\mid x)=\frac{\pi_k\,\mathcal{N}(x\mid\mu_k,\Sigma)}{\sum_\ell\pi_\ell\,\mathcal{N}(x\mid\mu_\ell,\Sigma)},$$

which (shared Σ) also simplifies to a sigmoid of a *linear* function of x. **(d) How they differ.** Logistic regression is **discriminative** — it models $P(Y\mid X)$ directly and fits w,b. LDA is **generative** — it models $P(X\mid Y)$ (Gaussian per class) and $P(Y)=\pi_k$, then *derives* $P(Y\mid X)$ by Bayes' rule; this is why LDA needs the class means μ_k and the (shared) covariance Σ. **(e) Sample covariance and mean of**

the two feature rows. Points $(0,3),(1,2),(2,4),(3,4)$ (columns of $[X_1; X_2]$). Mean vector $\bar{x} = \left(\frac{0+1+2+3}{4}, \frac{3+2+4+4}{4}\right) = (1.5,\ 3.25)$. Deviations: $X_1 : [-1.5, -0.5, 0.5, 1.5]$, $X_2 : [-0.25, -1.25, 0.75, 0.75]$. With the unbiased $1/(N-1) = 1/3$,

$$\Sigma = \begin{bmatrix} \frac{1}{3}(2.25+0.25+0.25+2.25) & \frac{1}{3}(0.375+0.625+0.375+1.125) \\ \text{sym} & \frac{1}{3}(0.0625+1.5625+0.5625+0.5625) \end{bmatrix} = \begin{bmatrix} 1.667 & 0.833 \\ 0.833 & 0.917 \end{bmatrix}.$$

Q6 (question sheet). Decision-tree entropy and information gain ("Hungry")

Six examples; target Y (Will Wait) = Yes,No,Yes,No,No,Yes ⇒ 3 Yes, 3 No.

Derivation

(a) $H(Y)$. $H(Y) = -\frac{3}{6}\log_2\frac{3}{6} - \frac{3}{6}\log_2\frac{3}{6} = \boxed{1.0}$ bit. **(b) $H(Y \mid X)$ for X = Hungry.** Hungry = Yes: $\{X_1, X_2, X_3\}$ = Yes,No,Yes ⇒ 2 Yes/1 No, $H = -\frac{2}{3}\log_2\frac{2}{3} - \frac{1}{3}\log_2\frac{1}{3} = 0.9183$. Hungry = No: $\{X_4, X_5, X_6\}$ = No,No,Yes ⇒ 1 Yes/2 No, same $H = 0.9183$. Weighting each branch by 3/6: $H(Y \mid X) = \frac{1}{2}(0.9183) + \frac{1}{2}(0.9183) = \boxed{0.9183}$. **(c) Information gain.** IG $= H(Y) - H(Y \mid X) = 1.0 - 0.9183 = \boxed{0.0817}$. **(d) Assessment.** The gain is tiny (≈ 0.08 bit): splitting on Hungry barely reduces uncertainty about waiting, because both branches are still nearly a 2:1 mix. **Hungry is a poor attribute to split on**; a good split would drive each child close to pure (IG near 1).

E.3 Short answers: questions beyond the current chapters

These sit outside the book's chapters (SVM, clustering), but the exam asked them, so here are crisp, correct answers.

Primal. $\min\limits_{w,b,\xi} \frac{1}{2}\|w\|^2 + C\sum\limits_{i=1}^{N}\xi_i$ s.t. $y^{(i)}(w^\top x^{(i)} + b) \geq 1 - \xi_i$ and $\xi_i \geq 0$. **Lagrangian.** $\mathcal{L} = \frac{1}{2}\|w\|^2 + C\sum_i \xi_i - \sum_i \alpha_i\left[y^{(i)}(w^\top x^{(i)} + b) - 1 + \xi_i\right] - \sum_i \mu_i\xi_i$, $\alpha_i, \mu_i \geq 0$. **(b) Number of unknowns (primal):** $w \in \mathbb{R}^d$, $b \in \mathbb{R}$, $\xi \in \mathbb{R}^N$ ⇒ $d + 1 + N$. **(c) Parameters for the final boundary:** only w and b, i.e. $d + 1$ (the ξ_i are slack, discarded after training).

(a) K-means: only the cluster centres, $K \times d = 3 \times 2 = \boxed{6}$. **(b) GMM, *independent* parameters:** means $Kd = 6$; covariances $K \cdot \frac{d(d+1)}{2} = 3 \cdot 3 = 9$ (symmetry gives $d(d+1)/2$ per matrix); mixing weights $K - 1 = 2$ (the sum-to-one constraint removes one). Total $6 + 9 + 2 = \boxed{17}$.

(a) K-means does **hard** assignment (each point to exactly one cluster, by nearest centre); GMM does **soft** assignment (a responsibility/probability per cluster). **(b)** GMM has the multi-cluster assignment — its responsibilities $P(z = k \mid x)$ are probabilities summing to 1 across clusters (e.g. 0.55 and 0.45), so a point belongs partly to several. **(c)** GMM is better when clusters **overlap**: soft responsibilities express the genuine ambiguity a hard boundary

> would have to guess.

Pattern to notice across the whole paper: half of it is this book. Convexity and the Hessian (Ch on gradients & Hessians), PSD and eigen-as-optimisation (determinant/eigen chapter), least squares and the pseudoinverse, MLE and Gaussians, weighted least squares from heteroscedastic noise (the warn box in the MLE-for-the-line chapter), covariance and PCA (SVD chapter). Learn the derivations here and the "changed terms" on the paper are just the same recipe with new numbers.

Closing: The Whole Story in One Page

If you can reproduce the following list on blank paper, you have the course. Not recognise it — reproduce it.

1. **What ML is:** learning a function h_θ from examples by minimising a loss $J(\theta)$.
2. **Linear regression:** the straight-line model and its convex squared-error bowl.
3. **Why squares:** Gaussian leftovers $\Rightarrow$ MSE is MLE (extra; class probability is still later).
4. **Gradient descent:** $\theta := \theta - \alpha \nabla J$; the cost-gradient derivation; the learning rate.
5. **Class bowl (Lec 6):** $J = \theta_1^2 + \theta_2^2 + 4$, four α, three stopping tests, steepest-ascent proof.
6. **Gradients & Hessians:** vector calculus, and $\hat{\theta} = (X^\top X)^{-1} X^\top y$.
7. **Batch vs SGD:** how much data per step.
8. **Vectorization & scaling (Lec 10):** $h = X\theta$, house-size contours, train-only μ, σ.
9. **Ridge in full (Lec 11):** under/overfit, both λ conventions, shrinkage $1 - \alpha\lambda/n$, regularized NE.
10. **Hard constraint:** $w^\top w \le C$ becomes λ (extra).
11. **Lab 3 kit:** the pocket card with R-masked θ_0.
12. **Det / eigen (Lec 9):** $\nabla_A |A| = |A|(A^{-1})^\top$, $\nabla \log |A| = A^{-\top}$, Rayleigh, trace identities.
13. **Pseudoinverse / SVD:** Σ^+, $X^\dagger = V\Sigma^+ U^\top$, rank-1 example.
14. **Probability & the fork:** frequentist ($P(x;\theta)$, fixed θ) vs Bayesian ($P(x \mid \theta)$, random θ); i.i.d. $\Rightarrow$ likelihood is a product.
15. **MLE & MAP:** Bernoulli MLE $= s/n$, Beta prior, $\hat{\theta}_{\text{MAP}} = (s + \alpha - 1)/(n + \alpha + \beta - 2)$.
16. **Smoothing & fully Bayesian:** Laplace $\frac{s+1}{n+2} = \text{Beta}(1,1)$ posterior mean; Dirichlet for k outcomes; predictive $\int \theta\, p(\theta \mid D)\, d\theta = \frac{s+\alpha}{n+\alpha+\beta}$.
17. **MLE & MAP for the line (Lec 14):** Gaussian noise $\Rightarrow$ least squares; Gaussian prior $\Rightarrow$ ridge; $\lambda = \sigma^2/\sigma_\theta^2$.
18. **Basis functions & generalization (Lec 15):** $h_\theta = \sum_j \theta_j \varphi_j(x)$, polynomial/Gaussian/sigmoid, local vs global, the train/test gap and capacity.
19. **Paradigms (uncut):** supervised / unsupervised / RL, after the learner exists.

20. **Maths camp:** functions, slopes, vectors, matrices, sums — Appendix A.

Every later method is a new choice of *model*, *loss*, or *optimiser* — the loop does not change. After the MLE chapter you also know the *probabilistic* loop: pick a likelihood, maybe a prior, and extremise.

Free resources used to build this book

Openly available references that reinforce these chapters (cited, not copied). Local copies live in `references/` when downloaded. Nothing below is pasted into the text.

Course-adjacent, already on disk.

Stanford CS229 — Notes 1 and Main Notes.
Linear regression, GD, least squares as Gaussian MLE. https://cs229.stanford.edu/notes2020fall/notes2020fall/cs229-notes1.pdf

Princeton COS 324 notes (Arora / Chen).
LR then GD; math in appendices. https://princeton-introml.github.io/

Berkeley CS189 TA guide / Shewchuk Concise ML.
OLS then ridge; pinv with the normal equation.

UCLA CS260 (Hsieh), linear regression note.
Hessian $X^\top X$, closed form vs iterate.

Caltech Learning From Data, lecture 12 library.
Hard constraint → weight decay.

Mathematics for Machine Learning (Deisenroth et al.).
https://mml-book.github.io/book/mml-book.pdf

Boyd & Vandenberghe, Convex Optimization.
https://web.stanford.edu/~boyd/cvxbook/

Boyd & Vandenberghe, VMLS.
https://web.stanford.edu/~boyd/vmls/

S. Ruder, Gradient Descent Optimization Algorithms.
https://arxiv.org/abs/1609.04747

Kingma & Ba, Adam.
https://arxiv.org/abs/1412.6980

Loshchilov & Hutter, AdamW.
https://arxiv.org/abs/1711.05101

Dive into Deep Learning (d2l.ai).
https://d2l.ai/

Non-generic sources in circulation as of August 2026 (Reddit / HN / course-reading-list pointers; we cite the papers, not the threads).

Petersen & Pedersen, The Matrix Cookbook.
Det / log-det / trace gradients.
https://www.math.uwaterloo.ca/~hwolkowi/matrixcookbook.pdf

Parr & Howard, The Matrix Calculus You Need For Deep Learning.
https://arxiv.org/abs/1802.01528

Barata & Hussein, Moore–Penrose tutorial.
https://arxiv.org/abs/1110.6882

Hastie, Ridge Regularization (2020).
https://arxiv.org/abs/2006.00371

van Wieringen, Lecture notes on ridge regression.
https://arxiv.org/abs/1509.09169

Murphy, Conjugate Bayesian analysis of the Gaussian.
MAP = ridge. https://www.cs.ubc.ca/~murphyk/Papers/bayesGauss.pdf

Francis Bach, Learning Theory from First Principles.
https://www.di.ens.fr/~fbach/ltfp_book.pdf

Simon Prince, Understanding Deep Learning (PDF 09 Feb 2026).
https://udlbook.github.io/udlbook/

Fleuret, The Little Book of Deep Learning.
https://fleuret.org/public/lbdl.pdf

Petersen & Zech, Mathematical Theory of Deep Learning.
https://arxiv.org/abs/2407.18384

Osher et al., Mathematical Foundations of Deep Learning (2026).
https://arxiv.org/abs/2603.18387

Strawn / Mathematics of Data Science (2026).
https://arxiv.org/abs/2607.11938

Power et al., Grokking.
https://arxiv.org/abs/2201.02177

Nakkiran et al., Deep Double Descent.
https://arxiv.org/abs/1912.02292

Jordan et al., Muon (2025).
https://arxiv.org/abs/2502.16982

Watch these: hand-picked video derivations (non-generic)

Not the "ML in five minutes" channels. These are lecturers who actually turn the chalk and *derive* the MLE, MAP, ridge-as-a-prior, basis-function, generalization, and linear-algebra results this book leans on. Every link below was checked to resolve. Watch the one next to whichever idea has not clicked yet.

MLE, and MLE for the line = least squares.

mathematicalmonk — "MLE for linear regression".
Gaussian-noise likelihood ⇒ least-squares weights.
https://www.youtube.com/watch?v=n5QZL1B8FaE

Nando de Freitas (UBC) — "Maximum likelihood and linear regression".
LR *as* MLE, with the design matrix. https://www.youtube.com/watch?v=voN8omBe2r4

Ben Lambert — "Deriving the Least Squares Estimators" (parts 1–5).
OLS from first principles. https://www.youtube.com/watch?v=Hi5EJnBHFB4

Kilian Weinberger (Cornell CS4780) — Lecture 7, "MLE".
Rigorous, intuitive; sets up MAP too. https://www.youtube.com/watch?v=RIawrYLVdIw

Herman Kamper — "Maximum likelihood estimation".
The write / log / differentiate recipe, clean.
https://www.youtube.com/watch?v=i6Rp0eiINgM

MAP: Gaussian prior = ridge, $\lambda = \sigma^2/\sigma_\theta^2$.

ritvikmath — "MAP: Why L2 Regularization is Bayesian in Disguise".
Zero-mean Gaussian prior ⇒ ridge; λ as a variance ratio.
https://www.youtube.com/watch?v=0vuszur2uzk

ritvikmath — "Bayesian Linear Regression".
Ridge/LASSO as Gaussian/Laplace priors.
https://www.youtube.com/watch?v=Z6HGJMUakmc

mathematicalmonk — "Posterior for linear regression".
MAP falls out as the ridge solution. https://www.youtube.com/watch?v=qz2U8coNwV4

Bayes, frequentist vs Bayesian, conjugate Beta–Bernoulli.

Ben Lambert — "A Student's Guide to Bayesian Statistics" (playlist).
Beta-conjugate-to-Bernoulli proof inside.
https://www.youtube.com/playlist?list=PLwJRxp3blEvZ8AKMXOy0fc0cqT61GsKCG

Meerkat Statistics — "Conjugate Priors" (playlist).
Rigorous conjugacy, Beta–Bernoulli included.
https://www.youtube.com/playlist?list=PLvCaViUWnpSmNesOOS2V3fWPO0YTvWWKo

ritvikmath — "Conjugate Priors".
Why conjugacy gives closed-form posteriors.
https://www.youtube.com/watch?v=mvGj4OKbDcY

Basis functions / feature maps / design matrix.

Herman Kamper — "Polynomial regression and basis functions".
The design matrix of transformed features; linear-in-parameters.
https://www.youtube.com/watch?v=TSFMepJbHa0

Herman Kamper — "Basis functions and regularisation".
Basis expansion into regularisation. https://www.youtube.com/watch?v=D_rIX0xaYno

Overfitting, generalization, bias–variance, capacity.

Kilian Weinberger (Cornell CS4780) — Lecture 19, "Bias–Variance Decomposition".
bias^2 + variance + noise of expected test error.
https://www.youtube.com/watch?v=zUJbRO0Wavo

Herman Kamper — "Overfitting".
Train-vs-test picture and capacity intuition.
https://www.youtube.com/watch?v=S7B3LQJrU0w

Herman Kamper — "Bias and variance".
Inductive bias/variance tied to over/under-fitting.
https://www.youtube.com/watch?v=djRh0Rkqygw

The linear-algebra spine: SVD, pseudoinverse, projections, the Gaussian.

3Blue1Brown — "Essence of Linear Algebra" (playlist).
Unmatched geometric intuition.
https://www.youtube.com/playlist?list=PLZHQObOWTQDPD3MizzM2xVFitgF8hE_ab

Steve Brunton — "Singular Value Decomposition" (playlist).
SVD from scratch to low-rank.
https://www.youtube.com/playlist?list=PLMrJAkhIeNNSVjnsviglFoY2nXildDCcv

Steve Brunton — "Least Squares Regression and the SVD".
Least squares, geometrically. https://www.youtube.com/watch?v=02QCtHM1qb4

Steve Brunton — "Linear Systems, Least Squares, Pseudoinverse".
Moore–Penrose and the projection view.
https://www.youtube.com/watch?v=PjeOmOz9jSY

Herman Kamper — "The multivariate Gaussian".
Mean vector, covariance, geometry. https://www.youtube.com/watch?v=azrTdjrA2bU

End of the Theory Book

Foundations of Machine Learning — one linear learner; every class derivation kept.
Prepared by Krushna Parmar (202611032), grounded in the course lectures and class notes.